UNIPÊ · Centro Universitário de João Pessoa
Análise e Projeto de Sistemas · Projeto Final · 2026.2

Rufus Bank

"Rufus, iluminando sua vida financeira"

Documento de Apoio (AID) · Da Concepção ao Dia 1 do Dev

Integrante	Papel
Pedro Oliveira	Analista / PO
Izaias Terto	Arquiteto de Software
Carlos Eduardo	Engenheiro / DBA

Professor: Israel Cunha · 04/11/2026

Sumário

A disciplina no projeto 4

Bloco 1 · Concepção e Escopo 6

1. Documento de Visão 6
2. Personas 9
3. Mapa de Stakeholders 11
4. Estudo de Viabilidade 13
4.4 Custo de infraestrutura item a item 16
5. Riscos iniciais 19

Bloco 2 · Requisitos 21

2.0 Processo de Engenharia de Requisitos 21
2.1 Requisitos Funcionais 23
2.2 Requisitos Não Funcionais 26
2.3 Histórias de Usuário 29
2.4 Regras de negócio 35
2.5 Design de interface 36

Bloco 3 · Arquitetura de Software 46

3.1 Architecture Decision Records (ADR) 46
3.2 Modelo C4 52
3.3 Stack tecnológica 62
3.4 Padrões de design 64
3.5 Modelo de domínio (fonte única) 65
3.6 Rotas da API (fonte única) 70

Bloco 4 · Diagramas UML 72

Bloco 5 · Banco de Dados 105

5.1 Decisão do banco de dados 105
5.2 Modelo conceitual (ER) 107
5.3 Modelo lógico 110
5.5 Dicionário de dados 118
5.6 Estratégia: backup, LGPD e migrações 133

Bloco 6 · Infraestrutura e DevOps 136

6.1 Diagrama de infraestrutura e custo por ambiente 136
6.2 Pipeline CI/CD 139
6.3 Containerização 141
6.4 Segurança 143
6.5 Observabilidade 145

Bloco 7 · Planejamento e Estimativas 147

7.1 WBS (Work Breakdown Structure) 147

7.2 Estimativa de esforço e cronograma 151

7.3 Composição da equipe e custo do projeto 156

7.4 Matriz de riscos 159

Bloco 8 · Documentação para o Dev 161

8.1 README do desenvolvedor (do clone ao localhost) 161

8.2 Variáveis de ambiente 163

8.3 Guia de padrões de código 165

8.4 Git workflow 167

8.5 Contrato de API (OpenAPI 3.0) 169

8.6 Sprint 1 detalhado 170

Bloco 9 · Qualidade e Processo 172

9.1 Glossário e modelo de domínio 172

9.2 Estratégia de testes 175

9.3 Metas de cobertura de código 177

9.4 Definition of Ready (DoR) 178

9.5 Definition of Done (DoD) 179

9.6 Metodologia de desenvolvimento 180

Fontes e referências 182

A disciplina no projeto

O Rufus Bank é a conclusão da disciplina: cada conceito das Aulas 01 a 05 e cada item do enunciado do Projeto Final aparece aplicado em algum bloco. Este mapa mostra onde.

As etapas da engenharia de software (Aula 01) percorridas pelo projeto

Etapa da Aula 01	Onde está
Levantamento de necessidades	Bloco 1: problema com dados do Banco Central, Inaf e Febraban, personas
Definição de requisitos	Bloco 2: 32 RF, 14 RNF, 24 histórias, 16 regras de negócio
Análise de viabilidade	Bloco 1, item 4: as 4 perguntas, técnica, operacional e custo real
Documentação dos procedimentos	Blocos 3 a 6 e o AID inteiro
Definição da metodologia	Bloco 9, item 9.6: ciclo de vida Incremental com Scrum
Desenvolvimento	Bloco 7 (9 sprints) e Bloco 8 (o código que roda no localhost)
Teste	Bloco 9: estratégia de testes, cobertura, DoD. Bloco 5: o SQL testado num PostgreSQL real
Implantação	Bloco 6: pipeline com homologação, smoke test e aprovação
Manutenção	Bloco 7: custo de manutenção de 15% a 20% ao ano. Bloco 9: sustentação em Kanban

Conceito por aula

Aula	Conceito	Como aparece no Rufus	Onde
01 · Introdução	Engenharia de software é escala, equipe e durabilidade, não "programar bem"	Projeto para 10.000 usuários, equipe de 8 pessoas e operação com disponibilidade de 99,9%	Blocos 1, 2 e 7
01 · Introdução	Custos: 60% desenvolvimento e 40% teste, manutenção de 15% a 20% ao ano, faixas por tipo de sistema	Estimativa do Rufus conferida contra as faixas da aula	Bloco 7, item 7.3
01 · Introdução	Os 3 P: pessoas, papéis e processos	Trio com papéis definidos (PO, Arquiteto, DBA) e processo Scrum	README, Bloco 9
02 · Processos	BPM (gestão) e BPMN (notação), os 5 elementos mínimos	Processos de negócio do Pix e da conta de menor em BPMN 2.0, com pools e raias	Bloco 4
02 · Processos	Caso Ariane 5: redundância de hardware não protege contra defeito de software	As 2 zonas rodam a mesma imagem. A proteção contra defeito vem de testes, regras no banco, dinheiro em inteiro e deploy que se desfaz sozinho	Bloco 9, item 9.2
03 · Ciclos de vida	NBR ISO/IEC 12207, as 3 etapas e os 5 modelos (Cascata, V, Incremental, Evolutivo, Espiral)	Incremental escolhido e justificado contra os outros 4, com a análise de risco do Espiral e o teste planejado do V	Bloco 9, item 9.6
04 · Requisitos	Definição do IEEE, RF e RNF	RF com MoSCoW e RNF com métrica	Bloco 2
04 · Requisitos	Processo de engenharia de requisitos em 6 passos e técnicas de elicitação	Cada passo aplicado, separando o que foi feito do que fica planejado	Bloco 2, item 2.0
04 · Requisitos	Estudo de viabilidade com as 4 perguntas, inclusive "comprar ou fazer"	Comprar o core bancário (parceiro BaaS) e fazer a experiência	Bloco 1, item 4
04 · Requisitos	Classificação dos RNF (Sommerville)	Cada RNF classificado como de produto, organizacional ou externo	Bloco 2, item 2.2
04 · Requisitos	Regras de negócio (a dinâmica da aula pedia 3)	16 regras, com origem e onde cada uma é garantida	Bloco 2, item 2.4

Aula	Conceito	Como aparece no Rufus	Onde
04 · Requisitos	Backlog: épico, história de usuário e tarefa	7 épicos, 24 histórias e 20 tarefas técnicas	Blocos 2 e 7
04 · Requisitos	PO como ponto único de contato, requisito validado precisa estar escrito	Definition of Ready com aprovação do PO	Bloco 9
05 · Interface	Personas, conteúdo e funcionalidades como base do projeto de interface	As telas partem das 3 personas e das 16 funcionalidades	Bloco 2, item 2.5
05 · Interface	Taxonomia, wireframe, Material Design, protótipo conceito, baixa e alta fidelidade, TER	Árvore de navegação, telas nos três níveis de protótipo e plano de Teste de Resposta Emocional	Bloco 2, item 2.5
05 · Interface	Análise SWOT (FOFA)	Forças, fraquezas, oportunidades e ameaças do Rufus	Bloco 1, item 1

O enunciado do Projeto Final, item por item

Bloco do enunciado	Exigência	Onde está
1 · Concepção	Visão, personas (mín. 3), stakeholders, viabilidade com custo real, riscos (mín. 5)	Bloco 1: 3 personas, 10 stakeholders, custo da tabela oficial da AWS, 8 riscos
2 · Requisitos	RF (mín. 20) com MoSCoW, RNF com métrica, histórias (mín. 15) com Given/When/Then e story points	Bloco 2: 32 RF, 14 RNF, 24 histórias
3 · Arquitetura	ADR, C4 nos 4 níveis, stack com alternativa descartada, padrões	Bloco 3: 13 ADRs, C4 completo, 23 camadas na stack, 13 padrões
4 · UML	Casos de uso, classes (mín. 10, tipadas), sequência x3, componentes, deployment, atividades x2	Bloco 4: os 9 diagramas, mais 2 processos em BPMN
5 · Banco	Decisão, conceitual, lógico, físico executável, dicionário, estratégia	Bloco 5: 27 tabelas, SQL testado em PostgreSQL 17 real, dicionário gerado do banco
6 · Infra	Diagrama com custo, CI/CD, Docker, segurança OWASP, observabilidade	Bloco 6, e os arquivos reais em `codigo/`
7 · Planejamento	WBS (3 níveis), estimativa, equipe com salário de mercado, riscos (mín. 8)	Bloco 7: WBS em 4 níveis, 221 SP em 9 sprints, 12 riscos
8 · Dev	README, .env.example, OpenAPI 3.0, padrões, git workflow, Sprint 1	Bloco 8: o README leva do clone ao localhost, testado
9 · Qualidade	Glossário, testes, cobertura, DoR, DoD, metodologia	Bloco 9
AID	PDF com sumário, os 9 blocos, diagramas com título, autores e data	Este documento, gerado pelo `ferramentas/aid/build.py`

Bloco 1 · Concepção e Escopo

1. Documento de Visão

Nome e slogan

Nome	**Rufus Bank**
Slogan	**"Rufus, iluminando sua vida financeira"**
Domínio	Fintech: conta digital, pagamentos e controle financeiro

Problema real

Bancos digitais têm interfaces complexas, cheias de jargão e com menus escondidos. Isso exclui quem tem pouca leitura, pouca familiaridade com tecnologia ou com finanças. O problema hoje não é ter conta: é conseguir usar a conta sozinho e com segurança.

Os dados mostram isso:

Dado	Número	Fonte
Adultos que não usaram nenhum produto financeiro em 2024 (nem Pix, nem boleto, nem crédito)	21,1 milhões (11,6% dos adultos). Predominam idosos e pessoas sem emprego formal	Banco Central, Relatório de Cidadania Financeira 2025, seção 1.1
Acerto médio nas perguntas de conhecimento financeiro	53%. O Brasil fica em 33º lugar, abaixo da média mundial	Idem, seção 1.2 (pesquisa OCDE/INFE de 2023)
Pessoas que erraram uma conta simples de juros	87%	Idem, seção 1.2
Relação do letramento financeiro com idade e escolaridade	Cai com a idade e sobe com a escolaridade	Idem, seção 1.2
Idosos (60+) que usam internet	59%, contra 43% em 2022	Idem, citando a TIC Domicílios 2024
Analfabetos funcionais (15 a 64 anos)	29%. Na faixa de 50 a 64 anos, 51%	Inaf 2024, divulgado em 05/05/2025
Analfabetos funcionais que usam internet	73%, mas 60% deles estão no nível mais baixo de habilidade digital	Idem
Golpe da falsa central ou falso funcionário de banco	139 mil relatos no 1º semestre de 2025, alta de 195,7% sobre 2024	Febraban, divulgado em 11/11/2025

Leitura dos dados: quase todo mundo já tem acesso, mas uma parte grande da população lê com dificuldade, entende pouco de juros e é alvo de golpes. São exatamente as pessoas que os apps atuais atendem mal.

Proposta de valor

Um banco digital com interface simples, linguagem clara e foco em quem os bancos tradicionais atendem mal.

Na prática, isso vira cinco princípios que valem para todas as telas e que os próximos blocos precisam cumprir:

1. **Ícone e áudio em toda tela.** Tudo que está escrito pode ser ouvido. Quem não lê bem não fica de fora.
2. **Poucos passos.** As ações principais (ver saldo, mandar Pix, pagar conta) terminam em no máximo 3 toques de navegação depois do login. A confirmação com a digital, que protege contra golpe, vem por último e não é cortada para economizar toque.
3. **Linguagem do dia a dia.** Frases curtas, sem jargão. "Dinheiro que você tem" no lugar de "saldo disponível".

4. **Proteção contra golpe por padrão.** O Rufus nunca liga para o cliente. Antes de mandar dinheiro, o app fala em voz alta para quem vai e quanto.
5. **Autonomia com rede de apoio.** O cliente usa sozinho, mas pode ter um contato de confiança e, no caso de menor de idade, um responsável vinculado.

Análise SWOT (FOFA)

Ferramenta de planejamento estratégico vista na Aula 05 para avaliar o cenário interno (forças e fraquezas) e externo (oportunidades e ameaças).

	Ajuda	Atrapalha
Interno	**Forças.** Interface com áudio e ícones em toda tela, coisa que os bancos atuais não fazem. Proteção contra golpe pensada para o público vulnerável. Conta de menor com responsável. Custo de infraestrutura baixo por usuário (R$ 0,38 por mês na AWS em produção)	**Fraquezas.** Marca nova, sem histórico de confiança. Depende de um parceiro BaaS para existir. Equipe pequena, de 8 pessoas
Externo	**Oportunidades.** 21,1 milhões de adultos fora do uso do sistema financeiro. Pix já popular, inclusive entre quem tem pouca leitura. Idosos usando mais a internet (59% em 2024, contra 43% em 2022)	**Ameaças.** Golpes crescendo (falsa central, +195,7% no 1º semestre de 2025). Grandes bancos podem copiar a ideia. Mudança nas regras do Pix ou da LGPD

Cada ameaça e cada fraqueza virou risco com plano de mitigação no item 5 (R01, R04, R06) e na matriz do Bloco 7.

Público-alvo

Adultos com baixa escolaridade ou pouca familiaridade digital, idosos e adolescentes de 16 e 17 anos com renda própria. As três personas do item 2 representam esses grupos.

Atores do sistema

O enunciado pede no mínimo 4 atores. O Rufus tem 7 humanos e 3 sistemas externos:

Ator	Tipo	O que faz
Cliente titular	Humano	Usa a conta: saldo, Pix, contas, metas
Cliente menor (16 e 17 anos)	Humano	Usa a conta dentro dos limites aprovados pelo responsável
Responsável legal	Humano	Autoriza a conta do menor, define limites e aprova pedidos
Contato de confiança	Humano	Recebe aviso de movimentação fora do padrão (familiar ou cuidador)
Atendente do SAC	Humano	Atende chamados abertos dentro do app
Analista antifraude	Humano	Analisa transações suspeitas e bloqueia contas
Administrador	Humano	Cadastra funcionários e define o papel de cada um nos painéis internos
Parceiro BaaS	Sistema externo	Instituição autorizada pelo Banco Central que mantém a conta e liquida o Pix
Serviço de voz	Sistema externo	Transforma texto em fala (Amazon Polly)
Serviço de notificação	Sistema externo	Envia push (Expo Push) e SMS (AWS)

Funcionalidades macro

O enunciado pede no mínimo 12. São 16, que o Bloco 2 detalha em requisitos com MoSCoW:

#	Funcionalidade	Persona que mais precisa
F01	Abertura de conta guiada por voz, com foto do documento e selfie	Antônio, Dona Marlene
F02	Entrada no app por biometria do celular ou senha de 6 números	Todas
F03	Saldo e extrato com ícones e leitura em voz alta	Antônio, Dona Marlene

#	Funcionalidade	Persona que mais precisa
F04	Enviar Pix por contato, chave ou QR Code, com confirmação falada	Todas
F05	Receber Pix por QR Code ou chave	Antônio
F06	Pagar boleto e contas de consumo apontando a câmera	Dona Marlene
F07	Conta de menor vinculada ao responsável	Camila
F08	Limites e pedidos de aprovação ao responsável	Camila
F09	Metas de economia com progresso em imagem ("cofrinho")	Camila
F10	Contato de confiança, avisado em movimentação fora do padrão	Dona Marlene
F11	Alertas contra golpe e botão "bloquear tudo agora"	Todas
F12	Lembrete de conta a vencer	Dona Marlene
F13	Dicas curtas de educação financeira em áudio	Camila, Antônio
F14	Atendimento humano iniciado só de dentro do app	Todas
F15	Painel antifraude para análise de transações suspeitas	Analista antifraude
F16	Painel do SAC com histórico do cliente	Atendente do SAC

Fora do escopo

Fora do escopo	Por quê
Atendimento presencial (agências, postos)	O produto é 100% digital. O atendimento humano existe, mas dentro do app (F14)
Crédito, empréstimo e cartão de crédito	Endividamento é um risco alto para o público-alvo, e crédito exige outra regulação. Fica para uma fase futura
Investimentos	Exige outra autorização e outro nível de letramento. Fica para uma fase futura
Conta para empresa (PJ ou MEI)	O público é pessoa física
Cartão físico e saque em dinheiro	Dependem de logística e de rede de terceiros. O MVP fica no Pix e no boleto
Câmbio, conta em outra moeda e criptoativos	Fora do problema que o projeto resolve
Versão web completa para o cliente	O público usa celular. Na web existe só o site institucional e os painéis internos (F15 e F16)
Core bancário próprio	A conta e a liquidação ficam com o parceiro BaaS (ver item 4)

2. Personas

Três personas, uma para cada grupo que os bancos atuais atendem mal: baixa leitura, menor de idade e idoso. Cada uma aponta para funcionalidades do Documento de Visão (F01 a F16).

Antônio, 47 anos

Profissão	Ajudante de pedreiro, recebe por diária
Escolaridade	Estudou até o 5º ano. Lê com dificuldade
Celular	Android de entrada, usa WhatsApp e áudio
Como lida com dinheiro hoje	Recebe em dinheiro vivo ou pelo cartão de um parente
O que já deu errado	Foi enganado por um golpista que se passou por atendente do banco
Dores	Tem vergonha de pedir ajuda. Textos longos o fazem desistir. Depende de outra pessoa para receber o próprio pagamento
Motivação	Ter uma conta própria que consiga usar por ícones, áudio e poucos passos, sem depender de ler textos longos nem de ninguém
O que o Rufus entrega	Abertura guiada por voz (F01), receber diária por QR Code (F05), saldo falado (F03), confirmação falada antes do Pix (F04), aviso de que o Rufus nunca liga (F11)

> "Eu só quero ver quanto eu tenho e mandar pro mercado sem pedir pra ninguém ler pra mim."

Camila, 17 anos

Profissão	Estudante do ensino médio e jovem aprendiz (contrato da Lei 10.097/2000)
Renda	Um salário pequeno de aprendiz
Celular	Usa bem, nativa digital
Objetivo	Juntar dinheiro para comprar um notebook
Situação legal	Menor de 16 a 18 anos precisa ser assistida pelos pais para atos da vida civil, como abrir conta
Dores	Os bancos atuais ou bloqueiam a conta dela ou a tratam como adulta, sem nenhuma educação financeira. Os pais precisam autorizar tudo
Motivação	Ter autonomia dentro de limites seguros, com conta vinculada ao responsável e metas de economia fáceis de entender
O que o Rufus entrega	Conta de menor vinculada (F07), limites e pedidos de aprovação ao responsável (F08), meta "notebook" com progresso em imagem (F09), dicas curtas em áudio (F13)

> "Eu quero decidir o que fazer com o meu dinheiro, mas sem meus pais terem que aprovar cada lanche."

Dona Marlene, 68 anos

Profissão	Aposentada
Celular	Usa só para WhatsApp
Como lida com o banco hoje	Depende do neto para mexer no app do banco
Dores	Tem medo de errar e perder dinheiro. Letra pequena e muitos menus confundem. Tem medo de golpe por telefone

Motivação	Ter autonomia para pagar as próprias contas sem pedir ajuda
O que o Rufus entrega	Pagar conta apontando a câmera (F06), lembrete de conta a vencer (F12), neto como contato de confiança (F10), botão "bloquear tudo agora" (F11), atendimento humano só pelo app (F14)

> "Eu pago minhas contas desde sempre. Só não quero ter que esperar meu neto ter tempo."

Como as personas viram requisito

As dores das três personas se repetem em quatro pontos, e cada ponto vira um requisito não funcional com número no Bloco 2:

Dor em comum	Personas	Vira no Bloco 2
Ler com dificuldade	Antônio, Dona Marlene	Toda tela com leitura em voz alta e texto em nível de leitura simples
Medo de errar	Dona Marlene, Antônio	Confirmação antes de mover dinheiro e desfazer enquanto não confirmado
Medo de golpe	Antônio, Dona Marlene	Regras antifraude e contato de confiança
Autonomia com limite	Camila, Dona Marlene	Papéis de responsável e de contato de confiança com permissões diferentes

3. Mapa de Stakeholders

Os stakeholders foram classificados pela matriz poder x interesse: **poder** é quanto a pessoa ou instituição pode mudar ou barrar o projeto, **interesse** é quanto o resultado a afeta. O quadrante define como o grupo vai se relacionar com cada um.

Tabela

Stakeholder	Tipo	Interesse no projeto	Influência sobre o projeto	Poder	Interesse	Estratégia
Sócios fundadores	Interno	Produto viável, que cresça e dê retorno	Decidem escopo, prioridade e orçamento	Alto	Alto	Gerenciar de perto
Investidores	Externo	Retorno do capital, crescimento da base de clientes	Liberam ou cortam o dinheiro. Cobram metas de adesão e custo	Alto	Alto	Gerenciar de perto
Parceiro BaaS (instituição autorizada)	Externo	Volume de contas e transações com baixo risco de fraude	Sem ele não existe conta nem Pix. Impõe requisitos de segurança e de prevenção à lavagem de dinheiro	Alto	Alto	Gerenciar de perto
Banco Central	Regulador	Inclusão financeira com segurança e sistema estável	Regula o parceiro e o Pix. Uma regra nova pode mudar funcionalidades	Alto	Médio	Manter satisfeito
ANPD	Regulador	Cumprimento da LGPD, com atenção a dados de menores e dados biométricos	Pode fiscalizar e multar. Exige comunicação de incidente	Alto	Baixo	Manter satisfeito
Usuários finais (Antônio, Camila, Dona Marlene)	Externo	Usar a conta sozinhos e sem cair em golpe	Decidem se o produto dá certo: adesão, uso e avaliação nas lojas	Médio	Alto	Manter informado e envolver nos testes
Famílias e cuidadores	Externo	Segurança do parente sem precisar fazer tudo por ele	Indicam ou desaconselham o app. São responsáveis legais e contatos de confiança	Médio	Alto	Manter informado
Equipe de desenvolvimento	Interno	Requisitos claros, prazo realista, stack bem decidida	Define o que é tecnicamente possível e em quanto tempo	Médio	Alto	Manter informado e envolver nas decisões
Equipe do SAC	Interno	Ferramentas para atender rápido e identificar golpe	Traz as reclamações reais. Influencia as melhorias	Baixo	Alto	Manter informado
Concorrentes (bancos e fintechs)	Externo	Manter a base de clientes	Podem copiar a ideia ou baixar preço	Médio	Baixo	Monitorar

O parceiro BaaS não estava na lista inicial do grupo e foi incluído porque o projeto depende dele para existir (ver item 4 e risco R04).

Matriz poder x interesse



Fonte do diagrama: diagramas/stakeholders.mmd.

Os quatro quadrantes, em uma linha cada:

- **Gerenciar de perto** (alto poder, alto interesse): sócios, investidores e parceiro BaaS. Reunião periódica e aprovação das decisões grandes.
- **Manter satisfeito** (alto poder, interesse menor): Banco Central e ANPD. Cumprir as regras e documentar, sem envolver no dia a dia.
- **Manter informado** (poder menor, alto interesse): usuários, famílias, equipe de desenvolvimento e SAC. Testes de uso, comunicação clara e canal de retorno.
- **Monitorar** (poder e interesse menores): concorrentes. Acompanhar lançamentos e preços.

4. Estudo de Viabilidade

Veredito: o projeto é viável. Tecnicamente, tudo que o Rufus precisa já existe em serviços maduros. Operacionalmente, a condição é ter um parceiro autorizado pelo Banco Central. A infraestrutura custa **R$ 5.446,39 por mês** (produção, homologação e serviços fora da AWS), mais **R$ 25.136,94 de custos únicos** antes do lançamento. Só a produção na AWS dá R$ 0,38 por usuário ativo.

As 4 perguntas do estudo de viabilidade

A Aula 04 define o estudo de viabilidade por 4 perguntas, e lembra que ele pode terminar em cancelar o projeto. As respostas do Rufus:

Pergunta	Resposta	Onde está a prova
O sistema contribui para os objetivos da organização?	Sim. O objetivo do Rufus é atender quem os bancos atendem mal. 21,1 milhões de adultos não usam nenhum produto financeiro e 29% da população de 15 a 64 anos é analfabeta funcional	Documento de Visão, item 1
Pode ser feito com a tecnologia atual, dentro de custo e prazo?	Sim. Toda necessidade tem tecnologia madura (4.1). O custo mensal de infraestrutura é de R$ 5.446,39 e o MVP cabe em 9 sprints com 8 pessoas	Itens 4.1 e 4.3, Bloco 7
Pode ser integrado aos sistemas que já existem?	Sim, por API do parceiro BaaS para conta, Pix e boleto, e por webhook para os eventos. A integração fica isolada numa porta com adaptador, para trocar de parceiro se precisar	ADR-002
Comprar pronto ou fazer aqui?	**Comprar o core bancário e fazer a experiência.** Construir conta, liquidação de Pix e autorização do Banco Central levaria anos e exigiria capital regulatório. Comprar isso de um parceiro BaaS e construir o que é o diferencial do Rufus (interface com áudio, proteção contra golpe, conta de menor) é o melhor custo-benefício	Item 4.2, fora do escopo do Bloco 1

O relatório de viabilidade é este item. Conclusão: seguir com o projeto. A condição para não seguir seria não conseguir nenhum parceiro BaaS (risco R04).

4.1 Viabilidade técnica

Cada necessidade do produto foi comparada com uma tecnologia que já está no mercado. Nenhuma exige pesquisa ou invenção.

Necessidade (do Documento de Visão)	Como se resolve	Maturidade
Conta, saldo e Pix (F03, F04, F05)	API do parceiro BaaS. O Rufus não constrói core bancário nem se conecta direto ao Pix	Modelo comum no mercado brasileiro
Pagar boleto pela câmera (F06)	Leitura do código de barras no próprio celular e pagamento pela API do parceiro	Madura
Leitura em voz alta (princípio 1)	Serviço de texto para fala. A tabela de preços da AWS em São Paulo lista o Amazon Polly a US$ 4 por milhão de caracteres. Frases fixas são geradas uma vez e guardadas no S3	Madura
Entrada por biometria (F02)	Biometria nativa do Android e do iOS. O dado biométrico fica no celular e não vai para o servidor	Madura
Abertura de conta com documento e selfie (F01)	Serviço de validação de identidade do parceiro ou de um fornecedor especializado	Madura
Saldo e movimentação sem erro	Banco relacional com transação ACID. PostgreSQL é a escolha preliminar. A decisão final, comparada com a alternativa, sai no ADR do Bloco 3 e no Bloco 5	Madura
99,9% de disponibilidade	Serviços gerenciados em 2 zonas de disponibilidade, banco Multi-AZ e balanceador de carga	Madura
Conta de menor e contato de confiança (F07, F08, F10)	Regra de negócio própria do Rufus: papéis e permissões sobre a mesma conta	Desenvolvimento próprio, sem risco técnico alto

Desenho preliminar de infraestrutura, só para calcular o custo: app mobile, uma API e um worker em containers no AWS Fargate, PostgreSQL no Amazon RDS, Application Load Balancer com WAF, tudo na região sa-east-1 (São Paulo). A região foi escolhida pela latência para o usuário brasileiro e por manter os dados no país. O Bloco 3 confirma ou muda cada peça com ADR, e o custo é recalculado se mudar.

4.2 Viabilidade operacional

Ponto	Como o Rufus opera
Modelo regulatório	O Rufus começa como empresa de tecnologia, sem autorização própria do Banco Central. A conta e o Pix ficam com um parceiro BaaS (Banking as a Service) que já é autorizado. O Rufus é a interface e a experiência
Regras que o produto precisa seguir	LGPD (Lei 13.709/2018), com cuidado especial com dados de adolescentes (art. 14) e dado biométrico, que é dado sensível. Regras do Pix e de segurança que o Banco Central impõe ao parceiro e que o parceiro repassa por contrato
Conta de menor	Adolescente de 16 e 17 anos abre conta assistido pelo responsável. O responsável aceita os termos e pode ver e limitar a movimentação
Atendimento	SAC humano por chat e telefone, sempre iniciado de dentro do app, depois que o cliente já entrou com biometria. Assim o cliente sabe que quem fala com ele é o Rufus de verdade
Prevenção a fraude	Regras automáticas no backend e analista antifraude para os casos suspeitos
Validação com usuário real	Teste de uso com pessoas do perfil das personas a cada sprint. A meta numérica entra como RNF no Bloco 2
Equipe	O tamanho e o custo da equipe, com pesquisa salarial, ficam no Bloco 7

O ponto mais frágil da operação é a dependência do parceiro BaaS. Ele vira o risco R04 no item 5.

4.3 Custo de infraestrutura mensal

Preços da AWS tirados da tabela oficial (Price List API pública, a mesma base da calculadora em calculator.aws), região South America (São Paulo), preço sob demanda, consultados em 04/11/2026. Preço do SMS tirado do CSV oficial de preços por país da AWS. Serviços fora da AWS (Expo, GitHub, domínio, lojas, teste de invasão) transcritos das páginas de preço de cada fornecedor. Dólar pela PTAX de venda do Banco Central de 04/11/2026: **R$ 5,4778**. Cada fonte, com o link, está no item 4.4.

Ambiente	Para que serve	USD por mês	R$ por mês
Produção (AWS)	Clientes reais, alta disponibilidade em 2 zonas	691,91	3.790,14
Homologação (AWS)	Testes antes de publicar, 1 zona, banco menor	207,50	1.136,64
Serviços fora da AWS	Build do app, push, GitHub, domínio e conta da Apple	94,86	519,62
Desenvolvimento	Cada dev roda tudo na própria máquina com Docker Compose	0,00	0,00
Total mensal		**994,27**	**5.446,39**

Custos únicos antes do lançamento: conta do Google Play (US$ 25) e teste de invasão externo (R$ 25.000, teto da faixa de mercado), total de **R$ 25.136,94**.

A tabela item a item (quantidade, preço unitário, total e fonte de cada serviço) está no item 4.4, em ferramentas/custo-infra/custo-gerado.md. Ela é gerada pelo script custo_infra.py, que baixa os preços oficiais e para com erro se algum preço não for encontrado.

Onde está o dinheiro em produção

Serviço	USD por mês	Por que custa isso
Banco PostgreSQL (RDS Multi-AZ, disco, backup e snapshots mensais)	284,40	Multi-AZ mantém uma cópia em outra zona e assume em caso de falha. É o que sustenta os 99,9%. Os snapshots mensais ficam 12 meses (Bloco 5)

Serviço	USD por mês	Por que custa isso
NAT Gateway (2)	137,64	Um por zona, para a API falar com o parceiro BaaS mesmo se uma zona cair
Containers (Fargate)	62,00	2 tarefas da API, uma em cada zona, e 1 worker
Leitura em voz alta (Polly)	48,00	Fala gerada na hora para saldo, valores e nomes
SMS para o Brasil	30,00	1.000 SMS por mês a US$ 0,03, só para quem ainda não tem o app (convites e avisos)
Restante (balanceador, WAF, logs, saída de dados, IPs, DNS e verificação externa, segredos, S3, ECR)	129,87	Rede, segurança e observabilidade

Premissas de uso

Premissa	Valor	De onde vem
Usuários ativos por mês no 1º ano	10.000	Meta inicial. O Bloco 2 usa o mesmo número no RNF de escalabilidade
Requisições à API por mês	6 milhões	10.000 usuários x 20 acessos x 30 requisições
Dados enviados para a internet	120 GB	6 milhões de respostas de 20 KB
Caracteres falados por mês	12 milhões	Metade dos usuários usa áudio: 5.000 x 30 falas x 80 caracteres
Logs	20 GB por mês, guardados por 90 dias	Estimativa para API e worker com log estruturado
SMS	1.000 por mês	Push é grátis; SMS só para convite do responsável e do contato de confiança e para aviso a quem não tem o app
Usuários que recebem atualização do app (Expo)	10.000	Todos os usuários ativos
Pessoas no GitHub	8	A equipe do Bloco 7
Disponibilidade	99,9%	Meta que obriga 2 zonas, banco Multi-AZ, 2 NAT e balanceador

O que este valor não inclui

- Tarifa do parceiro BaaS por conta, Pix, boleto e validação de identidade. Os parceiros não publicam tabela de preço: depende de contrato e de volume (risco R04).
- Salários da equipe (Bloco 7).
- Descontos de reserva (Savings Plans). Com uso estável depois do lançamento, o custo de containers e banco pode cair.

4.4 Custo de infraestrutura item a item

Anexo do Estudo de Viabilidade. Tabela gerada pelo script `ferramentas/custo-infra/custo_infra.py` a partir da tabela oficial de preços da AWS e das páginas de preço dos outros fornecedores; para mudar, altere as premissas no script e gere de novo.

- Região: AWS sa-east-1 (São Paulo), preço sob demanda, sem desconto de reserva.
- Tabelas de preço AWS publicadas entre 2026-09-11 e 2026-11-04.
- Câmbio: PTAX de venda de 2026-11-04, R$ 5,4778 por dólar (Banco Central).
- Gerado em 04/11/2026.

Produção

Item	Quantidade	Preço unitário (USD)	Total (USD)
API: 2 tarefas Fargate ARM (1 por zona), 0,5 vCPU cada	730 vCPU-h	0,0557	40,66
API: memória das 2 tarefas, 1 GB cada	1.460 GB-h	0,00612	8,94
Worker (filas e notificações): 1 tarefa, 0,25 vCPU	182 vCPU-h	0,0557	10,17
Worker: memória, 0,5 GB	365 GB-h	0,00612	2,23
PostgreSQL RDS db.t4g.medium Multi-AZ	730 h	0,275	200,75
Disco do banco: 50 GB gp3 Multi-AZ	50 GB-mês	0,438	21,90
Backup além da cota grátis (retenção de 35 dias)	50 GB-mês	0,095	4,75
Snapshots mensais guardados 12 meses (12 x 50 GB, Bloco 5)	600 GB-mês	0,095	57,00
Application Load Balancer	730 h	0,034	24,82
ALB: capacidade media de 1 LCU	730 LCU-h	0,011	8,03
NAT Gateway: 2, um por zona	1.460 h	0,093	135,78
NAT: tráfego de saída para o parceiro BaaS	20 GB	0,093	1,86
Saída para a internet (respostas da API)	120 GB	0,25	30,00
IPv4 público: 2 do ALB e 2 dos NAT	2.920 IP-h	0,005	14,60
WAF: 1 web ACL	1 ACL	5	5,00
WAF: 5 regras	5 regras	1	5,00
WAF: requisições inspecionadas	6.000.000 req	0,0000006	3,60
Secrets Manager: 10 segredos	10 segredos	0,4	4,00
CloudWatch Logs: ingestão	20 GB	0,9	18,00
CloudWatch Logs: armazenamento (90 dias)	60 GB-mês	0,0408	2,45
CloudWatch: 10 alarmes	10 alarmes	0,135	1,35
CloudWatch: 20 métricas de negócio	20 métricas	0,3	6,00
S3: comprovantes, documentos e cache de áudio	50 GB-mês	0,0405	2,02
ECR: imagens dos containers	5 GB-mês	0,1	0,50
Polly: leitura em voz alta (12 mi caracteres)	12.000.000 caracteres	0,000004	48,00
Route 53: 1 zona de DNS (rufusbank.com.br)	1 zona	0,5	0,50
Route 53: consultas de DNS	5.000.000 consultas	0,0000004	2,00
Route 53: verificação externa de saúde do /health (RNF03), grátis até 50	1 verificação	0	0,00
Route 53: opções HTTPS e conferência do texto da resposta	2 opções	1	2,00
SMS para o Brasil: convites e avisos a quem não tem o app	1.000 SMS	0,03	30,00

Item	Quantidade	Preço unitário (USD)	Total (USD)
Total			**691,91**

Em reais: **R$ 3.790,14 por mês**.

Homologação (staging)

Item	Quantidade	Preço unitário (USD)	Total (USD)
API: 1 tarefa Fargate ARM, 0,5 vCPU	365 vCPU-h	0,0557	20,33
API: memória, 1 GB	730 GB-h	0,00612	4,47
Worker: 1 tarefa, 0,25 vCPU	182 vCPU-h	0,0557	10,17
Worker: memória, 0,5 GB	365 GB-h	0,00612	2,23
PostgreSQL RDS db.t4g.small Single-AZ	730 h	0,069	50,37
Disco do banco: 20 GB gp3	20 GB-mês	0,219	4,38
Application Load Balancer	730 h	0,034	24,82
ALB: capacidade mínima	73 LCU-h	0,011	0,80
NAT Gateway: 1	730 h	0,093	67,89
NAT: tráfego	2 GB	0,093	0,19
IPv4 público: 2 do ALB e 1 do NAT	2.190 IP-h	0,005	10,95
Secrets Manager: 10 segredos	10 segredos	0,4	4,00
CloudWatch Logs: ingestão	3 GB	0,9	2,70
S3	5 GB-mês	0,0405	0,20
Polly: testes (1 mi caracteres)	1.000.000 caracteres	0,000004	4,00
Total			**207,50**

Em reais: **R$ 1.136,64 por mês**.

Serviços fora da AWS

Preços transcritos da página oficial de cada fornecedor em 04/11/2026.

Item	Valor	Periodicidade	Conta	Equivalente mensal (R$)	Fonte
Expo EAS, plano Starter (build do app e atualizações)	US$ 54,00	mês	US$ 19 + 7.000 usuários acima dos 3.000 incluídos x US$ 0,005	295,80	Expo, página de preços
Notificações push (Expo Push)	US$ 0,00	mês	Sem custo por envio	0,00	Expo, FAQ de push
GitHub Team (código, revisão e CI)	US$ 32,00	mês	8 pessoas x US$ 4; inclui 3.000 minutos de GitHub Actions	175,29	GitHub, página de preços
Domínio rufusbank.com.br	R$ 40,00	ano	Anuidade do .com.br	3,33	Registro.br
Conta de desenvolvedor Apple	US$ 99,00	ano	Anuidade	45,19	Apple Developer Program
Conta de desenvolvedor Google Play	US$ 25,00	único	Taxa única de registro	custo único de 136,94	Google Play Console
Teste de invasão externo antes do lançamento	R$ 25.000,00	único	Teto da faixa de mercado para aplicação + API + autenticação (R$ 10.000 a R$ 25.000). Referência, não cotação	custo único de 25.000,00	FreelaSemCrise, tabela de preços 2026

Equivalente mensal dos serviços externos: **R$ 519,62**. Custos únicos: **R$ 25.136,94**.

Sem preço público, fora do total:

- Tarifa do parceiro BaaS (conta, Pix, boleto e validação de identidade): Os parceiros não publicam tabela de preço; depende de contrato e volume (risco R04).

Resumo por ambiente

Ambiente	USD por mês	R$ por mês
Produção	691,91	3.790,14
Homologação	207,50	1.136,64
Serviços fora da AWS	94,86	519,62
Desenvolvimento	0,00	0,00 (Docker Compose na máquina do dev)
Total	**994,27**	**5.446,39**

Custos únicos (antes do lançamento): **R$ 25.136,94**.

Fontes dos preços

Cada preço unitário da AWS foi lido destes arquivos oficiais (Price List API, pública, sem login), filtrando pelo código de uso (`usagetype`) indicado no script.

Serviço	Tabela oficial	Publicada em
AWSELB	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AWSELB/current/sa-east-1/index.json	2026-11-02
AWSSecretsManager	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AWSSecretsManager/current/sa-east-1/index.json	2026-09-11
AmazonCloudWatch	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonCloudWatch/current/sa-east-1/index.json	2026-11-04
AmazonEC2	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/sa-east-1/index.json	2026-11-04
AmazonECR	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonECR/current/sa-east-1/index.json	2026-09-11
AmazonECS	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonECS/current/sa-east-1/index.json	2026-10-19
AmazonPolly	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonPolly/current/sa-east-1/index.json	2026-10-22
AmazonRDS	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonRDS/current/sa-east-1/index.json	2026-10-29
AmazonRoute53	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonRoute53/current/index.json	2026-11-04
AmazonS3	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonS3/current/sa-east-1/index.json	2026-11-04
AmazonVPC	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonVPC/current/sa-east-1/index.json	2026-11-03
awswaf	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/awswaf/current/sa-east-1/index.json	2026-09-14
SMS para o Brasil (End User Messaging)	https://d1.awsstatic.com/onedam/marketing-channels/website/aws/en_US/business-applications/approved/documents/End-User-Messaging-SMS-Prices.ebc340b4d416d90832dd59629c4792b0deb6f8bc.csv	consultado em 04/11/2026

Câmbio: API de cotações PTAX do Banco Central, https://olinda.bcb.gov.br/olinda/servico/PTAX/versao/v1/odata/, cotação de venda de 2026-11-04.

Conferência manual: os mesmos itens e quantidades podem ser lançados na calculadora oficial, https://calculator.aws, região South America (São Paulo).

5. Riscos iniciais

Os dois riscos mais graves são **golpe contra o usuário vulnerável (R01)** e **baixa adesão (R06)**: os dois atacam o motivo de o Rufus existir. Seis riscos vieram do grupo e dois foram acrescentados (R07 e R08) para cobrir as categorias equipe e regulatório, que a matriz do Bloco 7 exige (mínimo de 8).

Escala

Nota	Probabilidade	Impacto
1	Baixa: pode não acontecer no 1º ano	Baixo: atrasa ou incomoda, sem dano ao cliente
2	Média: pode acontecer no 1º ano	Médio: perda de dinheiro ou prazo que dá para recuperar
3	Alta: deve acontecer no 1º ano	Alto: dano ao cliente, multa ou ameaça à existência do produto

Severidade = probabilidade x impacto (de 1 a 9). De 6 para cima, o risco tem dono e plano desde o início.

Riscos

ID	Risco	Categoria	Prob.	Impacto	Severidade	Mitigação	Sinal de alerta
R01	Fraudes e golpes contra usuários vulneráveis	Segurança	3	3	**9**	O app avisa em toda tela de dinheiro que o Rufus nunca liga pedindo senha. Atendimento só começa de dentro do app. Antes do Pix, o app fala o nome de quem recebe e o valor. Limite menor para destinatário novo e à noite. Contato de confiança avisado em movimentação fora do padrão. Botão "bloquear tudo agora". Analista antifraude para casos suspeitos	Pedidos de devolução de Pix e contestações por mês
R02	Vazamento de dados pessoais	Segurança e regulatório	2	3	**6**	Criptografia em trânsito e em repouso. Coletar só o dado necessário. Segredos no Secrets Manager, nunca no código. WAF na entrada. Acesso administrativo com dois fatores. Log sem dado pessoal. Teste de invasão antes do lançamento. Plano de resposta a incidente com comunicação à ANPD e aos titulares	Alertas de acesso anormal, achados de teste de invasão
R03	Usuários não conseguem usar o app (o oposto do objetivo)	Negócio	2	3	**6**	Teste de uso com pessoas do perfil das personas a cada sprint. Áudio e ícone em toda tela. Linguagem simples revisada. Medir onde as pessoas desistem em cada fluxo	Taxa de abandono por tela, chamados de "não consigo" no SAC
R04	Não conseguir parceria regulatória (BaaS)	Regulatório e negócio	2	3	**6**	Negociar com pelo menos 2 parceiros ao mesmo tempo. Isolar a integração com o parceiro numa camada própria, para trocar de parceiro sem reescrever o sistema (vira ADR no Bloco 3). Começar a conversa antes do desenvolvimento	Nenhum contrato assinado até o fim do 1º sprint
R05	Custo de infraestrutura muito maior que o esperado	Técnico e financeiro	2	2	**4**	Alerta de orçamento na AWS quando o gasto passar de 80% do previsto (R$ 3.790,14 em produção na AWS). Tag de custo por ambiente. Homologação desligada fora do horário de trabalho. Frases fixas em áudio geradas uma vez e guardadas. Reserva de capacidade depois de 3 meses de uso estável	Fatura parcial acima de 80% do orçamento antes do dia 20
R06	Baixa adesão	Negócio	3	3	**9**	Conta sem tarifa. Chegar ao público por quem ele confia: a família (o neto convida a avó), associações, igrejas e quem paga diária na construção civil. Programa de indicação. Medir o uso, não só o download	Contas abertas e usuários ativos por mês abaixo da meta de 10.000

ID	Risco	Categoria	Prob.	Impacto	Severidade	Mitigação	Sinal de alerta
R07	Equipe sem experiência em acessibilidade e em produto financeiro	Equipe	2	2	**4**	Guia de acessibilidade no padrão do projeto (Bloco 8). Revisão de acessibilidade na Definition of Done (Bloco 9). Pelo menos uma pessoa com experiência em fintech na equipe (Bloco 7)	Telas reprovadas no teste de uso por motivo de acessibilidade
R08	Erro no tratamento de dados de menores e de dado biométrico	Regulatório	2	3	**6**	Consentimento do responsável registrado com data e versão do termo. Biometria só no celular, sem guardar no servidor. Campos com dado pessoal marcados no dicionário de dados (Bloco 5)	Pedido de titular ou da ANPD sem resposta no prazo

Matriz de risco

	Impacto 1	Impacto 2	Impacto 3
Probabilidade 3			R01, R06
Probabilidade 2		R05, R07	R02, R03, R04, R08
Probabilidade 1			

Nenhum risco ficou com probabilidade baixa: o projeto lida com dinheiro de um público que já é alvo de golpe, então o grupo preferiu ser conservador.

Bloco 2 · Requisitos

2.0 Processo de Engenharia de Requisitos

Os requisitos deste bloco saíram do processo de Engenharia de Requisitos em 6 passos visto na Aula 04: elicitação, estudo de viabilidade, análise e negociação, especificação, validação e gerência. A tabela diz o que foi feito em cada passo e o que fica planejado para quando houver equipe e usuários reais, sem misturar os dois.

> Requisito, pela definição do IEEE (2008): uma capacidade que um usuário precisa para resolver um problema ou atingir um objetivo, ou que o sistema precisa ter para cumprir um contrato, padrão ou norma.

Os 6 passos no Rufus

Passo	O que é	O que fizemos	O que fica planejado
1. Elicitação	Investigar, descobrir e extrair as necessidades reais, os problemas ocultos e as restrições. Não é só anotar pedidos	**Brainstorming** do grupo para o problema e as personas. **Pesquisa documental** em fontes oficiais: Banco Central, Inaf, Febraban (Bloco 1). **Análise das regras** que restringem o produto: LGPD, Lei 9.613, Resolução BCB 142, Código Civil	**Entrevistas** com 5 pessoas de cada perfil de persona. **Observação** (etnografia): acompanhar uma pessoa idosa pagando uma conta no app do banco atual, para ver onde ela trava. **Prototipação** com os protótipos do item 2.5
2. Estudo de viabilidade	Responder se vale a pena e se dá para fazer	As 4 perguntas da disciplina respondidas no Bloco 1, item 4, com custo real e a decisão de comprar (parceiro BaaS) em vez de fazer o core bancário	Revisar a viabilidade quando o parceiro BaaS mandar a tabela de tarifas
3. Análise e negociação	Achar conflitos entre requisitos e chegar a um acordo	Os 6 conflitos da tabela abaixo, cada um com a decisão	Rodada de negociação com sócios e parceiro BaaS antes do sprint 0
4. Especificação	Registrar e oficializar os requisitos	RF com MoSCoW (2.1), RNF com métrica e classificação (2.2), histórias com Given/When/Then (2.3), regras de negócio (2.4) e matriz de rastreabilidade	Manter no repositório, versionado no git
5. Validação	Mostrar, com o cliente e o usuário, que os requisitos descrevem o sistema certo	Revisão cruzada no trio: cada requisito conferido contra as personas e as fontes. Checklist: completo, sem ambiguidade, mensurável, rastreável	O PO valida e aprova cada história antes do sprint (Definition of Ready, Bloco 9). Teste de uso com 5 pessoas do perfil a cada sprint (RNF08) e TER dos protótipos (item 2.5)
6. Gerência de requisitos	Controlar mudanças, que são regra e não exceção	Rastreabilidade de ponta a ponta: funcionalidade (F) → RF → história (US) → teste. Histórico de mudanças no git	Toda mudança entra como item do backlog. O PO avalia o impacto pela rastreabilidade e reprioriza. Nada muda sem passar pelo PO

Conflitos encontrados e como foram resolvidos

Conflito	Lados	Decisão	Onde ficou
Facilidade de uso x segurança	"Poucos passos" (Antônio) x proteção contra golpe	A segurança entra sem passo extra: biometria no lugar de senha, confirmação falada na mesma tela do Pix, limite só para destinatário novo e à noite	RF003, RF009, RF019
Autonomia do menor x controle dos pais	Camila quer decidir; os pais respondem por ela	Limite diário e mensal definido pelo responsável, e pedido de aprovação só acima do limite	RF015, RF016

Conflito	Lados	Decisão	Onde ficou
Autonomia do idoso x rede de apoio	Dona Marlene não quer depender do neto; a família quer protegê-la	O contato de confiança é só avisado, sem ver saldo nem movimentar	RF018
Voz de qualidade x custo	Áudio em toda tela x orçamento de infraestrutura	Voz gerada no servidor com cache das frases fixas; a voz do celular fica como reserva	ADR-011
Escopo amplo x prazo	Crédito e investimento atrairiam clientes x risco ao público e outra regulação	Fora do escopo do MVP	Bloco 1, fora do escopo
Estágio x jovem aprendiz	O texto inicial da persona Camila citava os dois contratos	Jovem aprendiz (Lei 10.097/2000), que é o caso de renda mensal com carteira	Persona Camila

O que aprendemos com as "situações da vida real" da Aula 04

- **Um ponto único de contato.** O Product Owner é quem mantém o backlog valorado e priorizado. Pedido que chega por outro caminho vai para ele.
- **Requisito validado precisa estar escrito.** O que não está no repositório não existe. Por isso a Definition of Ready exige a história escrita, com critérios de aceite e aprovada pelo PO.
- **Nada é óbvio.** "É só um login" virou 3 requisitos (RF003 a RF005), um ADR (ADR-010) e um diagrama de sequência.

2.1 Requisitos Funcionais

32 requisitos, numerados de RF001 a RF032, cada um ligado a uma funcionalidade do Documento de Visão (F01 a F16) e priorizado por MoSCoW.

Critério de prioridade

Prioridade	Quando usamos	Quantos
Must	Sem ele o MVP não resolve o problema do Bloco 1, não passa pelo parceiro BaaS ou descumpre a lei	22
Should	Importante para as personas, mas o MVP funciona sem ele por algumas semanas	6
Could	Melhora a experiência. Entra se sobrar capacidade	2
Won't	Fica fora desta versão, de forma consciente	2

Requisitos

ID	Requisito	F	Prioridade	Justificativa
RF001	O sistema deve permitir abrir conta com foto do documento e selfie, com cada passo explicado em áudio	F01	Must	Sem conta não há produto. O áudio é o que permite Antônio abrir sozinho
RF002	O sistema deve validar a identidade do cliente no parceiro BaaS antes de liberar a conta	F01	Must	Exigência do parceiro e das regras de prevenção à lavagem de dinheiro
RF003	O sistema deve pedir uma senha de 6 números e oferecer a biometria do celular no primeiro acesso	F02	Must	Senha numérica é mais fácil para quem lê pouco. Biometria evita digitar
RF004	O sistema deve permitir entrar com biometria ou senha e bloquear o acesso por 30 minutos após 5 senhas erradas	F02	Must	Proteção contra tentativa de adivinhar a senha
RF005	O sistema deve manter um único celular ativo por conta e exigir nova selfie para trocar de aparelho	F02	Must	Impede que o golpista instale a conta da vítima no próprio celular (risco R01)
RF006	O sistema deve mostrar o saldo e o extrato com ícone por tipo de movimentação	F03	Must	É a ação mais usada. Ícone reduz a necessidade de leitura
RF007	O sistema deve ler em voz alta o conteúdo de qualquer tela pelo botão fixo "Ouvir"	F03	Must	Princípio 1 do Documento de Visão. Sem isso Antônio e Dona Marlene ficam de fora
RF008	O sistema deve permitir enviar Pix por chave, por contato salvo ou por QR Code	F04	Must	Pix é o meio de pagamento do dia a dia das três personas
RF009	O sistema deve mostrar e falar o nome do recebedor, o banco e o valor antes da confirmação do Pix	F04	Must	Última barreira contra golpe e contra erro de digitação (R01)
RF010	O sistema deve permitir receber Pix por chave e por QR Code com valor	F05	Must	Antônio recebe a diária por Pix. Sem isso ele continua dependendo do parente
RF011	O sistema deve permitir cadastrar e excluir chaves Pix de CPF, celular e aleatória	F05	Must	Pré-requisito para receber Pix por chave
RF012	O sistema deve gerar comprovante de Pix que pode ser ouvido e compartilhado pelo WhatsApp	F04	Should	O comprovante é pedido pelo patrão e pela família. O Pix funciona sem ele nas primeiras semanas
RF013	O sistema deve permitir pagar boleto lendo o código pela câmera ou digitando	F06	Must	Motivação principal da Dona Marlene
RF014	O sistema deve permitir abrir conta para adolescente de 16 ou 17 anos com aceite do responsável legal	F07	Must	Persona Camila. O aceite do responsável é exigido por lei
RF015	O sistema deve permitir que o responsável defina limite diário e mensal de saída da conta do menor	F08	Must	Garante a "autonomia com limite" que a Camila e os pais querem
RF016	O sistema deve permitir que o menor peça aprovação para uma saída acima do limite e que o responsável aprove ou recuse no app	F08	Should	Evita o "tudo bloqueado" dos bancos atuais. O limite fixo já protege no início

ID	Requisito	F	Prioridade	Justificativa
RF017	O sistema deve permitir criar meta de economia com nome, valor, data e imagem, guardar e resgatar dinheiro da meta	F09	Should	Persona Camila (notebook). Não é necessário para movimentar a conta
RF018	O sistema deve permitir cadastrar um contato de confiança, que recebe aviso de movimentação fora do padrão	F10	Should	Rede de apoio da Dona Marlene sem tirar a autonomia dela
RF019	O sistema deve aplicar limite menor para destinatário novo nas primeiras 24 horas e para Pix entre 20h e 6h	F11	Must	Os golpes acontecem com destinatário novo e com urgência (R01)
RF020	O sistema deve oferecer o botão "bloquear tudo agora", que suspende na hora todas as saídas de dinheiro	F11	Must	Resposta imediata quando o cliente percebe o golpe (R01)
RF021	O sistema deve mostrar o aviso "o Rufus nunca liga para você" nas telas de dinheiro e no início do atendimento	F11	Must	Ataca direto o golpe da falsa central, o que mais cresceu em 2025
RF022	O sistema deve lembrar o cliente de contas a vencer que ele cadastrou	F12	Could	Ajuda a Dona Marlene, mas ela já paga as contas sem o lembrete
RF023	O sistema deve oferecer dicas de educação financeira em áudio de até 60 segundos	F13	Could	Diferencial de educação. Não afeta a movimentação
RF024	O sistema deve permitir abrir chamado de atendimento por chat de dentro do app, já identificado	F14	Must	É o único canal humano. Começar dentro do app prova que é o Rufus de verdade
RF025	O sistema deve permitir que o atendente veja o histórico do cliente e responda chamados no painel do SAC	F16	Must	Sem o painel o chamado do RF024 não tem quem responda
RF026	O sistema deve dar uma nota de risco a cada Pix enviado e reter para análise os que passarem do limite de risco	F15	Must	Prevenção de fraude exigida pelo parceiro e pelo risco R01
RF027	O sistema deve permitir que o analista antifraude libere, recuse ou bloqueie a conta a partir do painel	F15	Must	Decisão humana para o Pix retido no RF026
RF028	O sistema deve enviar notificação push a cada entrada e saída de dinheiro	F03	Should	O cliente percebe na hora uma saída que não fez. O extrato já mostra o mesmo
RF029	O sistema deve permitir encerrar a conta e baixar os próprios dados pessoais	F14	Must	Direitos do titular da LGPD (art. 18)
RF030	O sistema deve permitir cadastrar funcionários e atribuir papéis (atendente, analista antifraude, administrador)	F16	Should	Controle de acesso dos painéis. No início os papéis podem ser cadastrados por script
RF031	Agendar Pix para data futura	F04	Won't	Pouco valor para as personas e aumenta a superfície de golpe. Fica para a próxima versão
RF032	Entrar no app por reconhecimento de voz	F02	Won't	A biometria nativa do celular já resolve, sem custo e com mais precisão

Rastreabilidade

Cada funcionalidade do Bloco 1 tem pelo menos um requisito:

F	Requisitos
F01 Abertura de conta	RF001, RF002
F02 Entrada no app	RF003, RF004, RF005, RF032
F03 Saldo e extrato	RF006, RF007, RF028
F04 Enviar Pix	RF008, RF009, RF012, RF031
F05 Receber Pix	RF010, RF011
F06 Pagar contas	RF013
F07 Conta de menor	RF014
F08 Limites e aprovação	RF015, RF016
F09 Metas	RF017
F10 Contato de confiança	RF018

F	Requisitos
F11 Proteção contra golpe	RF019, RF020, RF021
F12 Lembrete de contas	RF022
F13 Educação financeira	RF023
F14 Atendimento e direitos	RF024, RF029
F15 Antifraude	RF026, RF027
F16 Painel do SAC	RF025, RF030

2.2 Requisitos Não Funcionais

14 requisitos, todos com um número que pode ser medido e com a forma de medir. Cada um diz de onde veio o número e qual bloco seguinte ele obriga.

A coluna de classificação segue a taxonomia de Sommerville vista na Aula 04: requisitos de **produto** (como o software se comporta: eficiência, confiabilidade, proteção, usabilidade, portabilidade), **organizacionais** (vêm das políticas e processos da empresa: operação e desenvolvimento) e **externos** (vêm de fora do sistema e da empresa: leis e regulação).

Classificação	Quantos
Produto	9
Organizacional	3
Externo	2

ID	Categoria	Classificação (Sommerville)	Requisito com métrica	Como medir	Prioridade	Por que esse número	Obriga
RNF01	Desempenho	Produto · eficiência	Rotas de leitura da API (saldo, extrato, metas) respondem em menos de **300 ms no p95**, sem contar o tempo do parceiro BaaS	Métrica de latência por rota no CloudWatch e teste de carga	Must	Acima de 1 s o usuário perde a sensação de resposta imediata. 300 ms deixa folga para a rede móvel	Bloco 6: alarme de p95
RNF02	Desempenho	Produto · eficiência	Pix enviado fica confirmado ou recusado na tela em até **10 s no p95**, do toque em "confirmar" à resposta	Métrica de tempo entre pedido e resposta do parceiro	Must	Dona Marlene tem medo de errar. Ficar sem resposta gera clique repetido e Pix duplicado	Bloco 3: chamada ao parceiro com timeout e idempotência
RNF03	Disponibilidade	Produto · confiabilidade	**99,9% ao mês** (até 43 minutos fora do ar por mês)	Verificação externa a cada 1 minuto no endpoint de saúde	Must	Conta é serviço essencial: quem recebe a diária por Pix precisa do app disponível	Bloco 6: 2 zonas, RDS Multi-AZ, balanceador. Já custeado no Bloco 1
RNF04	Escalabilidade	Produto · eficiência	Suportar **10.000 usuários ativos por mês** e picos de **100 requisições por segundo** mantendo o RNF01	Teste de carga com k6 antes de cada versão grande	Must	10.000 é a meta do 1º ano (Bloco 1). 100 req/s é cerca de 40 vezes a média de 6 milhões de requisições por mês	Bloco 6: auto scaling de 2 a 6 tarefas
RNF05	Segurança	Produto · proteção	Senha guardada com **Argon2id**. Token de acesso vale **15 minutos**, token de renovação **30 dias**, trocado a cada uso. Toda conexão em **TLS 1.2 ou superior**	Revisão de código e teste automatizado de configuração	Must	Padrão recomendado pela OWASP para senha e sessão em app financeiro	Bloco 6: seção de segurança
RNF06	Segurança	Organizacional · desenvolvimento	**Nenhuma vulnerabilidade crítica ou alta** conhecida em dependência ou imagem no momento do deploy	Varredura no pipeline (npm audit e Trivy)	Must	Vulnerabilidade conhecida é a porta mais fácil para vazamento (R02)	Bloco 6: o pipeline bloqueia o deploy

ID	Categoria	Classificação (Sommerville)	Requisito com métrica	Como medir	Prioridade	Por que esse número	Obriga
RNF07	LGPD	Externo · legal	**100% dos campos com dado pessoal marcados** no dicionário de dados. CPF e data de nascimento **criptografados** no banco. Pedido do titular respondido em até **15 dias**	Revisão do dicionário de dados e prazo dos pedidos registrados	Must	A LGPD (art. 19) dá 15 dias para a resposta completa ao titular	Bloco 5: dicionário com coluna LGPD
RNF08	Usabilidade	Produto · usabilidade	Ver saldo, enviar Pix e pagar boleto em **no máximo 3 toques de navegação** depois do login. Não contam digitar o valor nem o toque final de confirmação com a digital, que é a barreira de segurança do RF009. Em teste com 5 pessoas do perfil das personas, **pelo menos 4 pagam um boleto sem ajuda em até 3 minutos**	Teste de uso a cada sprint, com cronômetro	Must	É a promessa do produto. 5 pessoas por rodada é o número clássico de teste de usabilidade (Nielsen)	Bloco 9: o teste entra na DoD das telas
RNF09	Acessibilidade	Produto · usabilidade	Seguir a **WCAG 2.2 nível AA**: contraste de pelo menos **4,5:1**, área de toque de pelo menos **48 dp**, texto de leitura (corpo) em **18 sp** e nenhum rótulo abaixo de **14 sp**, funcionar com TalkBack e VoiceOver. **100% das telas** com o botão "Ouvir"	Checklist automático e teste manual com leitor de tela	Must	Antônio e Dona Marlene leem com dificuldade. 48 dp é o mínimo do guia do Android	Bloco 9: DoD
RNF10	Compatibilidade	Produto · portabilidade	Funcionar em **Android 10 ou superior** e **iOS 16 ou superior**, em celular com **3 GB de RAM**	Teste em aparelho real de entrada antes da publicação	Must	Antônio usa um Android de entrada	Bloco 3: escolha do framework mobile
RNF11	Integridade	Produto · confiabilidade	**Zero Pix duplicado** por clique repetido ou falha de rede. Diferença entre o nosso razão contábil e o parceiro **detectada em até 24 horas**	Teste automatizado de idempotência e conciliação diária	Must	Erro de dinheiro destrói a confiança de quem já tem medo de errar	Bloco 3: chave de idempotência. Bloco 5: razão com partidas dobradas
RNF12	Recuperação	Organizacional · operacional	Perda máxima de dados (RPO) de **5 minutos** e volta ao ar (RTO) em até **1 hora** após falha grave	Teste de restauração do backup a cada 3 meses	Must	O RDS restaura para um ponto de até 5 minutos atrás. 1 hora cabe nos 43 minutos do RNF03 quando a falha é rara	Bloco 5: estratégia de backup

ID	Categoria	Classificação (Sommerville)	Requisito com métrica	Como medir	Prioridade	Por que esse número	Obriga
RNF13	Observabilidade	Organizacional · operacional	**100% das requisições** com identificador de correlação no log. Alerta em até **5 minutos** quando a taxa de erro passar de **2%**	Configuração dos alarmes no CloudWatch	Should	Sem isso uma falha só é percebida pela reclamação do cliente	Bloco 6: observabilidade
RNF14	Auditoria	Externo · legal	Toda ação de funcionário (consulta e alteração) registrada e guardada por **5 anos**	Consulta à tabela de auditoria	Must	A Lei 9.613/1998 (prevenção à lavagem de dinheiro) exige guardar registros por 5 anos	Bloco 5: tabela de auditoria

Por que esses são Must

Treze dos 14 são Must porque cada um protege diretamente o dinheiro do cliente (RNF02, RNF05, RNF06, RNF11, RNF12), a promessa do produto (RNF08, RNF09, RNF10), a lei (RNF07, RNF14) ou a meta de operação do Bloco 1 (RNF01, RNF03, RNF04). O RNF13 é Should porque a observabilidade completa pode amadurecer depois do lançamento; o mínimo de alarmes já entra no Must do RNF03.

2.3 Histórias de Usuário

24 histórias no formato "Como [persona], quero [ação], para [objetivo]", cada uma com critérios de aceite em Given/When/Then (Dado/Quando/Então) e estimativa em story points.

Escala de story points

Sequência de Fibonacci (1, 2, 3, 5, 8, 13), estimada pela equipe por comparação. A história de referência é a **US13 (cadastrar contato de confiança) = 3 pontos**: um formulário, uma regra e uma notificação. Uma história de 13 pontos é grande demais para um sprint e precisa ser quebrada antes de entrar.

Do épico à tarefa

O backlog segue a hierarquia da Aula 04: **épico → história de usuário → tarefa**. O épico é grande demais e incerto demais para virar incremento, por isso é fatiado em histórias; a história vira incremento num sprint; as tarefas técnicas são o trabalho para entregar a história (Bloco 7). Os 7 épicos são os mesmos pacotes de produto do WBS (Bloco 7, item 1.2).

Épico	Histórias	SP
E1 Conta e acesso	US01, US02, US03, US17	23
E2 Dinheiro do dia a dia	US04, US05, US06, US07, US08, US21	32
E3 Proteção contra golpe	US11, US12, US13, US16	17
E4 Família	US09, US10, US18	16
E5 Metas e educação	US19, US23, US24	10
E6 Atendimento e avisos	US14, US15, US20	13
E7 Backoffice	US22	3
Total	**24 histórias**	**114**

Exemplo de fatiamento, do épico à tarefa: o épico **E3 Proteção contra golpe** ("Como Dona Marlene, quero estar protegida de golpes sem perder a autonomia") vira a história **US12 Bloquear tudo na hora**, que vira as tarefas: rota `POST /v1/contas/me/bloqueio-total`, atualização de `conta.status` com teste da regra RN07, botão na tela inicial com confirmação falada, aviso ao contato de confiança pelo outbox.

Resumo

ID	Persona	História	RF	MoSCoW	SP
US01	Antônio	Abrir a conta ouvindo cada passo	RF001, RF002	Must	8
US02	Antônio	Entrar com a digital	RF003, RF004	Must	5
US03	Dona Marlene	Trocar de celular com selfie	RF005	Must	5
US04	Antônio	Ouvir o saldo e o extrato	RF006, RF007	Must	5
US05	Antônio	Receber a diária por QR Code	RF010, RF011	Must	5
US06	Dona Marlene	Enviar Pix ouvindo para quem vai	RF008, RF009	Must	8
US07	Antônio	Não mandar Pix em dobro	RF008, RNF11	Must	3
US08	Dona Marlene	Pagar boleto pela câmera	RF013	Must	8
US09	Camila	Abrir conta com aceite do pai	RF014	Must	8
US10	Responsável da Camila	Definir limite da conta da filha	RF015	Must	3
US11	Antônio	Ter limite menor com destinatário novo e à noite	RF019	Must	3
US12	Dona Marlene	Bloquear tudo na hora	RF020, RF021	Must	3

ID	Persona	História	RF	MoSCoW	SP
US14	Antônio	Falar com uma pessoa pelo chat do app	RF024	Must	5
US15	Atendente do SAC	Responder chamado vendo o histórico	RF025	Must	5
US16	Analista antifraude	Analisar Pix retido	RF026, RF027	Must	8
US17	Antônio	Encerrar a conta e baixar meus dados	RF029	Must	5
US13	Dona Marlene	Cadastrar o neto como contato de confiança	RF018	Should	3
US18	Camila	Pedir aprovação acima do limite	RF016	Should	5
US19	Camila	Criar a meta do notebook	RF017	Should	5
US20	Camila	Receber aviso de cada movimentação	RF028	Should	3
US21	Dona Marlene	Ouvir e compartilhar o comprovante	RF012	Should	3
US22	Administrador	Cadastrar funcionário com papel	RF030	Should	3
US23	Dona Marlene	Lembrete de conta a vencer	RF022	Could	3
US24	Camila	Ouvir dicas de dinheiro	RF023	Could	2
		Total			**114**

Must: 87 pontos em 16 histórias. Should: 22 pontos em 6 histórias. Could: 5 pontos em 2 histórias. O Bloco 7 soma a isso as tarefas técnicas (infraestrutura, pipeline, integração com o parceiro) e calcula os sprints.

Histórias com critérios de aceite

US01 · Abrir a conta ouvindo cada passo (8 SP, Must)

Como **Antônio**, quero abrir minha conta ouvindo o que fazer em cada passo, para não depender de ninguém para ler as instruções.

- **Dado** que instalei o app e toquei em "Abrir conta", **quando** cada tela aparece, **então** a instrução é falada automaticamente e pode ser repetida pelo botão "Ouvir".
- **Dado** que enviei foto do documento e selfie, **quando** o parceiro aprova a identidade, **então** a conta fica ativa e o app avisa por voz e por push.
- **Dado** que a foto ficou escura ou cortada, **quando** o envio é recusado, **então** o app fala o motivo em uma frase ("a foto ficou escura, tente perto da janela") e permite tentar de novo.

US02 · Entrar com a digital (5 SP, Must)

Como **Antônio**, quero entrar no app com a minha digital, para não precisar lembrar nem digitar senha.

- **Dado** que criei a senha de 6 números, **quando** aceito usar a biometria, **então** os próximos acessos pedem só a digital.
- **Dado** que errei a senha 5 vezes, **quando** tento a 6ª, **então** o acesso fica bloqueado por 30 minutos e o app explica por voz.
- **Dado** que meu celular não tem biometria, **quando** entro, **então** o app pede a senha de 6 números com teclado grande.

US03 · Trocar de celular com selfie (5 SP, Must)

Como **Dona Marlene**, quero que só o meu celular acesse a minha conta, para que ninguém instale minha conta em outro aparelho.

- **Dado** que minha conta já está ativa num celular, **quando** alguém tenta entrar em outro aparelho, **então** o app exige uma selfie nova e só libera se ela bater com a do cadastro.
- **Dado** que a troca de aparelho foi aprovada, **quando** a nova sessão começa, **então** o aparelho antigo é desconectado e eu recebo aviso no contato cadastrado.

US04 · Ouvir o saldo e o extrato (5 SP, Must)

Como **Antônio**, quero ouvir quanto dinheiro eu tenho e o que entrou e saiu, para saber das minhas contas sem ler.

- **Dado** que entrei no app, **quando** a tela inicial abre, **então** o saldo aparece em número grande com o botão "Ouvir" ao lado.
- **Dado** que toquei em "Ouvir" no extrato, **quando** o áudio toca, **então** cada movimentação é falada como "Recebeu 120 reais de José, ontem".
- **Dado** que estou na tela inicial, **quando** quero ver o extrato, **então** chego nele com 1 toque.

US05 · Receber a diária por QR Code (5 SP, Must)

Como **Antônio**, quero mostrar um QR Code para o mestre de obras me pagar, para receber a diária na minha conta.

- **Dado** que toquei em "Receber", **quando** digito o valor (ou deixo em branco), **então** o app mostra um QR Code Pix em tela cheia.
- **Dado** que o pagamento caiu, **quando** o parceiro confirma, **então** o celular vibra, fala "Recebeu 150 reais de Carlos" e o saldo é atualizado.

US06 · Enviar Pix ouvindo para quem vai (8 SP, Must)

Como **Dona Marlene**, quero ouvir o nome de quem vai receber e o valor antes de confirmar, para ter certeza de que não estou errando.

- **Dado** que informei a chave e o valor, **quando** chego na tela de confirmação, **então** o app mostra e fala o nome do recebedor, o banco e o valor.
- **Dado** que confirmei com a biometria, **quando** o parceiro aceita o Pix, **então** o app mostra "Pix enviado" com ícone verde em até 10 segundos (RNF02).
- **Dado** que o saldo não é suficiente, **quando** tento confirmar, **então** o app avisa antes de pedir a biometria e não envia nada.

US07 · Não mandar Pix em dobro (3 SP, Must)

Como **Antônio**, quero que o Pix não saia duas vezes se eu apertar o botão de novo, para não perder dinheiro.

- **Dado** que toquei em "confirmar" e a internet caiu, **quando** o app reenvia o pedido, **então** o servidor reconhece a mesma chave de idempotência e devolve o resultado do primeiro pedido.
- **Dado** que toquei duas vezes seguidas, **quando** os dois pedidos chegam, **então** só um Pix é criado.

US08 · Pagar boleto pela câmera (8 SP, Must)

Como **Dona Marlene**, quero apontar a câmera para a conta de luz e pagar, para não precisar digitar aquele número enorme.

- **Dado** que abri "Pagar conta", **quando** aponto a câmera para o código de barras, **então** o app lê o código e mostra e fala a empresa, o valor e o vencimento.
- **Dado** que o código não foi lido em 15 segundos, **quando** o tempo acaba, **então** o app oferece digitar o código com teclado grande.
- **Dado** que o boleto está vencido, **quando** o valor é calculado, **então** o app mostra os juros separados e fala o total.

US09 · Abrir conta com aceite do pai (8 SP, Must)

Como **Camila**, quero abrir minha conta com meu pai aceitando pelo celular dele, para não precisar ir a lugar nenhum.

- **Dado** que informei que tenho 17 anos, **quando** termino meu cadastro, **então** o app pede o CPF e o celular do responsável e envia um convite para ele.
- **Dado** que meu pai abriu o convite, **quando** ele confirma a identidade e aceita os termos, **então** minha conta fica ativa vinculada a ele.
- **Dado** que o responsável não aceitou em 7 dias, **quando** o prazo vence, **então** o pedido é cancelado e os dados enviados são apagados.

US10 · Definir limite da conta da filha (3 SP, Must)

Como **responsável da Camila**, quero definir quanto ela pode gastar por dia e por mês, para ela ter autonomia sem risco.

- **Dado** que sou o responsável vinculado, **quando** defino o limite diário e o mensal, **então** os novos limites valem na hora.
- **Dado** que a Camila tenta um Pix acima do limite, **quando** ela confirma, **então** o Pix não é enviado e ela vê a opção de pedir aprovação (US18).

US11 · Limite menor com destinatário novo e à noite (3 SP, Must)

Como **Antônio**, quero que o app limite Pix para alguém novo e à noite, para que um golpista não leve tudo de uma vez.

- **Dado** que o destinatário nunca recebeu Pix meu, **quando** envio nas primeiras 24 horas, **então** o limite é de R$ 300 por Pix.
- **Dado** que são entre 20h e 6h, **quando** envio um Pix, **então** o limite é de R$ 1.000 por período, como na regra do Pix noturno.
- **Dado** que bati o limite, **quando** a mensagem aparece, **então** o app explica o motivo em uma frase e diz quando libera.

US12 · Bloquear tudo na hora (3 SP, Must)

Como **Dona Marlene**, quero um botão para travar a conta na hora, para me proteger se eu achar que caí num golpe.

- **Dado** que toquei em "bloquear tudo agora" e confirmei, **quando** o bloqueio é aplicado, **então** nenhuma saída de dinheiro é aceita e o contato de confiança é avisado.
- **Dado** que a conta está bloqueada, **quando** eu quiser desbloquear, **então** preciso de selfie e de 1 hora de espera.
- **Dado** que estou em qualquer tela de dinheiro, **quando** ela abre, **então** aparece o aviso "o Rufus nunca liga para você".

US13 · Cadastrar o neto como contato de confiança (3 SP, Should)

Como **Dona Marlene**, quero que meu neto seja avisado se sair dinheiro estranho da minha conta, para ter ajuda sem dar a ele acesso à conta.

- **Dado** que informei o nome e o celular do meu neto, **quando** ele aceita o convite, **então** ele vira meu contato de confiança.
- **Dado** que sai um Pix acima de 3 vezes a minha média, **quando** o Pix é confirmado, **então** meu neto recebe aviso com o valor, sem ver meu saldo.

US14 · Falar com uma pessoa pelo chat do app (5 SP, Must)

Como **Antônio**, quero falar com uma pessoa de dentro do app, para ter certeza de que é o banco mesmo.

- **Dado** que toquei em "Falar com uma pessoa", **quando** o chat abre, **então** o atendente já sabe quem eu sou e eu não preciso informar senha nem código.
- **Dado** que prefiro falar, **quando** toco no microfone, **então** posso mandar mensagem de áudio no chat.

US15 · Responder chamado vendo o histórico (5 SP, Must)

Como **atendente do SAC**, quero ver o histórico do cliente ao abrir o chamado, para resolver sem fazer o cliente repetir tudo.

- **Dado** que abri um chamado, **quando** o painel carrega, **então** vejo os chamados anteriores e as últimas 10 movimentações do cliente.
- **Dado** que respondi o chamado, **quando** o cliente lê, **então** a resposta também pode ser ouvida no app.
- **Dado** que consultei os dados do cliente, **quando** a consulta acontece, **então** ela fica registrada na auditoria (RNF14).

US16 · Analisar Pix retido (8 SP, Must)

Como **analista antifraude**, quero ver os Pix retidos com o motivo da nota de risco, para decidir rápido se libero ou bloqueio.

- **Dado** que um Pix passou da nota de risco, **quando** ele é retido, **então** aparece na fila do painel com o motivo (destinatário novo, valor fora do padrão, horário).
- **Dado** que liberei o Pix, **quando** a decisão é salva, **então** o Pix segue para o parceiro e o cliente é avisado.
- **Dado** que recusei e bloqueei, **quando** a decisão é salva, **então** o valor volta ao saldo, a conta fica bloqueada e o cliente recebe aviso para falar com o SAC pelo app.

US17 · Encerrar a conta e baixar meus dados (5 SP, Must)

Como **Antônio**, quero encerrar minha conta e receber meus dados, para exercer meus direitos.

- **Dado** que pedi para baixar meus dados, **quando** o arquivo fica pronto, **então** recebo no app um arquivo com meus dados pessoais e movimentações.
- **Dado** que pedi o encerramento com saldo zero, **quando** confirmo, **então** a conta é encerrada e os dados pessoais são anonimizados, mantidos só os registros que a lei obriga guardar.

US18 · Pedir aprovação acima do limite (5 SP, Should)

Como **Camila**, quero pedir ao meu pai para liberar um gasto acima do limite, para não ficar travada quando preciso.

- **Dado** que meu Pix passou do limite, **quando** toco em "pedir aprovação", **então** meu responsável recebe push com valor e destinatário.
- **Dado** que ele aprovou em até 24 horas, **quando** a aprovação chega, **então** o Pix é enviado e eu sou avisada.
- **Dado** que ele recusou ou não respondeu em 24 horas, **quando** o prazo acaba, **então** o pedido é cancelado.

US19 · Criar a meta do notebook (5 SP, Should)

Como **Camila**, quero criar uma meta com a foto do notebook e ver quanto falta, para ter motivação para guardar.

- **Dado** que criei a meta com nome, valor e data, **quando** guardo dinheiro nela, **então** o valor sai do saldo e a barra de progresso avança.
- **Dado** que resgatei da meta, **quando** confirmo, **então** o valor volta para o saldo.

US20 · Receber aviso de cada movimentação (3 SP, Should)

Como **Camila**, quero receber notificação de cada entrada e saída, para saber na hora se algo estranho aconteceu.

- **Dado** que entrou ou saiu dinheiro, **quando** o lançamento é confirmado, **então** recebo push em até 30 segundos com valor e tipo.

US21 · Ouvir e compartilhar o comprovante (3 SP, Should)

Como **Dona Marlene**, quero mandar o comprovante pelo WhatsApp, para provar que paguei.

- **Dado** que o Pix ou boleto foi concluído, **quando** toco em "comprovante", **então** vejo o comprovante, posso ouvi-lo e compartilhar como imagem.

US22 · Cadastrar funcionário com papel (3 SP, Should)

Como **administrador**, quero cadastrar funcionários com um papel, para que cada um veja só o que precisa.

- **Dado** que cadastrei um atendente, **quando** ele entra no painel, **então** vê os chamados mas não a fila antifraude.
- **Dado** que o funcionário entra pela primeira vez, **quando** faz login, **então** é obrigado a configurar o segundo fator.

US23 · Lembrete de conta a vencer (3 SP, Could)

Como **Dona Marlene**, quero ser lembrada da conta de água, para não pagar multa.

- **Dado** que cadastrei a conta com dia de vencimento, **quando** faltam 3 dias, **então** recebo push e o lembrete aparece na tela inicial.

US24 · Ouvir dicas de dinheiro (2 SP, Could)

Como **Camila**, quero ouvir dicas curtas sobre dinheiro, para aprender a guardar melhor.

- **Dado** que abri "Dicas", **quando** escolho uma, **então** ouço um áudio de até 60 segundos e posso ler o texto.

2.4 Regras de negócio

Regras de negócio são as políticas do Rufus que valem independentemente da tela ou da tecnologia. São 16, cada uma com a origem e o lugar onde é garantida. Quando a regra vive no banco de dados, nem um erro no código consegue quebrá-la.

ID	Regra	Origem	Garantida em
RN01	Titular precisa ter 18 anos ou mais. De 16 a 17 anos, a conta só existe vinculada a um responsável legal. Menor de 16 não abre conta	Código Civil, art. 4º (16 a 18 anos assistidos pelos pais)	API (`FabricaDeConta`) e banco (vínculo obrigatório)
RN02	Cada cliente tem um único celular ativo. Trocar de aparelho exige selfie aprovada pelo parceiro, desconecta o aparelho anterior e avisa o contato de confiança	RF005, risco R01	Banco (índice único parcial `ux_dispositivo_um_ativo_por_cliente`)
RN03	A senha tem 6 números. Depois de 5 erros seguidos, o acesso fica bloqueado por 30 minutos	RF004	API (`Credencial`)
RN04	Pix para um destinatário que nunca recebeu do cliente: até R$ 300 por Pix nas primeiras 24 horas	RF019, risco R01	API (`MotorAntifraude`)
RN05	Entre 20h e 6h, o total de Pix entre pessoas físicas fica limitado a R$ 1.000 por conta	Resolução BCB 142/2021, art. 2º, I	API (`MotorAntifraude`)
RN06	Pix com nota de risco de 70 ou mais fica retido e só segue com a decisão de um analista	RF026	Banco (restrição na `analise_risco`) e API
RN07	"Bloquear tudo agora" suspende todas as saídas de dinheiro na hora. Desbloquear exige selfie e só vale 1 hora depois	RF020	Banco (`conta.status`) e API
RN08	Na conta de menor, o responsável define o limite diário e o mensal. O diário nunca passa do mensal	RF015	Banco (restrição de verificação no `vinculo_responsavel`)
RN09	O pedido de aprovação do menor expira em 24 horas. Sem resposta, conta como recusado	RF016, US18	Worker (evento de expiração)
RN10	O convite ao responsável expira em 7 dias. Se expirar, o cadastro do menor é cancelado e os dados enviados são apagados	US09, LGPD art. 14	Worker e banco (`expira_em`)
RN11	O contato de confiança é avisado quando sai um Pix acima de 3 vezes a média dos Pix enviados nos últimos 90 dias. Ele nunca vê o saldo nem movimenta a conta	RF018, US13	API (`familia`)
RN12	Toda movimentação gera lançamentos que somam zero. O saldo é a soma dos lançamentos e nunca é editado	ADR-007, RNF11	Banco (gatilho adiado de partidas dobradas)
RN13	O mesmo pedido de dinheiro, com a mesma chave de idempotência na mesma conta, nunca gera duas transações	RNF11, US07	Banco (índice único `ux_transacao_idempotencia`)
RN14	A conta só é encerrada com saldo zero. Os dados pessoais são anonimizados e os registros financeiros ficam guardados por 5 anos	RF029, Lei 9.613/1998, art. 10, §2º	API e banco (lançamentos e auditoria só aceitam inserção)
RN15	O Rufus nunca liga para o cliente. O atendimento humano só começa de dentro do app, com o cliente já autenticado	RF021, RF024	App e processo do SAC
RN16	O funcionário só vê o que o papel dele permite, e toda consulta a dado de cliente gera um registro de auditoria	RF030, RNF14	API (RBAC) e banco (`registro_auditoria`)

As regras RN02, RN06, RN08, RN12 e RN13 foram testadas no PostgreSQL real: os casos que deveriam ser recusados foram recusados (Bloco 5, `testes/resultado.txt`).

2.5 Design de interface

No Rufus a interface é o produto. A interface é a ligação do homem com o software (Aula 05), e o problema do Bloco 1 está nessa ligação: as pessoas já têm conta, mas não conseguem usá-la sozinhas. Por isso o design foi feito antes do código, em três níveis de protótipo, e cada decisão de tela sai de uma persona, de um RF ou de um RNF.

Os três aspectos que guiaram o projeto

A Aula 05 pede conhecer três aspectos antes de desenhar. No Rufus:

Aspecto	No Rufus	De onde vem
Clientes (personas)	Antônio (lê com dificuldade), Camila (17 anos, conta com limite), Dona Marlene (68 anos, medo de errar)	Bloco 1, item 2
Conteúdo (entrada e saída)	Saída: saldo, extrato, nome de quem recebe, comprovante, sempre também em áudio. Entrada: valor, chave Pix, código de barras pela câmera, voz no chat	RF006 a RF013, RF024
Funcionalidades (features)	As 16 funcionalidades F01 a F16, organizadas na taxonomia abaixo	Bloco 1, item 1

Decisões de design

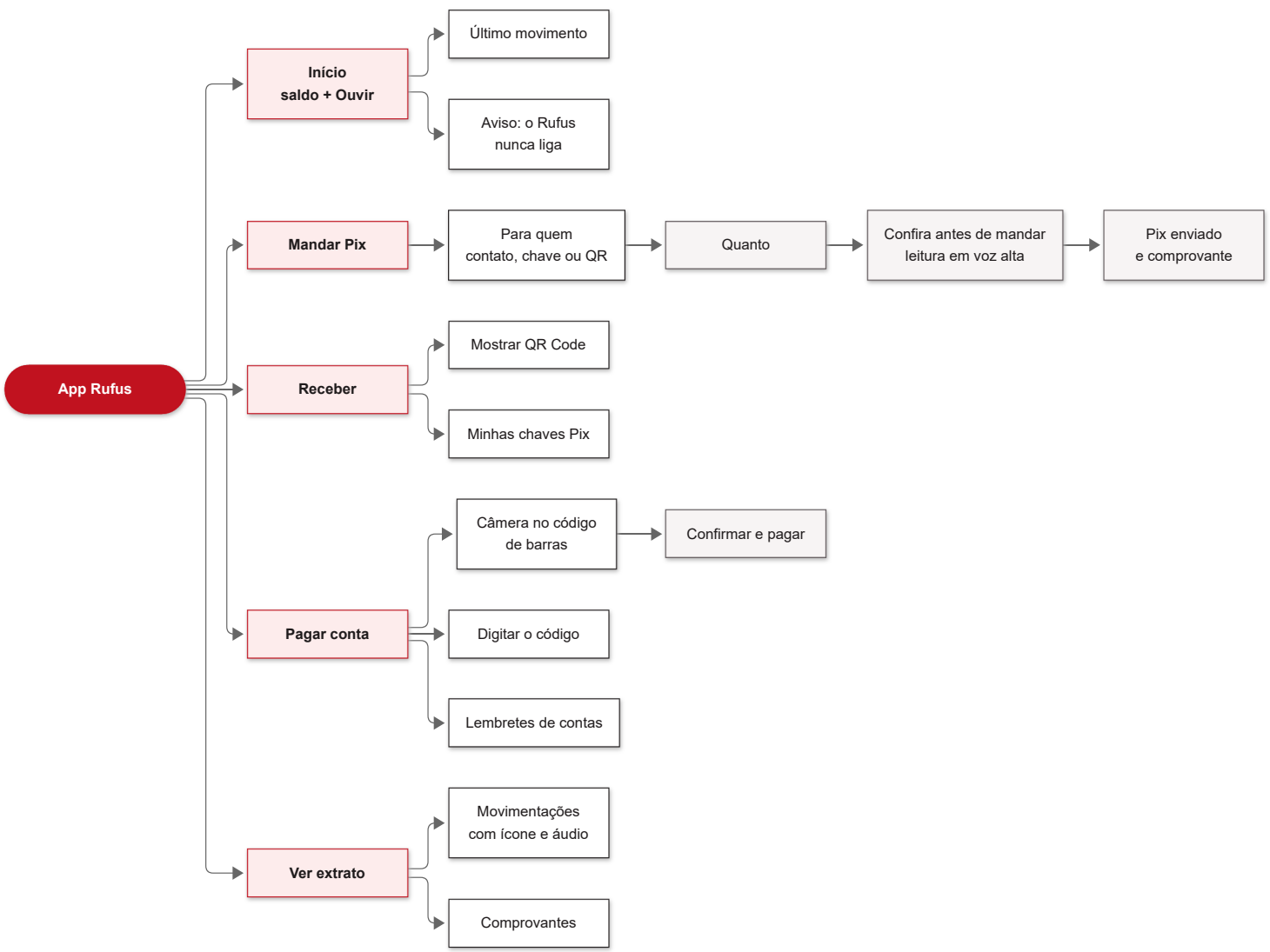
Decisão	Por quê	Persona	Requisito
Botão "Ouvir" fixo no topo de toda tela, sempre no mesmo lugar	Quem lê pouco precisa achar o áudio sem procurar	Antônio, Dona Marlene	RF007, RNF09
Linguagem do dia a dia: "Dinheiro que você tem", "Mandar Pix", "Pagar conta"	Jargão ("saldo disponível", "transferência") exclui quem tem pouco letramento financeiro	Todas	Princípio 3 do Bloco 1
Ícone vermelho dentro de um círculo vermelho suave em cada ação	O ícone reduz a leitura; o círculo é o motivo visual da identidade	Antônio	RF006, identidade visual
Vermelho só na marca e nos botões; valores em dinheiro sempre em grafite	*Rufus* vem do latim e quer dizer vermelho, ruivo: é a cor da marca. Mas valor vermelho num banco se lê como dinheiro saindo ou saldo negativo	Todas	Identidade visual
Tela "Confira antes de mandar" com nome, banco e valor, lida em voz alta automaticamente	Última barreira contra golpe e erro de digitação	Dona Marlene	RF009, risco R01
Aviso "o Rufus nunca liga para você" nas telas de dinheiro e no início do chat	Ataca o golpe da falsa central, o que mais cresceu em 2025	Antônio, Dona Marlene	RF021
"Bloquear tudo" visível já na tela inicial, em vermelho escuro com cadeado, diferente do vermelho da marca	Quando a pessoa percebe o golpe, cada segundo conta	Dona Marlene	RF020
Câmera que lê o código de barras sozinha, com "Prefiro digitar o código" como saída	Digitar os 47 ou 48 números da linha digitável é a maior barreira para pagar conta	Dona Marlene	RF013, US08
Limite e meta da Camila na própria tela inicial dela	Autonomia com limite visível: ela sabe quanto ainda pode gastar	Camila	RF015, RF017
Responsável decide num cartão só: valor, para quem, o que ela escreveu, prazo	Decisão rápida, sem navegar	Responsável da Camila	RF016, US18
Chat com botão de microfone grande	Antônio prefere falar a escrever	Antônio	RF024, US14
Texto base de 18, valores de dinheiro de 32 a 40, mínimo de 14 só em rótulos curtos	Legibilidade para quem lê com dificuldade	Todas	RNF09
Toda área de toque com 48 dp ou mais (botões de 56, "Ouvir" de 48, ações de 112, microfone de 72)	Mínimo do guia de acessibilidade do Android	Dona Marlene	RNF09

Taxonomia

A taxonomia faz duas coisas (Aula 05): **decompõe o conteúdo** em entidades da interface e **ordena os fluxos** em camadas. No Rufus são três camadas: a seção (o que a pessoa quer fazer), a tela e o passo do fluxo. Toda funcionalidade fica no máximo a um toque de distância da tela inicial.

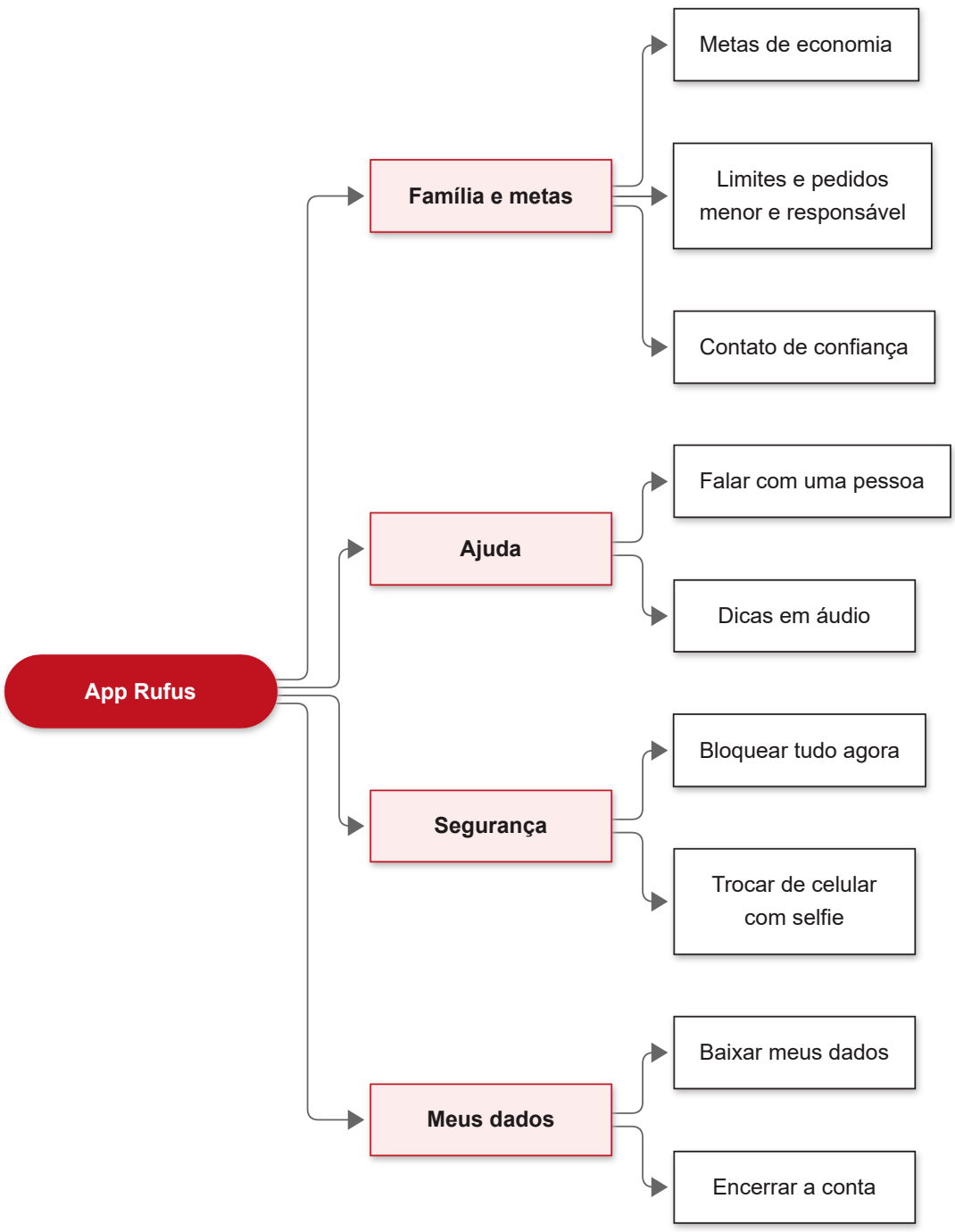
Taxonomia do app Rufus, parte 1 de 2: dinheiro do dia a dia

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Taxonomia do app Rufus, parte 2 de 2: família, ajuda, segurança e dados

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Fontes: diagramas/taxonomia-1-dinheiro.mmd e diagramas/taxonomia-2-apoio-e-seguranca.mmd. As 9 seções, divididas em duas partes para caber legível.

Material Design e tipografia

O protótipo segue o Material Design, a abordagem que padroniza temas, formas e linhas e que tem a tipografia como pilar (Aula 05). No app real a fonte é a Roboto do Android; nos protótipos foi usada a Arial, que tem as mesmas proporções e estava instalada na máquina.

Uso	Tamanho	Peso
Valor em dinheiro	32 a 40	Negrito
Título de tela grande ("Bloquear tudo agora?")	24	Negrito
Título da barra do topo	20	Negrito
Texto e botões	18	Normal e negrito
Texto de apoio e avisos	16	Normal
Rótulos curtos (hora, nome no chat)	14 a 15	Normal

Cores da identidade (`ferramentas/identidade/identidade.json`), todas com contraste medido pela fórmula da WCAG 2.2:

Par de cores na tela	Contraste	Mínimo AA
Texto grafite sobre branco (títulos e valores)	17,2:1	4,5:1
Texto branco sobre vermelho (botão "Ouvir" e botão principal)	6,2:1	4,5:1
Texto branco sobre grafite (cartão de saldo)	17,2:1	4,5:1
Rótulo claro sobre grafite ("Dinheiro que você tem")	12,8:1	4,5:1
Ícone vermelho sobre vermelho suave (círculo das ações)	5,4:1	4,5:1
Texto de apoio cinza sobre branco	7,3:1	4,5:1
Texto do aviso laranja sobre fundo laranja claro	9,0:1	4,5:1
Texto branco sobre verde (Aprovar, Pix enviado)	5,4:1	4,5:1
Texto branco sobre vermelho escuro (Sim, bloquear tudo)	9,8:1	4,5:1

Os três níveis de protótipo

Os três níveis saem de uma única descrição de cada tela (`prototipos/src/gerar_prototipos.py`). Assim o wireframe e a tela final nunca divergem de layout: só muda o acabamento.

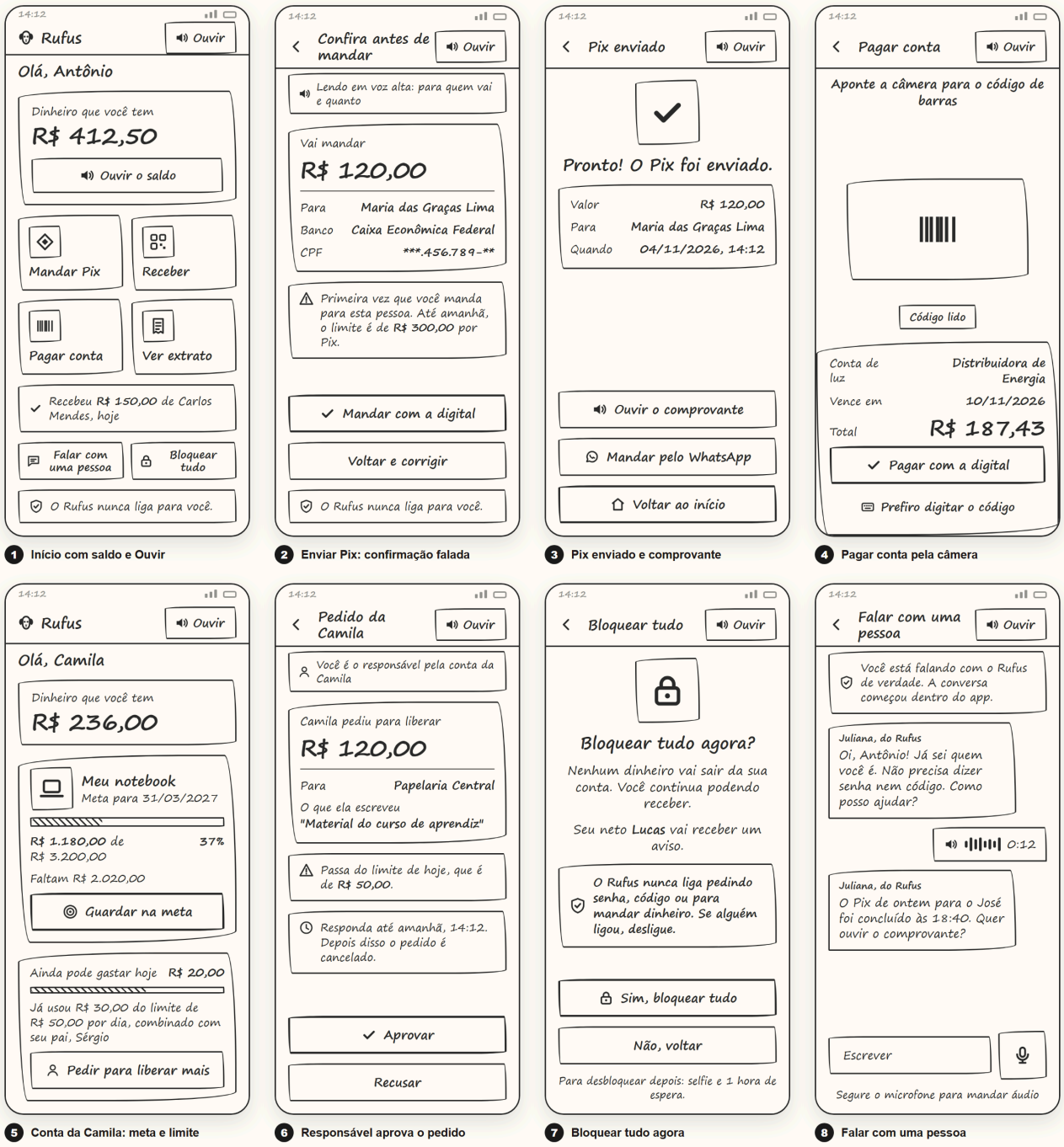
Nível	O que é (Aula 05)	Para que usamos
Protótipo conceito (sketch)	Lápis, caneta e papel: a primeira versão da solução	Discutir a ideia com o grupo e com o PO, sem gastar tempo com cor
Baixa fidelidade (wireframe)	O conceito com preto e branco, caixas e símbolos	Validar conteúdo, ordem e fluxo no teste de uso, sem a cor influenciar a opinião
Alta fidelidade	Cores, padrão visual e identidade completa	TER, aprovação final do PO e referência para o dev implementar

Protótipo conceito

Protótipo conceito (sketch) · Rufus Bank

Traço de caneta: a primeira versão da solução, sem cor nem detalhe.

Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

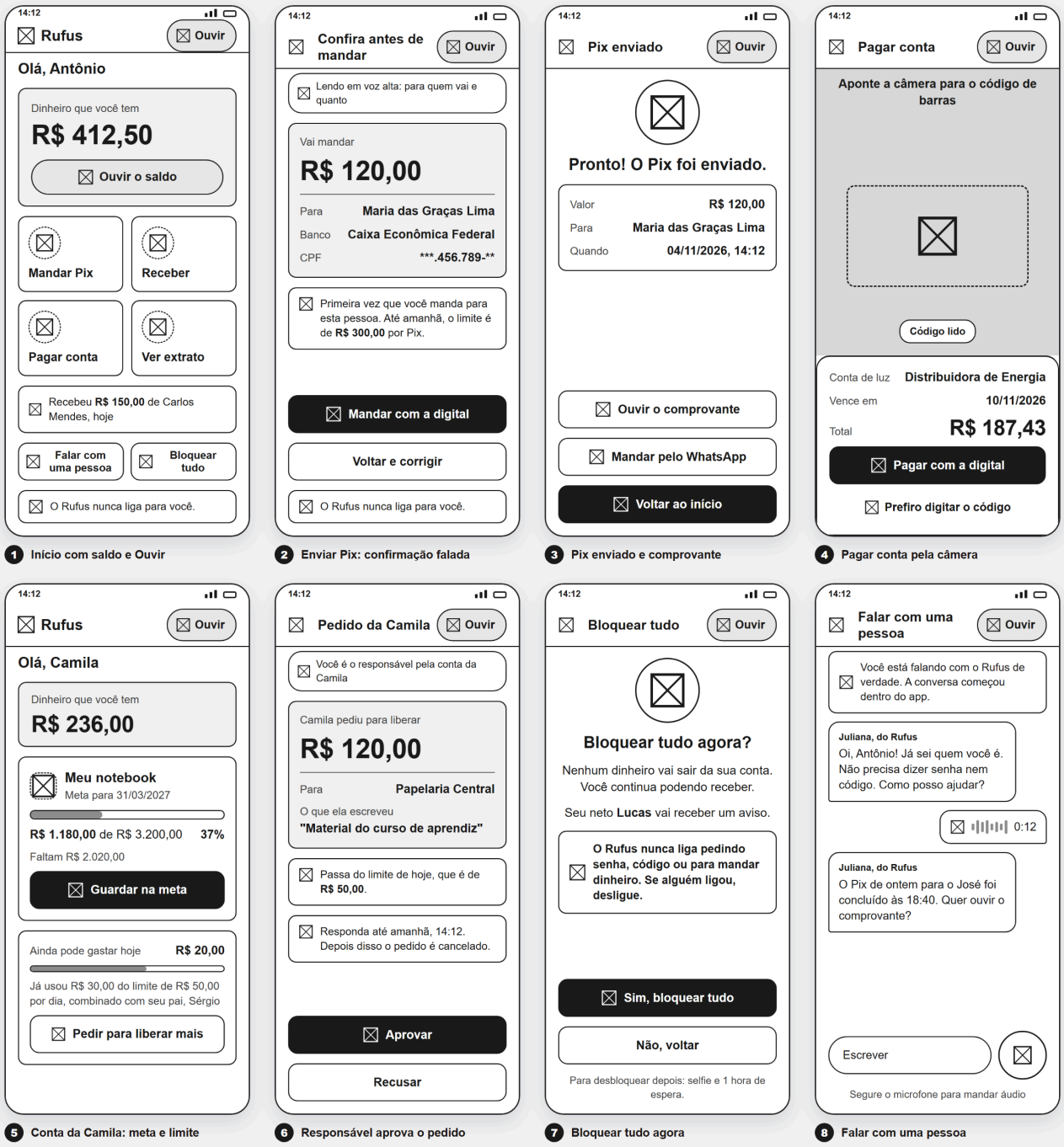


Baixa fidelidade

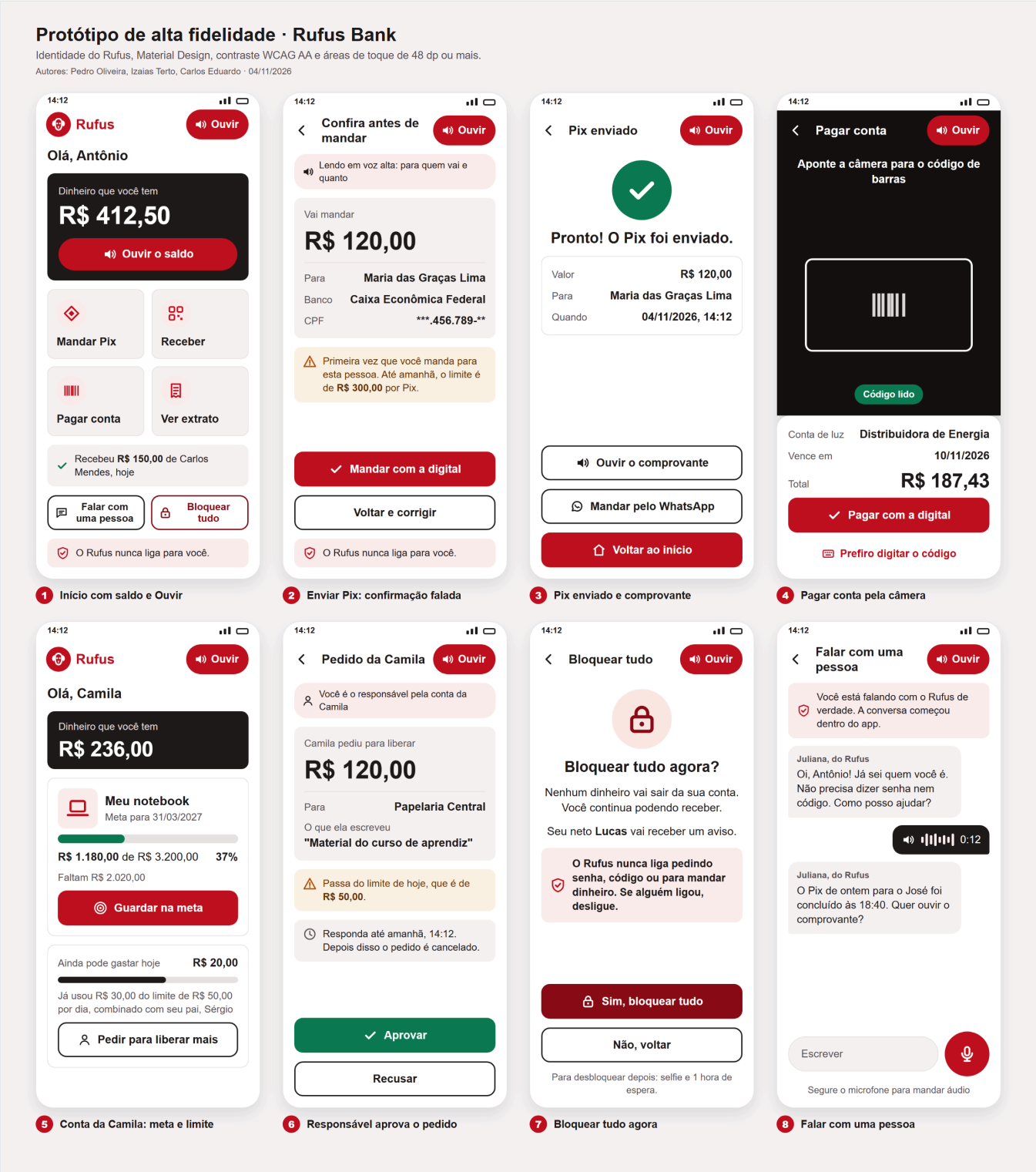
Protótipo de baixa fidelidade (wireframe) · Rufus Bank

Preto e branco, caixas e símbolos: valida layout, conteúdo e fluxo.

Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Alta fidelidade



As telas avulsas, em 720 x 1560 pixels, estão em `prototipos/img/` (`alta-01-inicio.png` a `alta-08-falar-com-pessoa.png`, e o mesmo para `baixa-` e `conceito-`).

Quantos toques cada fluxo leva

O RNF08 promete ver saldo, mandar Pix e pagar boleto em no máximo 3 toques depois do login. A contagem abaixo considera os toques em botões a partir da tela inicial; digitar um valor e pôr a digital no sensor aparecem à parte.

Fluxo	Toques	Caminho	Dentro do RNF08?
Ver o saldo	0	Já aparece na tela inicial	Sim
Ouvir o saldo	1	Ouvir o saldo	Sim
Pagar conta pela câmera	2 + digital	Pagar conta, a câmera lê sozinha, Pagar com a digital	Sim
Mandar Pix para um contato salvo	4 + valor + digital	Mandar Pix, escolher o contato, digitar o valor e Continuar, Mandar com a digital	Sim: 3 de navegação mais a confirmação com a digital (ver abaixo)
Receber por QR Code	2	Receber, Mostrar QR Code	Sim
Ver extrato	1	Ver extrato	Sim
Bloquear tudo agora	2	Bloquear tudo, Sim, bloquear tudo	Sim
Falar com uma pessoa	1	Falar com uma pessoa	Sim
Guardar na meta (Camila)	2 + valor	Guardar na meta, digitar o valor e Confirmar	Sim
Responsável aprovar pedido (pai da Camila)	2	Tocar na notificação, Aprovar	Sim

O Pix tem 4 toques, e o quarto é o "Mandar com a digital" da tela de conferência. Essa tela é obrigatória por segurança (RF009): é ela que lê em voz alta para quem vai o dinheiro. A decisão foi não cortar a proteção para bater a meta, e sim deixar a regra de contagem explícita no RNF08: contam os toques de navegação; digitar o valor e o toque final de confirmação com a digital não contam. Pela regra, o Pix fica em 3 toques.

Melhoria registrada para depois do MVP: um atalho "Mandar de novo" na tela inicial para as 3 pessoas mais frequentes, que tira mais um toque do caminho.

TER: Teste de Resposta Emocional

O TER mede, antes de implementar, se o conceito agrada ao público (Aula 05). Ele é feito com o protótipo de alta fidelidade, antes do Sprint 1 de telas.

Item	Como fazemos
Quem	3 pessoas de cada persona: 3 com pouca leitura, 3 adolescentes de 16 ou 17 anos, 3 pessoas com 60 anos ou mais
O que mostramos	As 8 telas de alta fidelidade, impressas em tamanho real ou no celular
O que perguntamos	"O que você sente olhando esta tela?" e uma escala de 5 rostos (de muito tranquilo a muito nervoso) por tela
O que medimos	A palavra espontânea mais repetida (esperado: "fácil", "claro", "seguro") e a média da escala por tela
Critério	Tela com média abaixo de "tranquilo" ou com a palavra "confuso" em 2 ou mais pessoas volta para o protótipo
Quem decide	O PO, que mantém o backlog priorizado

Teste de uso (RNF08)

Item	Como fazemos
Quando	A cada sprint que entrega tela nova, com o protótipo de baixa fidelidade antes e com o app depois
Quem	5 pessoas do perfil das personas por rodada (Nielsen: 5 usuários acham a maior parte dos problemas)
Tarefas	Ver quanto tem, mandar R$ 20,00 para um contato, pagar uma conta de luz de exemplo, bloquear a conta
Meta	Pelo menos 4 das 5 pessoas pagam o boleto sem ajuda em até 3 minutos (RNF08)
Registro	Tempo de cada tarefa, número de toques, onde a pessoa hesitou e frases ditas em voz alta
Saída	Problema encontrado vira item do backlog. A Definition of Done das telas exige a rodada aprovada (Bloco 9)

Como gerar de novo

```
python -I bloco-2-requisitos/prototipos/src/gerar_prototipos.py
python -I ferramentas/diagramas/gerar.py bloco-2
```

O primeiro comando gera as 24 telas e as 3 pranchas com o Chrome headless. O segundo gera a taxonomia. Os dados das telas são fictícios e seguem as personas e as regras do projeto: limite de R$ 300,00 para destinatário novo nas primeiras 24 horas, R$ 50,00 por dia na conta da Camila, prazo de 24 horas para o responsável decidir e desbloqueio com selfie e 1 hora de espera.

Bloco 3 · Arquitetura de Software

3.1 Architecture Decision Records (ADR)

Um ADR por decisão relevante. Cada um tem contexto, decisão, alternativas descartadas com o motivo e consequências. Os motivos saem dos requisitos do Bloco 2 (RF e RNF) e das premissas do Bloco 1, nunca de "é popular" ou "conhecemos".

ADR	Decisão	Status
ADR-001	Monolito modular	Aceito
ADR-002	Parceiro BaaS atrás de uma porta com adaptador	Aceito
ADR-003	REST com JSON e contrato OpenAPI 3.0	Aceito
ADR-004	Backend em TypeScript com NestJS sobre Node.js 22	Aceito
ADR-005	App em React Native com Expo	Aceito
ADR-006	PostgreSQL 17 no Amazon RDS Multi-AZ	Aceito
ADR-007	Razão contábil com partidas dobradas	Aceito
ADR-008	Eventos por outbox transacional no próprio PostgreSQL	Aceito
ADR-009	Containers no ECS Fargate	Aceito
ADR-010	Autenticação própria com JWT curto e celular vinculado	Aceito
ADR-011	Voz gerada no servidor, com cache	Aceito
ADR-012	Painéis internos servidos pela própria API	Aceito
ADR-013	AWS na região de São Paulo	Aceito

ADR-001 · Monolito modular, e não microsserviços

Contexto. O MVP atende 10.000 usuários ativos por mês com picos de 100 requisições por segundo (RNF04). A equipe tem 2 desenvolvedores de backend (Bloco 7). Os módulos (contas, Pix, família, antifraude, atendimento) trocam dados o tempo todo e várias operações precisam mexer em mais de um deles na mesma transação: um Pix debita a carteira, passa pelo antifraude e grava o evento de notificação (RNF11, zero Pix duplicado).

Decisão. Uma aplicação única (API) e um worker, os dois do mesmo código, divididos em módulos com fronteiras explícitas: cada módulo só expõe um serviço público e só ele escreve nas próprias tabelas. Um único banco PostgreSQL.

Alternativas descartadas.

Alternativa	Por que não
Microsserviços, um por módulo	Cada serviço teria o próprio banco, e o Pix viraria uma transação distribuída entre 3 serviços (saga com compensação em cada um). Para 100 req/s não há ganho de escala que pague isso. Exigiria 6 bancos ou mais, o que muda o custo do Bloco 1. 2 devs de backend não sustentam a operação de vários serviços
Monolito sem módulos	Resolve hoje, mas mistura as regras de antifraude com as de conta e impede extrair um módulo depois

Consequências. Transações locais garantem o RNF11 com uma transação do banco. Deploy único e simples. Se um módulo precisar escalar sozinho (o antifraude, por exemplo), ele já tem fronteira e pode virar serviço. Risco: alguém acessar a tabela de outro módulo. Mitigação: regra de lint de dependência entre módulos no pipeline (Bloco 6).

ADR-002 · Parceiro BaaS atrás de uma porta com adaptador

Contexto. Conta, Pix e boleto dependem de um parceiro autorizado pelo Banco Central (Bloco 1). O parceiro ainda não está contratado e existe o risco de não conseguir ou de precisar trocar (risco R04).

Decisão. Padrão Portas e Adaptadores (arquitetura hexagonal) para tudo que é externo. O domínio conversa com interfaces nossas (`PortaBancaria` , `PortaVoz` , `PortaNotificacao` , `PortaIdentidade`). Cada fornecedor ganha um adaptador que traduz o modelo dele para o nosso (camada anticorrupção). Em desenvolvimento e teste roda um adaptador falso, sem rede.

Alternativas descartadas.

Alternativa	Por que não
Usar o SDK do parceiro direto nos serviços	Prende o domínio ao parceiro. Trocar de parceiro (R04) obrigaria a reescrever Pix, boleto e conta
Contratar um agregador de vários bancos	Coloca mais um intermediário cobrando por transação. Não elimina a dependência, só muda de fornecedor

Consequências. Trocar de parceiro vira escrever um adaptador novo. O dev pode rodar tudo no localhost sem contrato assinado. Custo: uma camada a mais de tradução para manter.

ADR-003 · REST com JSON e contrato OpenAPI 3.0

Contexto. Os clientes da API são o app mobile e os painéis web. Os parceiros BaaS do mercado expõem REST e avisam eventos por webhook HTTP. O enunciado pede contrato OpenAPI 3.0 (Bloco 8).

Decisão. API REST com JSON, versionada no caminho (`/v1`), descrita em OpenAPI 3.0. Operações que movem dinheiro exigem o cabeçalho `Idempotency-Key` .

Alternativas descartadas.

Alternativa	Por que não
gRPC	O ganho de desempenho do protocolo binário não é necessário para 100 req/s com p95 de 300 ms (RNF01). Navegador não fala gRPC sem proxy, e os painéis web precisariam de gRPC-Web. Webhooks do parceiro são HTTP de qualquer forma
GraphQL	As telas são simples e fixas, por isso não há o problema de buscar dados demais. Dificulta cache e limite por rota no WAF. Idempotência por operação é mais natural em REST

Consequências. O cliente do app é gerado a partir do OpenAPI, então o contrato não diverge do código. Ferramentas de teste e de documentação (Swagger UI) prontas.

ADR-004 · Backend em TypeScript com NestJS sobre Node.js 22

Contexto. A carga é de entrada e saída (banco, parceiro, voz), não de cálculo pesado. Precisamos de tipagem forte para dinheiro e de módulos com fronteira clara (ADR-001). O app mobile é em TypeScript (ADR-005). Cada tarefa no Fargate tem 1 GB de memória (custo do Bloco 1).

Decisão. Node.js 22 LTS, TypeScript em modo estrito e framework NestJS. Valores em dinheiro sempre em centavos inteiros (`bigint` no banco).

Alternativas descartadas.

Alternativa	Por que não
Java 21 com Spring Boot	Muito usado em bancos e igualmente adequado. Descartado porque uma aplicação Spring com JVM costuma precisar de mais memória por tarefa que os 1 GB custeados, e porque exigiria uma segunda linguagem na equipe: os tipos do contrato não seriam compartilhados com o app
Go	Ótimo desempenho e baixo consumo de memória, mas sem framework modular equivalente ao NestJS: módulos, injeção de dependência e validação teriam de ser montados à mão. Mesma questão da segunda linguagem

Consequências. Uma linguagem só no app, na API e no worker. O NestJS dá módulos, injeção de dependência (que permite trocar adaptadores do ADR-002) e validação de entrada. Cuidado: número em JavaScript perde precisão acima de 2^53, por isso dinheiro trafega como inteiro em centavos e o banco usa `bigint`.

ADR-005 · App em React Native com Expo

Contexto. Android e iOS (RNF10), celular de entrada com 3 GB de RAM, leitor de tela TalkBack e VoiceOver obrigatório (RNF09), câmera para boleto (RF013), biometria (RF003).

Decisão. React Native com Expo, em TypeScript.

Alternativas descartadas.

Alternativa	Por que não
Flutter	Desenha a própria interface em vez de usar os componentes nativos, então a acessibilidade depende de anotar a árvore de semântica em cada widget. No React Native os componentes já são os nativos e o TalkBack e o VoiceOver os leem sem trabalho extra, que é o RNF mais importante do produto. Também usaria Dart, uma segunda linguagem
Dois apps nativos (Kotlin e Swift)	Melhor acesso a cada plataforma, mas dobra o trabalho mobile e exige dois perfis diferentes na equipe

Consequências. Um código para os dois sistemas. Biometria, câmera e armazenamento seguro (Keystore e Keychain) por bibliotecas do Expo. Atualização de correção sem esperar a loja (EAS Update), útil para corrigir texto de acessibilidade.

ADR-006 · PostgreSQL 17 no Amazon RDS Multi-AZ

Contexto. Dinheiro exige transação ACID entre várias linhas (RNF11), chaves estrangeiras e restrições. Disponibilidade de 99,9% (RNF03) e RPO de 5 minutos (RNF12).

Decisão. PostgreSQL 17 gerenciado no Amazon RDS, com réplica síncrona em outra zona (Multi-AZ). A comparação completa com a alternativa descartada e o modelo estão no Bloco 5.

Alternativas descartadas. MongoDB (documento) e Amazon Aurora PostgreSQL. Motivos detalhados no Bloco 5, item 5.1.

Consequências. Failover automático para a réplica. Backup com restauração a qualquer ponto dos últimos 35 dias. Custo já no Bloco 1 (US$ 284,40 por mês em produção, com os snapshots mensais).

ADR-007 · Razão contábil com partidas dobradas

Contexto. Saldo errado é o pior erro possível para o público do Rufus. A diferença entre o nosso registro e o do parceiro precisa ser detectada em até 24 horas (RNF11).

Decisão. Toda movimentação gera lançamentos em partidas dobradas: cada transação tem pelo menos um débito e um crédito, e a soma é zero. O saldo de uma carteira é a soma dos lançamentos dela, nunca um número editado. Um gatilho no banco recusa transação desequilibrada.

Alternativas descartadas.

Alternativa	Por que não
Coluna `saldo` atualizada a cada operação	Duas operações ao mesmo tempo podem sobrescrever uma à outra, e quando o saldo diverge não há trilha para explicar o porquê
Guardar só o saldo informado pelo parceiro	Deixa o Rufus sem registro próprio para conciliar e para mostrar o extrato quando o parceiro está fora

Consequências. Extrato e auditoria saem do mesmo registro. A conciliação diária compara a soma por carteira com o extrato do parceiro. Custo: a consulta de saldo soma lançamentos; com índice por carteira e saldo de fechamento diário,

fica dentro dos 300 ms (RNF01).

ADR-008 · Eventos por outbox transacional no próprio PostgreSQL

Contexto. Depois de um Pix é preciso enviar ao parceiro, notificar o cliente, avisar o contato de confiança e o responsável. Se a API gravar o Pix e cair antes de avisar, o aviso se perde. Se avisar antes de gravar, avisa algo que não aconteceu.

Decisão. Padrão Transactional Outbox: na mesma transação que grava o Pix, a API grava o evento na tabela `outbox_evento`. O worker lê os eventos pendentes com `FOR UPDATE SKIP LOCKED`, executa e marca como processado, com nova tentativa e espera crescente em caso de erro.

Alternativas descartadas.

Alternativa	Por que não
Amazon SQS	Gravar no banco e publicar na fila são duas operações que podem falhar separadas, e o problema voltaria. Seria preciso o outbox de qualquer forma
Kafka (Amazon MSK)	Feito para milhões de eventos por segundo. Para o volume do Rufus seria um cluster caro e mais uma peça para operar

Consequências. Garantia de "pelo menos uma vez": cada consumidor é idempotente. Nenhum serviço novo na infraestrutura do Bloco 1. Se o volume crescer muito, o worker pode passar a publicar do outbox para uma fila.

ADR-009 · Containers no ECS Fargate

Contexto. Duas cargas: a API, sempre ligada e com p95 de 300 ms (RNF01), e o worker, que fica lendo o outbox (ADR-008). 99,9% exige pelo menos 2 zonas (RNF03).

Decisão. Amazon ECS com Fargate: containers sem gerenciar servidor, 2 tarefas da API (uma por zona) e 1 do worker, com auto scaling de 2 a 6 tarefas da API.

Alternativas descartadas.

Alternativa	Por que não
Kubernetes (EKS)	O plano de controle custa US$ 0,10 por hora por cluster em São Paulo, US$ 73 por mês, e seriam 2 clusters (produção e homologação). Exige uma pessoa que domine Kubernetes. Os recursos extras não são usados por 2 serviços
AWS Lambda	O worker fica em execução contínua e a API mantém conexões com o PostgreSQL. Em Lambda, a partida a frio ameaça o p95 de 300 ms e cada execução abriria conexão nova com o banco, exigindo RDS Proxy
EC2 com Docker	Mais barato por hora, mas a equipe teria de cuidar de sistema operacional, atualização de segurança e substituição de máquina com defeito

Consequências. O mesmo container roda no localhost (Docker Compose) e na AWS. Custo de containers no Bloco 1: US$ 62,00 por mês em produção.

ADR-010 · Autenticação própria com JWT curto e celular vinculado

Contexto. Só um celular ativo por conta e troca de aparelho com selfie (RF005). Senha de 6 números e biometria (RF003, RF004). Sessão segura (RNF05). Funcionários precisam de papéis diferentes (RF030) e de segundo fator.

Decisão. Módulo de identidade próprio. No cadastro o app gera um par de chaves no Keystore ou Keychain do celular e envia a chave pública. O login exige senha (guardada com Argon2id) e a assinatura do aparelho. A API emite token de acesso JWT de 15 minutos e token de renovação de 30 dias, trocado a cada uso e guardado só como hash. A biometria destrava a chave no celular e nunca sai dele. Funcionários entram com senha e código TOTP e recebem papéis (RBAC). A relação entre menor e responsável é checada por atributo do vínculo (ABAC).

Alternativas descartadas.

Alternativa	Por que não
Amazon Cognito	Resolve usuário e senha, mas não tem vínculo de um único aparelho com assinatura por chave do celular nem as regras de menor e responsável. Teríamos de escrever essas regras de qualquer forma, e com duas fontes de verdade
Sessão no servidor com cookie	App mobile não usa cookie de forma natural e cada requisição consultaria a sessão no banco

Consequências. Controle total do fluxo de troca de aparelho, que é a principal barreira contra golpe (R01). Responsabilidade maior: o módulo de identidade tem a meta de cobertura de testes mais alta (Bloco 9).

ADR-011 · Voz gerada no servidor, com cache

Contexto. Toda tela precisa poder ser ouvida (RF007, RNF09). Os celulares de entrada nem sempre têm voz em português instalada e a qualidade varia de aparelho para aparelho.

Decisão. A API gera o áudio com o Amazon Polly. Frases fixas (instruções de tela) são geradas uma vez e guardadas no S3. Frases com dados (saldo, nome do recebedor) são geradas na hora e não são guardadas. Se o celular estiver sem internet, o app usa a voz do próprio aparelho.

Alternativas descartadas.

Alternativa	Por que não
Só a voz do celular	Custo zero, mas a voz pode faltar ou soar diferente em cada aparelho. Para quem depende do áudio, a voz é a interface
Gravar todas as frases com locutor	Não cobre frases com dados (valores, nomes)

Consequências. Mesma voz em todos os aparelhos. Custo no Bloco 1: US$ 48 por mês para 12 milhões de caracteres. A tabela de preços de São Paulo lista só a voz padrão do Polly; a voz neural fica como melhoria.

ADR-012 · Painéis internos servidos pela própria API

Contexto. SAC, antifraude e administração precisam de painéis web (RF025, RF027, RF030). São poucos usuários internos.

Decisão. Uma aplicação React (Vite) compilada como arquivos estáticos e servida pela API em `/admin`. O acesso ao caminho `/admin` só é aceito a partir dos IPs da empresa (regra do WAF), com senha e TOTP.

Alternativas descartadas.

Alternativa	Por que não
Hospedar no S3 com CloudFront	Mais uma distribuição para configurar e proteger, para poucos usuários internos
Ferramenta pronta de painel (low-code)	Os dados de cliente sairiam para um terceiro, o que piora o risco R02 e a LGPD

Consequências. Nenhum serviço novo na infraestrutura. O painel usa a mesma API, com rotas `/v1/admin` protegidas por papel.

ADR-013 · AWS na região de São Paulo

Contexto. Os dados são de brasileiros, incluindo menores e biometria. A latência para o usuário afeta o RNF01. Todos os serviços da arquitetura precisam estar na mesma região.

Decisão. AWS, região sa-east-1 (São Paulo), para todos os ambientes em nuvem.

Alternativas descartadas.

Alternativa	Por que não
Região dos EUA (us-east-1), mais barata	Dados pessoais fora do país pedem salvaguardas de transferência internacional na LGPD (art. 33) e somam latência de ida e volta a cada requisição
Outra nuvem com região em São Paulo (GCP ou Azure)	Também atenderia à latência. Ficamos com a AWS porque todos os serviços escolhidos (Fargate, RDS PostgreSQL Multi-AZ, Polly, WAF, Secrets Manager) existem na região de São Paulo e têm preço publicado na tabela oficial, que o script do Bloco 1 consulta para recalcular o custo a cada mudança

Consequências. Preço mais alto que nos EUA, já refletido no custo do Bloco 1.

3.2 Modelo C4

Os 4 níveis do C4, do mais amplo ao mais detalhado. Cada nível aproxima o zoom de uma parte do nível anterior: o sistema (nível 1) é aberto em containers (nível 2), o container API é aberto em componentes (nível 3) e o componente crítico ServicoEnvioPix é aberto em código (nível 4).

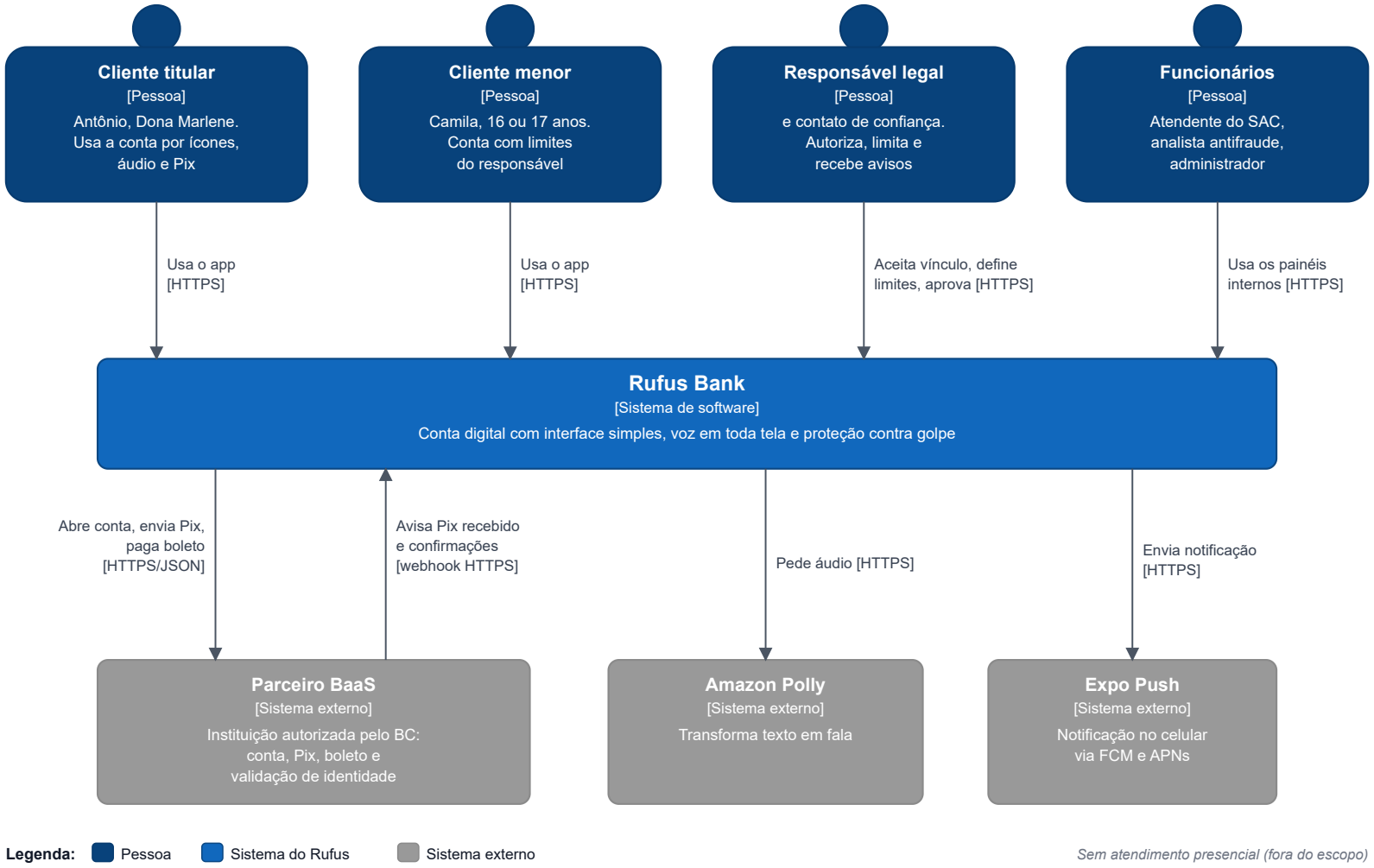
Cores na convenção do C4: azul escuro para pessoas, azul para o que o Rufus constrói, azul claro para componentes e cinza para sistemas externos. Toda seta diz o que trafega e o protocolo, entre colchetes.

Os diagramas são gerados por `python ferramentas/diagramas/gerar.py bloco-3` a partir das fontes em diagramas/. Título, autores e data entram sozinhos, a partir de `projeto.json`.

Nível 1 · Contexto

C4 nível 1 · Contexto do sistema Rufus Bank

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. Quem usa o Rufus e com quais sistemas de fora ele conversa. É a visão para quem não é técnico: o Rufus é uma caixa só.

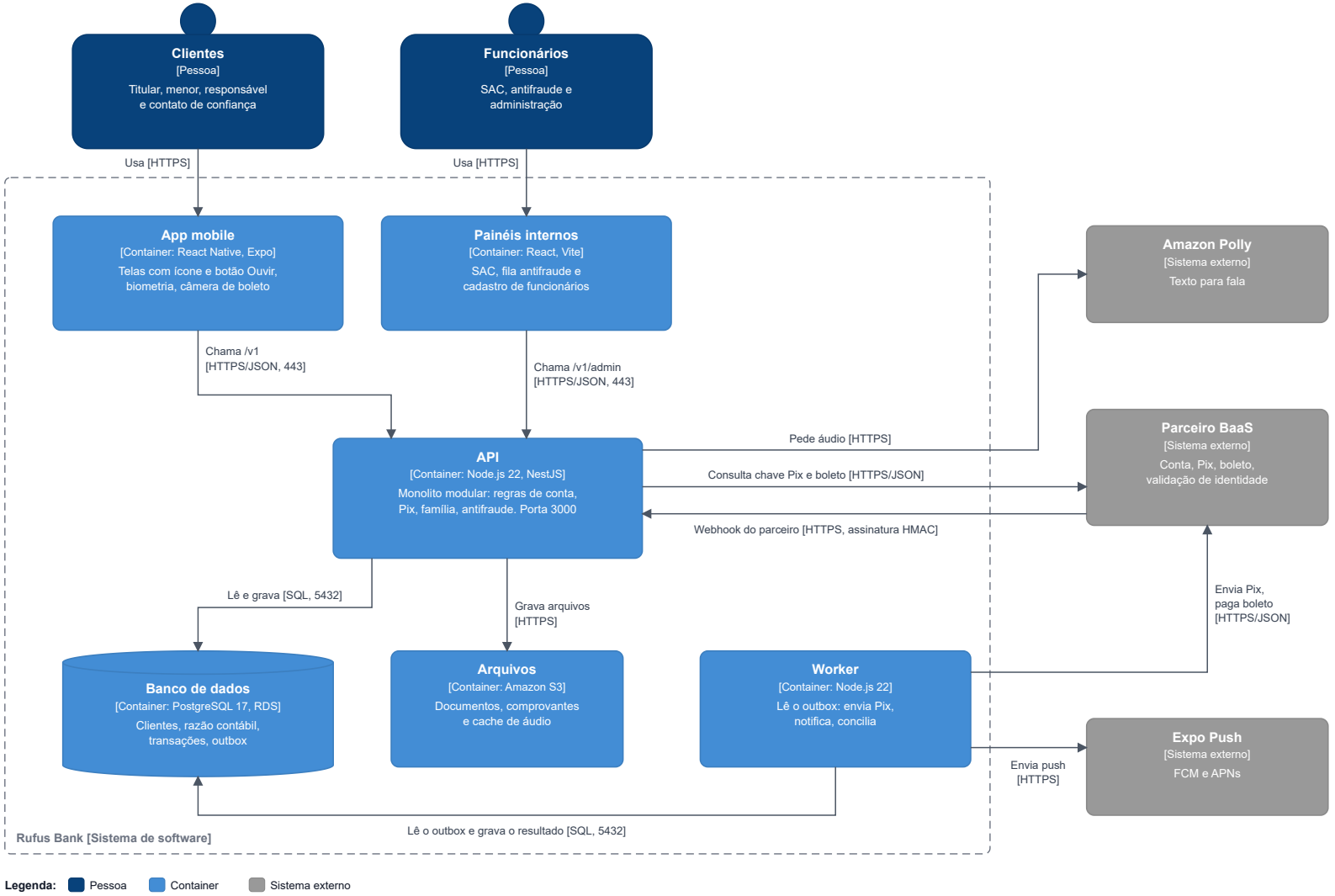
Como ler. No alto estão as quatro pessoas, que são os atores humanos do Documento de Visão agrupados. Clientes titular e menor usam o app. Responsável e contato de confiança aparecem juntos porque os dois só reagem ao que o cliente faz (aceitar, limitar, receber aviso). Funcionários (atendente, analista antifraude e administrador) usam os painéis. Embaixo estão os três sistemas externos. O parceiro BaaS tem duas setas, porque o Rufus chama o parceiro e o parceiro também chama o Rufus por webhook quando um Pix chega.

Decisões que aparecem aqui. - Conta, Pix e boleto ficam no parceiro BaaS, e o Rufus não tem core bancário próprio (Bloco 1, fora do escopo, e ADR-002). - A voz vem de um serviço externo, e não do celular (ADR-011). - Não há canal presencial (fora do escopo do Bloco 1).

Nível 2 · Containers

C4 nível 2 · Containers do Rufus Bank

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. As peças que rodam separadas e se comunicam pela rede: dois front-ends, a API, o worker, o banco e o armazenamento de arquivos.

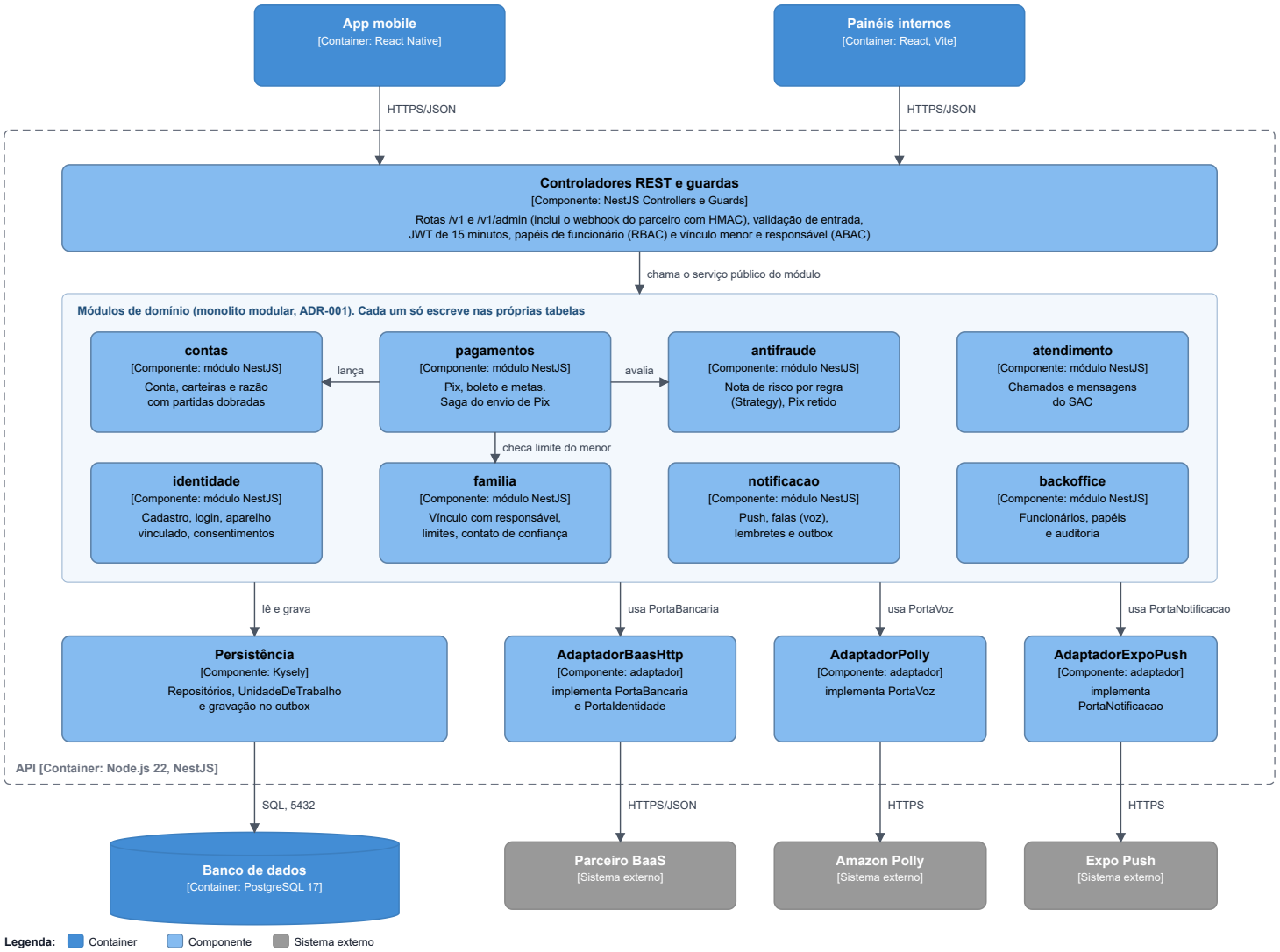
Como ler. - O **app mobile** e os **painéis internos** só falam com a **API**, por HTTPS na porta 443 do balanceador. Nenhum front-end acessa banco ou parceiro direto. - A **API** recebe as requisições na porta 3000 do container, grava no **PostgreSQL** (porta 5432) e no **S3**. Ela chama o parceiro só nas consultas síncronas: dados do recebedor do Pix e do boleto, que o cliente precisa ver antes de confirmar. - O **worker** faz o trabalho que não precisa segurar o cliente esperando. Ele lê o outbox, envia o Pix e o pagamento de boleto ao parceiro e manda os push. - O parceiro avisa a API por **webhook** assinado com HMAC. A API confere a assinatura antes de aceitar.

Decisões que aparecem aqui. - API e worker são dois containers do **mesmo código**, o monolito modular do ADR-001. Separar em dois processos deixa o envio ao parceiro fora da requisição do cliente. - Há um **único banco**, coerente com o monolito. Com microsserviços seria um banco por serviço, e essa opção foi descartada no ADR-001. - A fila é a tabela de outbox no próprio banco (ADR-008). Por isso não aparece fila separada. - Os painéis são um container próprio no desenho, mas são servidos pela API em `/admin` (ADR-012).

Nível 3 · Componentes da API

C4 nível 3 · Componentes do container API

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. O que existe dentro do container API: a camada HTTP, os 8 módulos de domínio e a camada de infraestrutura com as portas e adaptadores.

Como ler, de cima para baixo. 1. **Controladores REST e guardas** recebem a requisição, validam a entrada, conferem o JWT e aplicam o controle de acesso: papéis de funcionário (RBAC) e vínculo entre menor e responsável (ABAC). O webhook do parceiro também entra por aqui, com conferência do HMAC. 2. Os **módulos de domínio** têm as regras. Cada um só escreve nas próprias tabelas e expõe um serviço público. As setas mostram as dependências principais do Pix: `pagamentos` lança no razão de `contas`, pede a nota de risco ao `antifraude` e confere o limite do menor em `familia`. 3. A **persistência** e os **adaptadores** ficam embaixo. Os módulos não conhecem o parceiro, o Polly nem o Expo: dependem só das interfaces `PortaBancaria`, `PortaIdentidade`, `PortaVoz` e `PortaNotificacao`.

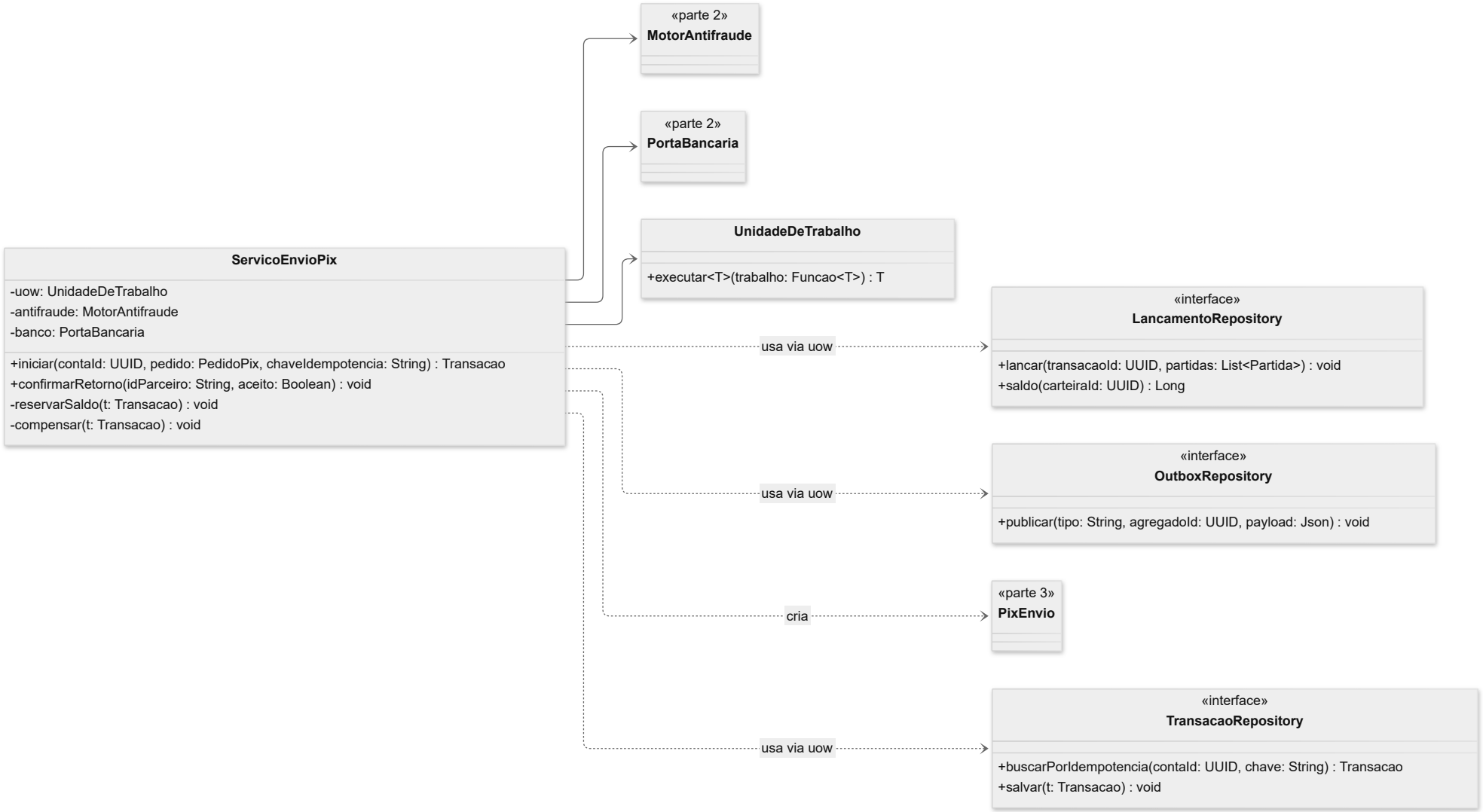
Decisões que aparecem aqui. - **Portas e adaptadores** (ADR-002): trocar de parceiro BaaS significa escrever um adaptador novo, sem tocar nos módulos. Isso mitiga o risco R04. No localhost os adaptadores reais dão lugar a adaptadores falsos. - **Fronteira de módulo** (ADR-001): a regra "cada módulo só escreve nas próprias tabelas" é verificada por lint de dependência no pipeline. - O Pix depende de três módulos, mas tudo acontece numa transação local do banco, pela UnidadeDeTrabalho. Isso garante o RNF11 (zero Pix duplicado) sem transação distribuída.

Nível 4 · Código do componente crítico ServicoEnvioPix

O nível 4 está em 3 partes, para o texto continuar legível impresso. A classe que pertence a outra parte aparece só com o nome e o estereótipo «parte N».

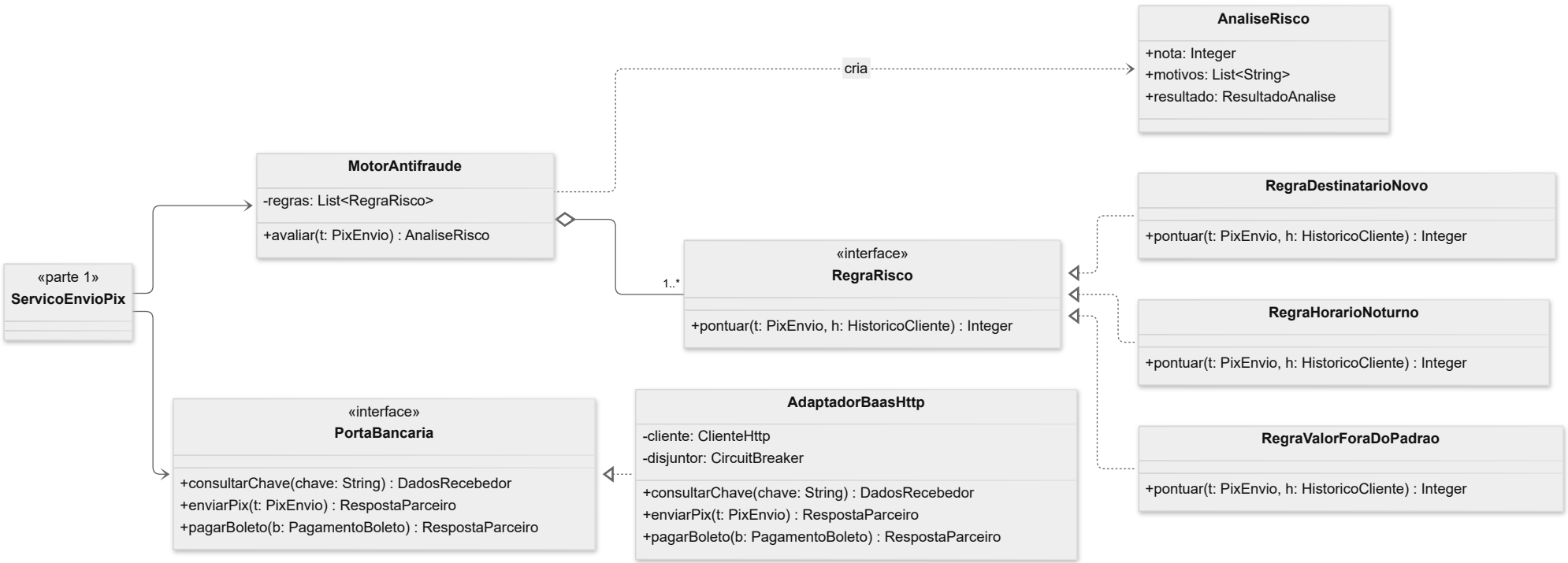
C4 nível 4, parte 1 de 3 · ServicoEnvioPix: orquestração e persistência

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



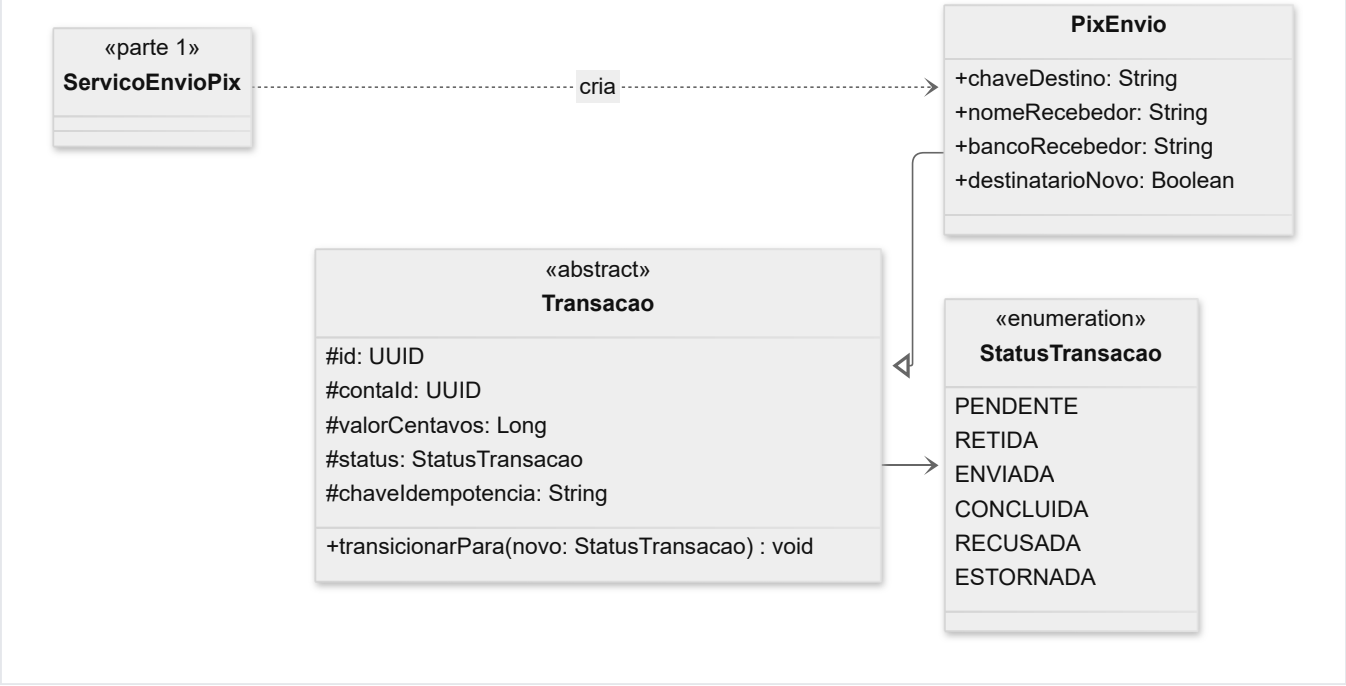
C4 nível 4, parte 2 de 3 · ServicoEnvioPix: antifraude e porta bancária

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



C4 nível 4, parte 3 de 3 · ServicoEnvioPix: a transação que ele cria

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Por que este componente. O envio de Pix é a operação que move dinheiro para fora, envolve um sistema externo que pode falhar e é o alvo dos golpes (risco R01). Um erro aqui é o pior erro possível para o produto.

Como funciona `iniciar(contaId, pedido, chaveIdempotencia)`. 1. Procura uma transação com a mesma chave de idempotência (`TransacaoRepository.buscarPorIdempotencia`). Se existir, devolve a mesma, sem débito novo (RNF11, US07). 2. Cria o `PixEnvio` e pede a nota ao `MotorAntifraude` , que soma as notas de cada `RegraRisco` . Com nota 70 ou mais a transação fica RETIDA (RF026). 3. Dentro da `UnidadeDeTrabalho` , numa única transação do banco: grava a transação PENDENTE, reserva o saldo com dois lançamentos (`reservarSaldo`) e publica o evento no outbox. 4. Depois o worker envia o Pix pela `PortaBancaria` . Se o parceiro recusar, `compensar` estorna a reserva. É a saga orquestrada do Bloco 3 (padrões de design) e do diagrama de sequência 3 do Bloco 4.

Notação e decisões. - `PortaBancaria` , `RegraRisco` e os repositórios são **interfaces**. A seta tracejada com triângulo vazio é **realização**: `AdaptadorBaasHttp` implementa `PortaBancaria` , e cada regra implementa `RegraRisco` (padrão Strategy). - `MotorAntifraude` tem as regras por **agregação** (losango vazio). As regras existem e são testadas sozinhas, e o motor recebe a lista por injeção de dependência. Não é composição porque a vida das regras não depende do motor. - `Transacao` é **abstrata** e `PixEnvio` a especializa (herança). Os atributos de `Transacao` são **protegidos** (`#`) porque as subclasses os usam. Os campos internos do serviço são **privados** (`-`). - `AdaptadorBaasHttp` guarda um `CircuitBreaker` . Quando o parceiro falha em sequência, o disjuntor para de chamar por 30 s (padrão Circuit Breaker).

3.3 Stack tecnológica

Todas as camadas, cada uma com a escolha, a alternativa descartada e o motivo técnico, ligado ao requisito ou ao ADR de onde veio.

Camada	Escolha	Alternativa descartada	Motivo técnico
App mobile	React Native 0.8x com Expo SDK, TypeScript	Flutter	Componentes nativos lidos pelo TalkBack e VoiceOver sem anotação extra (RNF09). Mesma linguagem da API (ADR-005)
Armazenamento seguro no celular	Expo SecureStore (Keystore no Android, Keychain no iOS)	AsyncStorage	Token e chave do aparelho precisam de armazenamento cifrado pelo sistema (RNF05, ADR-010)
Biometria	expo-local-authentication	Biometria própria	A biometria nativa não tira o dado do celular. Nada biométrico chega ao servidor (risco R08)
Leitura de boleto	expo-camera com leitura de código de barras	Biblioteca de terceiros paga	Lê o padrão ITF dos boletos sem custo por leitura (RF013)
Painéis internos	React 19 com Vite, TypeScript	Next.js	Painel interno não precisa de renderização no servidor nem de SEO. Arquivos estáticos servidos pela API (ADR-012)
API	Node.js 22 LTS, NestJS 11, TypeScript estrito	Java 21 com Spring Boot	Cabe em 1 GB por tarefa, mesma linguagem do app, módulos com injeção de dependência (ADR-004)
Validação de entrada	class-validator com DTOs do NestJS	Validação manual	Toda entrada validada antes do domínio, que é o item A03 (injeção) da OWASP
Acesso a dados	Kysely (construtor de SQL tipado) com SQL explícito	ORM completo (TypeORM ou Prisma)	O razão contábil usa gatilho, `SELECT ... FOR UPDATE` e `SKIP LOCKED` (ADR-007, ADR-008). SQL explícito evita consulta escondida pelo ORM e deixa o p95 previsível (RNF01)
Migrações	Arquivos SQL versionados executados pelo node-pg-migrate	Migração automática do ORM	Migração revisada linha a linha e no padrão expandir e contrair, sem parar o sistema (Bloco 5)
Banco de dados	PostgreSQL 17 no Amazon RDS Multi-AZ	MongoDB	Transação ACID entre várias linhas, chave estrangeira e restrição (RNF11, ADR-006, Bloco 5)
Fila de eventos	Tabela outbox no PostgreSQL e worker	Amazon SQS	Gravar o dado e o evento na mesma transação (ADR-008)
Voz	Amazon Polly e cache no S3	Só a voz do celular	Mesma voz em todos os aparelhos (ADR-011)
Notificação push	Expo Push (sobre FCM e APNs)	Amazon SNS mobile push	Já integrado ao Expo, sem custo por mensagem
Autenticação	Módulo próprio: Argon2id, JWT de 15 min, renovação de 30 dias, chave do aparelho	Amazon Cognito	Vínculo de um único aparelho e regras de menor e responsável (ADR-010)
Log	pino com saída em JSON para o CloudWatch Logs	Log em texto livre	Log estruturado com identificador de correlação em 100% das requisições (RNF13)
Containers	Docker com build em múltiplos estágios	Imagem única com ferramentas de build	Imagem final menor e sem compilador, menos superfície de ataque (RNF06)
Execução na nuvem	Amazon ECS com Fargate	Kubernetes (EKS)	Sem custo de plano de controle e sem operar cluster (ADR-009)
Rede e borda	Application Load Balancer com AWS WAF	Exposição direta das tarefas	TLS na borda, distribuição entre 2 zonas (RNF03) e regras contra ataques comuns (R02)
Segredos	AWS Secrets Manager	Variável de ambiente no código	Segredo fora do repositório, com rotação (R02)
Infraestrutura como código	Terraform	Criar recursos pelo console	Ambientes reproduzíveis e revisados em pull request. Homologação e produção com o mesmo código
CI/CD	GitHub Actions	Jenkins	O código já está no GitHub e não há servidor de CI para manter

Camada	Escolha	Alternativa descartada	Motivo técnico
Testes	Jest (unidade e integração), Testcontainers (PostgreSQL real), Maestro (E2E mobile), k6 (carga)	Mock do banco em todos os testes	O razão e o outbox só são testados de verdade contra um PostgreSQL real (Bloco 9)
Desenvolvimento local	Docker Compose	Ambiente compartilhado na nuvem	Custo zero de desenvolvimento (Bloco 1) e cada dev com o próprio banco

3.4 Padrões de design

Quais padrões o Rufus aplica, onde e por quê. No fim, os padrões que avaliamos e **não** aplicamos, também com o motivo.

Aplicados

Padrão	Onde	Por quê
Monolito modular	Estrutura da API: módulos `identidade`, `contas`, `pagamentos`, `familia`, `antifraude`, `atendimento`, `notificacao`, `backoffice`	Fronteiras claras sem o custo de microsserviços (ADR-001)
Portas e Adaptadores (hexagonal)	`PortaBancaria`, `PortaVoz`, `PortaNotificacao`, `PortaIdentidade`, cada uma com adaptador real e falso	Trocar de parceiro BaaS sem tocar no domínio (R04, ADR-002) e rodar tudo no localhost
Adapter (camada anticorrupção)	`AdaptadorBaasHttp` traduz o modelo do parceiro para `Transacao` e `Conta`	O modelo do fornecedor não vaza para o domínio
Repository	Um repositório por agregado: `ContaRepository`, `TransacaoRepository`, `ChamadoRepository`	O domínio não conhece SQL. Nos testes de unidade o repositório é trocado por um em memória
Unit of Work	`UnidadeDeTrabalho` abre a transação do banco e passa os repositórios para o caso de uso	Pix, lançamentos e evento do outbox gravados juntos ou nada (RNF11)
Transactional Outbox	Tabela `outbox_evento` gravada na mesma transação, lida pelo worker	Evento nunca perdido nem enviado sem o dado gravado (ADR-008)
Saga orquestrada	Envio de Pix: reservar saldo, enviar ao parceiro, confirmar ou compensar (estorno da reserva)	A operação envolve um sistema externo que pode falhar no meio. A compensação devolve o dinheiro ao cliente (diagrama de sequência do erro crítico, Bloco 4)
Idempotency Key	Cabeçalho `Idempotency-Key` em `POST /v1/pix/envios` e `POST /v1/boletos/pagamentos`, com restrição única no banco	Zero Pix duplicado (RNF11, US07)
State	Ciclo de vida de `Transacao`: PENDENTE, RETIDA, ENVIADA, CONCLUIDA, RECUSADA, ESTORNADA	Cada estado define quais transições são válidas. Impede, por exemplo, estornar um Pix já recusado
Strategy	Regras do antifraude (`RegraDestinatarioNovo`, `RegraHorarioNoturno`, `RegraValorForaDoPadrao`) implementam `RegraRisco`	Regra nova entra sem mexer no motor. Cada regra é testada sozinha
Factory	`FabricaDeConta` cria conta de titular ou de menor, já com limites e vínculo corretos	Conta de menor tem regras diferentes (RF014, RF015) e não pode nascer sem vínculo
Domain Events (Observer)	`PixConcluido`, `ContaBloqueada`, `PedidoAprovacaoCriado` publicados no outbox	Notificação, contato de confiança e responsável reagem sem que o módulo de pagamentos conheça cada um
Circuit Breaker com nova tentativa	Chamadas ao parceiro BaaS e ao Polly	Se o parceiro cair, a API para de insistir por 30 s, responde rápido ao cliente e não esgota as conexões (RNF02)

Avaliados e não aplicados

Padrão	Por que não agora
CQRS com banco de leitura separado	A leitura (saldo e extrato) cabe no p95 de 300 ms com índice e saldo de fechamento diário. Um segundo banco aumentaria custo e criaria atraso entre escrita e leitura, ruim para quem confere o saldo logo depois do Pix
Event Sourcing completo	O razão com partidas dobradas (ADR-007) já guarda o histórico do dinheiro. Reconstruir todo o estado a partir de eventos traria complexidade sem requisito que peça
Saga coreografada	Com um fluxo central (Pix) e poucos participantes, a saga orquestrada deixa o passo a passo visível num lugar só, mais fácil de testar e de explicar

3.5 Modelo de domínio (fonte única)

Este arquivo é a **fonte única** das entidades, atributos, tipos e relações do Rufus. O diagrama de classes (Bloco 4), o banco de dados (Bloco 5), o contrato OpenAPI (Bloco 8) e o glossário (Bloco 9) seguem exatamente estes nomes. Mudou aqui, muda lá.

Convenções: nomes de classe em PascalCase e de atributo em camelCase no código; no banco, tabelas e colunas em snake_case com os mesmos nomes. Dinheiro sempre em centavos inteiros (`valorCentavos: Long` , `bigint` no banco). Identificadores são `UUID` . Datas e horas em `Instant` (UTC, `timestamptz`).

Módulos e entidades

Módulo	Entidades que ele é dono (só ele escreve)
identidade	Cliente, Dispositivo, Credencial, SessaoRenovacao, Consentimento
contas	Conta, Carteira, Lancamento
pagamentos	Transacao (e subtipos PixEnvio, PixRecebimento, PagamentoBoleto, MovimentoMeta), ChavePix, Meta
familia	VinculoResponsavel, PedidoAprovacao, ContatoConfianca
antifraude	AnaliseRisco
atendimento	Chamado, MensagemChamado
backoffice	Funcionario, RegistroAuditoria
notificacao	Notificacao, OutboxEvento, LembreteConta

Entidades

Cliente (identidade)

Atributo	Tipo	Regra
id	UUID	
nomeCompleto	String	
cpf	Cpf	Único. Cifrado em repouso, com hash para busca. PII
dataNascimento	LocalDate	Cifrado. PII
celular	String	PII
email	String	Opcional. PII
tipo	TipoCliente	TITULAR ou MENOR
status	StatusCliente	EM_ANALISE, ATIVO, BLOQUEADO, ENCERRADO
criadoEm	Instant	

Dispositivo (identidade)

Atributo	Tipo	Regra
id	UUID	
clienteId	UUID	Só 1 ativo por cliente (RF005)
modelo	String	
chavePublica	String	Chave gerada no Keystore/Keychain (ADR-010)
ativo	Boolean	
vinculadoEm	Instant	

Credencial (identidade)

`clienteId: UUID` (1 para 1 com Cliente), `hashSenha: String` (Argon2id), `tentativasErradas: Integer`, `bloqueadaAte: Instant` (opcional).

SessaoRenovacao (identidade)

`id: UUID`, `dispositivoId: UUID`, `hashToken: String`, `expiraEm: Instant`, `revogadaEm: Instant` (opcional).

Consentimento (identidade)

`id: UUID`, `clienteId: UUID`, `tipo: TipoConsentimento` (TERMOS_DE_USO, PRIVACIDADE, BIOMETRIA, RESPONSAVEL_LEGAL), `versaoTermo: String`, `aceitoEm: Instant`, `aceitoPorClienteId: UUID` (o próprio ou o responsável).

Conta (contas)

Atributo	Tipo	Regra
id	UUID	
clienteId	UUID	1 conta por cliente
agencia	String	Dado do parceiro
numero	String	Único
idContaParceiro	String	Identificador no parceiro BaaS
status	StatusConta	ATIVA, BLOQUEADA_TOTAL, ENCERRADA
bloqueadaEm	Instant	Opcional. Preenchido pelo "bloquear tudo agora" (RF020)
criadaEm	Instant	

Carteira (contas)

`id: UUID`, `contaId: UUID` (vazio nas carteiras técnicas), `tipo: TipoCarteira` (PRINCIPAL, META, LIQUIDACAO_PIX, LIQUIDACAO_BOLETO). Toda conta tem 1 carteira PRINCIPAL e 1 carteira META por meta. As carteiras de liquidação são do sistema e representam o dinheiro em trânsito com o parceiro.

Lancamento (contas)

`id: UUID`, `transacaoId: UUID`, `carteiraId: UUID`, `valorCentavos: Long` (positivo = crédito, negativo = débito, nunca zero), `criadoEm: Instant`. **A soma dos lançamentos de uma transação é sempre zero** (ADR-007). Saldo da carteira = soma dos seus lançamentos.

Transacao (pagamentos), classe abstrata

Atributo	Tipo	Regra
id	UUID	
contaId	UUID	
tipo	TipoTransacao	PIX_ENVIO, PIX_RECEBIMENTO, BOLETO, META_GUARDAR, META_RESGATAR
valorCentavos	Long	Maior que zero
status	StatusTransacao	PENDENTE, RETIDA, ENVIADA, CONCLUIDA, RECUSADA, ESTORNADA
chaveIdempotencia	String	Única por conta (RNF11)
idParceiro	String	Opcional. Identificador fim a fim no parceiro
criadaEm	Instant	
concluidaEm	Instant	Opcional

Subtipos (herança, porque um Pix **é uma** transação e compartilha o ciclo de vida e os lançamentos): - **PixEnvio**: `chaveDestino: String`, `nomeRecebedor: String`, `documentoRecebedorMascarado: String`, `bancoRecebedor: String`, `destinatarioNovo: Boolean`. - **PixRecebimento**: `nomePagador: String`, `bancoPagador: String`. - **PagamentoBoleto**: `linhaDigitavel: String`, `beneficiario: String`, `vencimento: LocalDate`, `jurosCentavos: Long`. - **MovimentoMeta**: `metaId: UUID`.

ChavePix (pagamentos)

`id: UUID`, `contaId: UUID`, `tipo: TipoChavePix` (CPF, CELULAR, ALEATORIA), `valor: String` (único), `status: StatusChavePix` (ATIVA, EXCLUIDA), `criadaEm: Instant`.

Meta (pagamentos)

`id: UUID`, `contaId: UUID`, `carteiraId: UUID` (a carteira META, criada junto, composição), `nome: String`, `valorAlvoCentavos: Long`, `dataAlvo: LocalDate`, `imagemUrl: String`, `status: StatusMeta` (ATIVA, CONCLUIDA, CANCELADA).

VinculoResponsavel (familia)

`id: UUID`, `menorClienteId: UUID` (único), `responsavelClienteId: UUID`, `status: StatusVinculo` (CONVIDADO, ATIVO, ENCERRADO), `limiteDiarioCentavos: Long`, `limiteMensalCentavos: Long`, `convidadoEm: Instant`, `aceitoEm: Instant`, `expiraEm: Instant` (convite vale 7 dias, US09).

PedidoAprovacao (familia)

`id: UUID`, `vinculoId: UUID`, `transacaoId: UUID`, `status: StatusPedido` (PENDENTE, APROVADO, RECUSADO, EXPIRADO), `criadoEm: Instant`, `expiraEm: Instant` (24 horas, US18), `decididoEm: Instant`.

ContatoConfianca (familia)

`id: UUID`, `clienteId: UUID`, `nome: String` (PII), `celular: String` (PII), `status: StatusContato` (CONVIDADO, ATIVO, REMOVIDO), `aceitoEm: Instant`.

AnaliseRisco (antifraude)

`id: UUID`, `transacaoId: UUID` (1 para 1), `nota: Integer` (0 a 100), `motivos: List<String>`, `resultado: ResultadoAnalise` (LIBERADA_AUTOMATICA, RETIDA, LIBERADA_ANALISTA, RECUSADA_ANALISTA), `analistaId: UUID` (opcional), `decididaEm: Instant`. Pix com nota **igual ou acima de 70** fica RETIDA (RF026).

Chamado (atendimento)

`id: UUID`, `clienteId: UUID`, `atendenteId: UUID` (opcional), `assunto: String`, `status: StatusChamado` (ABERTO, EM_ATENDIMENTO, RESOLVIDO), `abertoEm: Instant`, `fechadoEm: Instant`.

MensagemChamado (atendimento)

`id: UUID`, `chamadoId: UUID`, `autor: AutorMensagem` (CLIENTE, ATENDENTE), `texto: String`, `audioUrl: String` (opcional), `enviadaEm: Instant`. Composição: a mensagem não existe sem o chamado.

Funcionario (backoffice)

`id: UUID`, `nome: String`, `email: String` (único), `papel: Papel` (ATENDENTE, ANALISTA_ANTIFRAUDE, ADMINISTRADOR), `hashSenha: String`, `segredoTotp: String` (cifrado), `ativo: Boolean`.

RegistroAuditoria (backoffice)

`id: Long`, `funcionarioId: UUID`, `acao: String`, `entidade: String`, `entidadeId: String`, `ip: String`, `ocorridoEm: Instant`. Só inserção, nunca alteração. Guardado 5 anos (RNF14).

Notificacao (notificacao)

`id: UUID`, `clienteId: UUID`, `canal: Canal` (PUSH, SMS), `tipo: String`, `conteudo: String`, `status: StatusNotificacao` (PENDENTE, ENVIADA, FALHOU), `criadaEm: Instant`, `enviadaEm: Instant`.

OutboxEvento (notificacao, usado por todos)

`id: Long`, `tipo: String` (ex.: `PixConcluido`), `agregadoId: UUID`, `payload: Json`, `criadoEm: Instant`, `processadoEm: Instant` (opcional), `tentativas: Integer`, `proximaTentativaEm: Instant`.

LembreteConta (notificacao)

`id: UUID`, `clienteId: UUID`, `descricao: String`, `diaVencimento: Integer` (1 a 31), `ativo: Boolean`.

Relações

De	Para	Cardinalidade	Tipo	Por quê
Cliente	Conta	1 para 1	Associação	Uma conta por cliente no MVP
Cliente	Dispositivo	1 para N (1 ativo)	Composição	Dispositivo não existe sem o cliente
Cliente	Credencial	1 para 1	Composição	
Cliente	Consentimento	1 para N	Composição	
Conta	Carteira	1 para N	Composição	Carteira não existe sem a conta
Carteira	Lancamento	1 para N	Associação	
Transacao	Lancamento	1 para N (no mínimo 2)	Composição	Lançamentos nascem e morrem com a transação
Conta	Transacao	1 para N	Associação	
Transacao	PixEnvio, PixRecebimento, PagamentoBoleto, MovimentoMeta	Generalização	Herança	Mesmo ciclo de vida (State) e mesmos lançamentos
Conta	ChavePix	1 para N	Agregação	A chave pode ser excluída sem afetar a conta
Conta	Meta	1 para N	Associação	
Meta	Carteira	1 para 1	Composição	A carteira da meta nasce com a meta
Cliente (menor)	VinculoResponsavel	1 para 1	Associação	Um responsável vinculado por menor no MVP
Cliente (responsável)	VinculoResponsavel	1 para N	Associação	Um pai pode ter mais de um filho
VinculoResponsavel	PedidoAprovacao	1 para N	Composição	
Cliente	ContatoConfianca	1 para N	Composição	
Transacao	AnaliseRisco	1 para 0..1	Associação	Só Pix enviado é analisado
Funcionario	AnaliseRisco	1 para N	Associação	Quem decidiu
Cliente	Chamado	1 para N	Associação	
Chamado	MensagemChamado	1 para N	Composição	
Funcionario	RegistroAuditoria	1 para N	Associação	

Como o modelo vira tabela (diferenças de propósito)

O diagrama de classes mostra o domínio; o banco (Bloco 5) guarda alguns campos de forma diferente por segurança e integridade:

No domínio	No banco	Por quê
`Cliente.cpf: Cpf`	`cpf_cifrado` + `cpf_hash`	CPF cifrado em repouso (RNF07). O hash permite buscar e garantir unicidade sem decifrar

No domínio	No banco	Por quê
`Cliente.dataNascimento: LocalDate`	`data_nascimento_cifrada`	Cifrado em repouso (RNF07)
`Funcionario.segredoTotp: String`	`segredo_totp_cifrado`	Segredo do segundo fator nunca em texto puro
Saldo da carteira (soma dos lançamentos)	Tabela extra `saldo_fechamento_diario`	Desnormalização de propósito: a consulta de saldo soma só os lançamentos do dia sobre o fechamento anterior e cabe no p95 de 300 ms (ADR-007, RNF01)
Subtipos de `Transacao` (herança)	Uma tabela por subtipo, com chave estrangeira composta (`transacao_id` , `tipo`)	Impede, por exemplo, um registro de Pix apontando para uma transação de boleto

Objeto de entrada do Pix

PedidoPix (objeto de valor, sem tabela): `chaveDestino: String` , `valorCentavos: Long` , `descricao: String` (opcional, até 140 caracteres), `assinaturaDispositivo: String` (o pedido assinado com a chave do aparelho, destravada pela biometria, ADR-010). É o corpo de `POST /v1/pix/envios` ; vira `PixEnvio` depois de validado.

Classes de serviço (para o diagrama de classes e o C4 nível 4)

Classe	Métodos principais
ServicoEnvioPix	`+iniciar(contaId: UUID, pedido: PedidoPix, chaveIdempotencia: String): Transacao` , `+confirmarRetorno(idParceiro: String, aceito: Boolean): void` , `-reservarSaldo(t: Transacao): void` , `-compensar(t: Transacao): void`
MotorAntifraude	`+avaliar(t: PixEnvio): AnaliseRisco` (soma as notas das `RegraRisco`)
RegraRisco (interface)	`+pontuar(t: PixEnvio, h: HistoricoCliente): Integer`
PortaBancaria (interface)	`+consultarChave(chave: String): DadosRecebedor` , `+enviarPix(t: PixEnvio): RespostaParceiro` , `+pagarBoleto(b: PagamentoBoleto): RespostaParceiro`
PortaVoz (interface)	`+falar(texto: String): UrlAudio`
UnidadeDeTrabalho	`+executar<T>(trabalho: Funcao<T>): T`

3.6 Rotas da API (fonte única)

Lista canônica das rotas. O contrato completo, com corpos de requisição e resposta, está em OpenAPI 3.0 no Bloco 8 (`bloco-8-dev/openapi.yaml`), que segue esta lista. Os diagramas de sequência (Bloco 4) usam estes mesmos caminhos.

Base: `https://api.rufusbank.com.br/v1` em produção, `http://localhost:3000/v1` no desenvolvimento. Todas as rotas, exceto as marcadas como públicas, exigem `Authorization: Bearer <token de acesso>`.

Método	Rota	Módulo	RF	Observação
POST	/onboarding/cadastros	identidade	RF001	Pública. Inicia o cadastro
POST	/onboarding/cadastros/{id}/documentos	identidade	RF001	Pública com token do cadastro. Foto do documento
POST	/onboarding/cadastros/{id}/selfie	identidade	RF001, RF002	Envia a selfie e dispara a validação no parceiro
GET	/onboarding/cadastros/{id}	identidade	RF002	Situação do cadastro
POST	/auth/login	identidade	RF003, RF004	Pública. CPF, senha e assinatura do aparelho
POST	/auth/renovacao	identidade	RNF05	Pública com token de renovação. Troca o token
POST	/auth/logout	identidade		Revoga a sessão
POST	/dispositivos/troca	identidade	RF005	Selfie nova para trocar de aparelho
GET	/contas/me	contas	RF006	Dados da conta
GET	/contas/me/saldo	contas	RF006	Saldo da carteira principal
GET	/contas/me/extrato	contas	RF006	Paginado por cursor
POST	/contas/me/bloqueio-total	contas	RF020	"Bloquear tudo agora"
DELETE	/contas/me/bloqueio-total	contas	RF020	Desbloqueio com selfie e 1 hora de espera
POST	/pix/consultas-chave	pagamentos	RF009	Devolve nome do recebedor e banco
POST	/pix/envios	pagamentos	RF008, RF019	Exige `Idempotency-Key`
GET	/pix/envios/{id}	pagamentos	RF008	Situação do Pix
GET	/pix/envios/{id}/comprovante	pagamentos	RF012	
POST	/pix/cobrancas	pagamentos	RF010	Gera QR Code para receber
GET	/pix/chaves	pagamentos	RF011	
POST	/pix/chaves	pagamentos	RF011	
DELETE	/pix/chaves/{id}	pagamentos	RF011	
POST	/boletos/consultas	pagamentos	RF013	Lê a linha digitável e devolve beneficiário, valor e vencimento
POST	/boletos/pagamentos	pagamentos	RF013	Exige `Idempotency-Key`
GET	/metas	pagamentos	RF017	
POST	/metas	pagamentos	RF017	
POST	/metas/{id}/depositos	pagamentos	RF017	Exige `Idempotency-Key`
POST	/metas/{id}/resgates	pagamentos	RF017	Exige `Idempotency-Key`
POST	/familia/convites-responsavel	familia	RF014	O menor convida o responsável
POST	/familia/vinculos/{id}/aceite	familia	RF014	O responsável aceita
PUT	/familia/vinculos/{id}/limites	familia	RF015	Só o responsável
POST	/familia/pedidos-aprovacao	familia	RF016	O menor pede

Método	Rota	Módulo	RF	Observação
POST	/familia/pedidos-aprovacao/{id}/decisao	familia	RF016	O responsável decide
POST	/contatos-confianca	familia	RF018	
DELETE	/contatos-confianca/{id}	familia	RF018	
POST	/falas	notificacao	RF007	Texto para áudio. Devolve URL do áudio
POST	/lembretes	notificacao	RF022	
POST	/chamados	atendimento	RF024	
GET	/chamados/{id}/mensagens	atendimento	RF024	
POST	/chamados/{id}/mensagens	atendimento	RF024	
POST	/titular/exportacoes	identidade	RF029	Gera arquivo com os dados do titular
POST	/titular/encerramento	identidade	RF029	
POST	/webhooks/baas	pagamentos	RF010, RF008	Pública, com assinatura HMAC do parceiro
POST	/admin/auth/login	backoffice	RF030	Pública. Senha e TOTP
GET	/admin/analises	antifraude	RF027	Papel ANALISTA_ANTIFRAUDE
POST	/admin/analises/{id}/decisao	antifraude	RF027	Papel ANALISTA_ANTIFRAUDE
GET	/admin/chamados	atendimento	RF025	Papel ATENDENTE
POST	/admin/chamados/{id}/mensagens	atendimento	RF025	Papel ATENDENTE
GET	/admin/clientes/{id}/historico	atendimento	RF025	Papel ATENDENTE. Gera registro de auditoria
POST	/admin/funcionarios	backoffice	RF030	Papel ADMINISTRADOR
GET	/health (fora do prefixo /v1)		RNF03	Pública. Usada pelo balanceador e pela verificação externa do Route 53. Também responde em /v1/health

Portas: a API escuta na **3000** dentro do container. O balanceador recebe na **443** (HTTPS) e redireciona a 80 para 443. O PostgreSQL usa a **5432**, acessível só de dentro da VPC.

Bloco 4 · Diagramas UML

São 9 diagramas UML, que cobrem os 6 tipos pedidos no enunciado, mais 2 processos de negócio em BPMN como complemento. Os diagramas grandes estão divididos em partes, para o texto continuar legível no PDF impresso (cerca de 7 pt ou mais). Cada imagem tem título, autores e data no cabeçalho e segue o modelo de domínio (05-modelo-de-dominio.md) e as rotas (06-rotas-da-api.md) do Bloco 3, com os mesmos nomes.

#	Diagrama	Tipo exigido	Requisitos cobertos
1	Casos de uso	Casos de Uso	RF001 a RF030
2	Classes do domínio	Classes (mín. 10)	23 classes, modelo de domínio inteiro
3	Sequência: autenticação	Sequência 1 de 3	RF003, RF004, RF005, RNF05
4	Sequência: Pix com sucesso	Sequência 2 de 3	RF008, RF009, RF019, RF026, RNF02
5	Sequência: erro crítico no Pix	Sequência 3 de 3	RNF11, US07, saga do Bloco 3
6	Componentes	Componentes	ADR-001, ADR-002
7	Deployment	Deployment	RNF03, ADR-009, ADR-013
8	Atividade: abertura de conta	Atividades 1 de 2 (negócio)	RF001, RF002, RF014, US09
9	Atividade: análise antifraude	Atividades 2 de 2 (administrativo)	RF026, RF027, US16
10	Processos de negócio em BPMN	Complemento (Aula 02)	RF008, RF014, RF026, RF027, US09

Gerar de novo: `python ferramentas/diagramas/gerar.py bloco-4`. As fontes ficam em diagramas/: `.mmd` para Mermaid (classes, sequência e atividades) e `.svg.py` para casos de uso, componentes, deployment e BPMN, que o Mermaid não desenha na notação padrão. As imagens ficam em `diagramas/img/`, em SVG (para o PDF) e PNG (para os slides).

---

1. Casos de uso

O diagrama está em 2 partes, para o texto continuar legível impresso: a parte 1 traz o cliente, os sistemas externos e todos os «include» e «extend»; a parte 2 traz os atores de gestão.

Diagrama de Casos de Uso, parte 1 de 2 · Cliente e sistemas externos

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

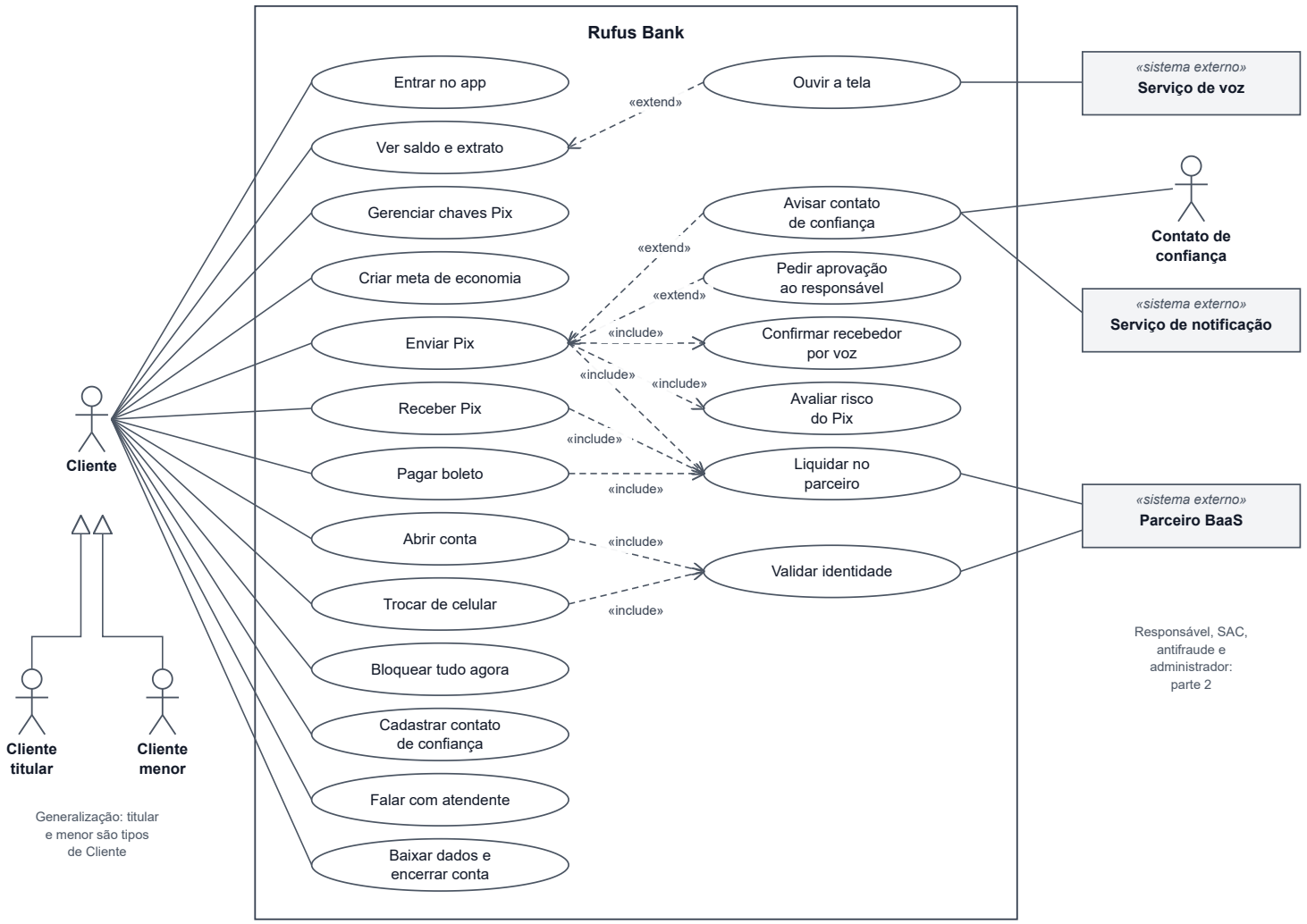
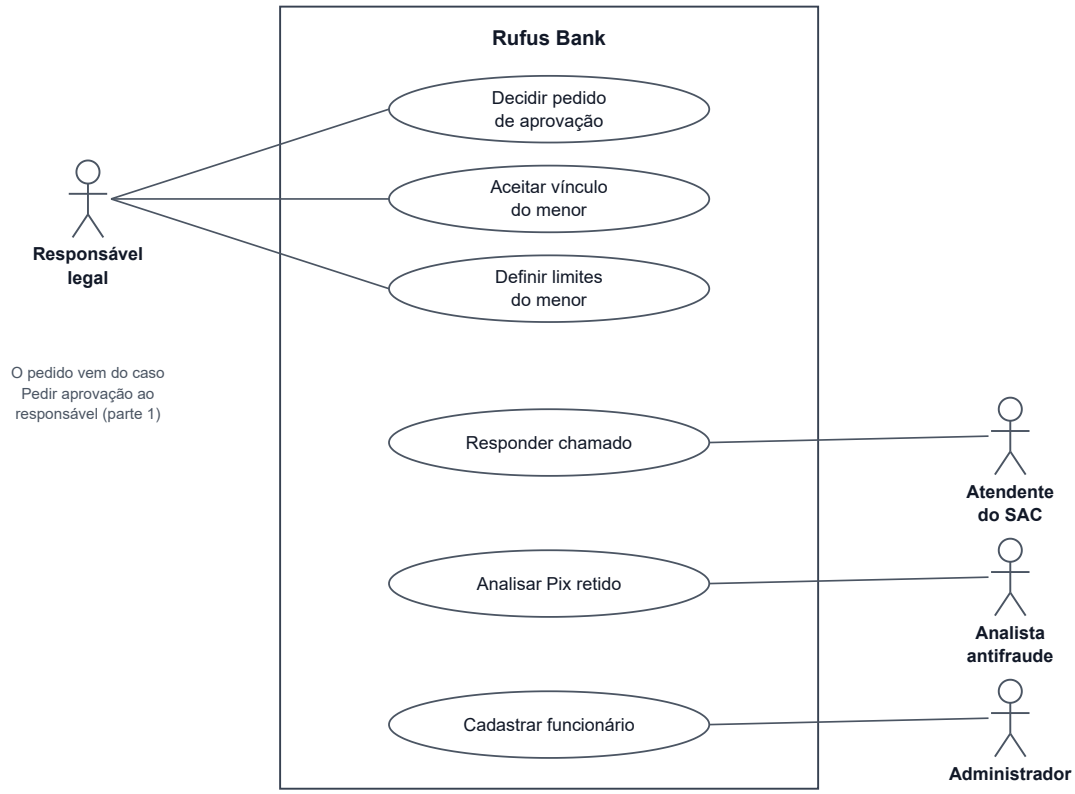


Diagrama de Casos de Uso, parte 2 de 2 · responsável legal, SAC, antifraude e administração

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. Tudo que cada ator faz no Rufus. A fronteira é o sistema Rufus Bank: dentro dela ficam os casos de uso, fora ficam os atores.

Atores. À esquerda, o **Cliente**, que é um ator abstrato. **Cliente titular** e **Cliente menor** são especializações dele (generalização, seta com triângulo vazio): os dois fazem tudo que o Cliente faz, e o menor tem limites. À direita ficam os atores que reagem ou administram: contato de confiança, responsável legal, atendente do SAC, analista antifraude e administrador. Os três sistemas externos (serviço de voz, serviço de notificação e parceiro BaaS) aparecem como atores com o estereótipo «sistema externo».

Relações entre casos de uso. - **«include»** (sempre acontece): Enviar Pix sempre inclui Confirmar recebedor por voz (RF009), Avaliar risco do Pix (RF026) e Liquidar no parceiro. Abrir conta e Trocar de celular sempre incluem Validar identidade (RF002, RF005). Receber Pix e Pagar boleto também incluem Liquidar no parceiro. A seta sai do caso base e aponta para o incluído. - **«extend»** (acontece só sob condição): Ouvir a tela estende Ver saldo e extrato, quando o cliente toca em "Ouvir" (RF007). Avisar contato de confiança estende Enviar Pix quando a movimentação foge do padrão (RF018). Pedir aprovação ao responsável estende Enviar Pix quando o menor passa do limite (RF016). A seta sai da extensão e aponta para o caso base. - **Generalização** entre atores: titular e menor herdam todos os casos do Cliente.

Por que o Administrador aparece. O RF030 e a US22 pedem cadastro de funcionários com papel, por isso ele é um ator.

2. Classes

Diagrama de Classes, parte A de D · identidade

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

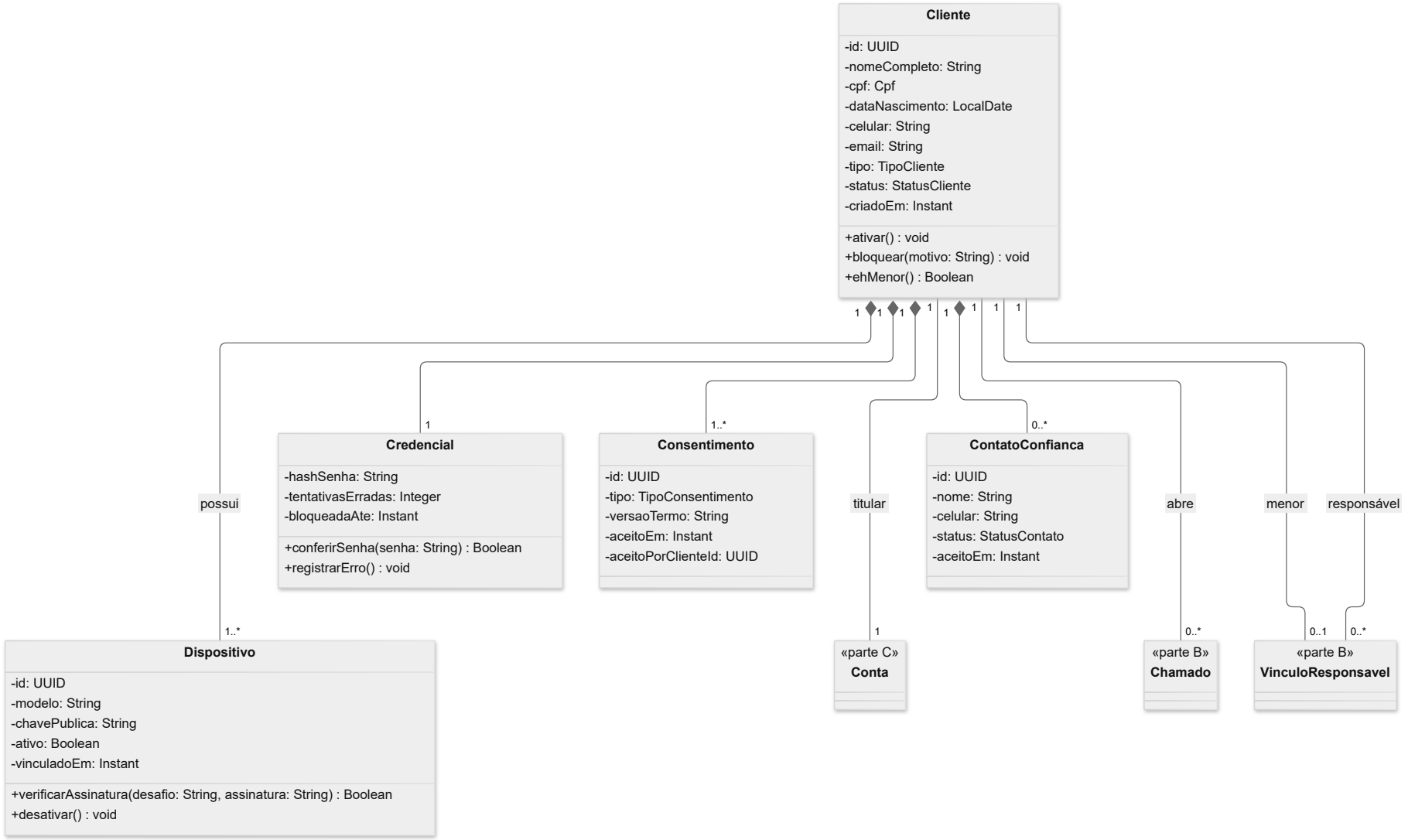


Diagrama de Classes, parte B de D · família, atendimento e backoffice

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

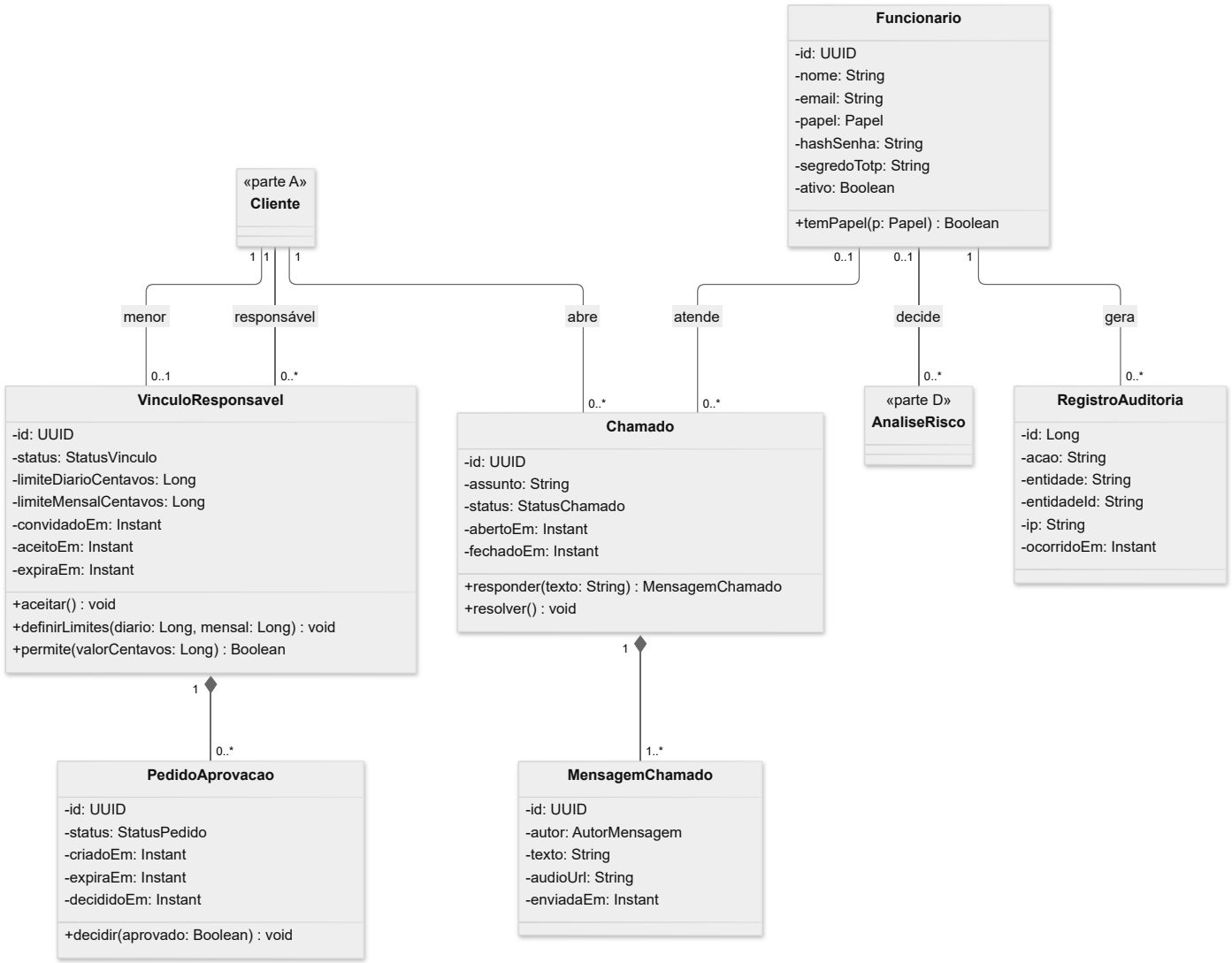


Diagrama de Classes, parte C de D · contas, carteiras e metas

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

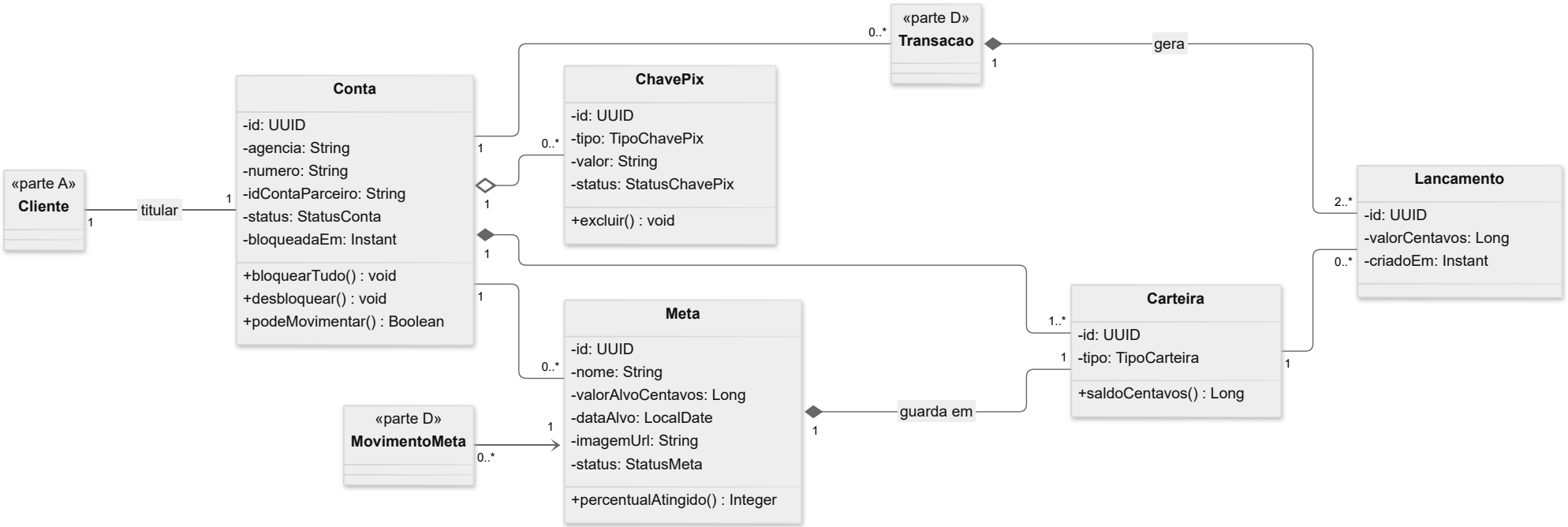
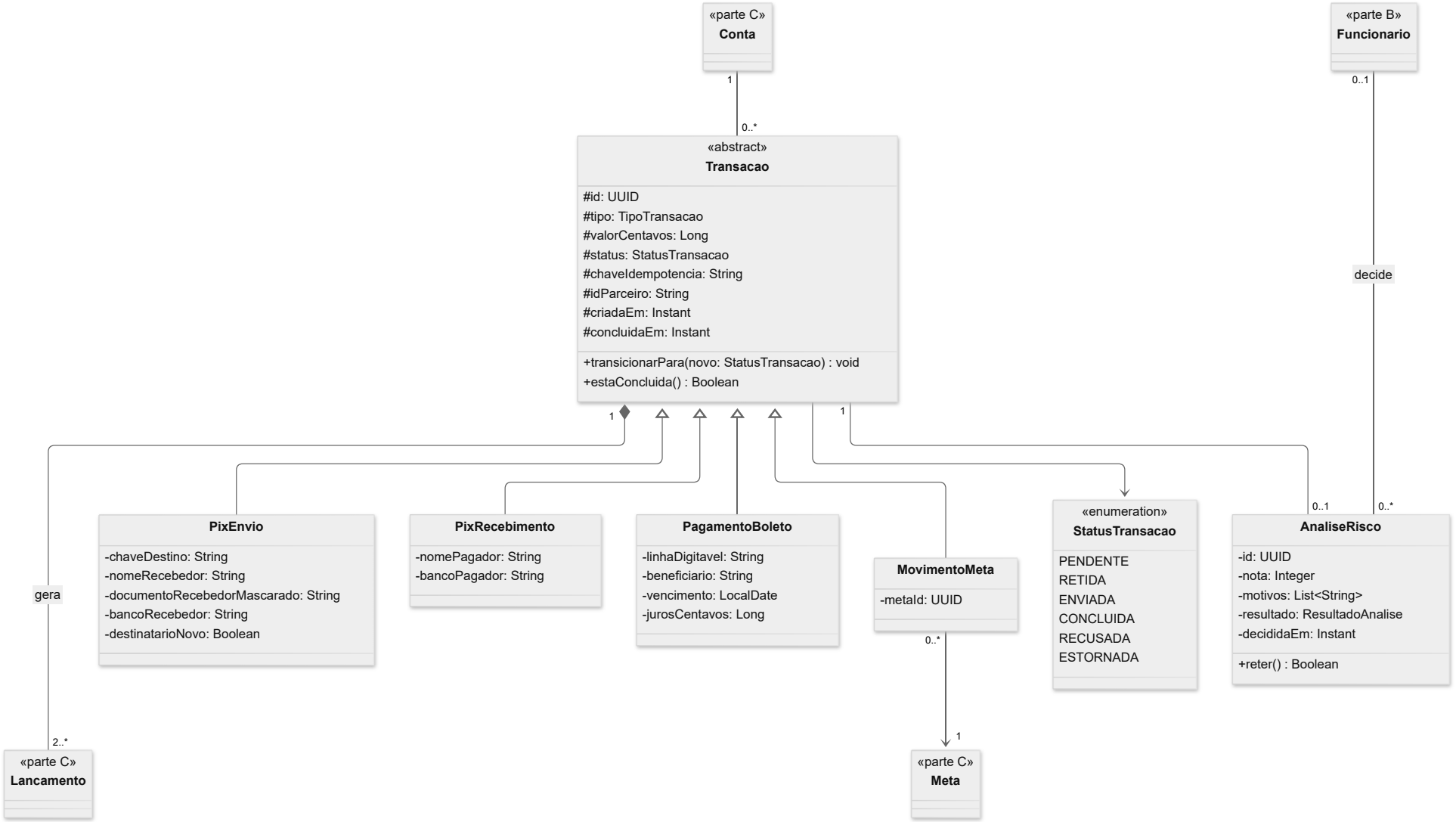


Diagrama de Classes, parte D de D · transações (herança) e antifraude

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. As 23 classes do domínio com atributos tipados, métodos e visibilidade, e todas as relações da tabela de relações do modelo de domínio. Para o texto continuar legível impresso, o diagrama está em 4 partes, uma por grupo de módulos (A identidade, B família, atendimento e backoffice, C contas e metas, D transações e antifraude). Uma classe que pertence a outra parte aparece só com o nome e o estereótipo «parte X», para a relação continuar visível. As 27 relações do modelo de domínio estão todas desenhadas; as duas que cruzam grupos aparecem nas duas partes.

Visibilidade. Atributos privados (`-`): o estado só muda pelos métodos públicos (`+`), por exemplo `Conta.bloquearTudo()` e `VinculoResponsavel.definirLimites(diario: Long, mensal: Long)` . Em `Transacao` os atributos são protegidos (`#`) porque as subclasses usam esses campos.

Tipos. Dinheiro sempre em `Long` com sufixo `Centavos` , porque ponto flutuante perde centavo. Datas e horas em `Instant` (UTC). Datas sem hora em `LocalDate` . Identificadores em `UUID` . Os estados são enumerações, como `StatusTransacao` .

As três decisões que a arguição costuma perguntar.

Onde	Relação usada	Por que essa e não outra
`Transacao` e `PixEnvio` , `PixRecebimento` , `PagamentoBoleto` , `MovimentoMeta`	**Herança** (generalização)	Um Pix **é uma** transação. Todos os subtipos compartilham o mesmo ciclo de vida (`StatusTransacao` , padrão State), a idempotência e os lançamentos no razão. Com composição (a transação "teria um" Pix), cada subtipo repetiria o ciclo de vida e as regras de estorno
`Chamado` e `MensagemChamado`	**Composição** (losango cheio)	A mensagem não existe sem o chamado: se o chamado for apagado por anonimização, as mensagens vão junto. A mesma regra vale para `Transacao` e `Lancamento` (um lançamento sem transação quebraria a soma zero das partidas dobradas) e para `Cliente` com `Dispositivo` , `Credencial` , `Consentimento` e `ContatoConfianca`
`Conta` e `ChavePix`	**Agregação** (losango vazio)	A chave Pix pertence à conta, mas pode ser excluída e cadastrada de novo sem afetar a conta (RF011). A conta continua existindo sem nenhuma chave

Outras relações. - `Cliente` tem duas associações com `VinculoResponsavel` , com papéis diferentes: como **menor** (0 ou 1 vínculo) e como **responsável** (0 ou mais vínculos, porque um pai pode ter mais de um filho). - `Transacao` gera **2 ou mais** `Lancamento` : as partidas dobradas exigem pelo menos um débito e um crédito (ADR-007). - `Meta` tem a própria `Carteira` por composição: o dinheiro guardado na meta é uma carteira separada no razão.

Do diagrama ao banco. Alguns atributos mudam de forma no banco por segurança. Por exemplo, `cpf` vira `cpf_cifrado` e `cpf_hash` . A tabela com essas diferenças está no fim do modelo de domínio do Bloco 3.

3. Sequência 1 · Autenticação

Diagrama de Sequência 1, parte A · Autenticação: login com celular vinculado e JWT

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

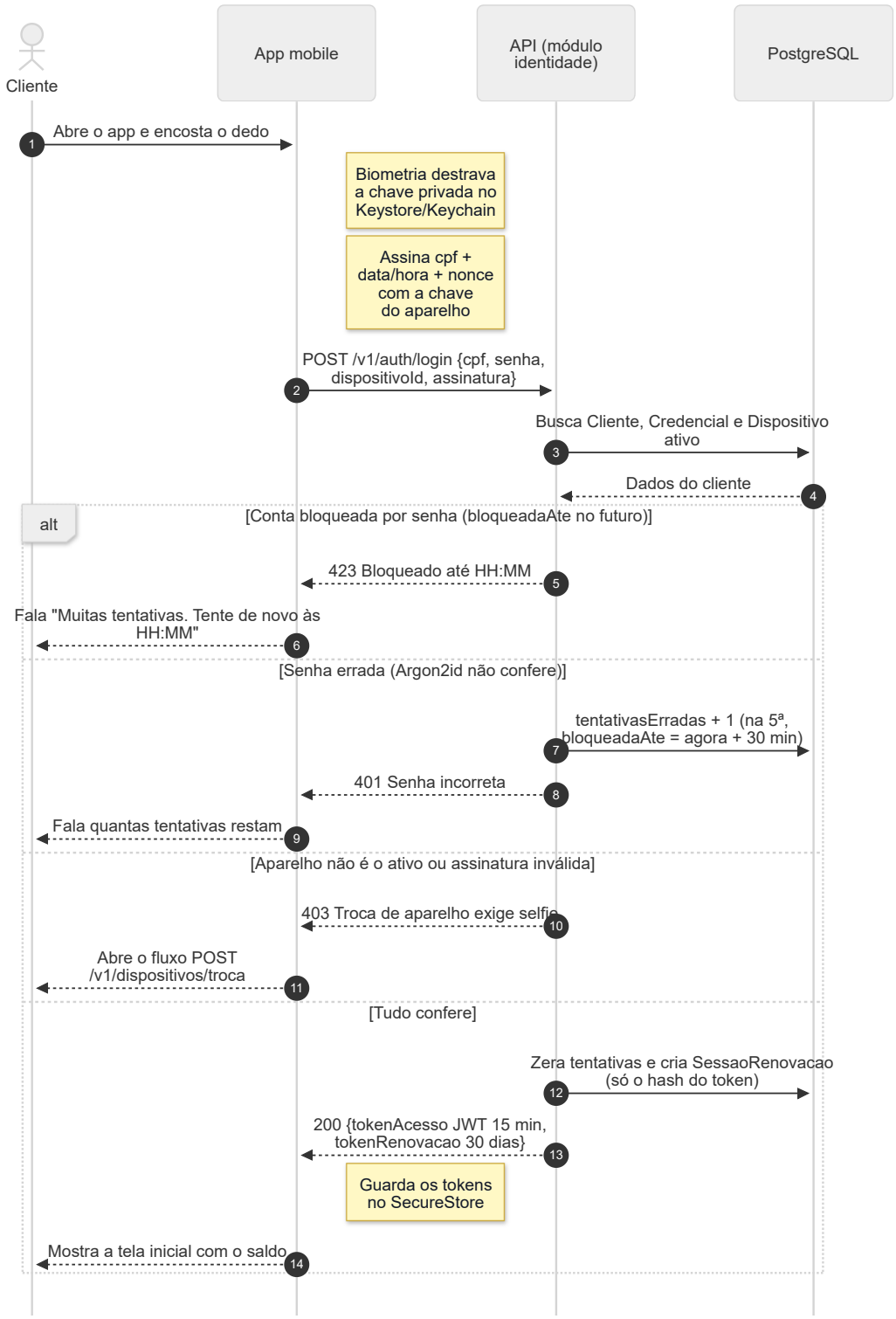
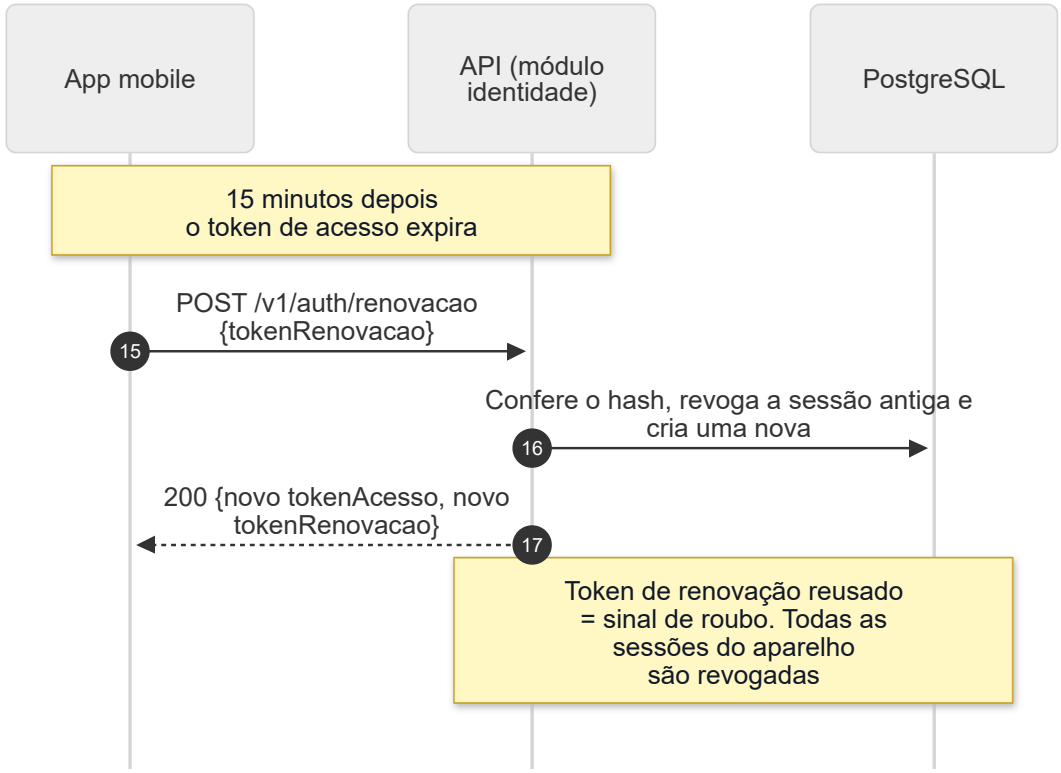


Diagrama de Sequência 1, parte B · Autenticação: renovação do token de acesso

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. O login com celular vinculado (ADR-010) e a renovação do token.

Como ler. - Passos 1 e 2 (e as notas): a biometria destrava, no próprio celular, a chave privada que fica no Keystore ou no Keychain. O app assina os dados do login com essa chave. A digital nunca sai do celular. - O `alt` mostra os quatro resultados possíveis da mesma requisição: - conta bloqueada por excesso de tentativas (423); - senha errada (401), com o bloqueio de 30 minutos na 5ª tentativa (RF004); - aparelho diferente do vinculado (403), que leva à troca de aparelho com selfie (RF005); - sucesso (200), com JWT de 15 minutos e token de renovação de 30 dias (RNF05). - Na parte B (passos 15 a 17), a renovação troca o token a cada uso. Se um token de renovação já usado aparece de novo, é sinal de roubo e todas as sessões do aparelho são revogadas.

4. Sequência 2 · Envio de Pix, caminho feliz

A sequência está em 3 partes (A, B e C), com a numeração dos passos continuando de uma para a outra. O MotorAntifraude aparece como chamada interna da API (passo 13), porque é um componente dela (ADR-001).

Diagrama de Sequência 2, parte A · Pix com sucesso: consulta da chave e fala do recebedor

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

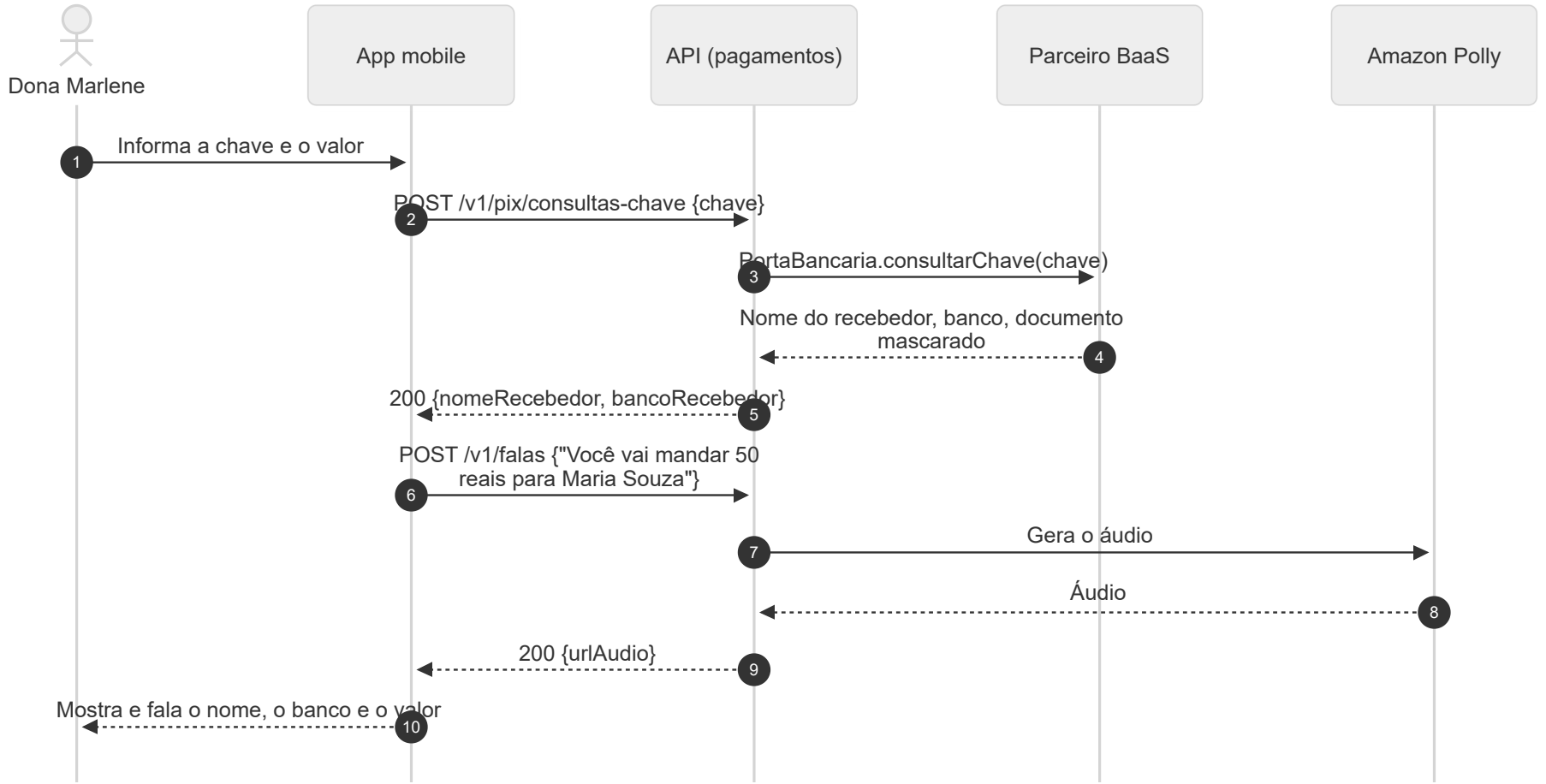


Diagrama de Sequência 2, parte B · Pix com sucesso: confirmação, antifraude e gravação

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

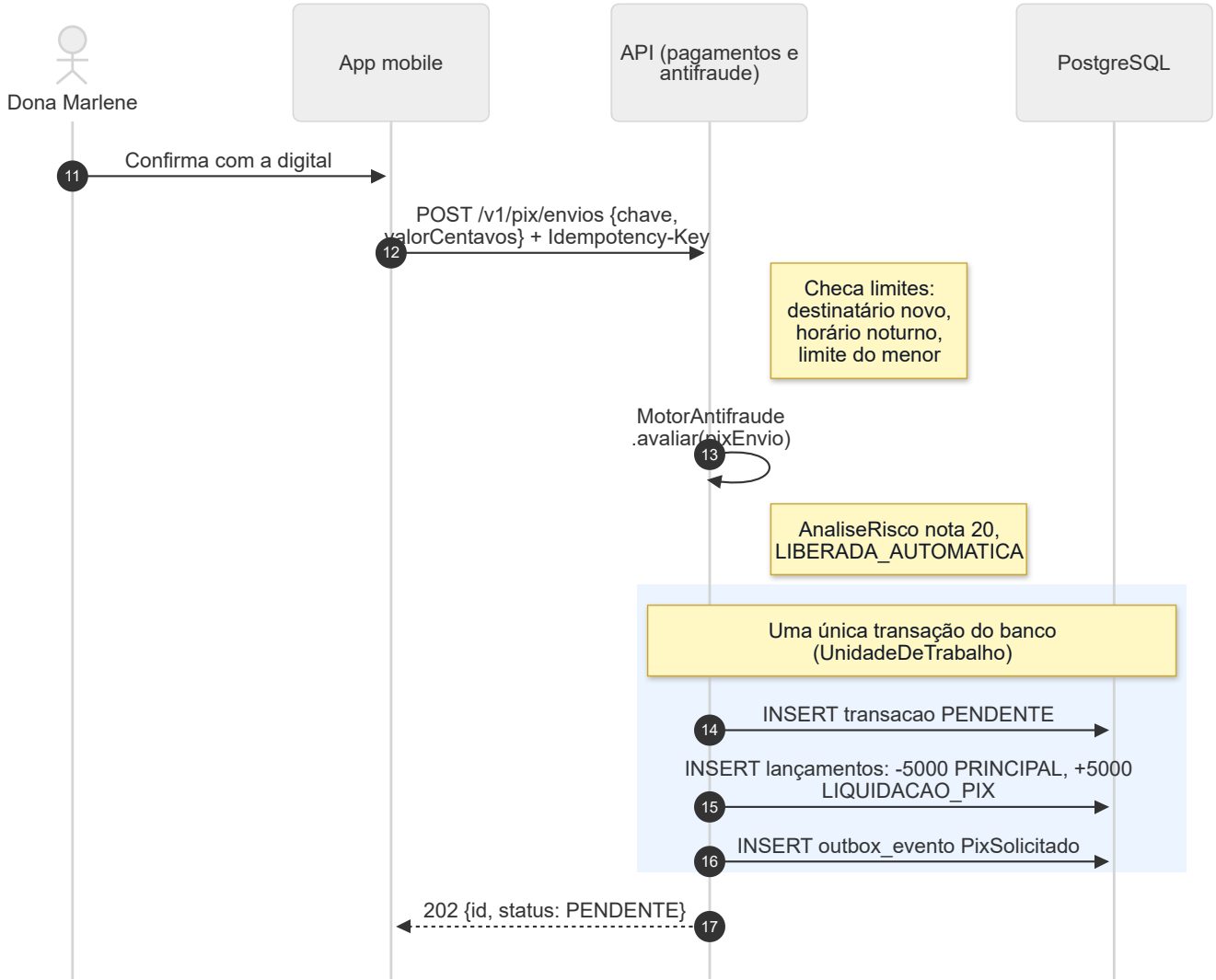
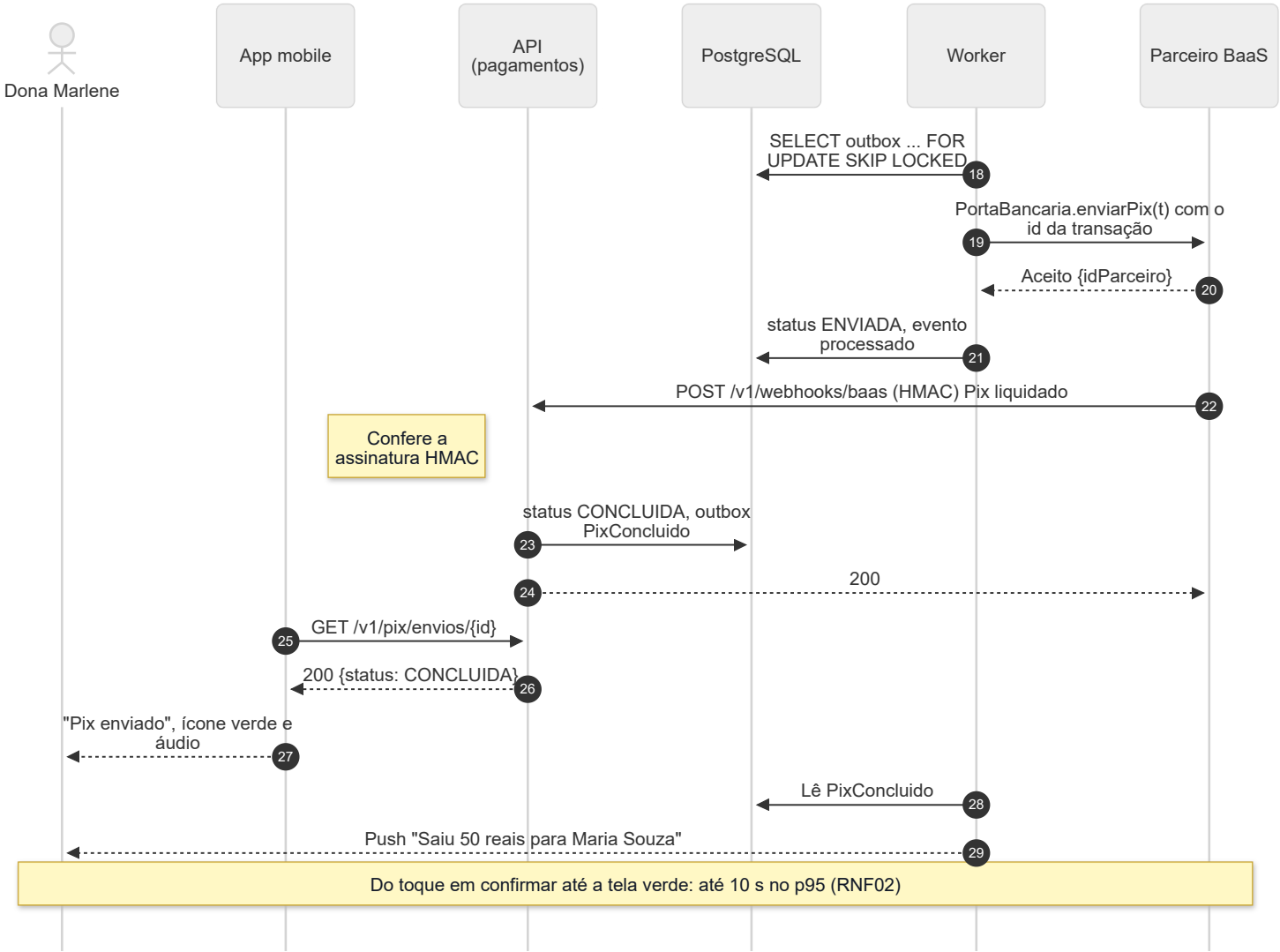


Diagrama de Sequência 2, parte C · Pix com sucesso: liquidação no parceiro e aviso

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. O fluxo principal do produto, com a Dona Marlene enviando 50 reais.

Como ler, em quatro partes. 1. **Antes de confirmar** (1 a 10, parte A): a API consulta o recebedor no parceiro e gera o áudio. O app mostra e fala o nome, o banco e o valor (RF009). É a barreira contra golpe e contra erro de digitação. 2. **Confirmação** (11 a 17, parte B): com a digital, o app envia o Pix com `Idempotency-Key`. A API confere os limites (RF019, limite do menor), pede a nota ao antifraude (RF026) e, numa única transação do banco (área azul), grava a transação, reserva o saldo com dois lançamentos e publica o evento no outbox. A resposta é 202: o Pix foi aceito para processamento. 3. **Envio ao parceiro** (18 a 24, parte C): o worker lê o outbox e envia. O parceiro confirma a liquidação por webhook assinado. 4. **Retorno ao cliente** (25 a 29, parte C): o app consulta a situação e mostra a tela verde com áudio, e o worker manda o push.

Por que o 202 e o worker. Se a API esperasse o parceiro dentro da requisição, uma lentidão do parceiro prenderia conexões e o clique repetido viraria risco de duplicidade. O cliente vê o resultado em até 10 s no p95 (RNF02).

5. Sequência 3 · Erro crítico: falha no parceiro e estorno

A sequência está em 3 partes, com a numeração continuando: A (passos 1 a 8), B (9 a 17) e C (18 a 24).

Diagrama de Sequência 3, parte A · Erro crítico: resposta perdida e reenvio idempotente

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

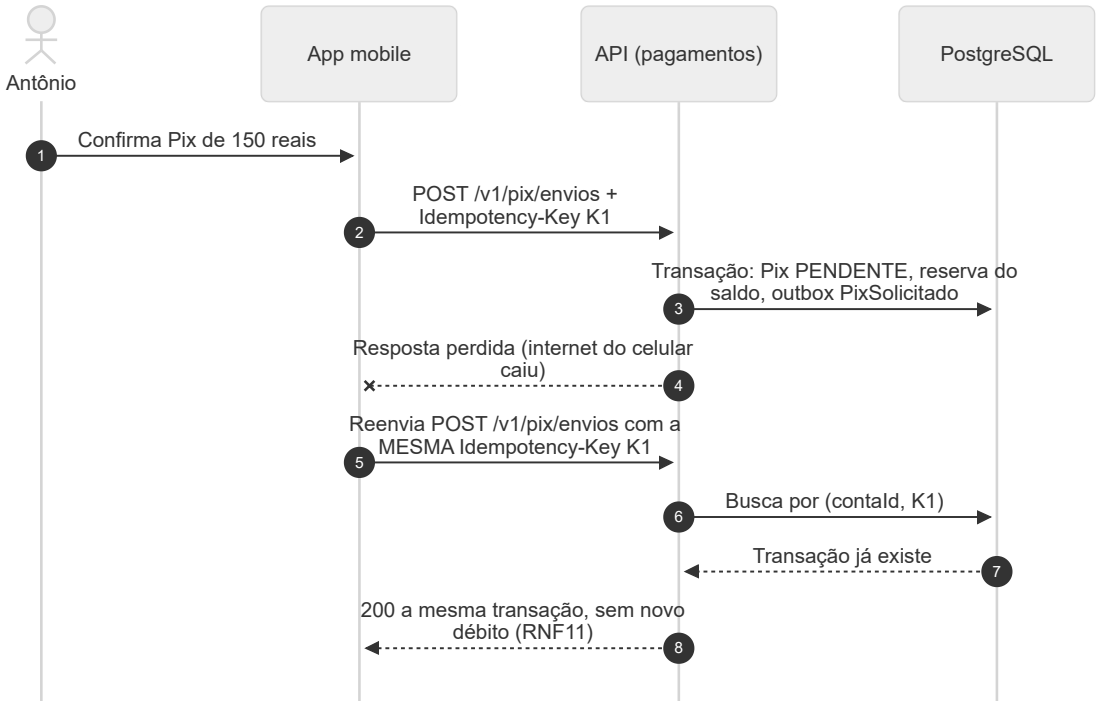


Diagrama de Sequência 3, parte B · Erro crítico: falha no parceiro, nova tentativa e compensação

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

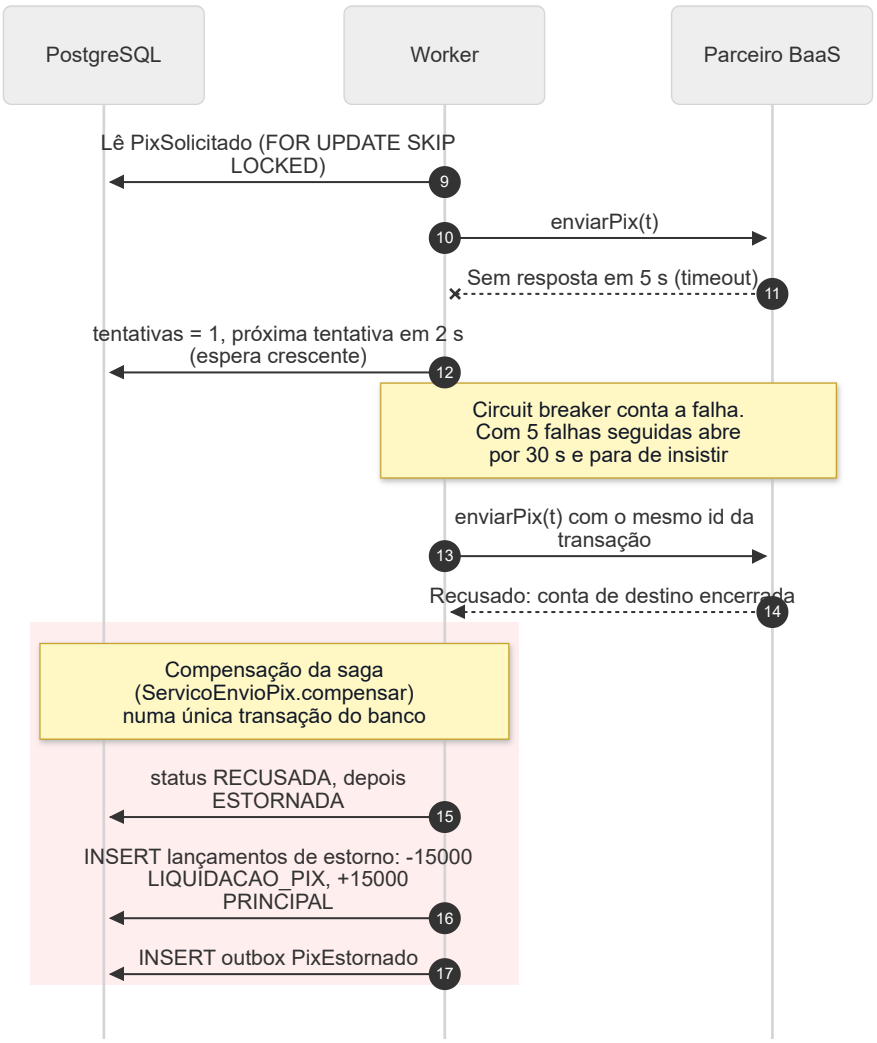
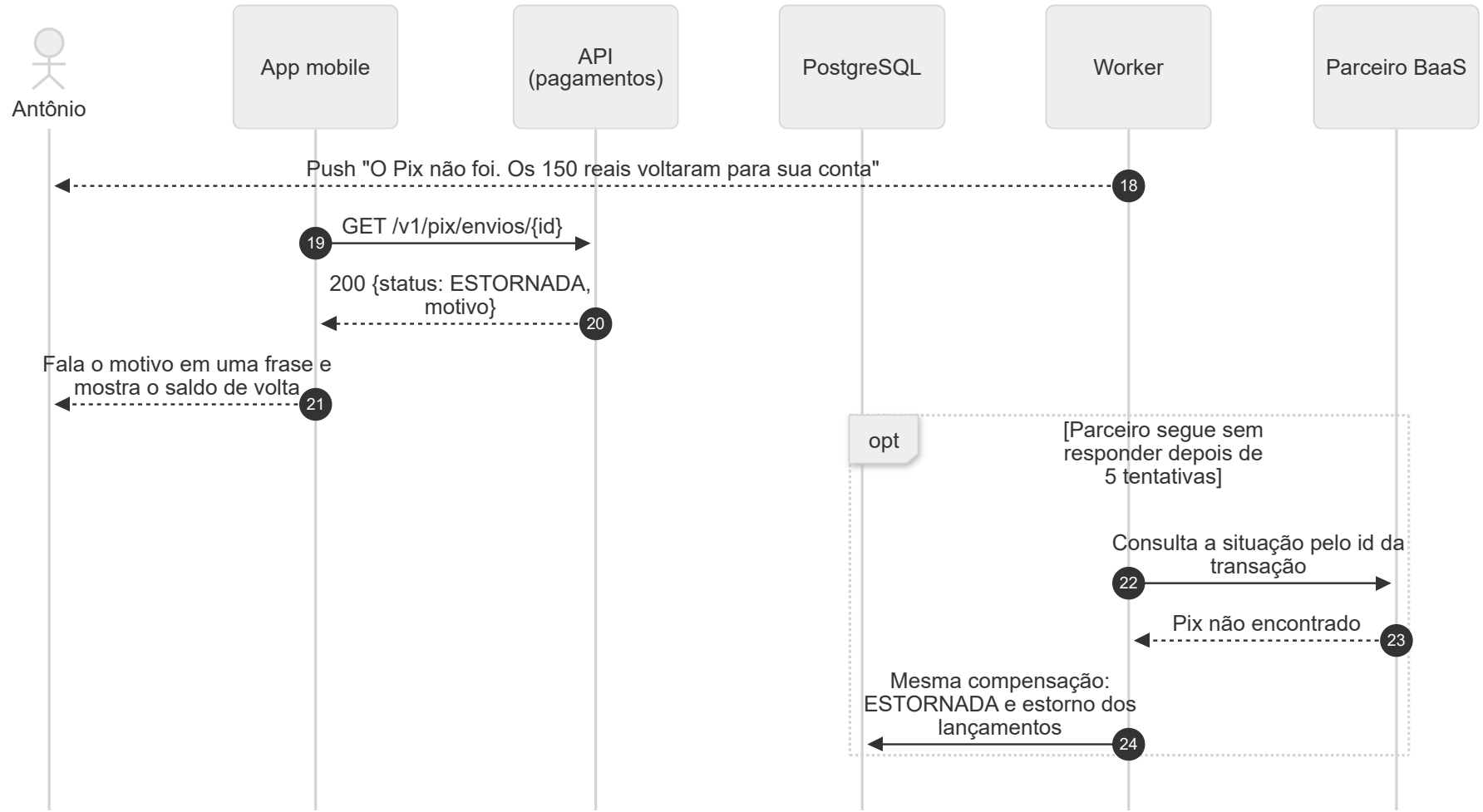


Diagrama de Sequência 3, parte C · Erro crítico: aviso ao cliente e consulta ao parceiro

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. O cenário de erro mais grave: dinheiro reservado e o parceiro falhando no meio. São dois problemas encadeados.

Problema 1, resposta perdida (passos 2 a 8). A internet do Antônio cai depois que a API já gravou o Pix (a seta com "x" é a mensagem perdida). O app reenvia com a **mesma** `Idempotency-Key`, a API encontra a transação existente e devolve a mesma, **sem novo débito**. É o que garante o RNF11 e a US07.

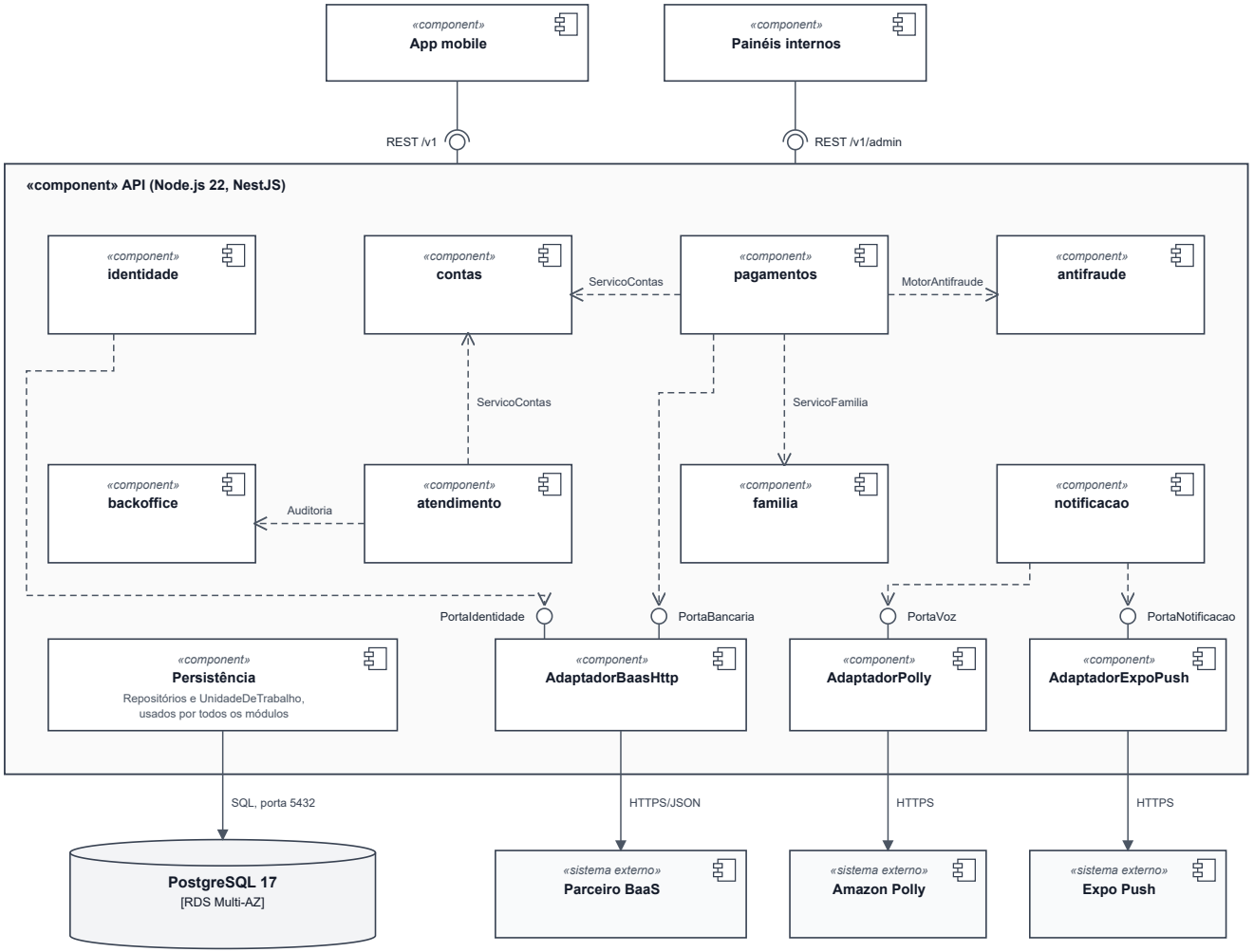
Problema 2, falha e recusa no parceiro (passos 9 a 21). - O worker não recebe resposta em 5 s, registra a tentativa e agenda outra com espera crescente. - O circuit breaker conta a falha. Com 5 falhas seguidas ele para de chamar o parceiro por 30 s, para não piorar uma queda. - Na segunda tentativa, com o mesmo id de transação (o parceiro não duplica), o parceiro recusa. - A **compensação da saga** (área vermelha), numa única transação do banco, marca a transação como RECUSADA e depois ESTORNADA, grava os lançamentos de estorno, que devolvem os 150 reais à carteira principal, e publica `PixEstornado`. - O Antônio recebe push e áudio com o motivo em uma frase.

O `opt` no fim. Se o parceiro continuar sem responder depois de 5 tentativas, o worker consulta a situação pelo id. Se o Pix não existe no parceiro, aplica a mesma compensação. O dinheiro do cliente nunca fica preso sem resposta.

6. Componentes

Diagrama de Componentes · API do Rufus Bank e sistemas externos

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Legenda: ○ interface fornecida · ◡ interface requerida · seta tracejada = dependência «use» da interface

O que mostra. Os componentes da API, as interfaces que cada um fornece e requer, e os sistemas externos.

Notação UML 2. - Componente: retângulo com o ícone de componente no canto. - Interface fornecida: círculo, o "pirulito". - Interface requerida: meia-lua, o "soquete". - Dependência de uma interface: seta tracejada.

Como ler. - No alto, a API fornece as interfaces **REST /v1**, usada pelo app, e **REST /v1/admin**, usada pelos painéis. O soquete de cada front-end se encaixa no pirulito da API. - Dentro da API, os 8 módulos. Cada seta tracejada leva o nome da interface pública usada: `ServicoContas`, `MotorAntifraude`, `ServicoFamilia` e `Auditoria`. - Embaixo, cada adaptador **fornece** uma porta: `AdaptadorBaasHttp` fornece `PortaBancaria` e `PortaIdentidade`, `AdaptadorPolly` fornece `PortaVoz` e `AdaptadorExpoPush` fornece `PortaNotificacao`. Os módulos **requerem** essas portas: `identidade` requer `PortaIdentidade`, `pagamentos` requer `PortaBancaria` e `notificacao` requer `PortaVoz` e `PortaNotificacao`.

Decisão que o diagrama prova. Nenhum módulo depende de um sistema externo diretamente: todos dependem de uma interface nossa. Trocar o parceiro BaaS (risco R04) é trocar um componente da última linha (ADR-002).

7. Deployment

O diagrama está em 2 partes: a produção na AWS (parte 1) e o desenvolvimento local com a homologação (parte 2).

Diagrama de Deployment, parte 1 de 2 · produção na AWS

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

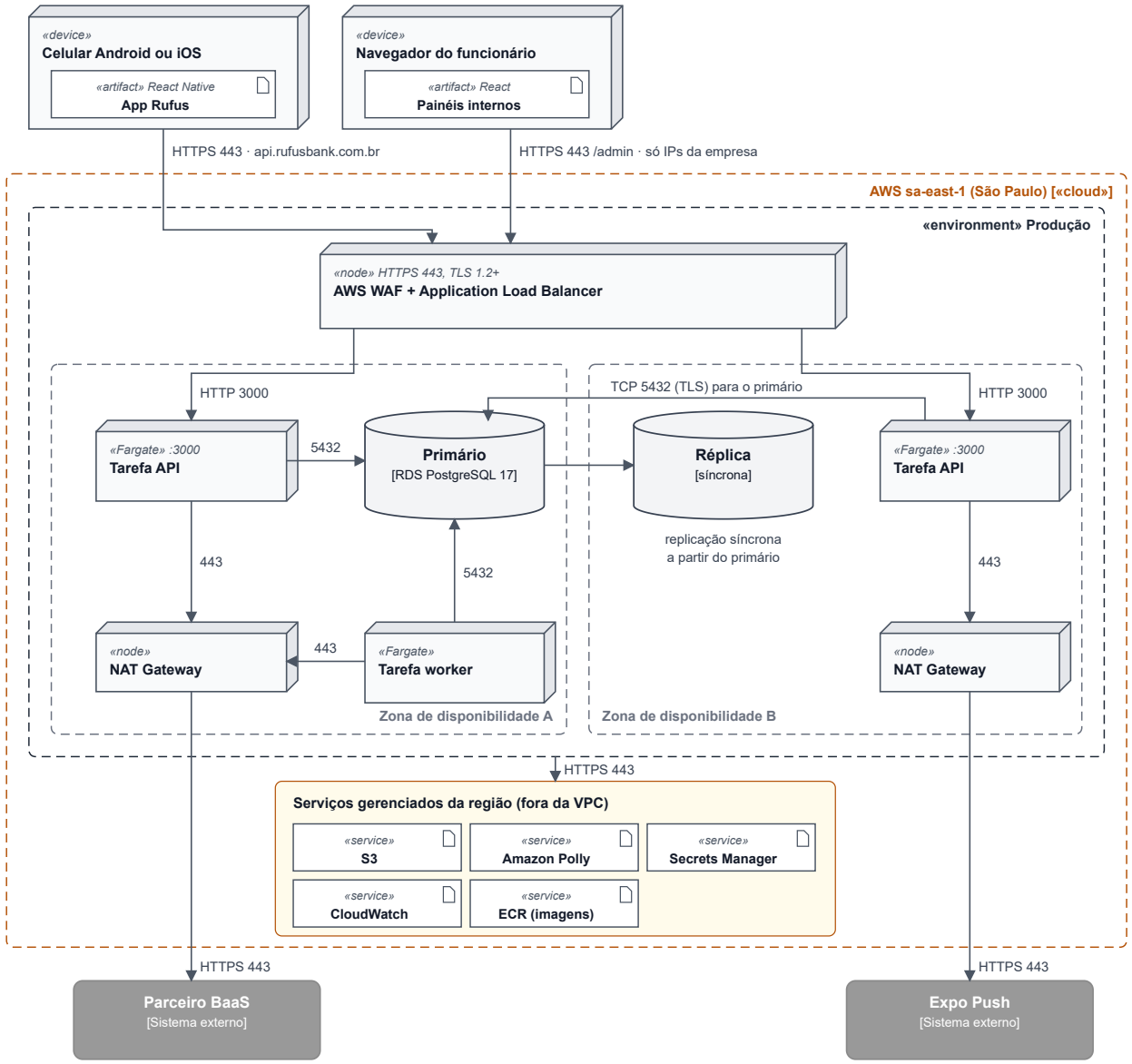
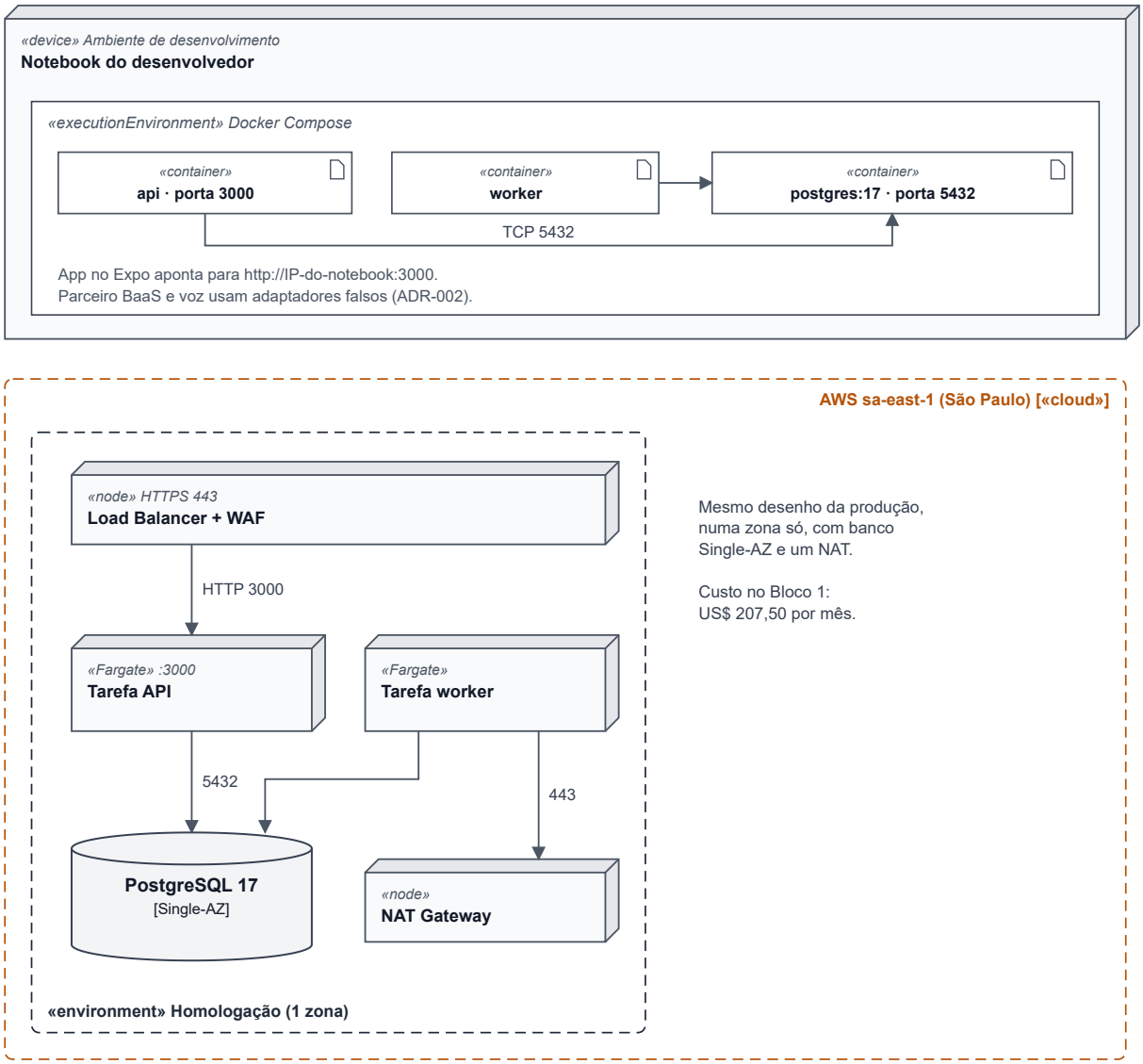


Diagrama de Deployment, parte 2 de 2 · desenvolvimento local e homologação

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. Onde cada peça roda nos três ambientes, com protocolos e portas.

Desenvolvimento (no alto à direita): tudo no notebook com Docker Compose. A API fica na porta 3000 e o PostgreSQL 17 na 5432, com custo zero de nuvem (Bloco 1). O app no Expo aponta para o IP do notebook, e o parceiro e a voz usam adaptadores falsos.

Produção (AWS sa-east-1, duas zonas de disponibilidade): - O celular e o navegador chegam ao **WAF e Application Load Balancer** por HTTPS na porta 443, com TLS 1.2 ou superior. O caminho `/admin` só aceita IPs da empresa (ADR-012). - O balanceador distribui por HTTP na porta 3000 para uma **tarefa da API em cada zona**. Se uma zona cair, a outra continua atendendo. - O **PostgreSQL primário** fica na zona A, com **réplica síncrona** na zona B (RDS Multi-AZ). As duas tarefas da API e o worker usam o primário pela porta 5432 com TLS. Se o primário cair, o RDS promove a réplica. - Cada zona tem o próprio **NAT Gateway** para sair à internet na porta 443 (parceiro BaaS e Expo Push). Assim a saída não depende de uma zona só. - S3, Polly, Secrets Manager, CloudWatch e ECR são serviços gerenciados da região, acessados por HTTPS.

Homologação (à direita): o mesmo desenho em uma zona só, com banco Single-AZ e um NAT. É mais barato e suficiente para testar antes de publicar.

Como isso garante os 99,9% (RNF03, até 43 minutos fora do ar por mês): não há ponto único de falha na produção. São duas tarefas em zonas diferentes, banco com failover automático e NAT por zona. São as mesmas peças custeadas no Bloco 1.

8. Atividade 1 · Abertura de conta

O diagrama está em 3 partes ligadas por **conectores** (círculo com letra, a notação UML para continuar o fluxo em outro lugar): A (cadastro e validação do adulto), B (ramo do menor, conector **M**) e C (abertura da conta, fim comum, que recebe o adulto pelo conector **P** e o menor pelo conector **V**).

Diagrama de Atividades 1, parte A · Abertura de conta: cadastro e validação do adulto

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

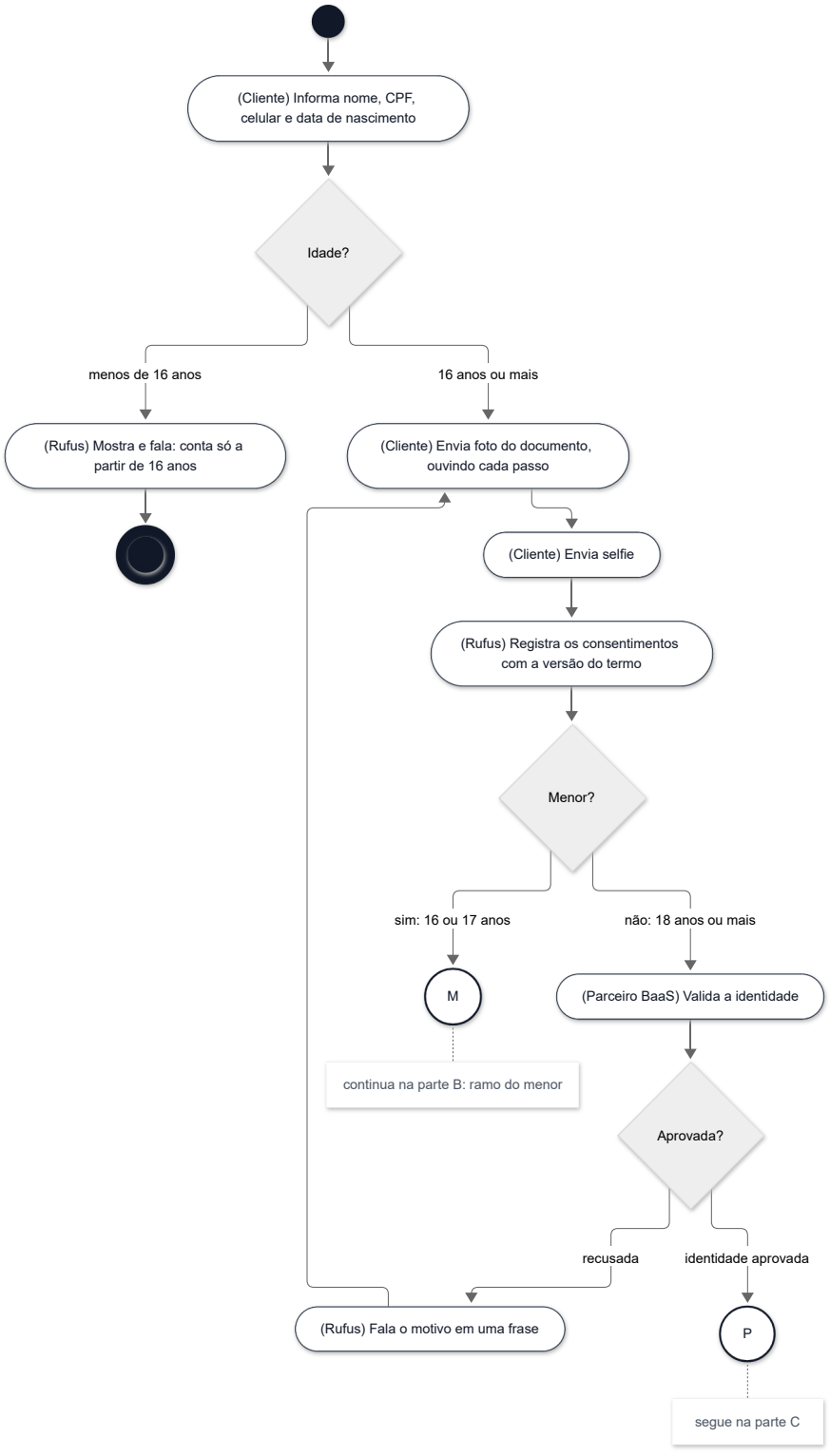


Diagrama de Atividades 1, parte B · Abertura de conta: ramo do menor com o responsável

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

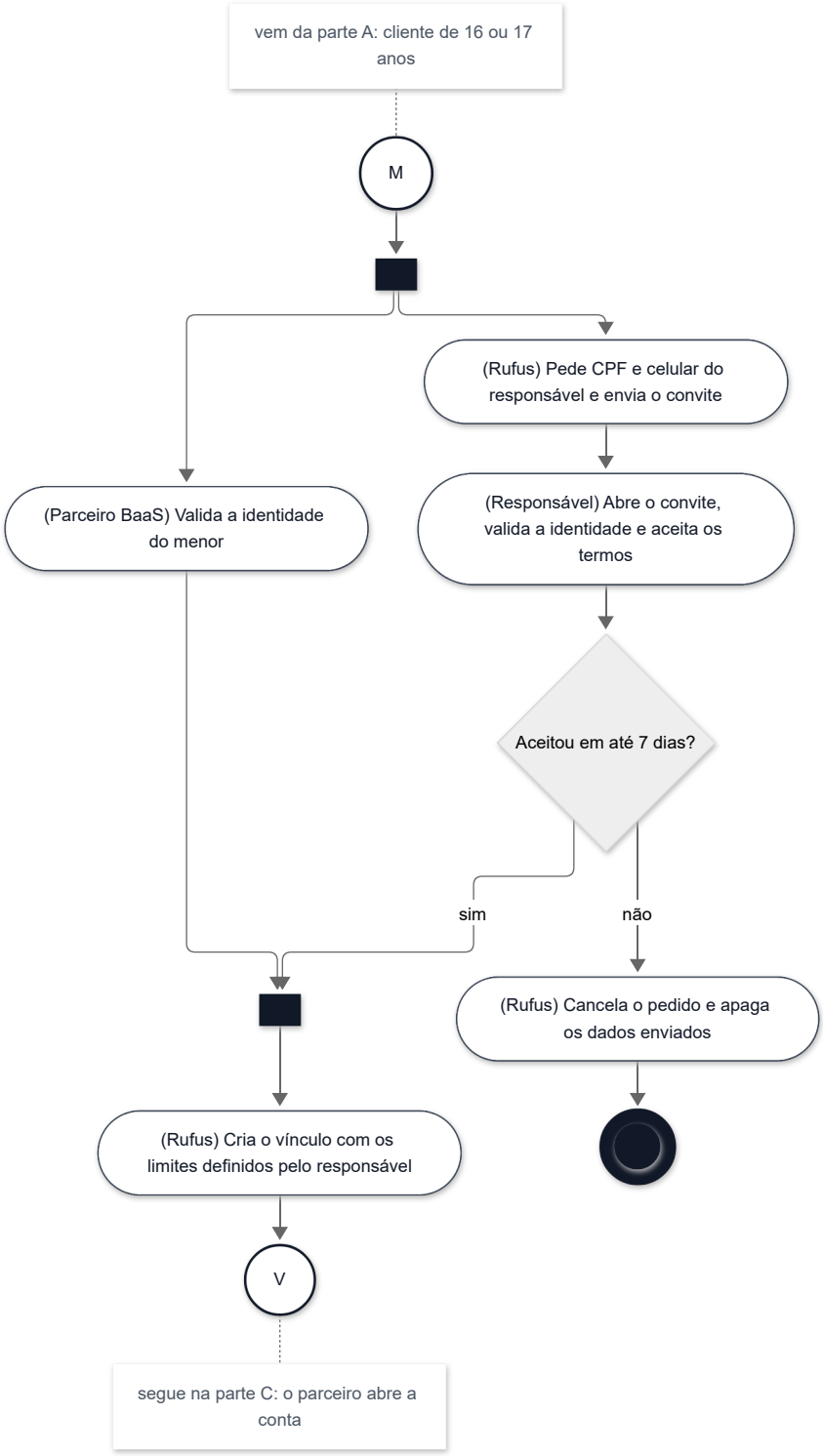
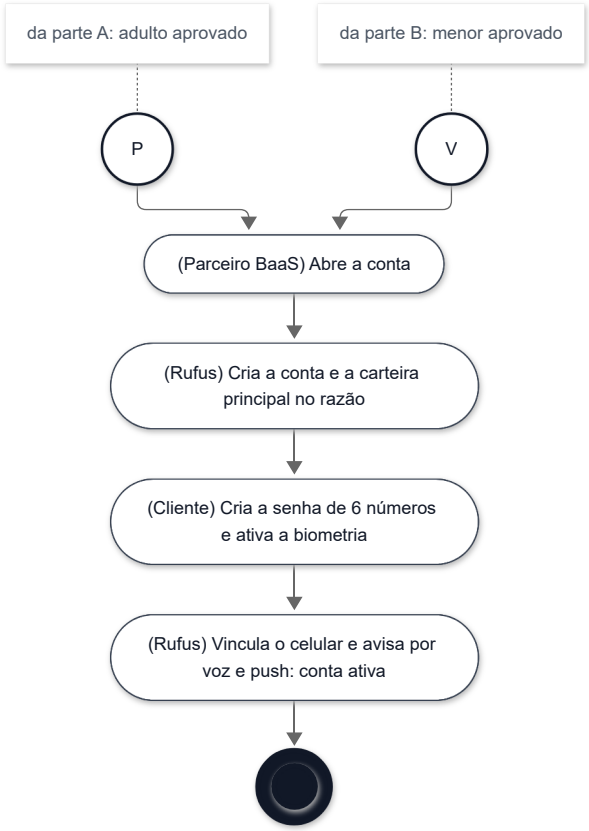


Diagrama de Atividades 1, parte C · Abertura de conta: abertura da conta, fim comum a adulto e menor

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. O processo de negócio principal: como uma pessoa vira cliente, incluindo o caso do menor.

Notação. Círculo cheio é o início. Círculo com borda é o fim. Losango é decisão, com cada saída rotulada. A barra preta é **fork** (divide em atividades paralelas) ou **join** (espera todas terminarem). Entre parênteses no começo de cada ação está a partição: quem executa (Cliente, Rufus, Parceiro BaaS ou Responsável).

Como ler. - Menos de 16 anos encerra na hora. Ninguém abaixo dessa idade é cliente. - Adulto: documento, selfie e consentimentos, depois a validação no parceiro. Se a foto for recusada, o Rufus fala o motivo e o cliente volta a enviar (o laço à esquerda, US01). - Menor de 16 ou 17 anos: depois dos consentimentos, um **fork** abre dois caminhos em paralelo: - o parceiro valida a identidade do menor; - o responsável recebe o convite, valida a própria identidade e aceita os termos. - O **join** só segue quando os dois terminam. Se o responsável não aceitar em 7 dias, o pedido é cancelado e os dados são apagados (US09, LGPD). - O fim é igual para todos: o parceiro abre a conta, o Rufus cria a carteira principal no razão, o cliente cria a senha e o celular é vinculado (RF003, RF005).

9. Atividade 2 · Análise antifraude de Pix retido

O diagrama está em 2 partes ligadas pelo conector **R**: A (retenção, análise e liberação) e B (recusa e bloqueio).

Diagrama de Atividades 2, parte A · Análise antifraude: retenção, análise e liberação

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

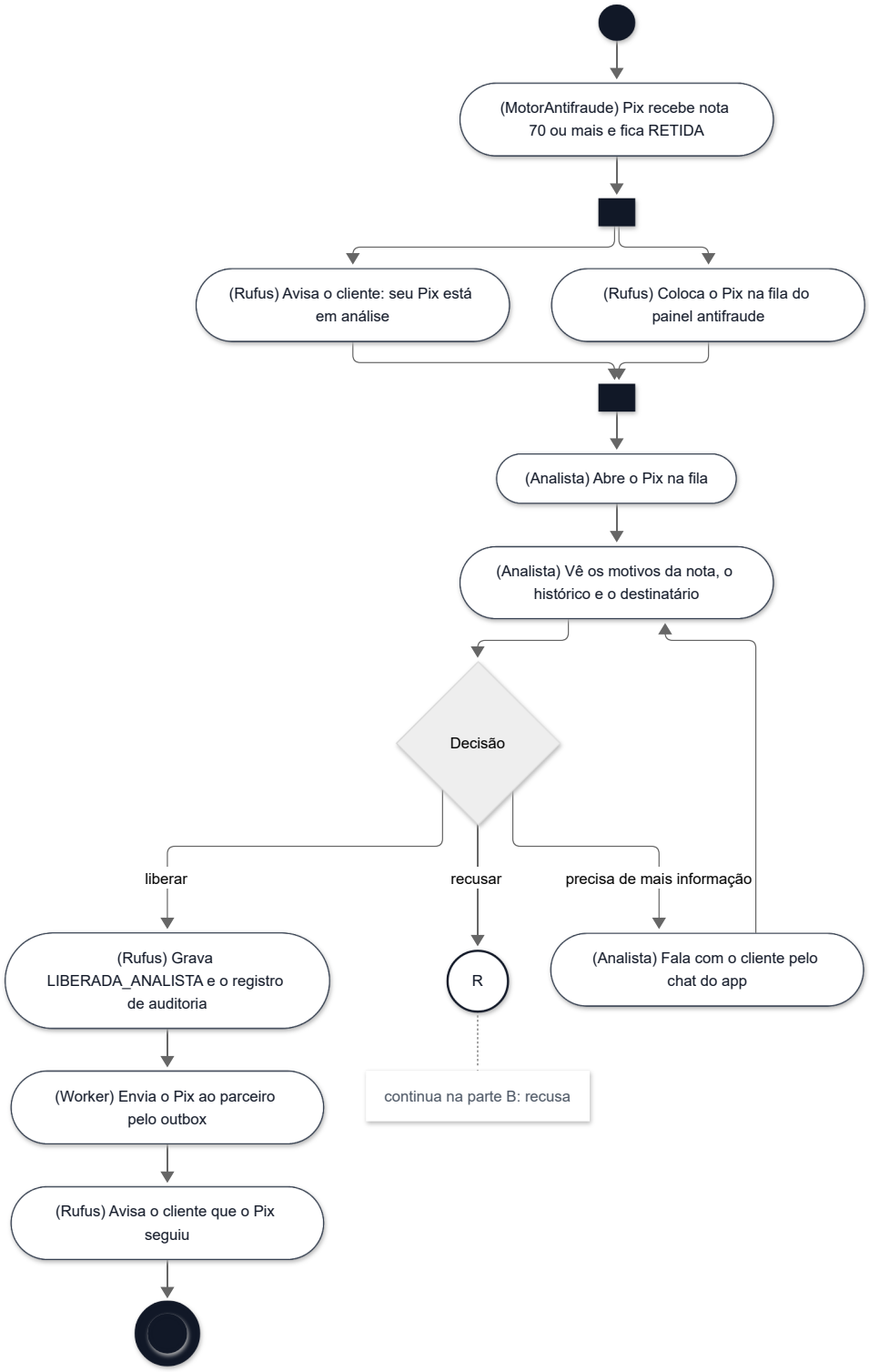
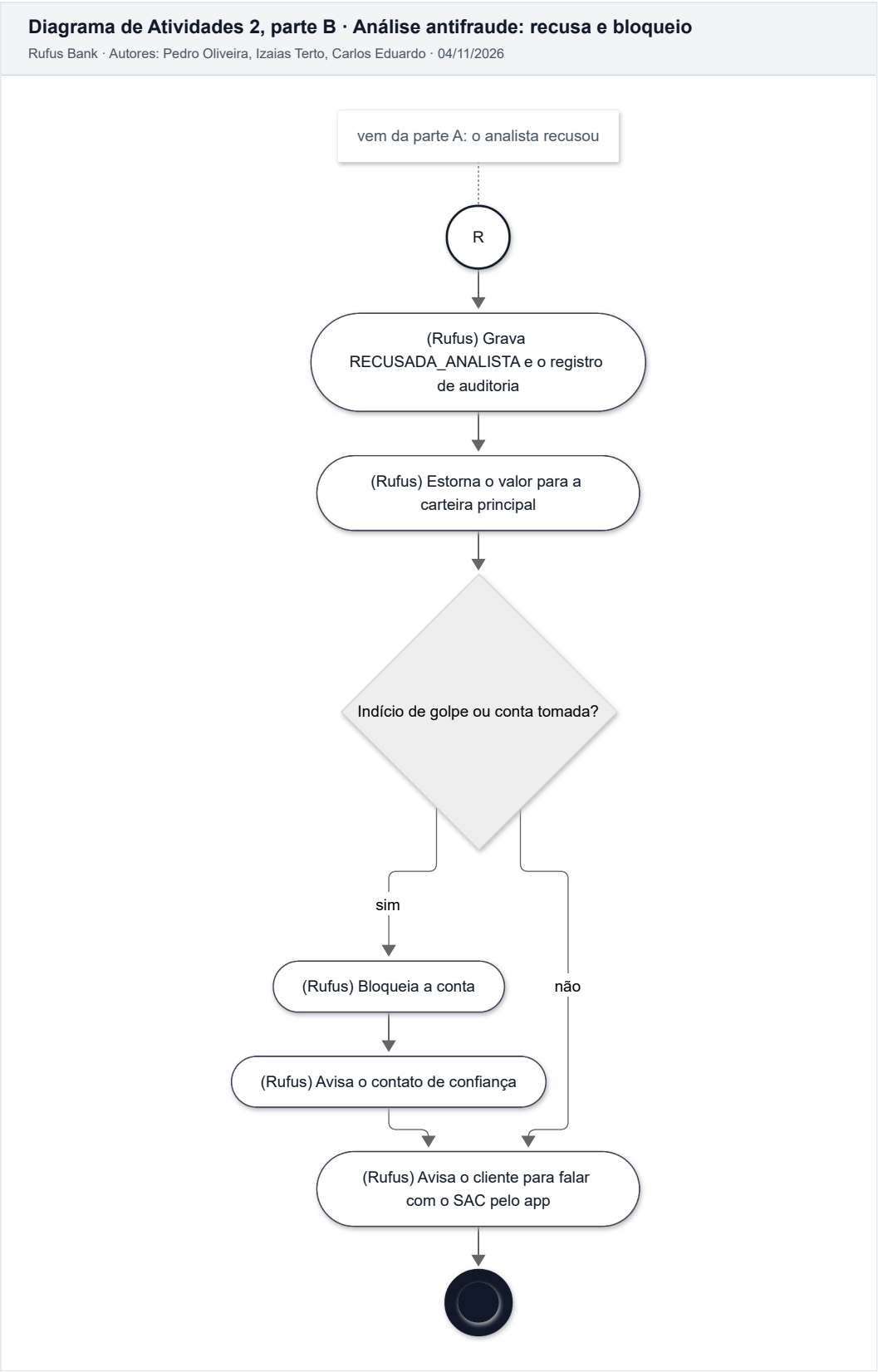


Diagrama de Atividades 2, parte B · Análise antifraude: recusa e bloqueio

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que mostra. O processo administrativo mais importante: o que acontece com um Pix suspeito até a decisão humana (RF026, RF027, US16).

Como ler. - Com nota 70 ou mais o Pix fica RETIDA. Um **fork** avisa o cliente e, ao mesmo tempo, coloca o Pix na fila do painel. O **join** junta os dois antes de o analista começar. - O analista vê os motivos da nota, o histórico e o destinatário. Se precisar de mais informação, fala com o cliente pelo chat do app e volta à análise (o laço à direita). - **Liberar**: a decisão fica gravada com auditoria (RNF14), o worker envia o Pix pelo outbox e o cliente é avisado. - **Recusar**: a decisão fica gravada com auditoria e o valor volta à carteira principal. Se houver indício de golpe ou de conta tomada, a conta é

bloqueada e o contato de confiança é avisado. Nos dois casos o cliente é orientado a falar com o SAC pelo app, nunca por telefone (RF021).

10. Processos de negócio em BPMN

Complemento aos 6 diagramas UML obrigatórios, com o conteúdo da Aula 02 da disciplina. Os diagramas de atividades (seções 8 e 9) mostram o fluxo **dentro** do sistema; o BPMN mostra o processo de negócio **entre os participantes**: cliente, Rufus, analista, responsável e parceiro BaaS, cada um no seu pool.

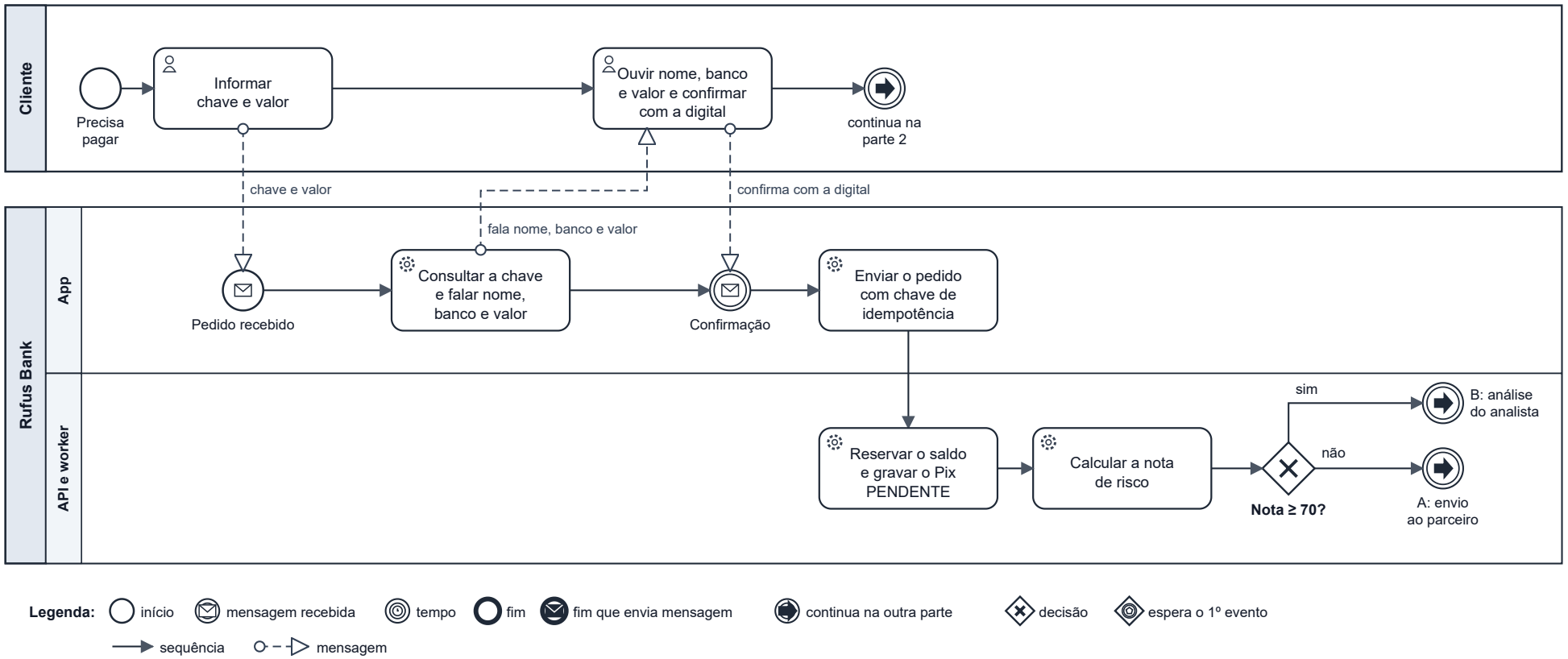
	BPM	**BPMN**
O que é	Disciplina de gestão: planejar, modelar, simular, executar, monitorar e otimizar processos	Notação gráfica padronizada (OMG) para desenhar o processo
Analogia da aula	A engenharia civil	A planta baixa
Onde aparece no Rufus	O ciclo vivo da operação: os sinais de alerta dos riscos (Bloco 7) e as métricas de negócio (Bloco 6) alimentam a otimização	Os 2 processos abaixo, na fase de **Modelagem** do ciclo do BPM

Notação usada. Pool (o participante) e raia (quem executa dentro dele); evento de início (círculo fino), intermediário (círculo duplo: mensagem ou tempo) e de fim (círculo grosso; com envelope cheio quando envia mensagem; com círculo preto quando encerra o processo todo); tarefa (retângulo arredondado, com ícone de pessoa ou de engrenagem para tarefa de usuário ou de sistema); gateway exclusivo (X), paralelo (+) e baseado em evento (pentágono); fluxo de sequência contínuo e fluxo de mensagem tracejado entre pools. Os 5 elementos mínimos da aula (início, tarefas, gateway, fim e fluxo de sequência) estão em todos. Cada processo está em 2 partes, ligadas por **eventos de link** (seta dentro do círculo: cheia onde o fluxo sai, vazada onde ele continua), o recurso do próprio BPMN para continuar o desenho em outra página.

Processo 1 · Envio de Pix com antifraude e estorno

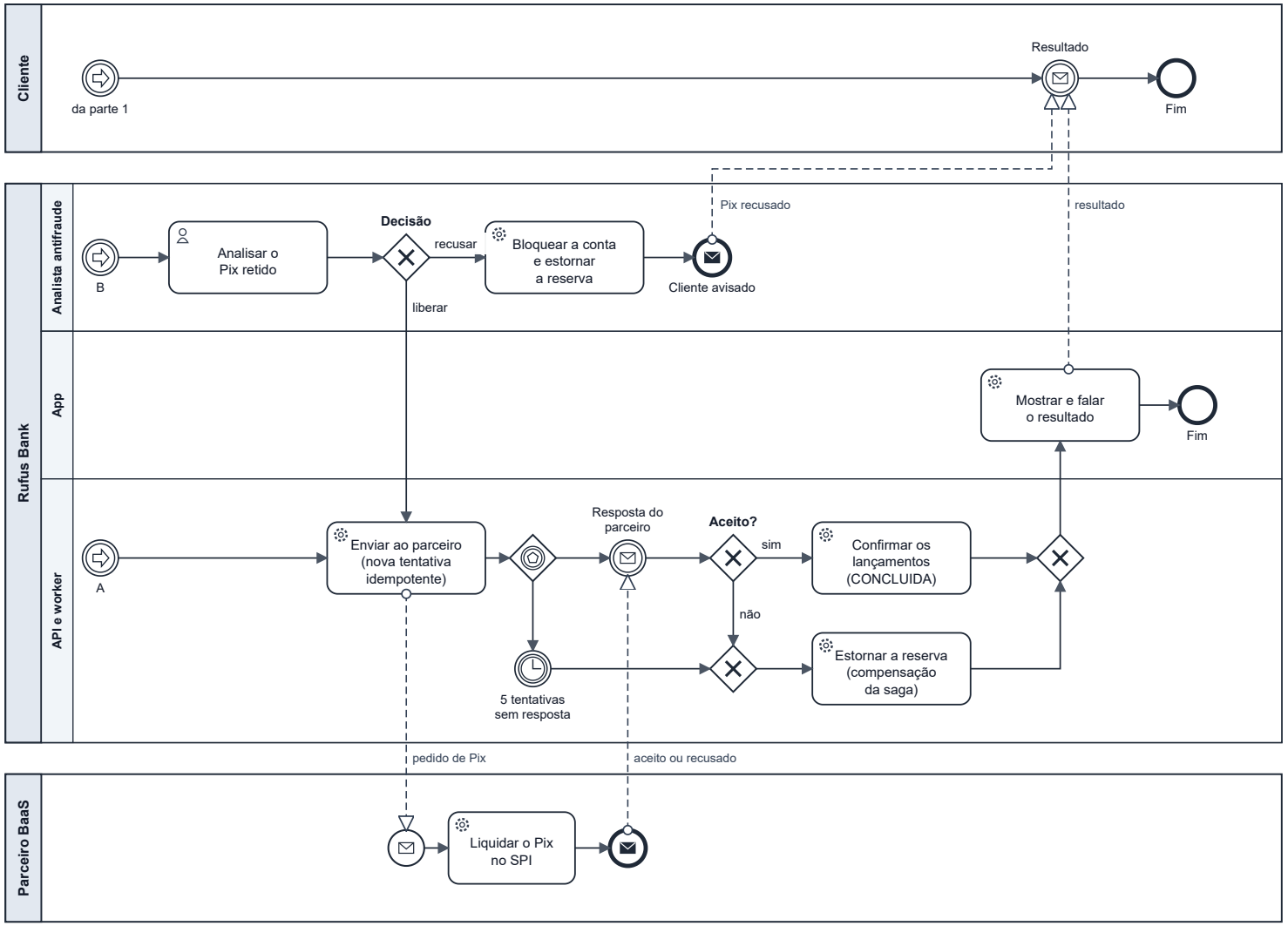
BPMN 2.0 · Envio de Pix, parte 1 de 2: do pedido à nota de risco

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



BPMN 2.0 · Envio de Pix, parte 2 de 2: análise, liquidação e estorno

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026

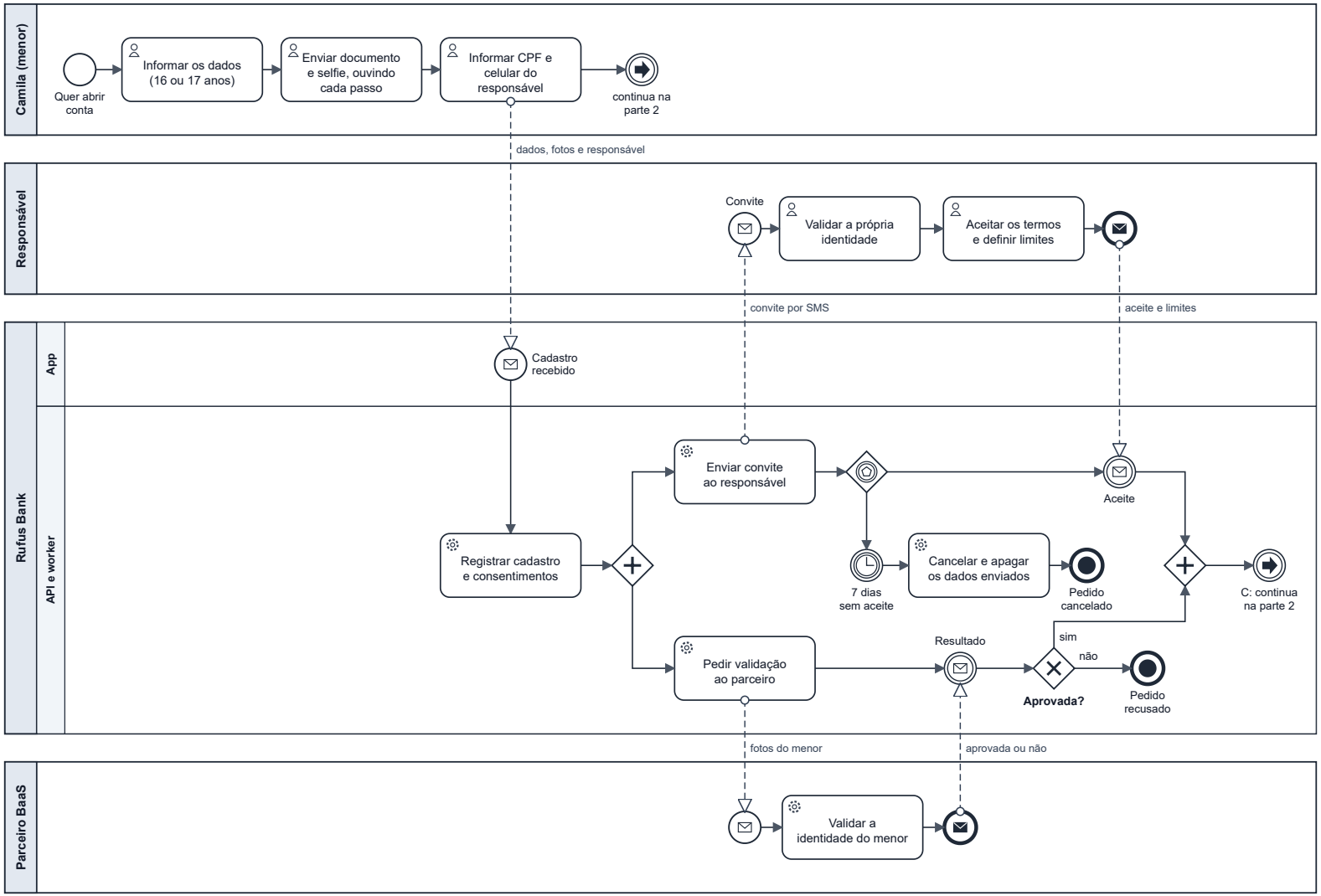


Como ler. - **Parte 1.** O cliente informa a chave e o valor. O app consulta a chave e fala nome, banco e valor (RF009); o cliente confirma com a digital. A API reserva o saldo, grava o Pix como PENDENTE e calcula a nota de risco. O gateway "Nota ≥ 70?" decide o caminho: **não** segue pelo link **A** (envio ao parceiro); **sim** segue pelo link **B** (análise humana, RF026). - **Parte 2, raia do analista** (link B). O analista analisa o Pix retido e decide. **Liberar** leva ao envio. **Recusar** bloqueia a conta e estorna a reserva; o fim com envelope avisa o cliente (RF027). - **Parte 2, envio** (link A). A API envia ao parceiro com nova tentativa idempotente. O **gateway baseado em evento** espera o que vier primeiro: a **resposta do parceiro** (mensagem) ou **5 tentativas sem resposta** (tempo). Aceito confirma os lançamentos (CONCLUIDA). Recusado ou sem resposta leva à **compensação da saga**: o estorno da reserva. É o mesmo comportamento da sequência de erro crítico (seção 5) e do ADR-008. - O resultado volta ao app, que mostra e fala o desfecho ao cliente.

Processo 2 · Abertura de conta de menor com aceite do responsável

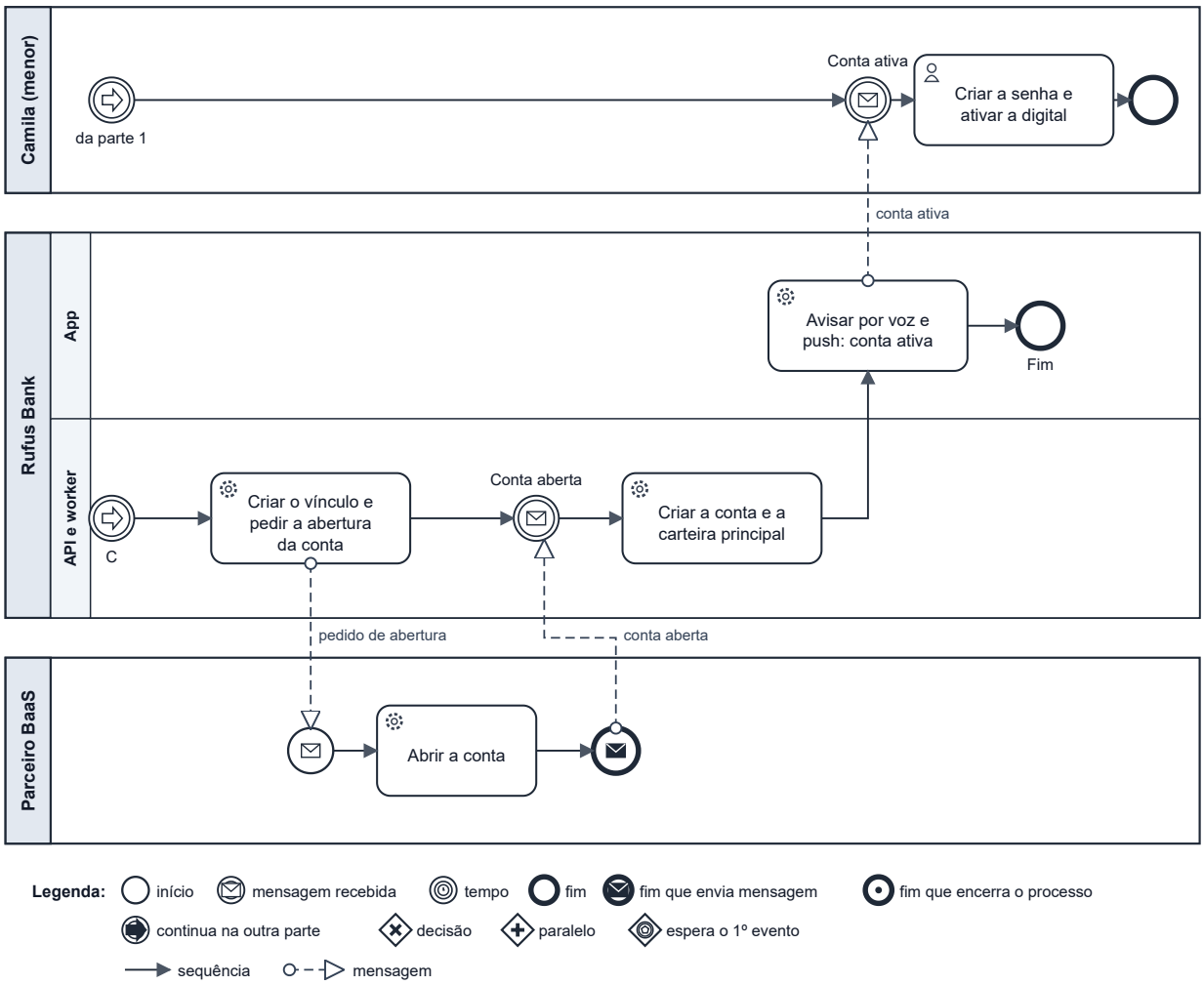
BPMN 2.0 · Conta de menor, parte 1 de 2: cadastro, aceite do responsável e validação

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



BPMN 2.0 · Conta de menor, parte 2 de 2: abertura da conta

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Como ler. - **Parte 1.** A Camila informa os dados, envia documento e selfie ouvindo cada passo e informa o CPF e o celular do responsável. A API registra o cadastro e os consentimentos e abre um **gateway paralelo** (+) com dois caminhos ao mesmo tempo: - **convite ao responsável:** o convite vai por SMS ao responsável, que valida a própria identidade, aceita os termos e define os limites. O gateway baseado em evento espera o **aceite** ou o **prazo de 7 dias** (evento de tempo, US09). Se o prazo vence, o pedido é cancelado e os dados enviados são apagados (LGPD), com um fim que **encerra o processo inteiro**; - **validação do menor:** o parceiro valida a identidade. Se for recusada, também encerra o processo. - O **gateway paralelo de junção** só segue quando os dois caminhos terminaram bem. Os fins que encerram o processo evitam que a junção fique esperando para sempre um caminho que já falhou, um erro comum de modelagem. - **Parte 2** (link C). A API cria o vínculo com os limites e pede a abertura da conta ao parceiro. Com a conta aberta, cria a conta e a carteira principal no razão, avisa por voz e push, e a Camila cria a senha e ativa a digital.

Bloco 5 · Banco de Dados

5.1 Decisão do banco de dados

Decisão: PostgreSQL 17 no Amazon RDS, com réplica síncrona em outra zona (Multi-AZ). O motivo central é que o Rufus movimenta dinheiro, e cada movimentação precisa gravar várias linhas de várias tabelas juntas ou nada (RNF11). Essa é a decisão do ADR-006, detalhada aqui.

O que os requisitos pedem do banco

Requisito	O que exige do banco
RNF11, zero Pix duplicado e razão sem divergência	Transação ACID envolvendo `transacao`, `lancamento` (2 ou mais linhas) e `outbox_evento` ao mesmo tempo. Restrição única para a chave de idempotência
ADR-007, partidas dobradas	Gatilho que confere a soma dos lançamentos no fim da transação (`DEFERRABLE`)
ADR-008, outbox	Leitura concorrente da fila com `SELECT ... FOR UPDATE SKIP LOCKED`
RF005, 1 celular ativo	Índice único parcial (`WHERE ativo`)
RNF03, 99,9%	Réplica síncrona e failover automático
RNF12, RPO de 5 min e RTO de 1 h	Restauração a qualquer ponto no tempo
RNF07, LGPD	Chaves estrangeiras para apagar ou anonimizar em cascata sem deixar órfão. Comentário por coluna para marcar dado pessoal
RNF14, auditoria por 5 anos	Tabela só de inserção, protegida por gatilho

Comparação

Critério	**PostgreSQL 17 (RDS Multi-AZ) escolhido**	MongoDB (Atlas ou DocumentDB)	Aurora PostgreSQL	DynamoDB
Transação entre várias linhas e tabelas	Nativa, ACID completa	Existe desde a versão 4.0, mas com limites de tempo e custo maior; o modelo incentiva um documento por operação	Nativa (é PostgreSQL)	Até 100 itens por transação, sem junção
Integridade referencial (chave estrangeira)	Sim	Não existe. A aplicação garante sozinha	Sim	Não existe
CHECK, gatilho adiado, índice único parcial	Sim	Validação de esquema limitada, sem gatilho adiado	Sim	Não
Consultas do antifraude e da conciliação (somas, junções, janelas por período)	SQL completo	Pipeline de agregação, sem junção eficiente entre coleções	SQL completo	Exige modelar cada consulta antes, sem consulta livre
Failover entre zonas	Multi-AZ, automático	Sim (conjunto de réplicas)	Sim, mais rápido	Sim, gerenciado

Critério	**PostgreSQL 17 (RDS Multi-AZ) escolhido**	MongoDB (Atlas ou DocumentDB)	Aurora PostgreSQL	DynamoDB
Custo no volume do MVP (10.000 usuários)	US$ 284,40/mês em produção (Bloco 1, preço oficial, com snapshots mensais)	Não calculado: descartado pelos critérios acima	Na configuração padrão cobra também por operação de entrada e saída, além da instância e do armazenamento; o ganho (failover mais rápido, réplicas de leitura) não é necessário para 100 req/s	Barato em volume baixo, mas os critérios de integridade já descartam
Encaixe na stack (ADR-004, Kysely)	Direto	Outro driver e outro modelo mental	Direto	SDK próprio
Resultado	**Atende a todos**	Descartado: sem chave estrangeira e sem gatilho, a integridade do dinheiro ficaria só no código	Descartado por agora: mesmas vantagens a um custo maior; vira opção se a carga crescer muito	Descartado: sem junção e sem integridade referencial

Por que não MongoDB (pergunta provável da arguição). O dado do Rufus é muito relacionado: cliente, conta, carteira, transação, lançamento, análise de risco, vínculo com responsável. O valor de um documento flexível não aparece aqui, e o que perderíamos (chave estrangeira, CHECK, gatilho que recusa lançamento desequilibrado) é justamente o que protege o dinheiro. No PostgreSQL, um lançamento desequilibrado é **recusado pelo banco**, como mostra o teste em testes/resultado.txt.

Normalização

O modelo está na **terceira forma normal (3FN)**:

- **1FN.** Todo campo é atômico. A única lista é `analise_risco.motivos` (`text[]`), que é um atributo descritivo da análise, nunca filtrado nem ligado a outra tabela. Ficar como array evita uma tabela só para códigos de motivo.
- **2FN.** Toda tabela com chave composta (`saldo_fechamento_diario`) tem os outros campos dependentes da chave inteira.
- **3FN.** Nenhum campo depende de outro campo que não seja a chave. Exemplo: o nome do titular não se repete na conta; a conta aponta para o cliente.

Por que 3FN e não menos: cada fato fica em um lugar só, então uma correção (o celular do cliente, por exemplo) não precisa ser feita em várias tabelas, e não existe o risco de duas cópias discordarem. Por que não mais (4FN, 5FN): não há dependências multivaloradas no domínio que justifiquem quebrar mais as tabelas.

Onde desnormalizamos de propósito

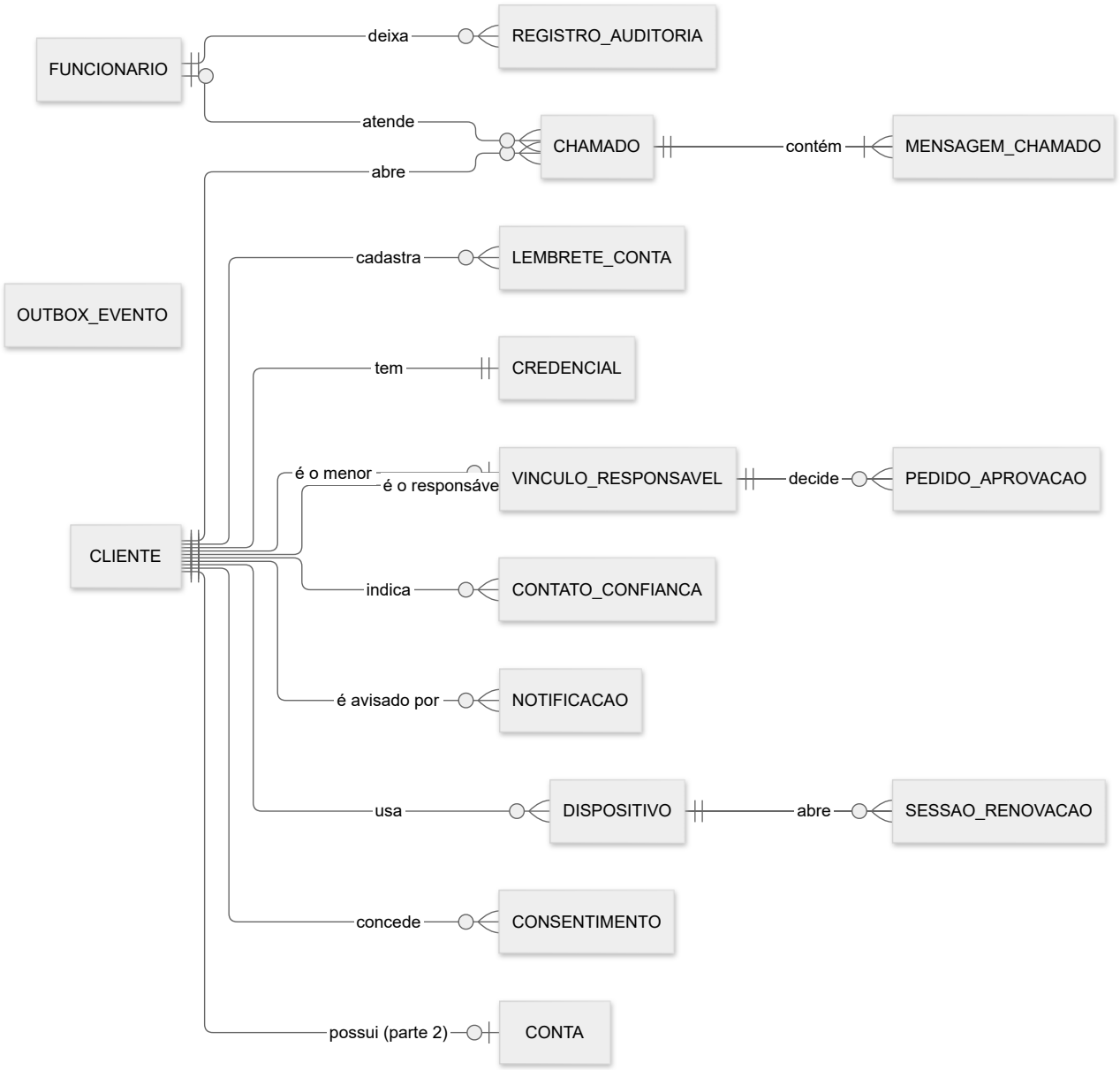
Onde	O que repete	Por quê
`saldo_fechamento_diario`	O saldo, que poderia ser sempre calculado somando os lançamentos	Com anos de lançamentos, somar tudo a cada consulta ameaçaria o p95 de 300 ms (RNF01). Saldo atual = último fechamento + lançamentos depois dele. O fechamento é recalculável a qualquer momento a partir do razão, então não há risco de perda
`pix_envio.nome_recebedor`, `documento_recebedor_mascarado`, `banco_recebedor`	Dados que pertencem ao recebedor, que não é cliente do Rufus	É o retrato do que foi mostrado e falado ao cliente no momento da confirmação (RF009). Se o recebedor mudar de nome depois, o comprovante continua dizendo a verdade daquele momento
`transacao.tipo` repetido em cada subtipo	O tipo, que já está na transação base	Permite a chave estrangeira composta `(transacao_id, tipo)`, que impede o banco de aceitar um `pix_envio` apontando para uma transação de boleto. Testado em testes/resultado.txt

5.2 Modelo conceitual (ER)

Todas as entidades do domínio e como se relacionam, sem atributos. Os nomes vêm do modelo de domínio.

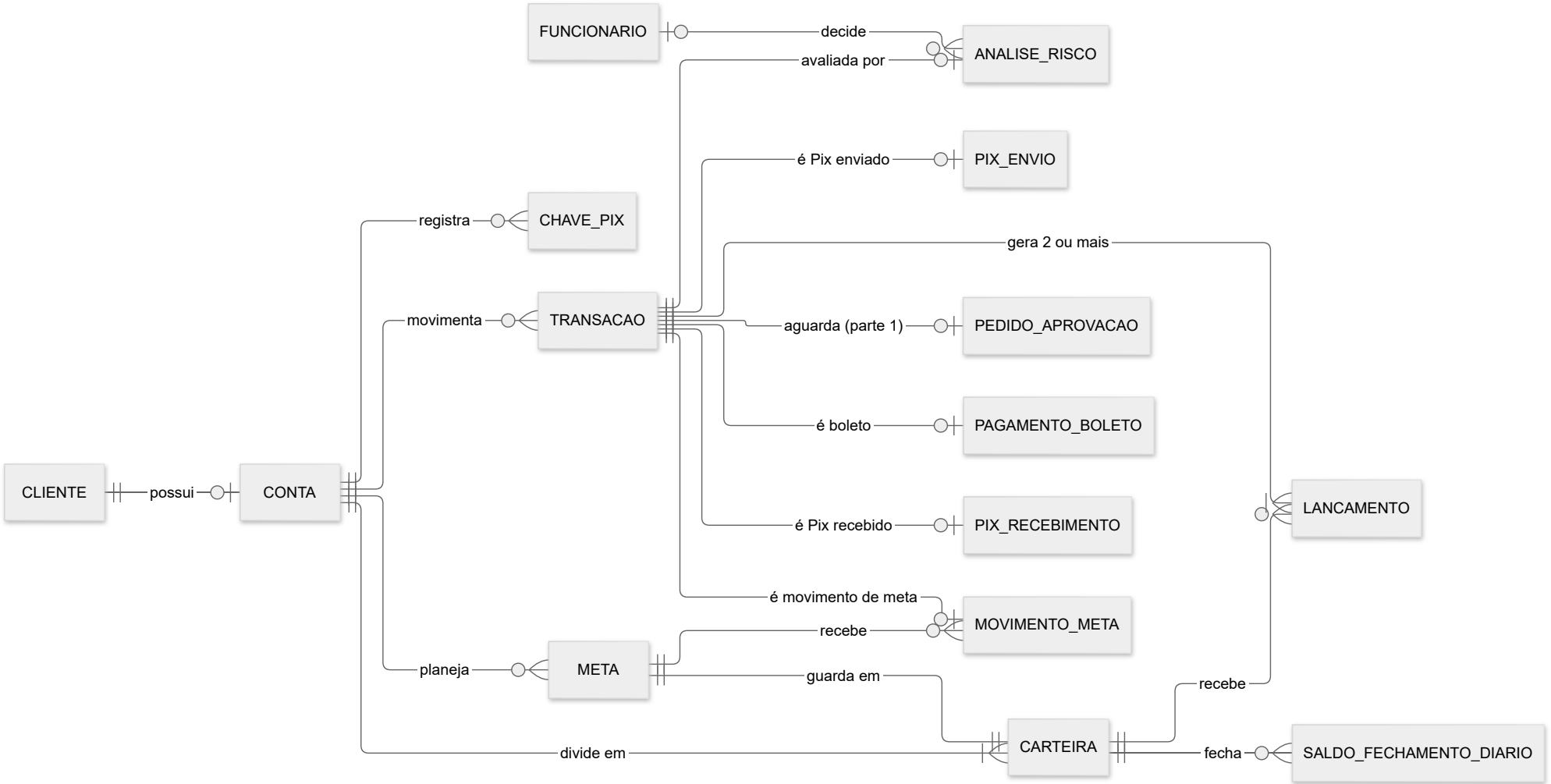
Modelo conceitual 1 de 2: pessoas, acesso, família e atendimento

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Modelo conceitual 2 de 2: conta, razão contábil e pagamentos

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Para caber legível, o modelo está em duas partes. Cliente, Conta, Transação, Funcionário e Pedido de aprovação aparecem nas duas, para mostrar a ligação.

Fontes: diagramas/conceitual-1-pessoas-e-atendimento.mmd e diagramas/conceitual-2-dinheiro.mmd. Notação pé de galinha: `||` exatamente um, `o|` zero ou um, `|{` um ou muitos, `o{` zero ou muitos.

Como ler o diagrama

O diagrama tem três centros:

1. **CLIENTE**, à esquerda: tudo que é da pessoa. Ela tem uma credencial, usa dispositivos (só um ativo), concede consentimentos, indica contatos de confiança, cadastra lembretes e possui no máximo uma conta. O vínculo com responsável aparece duas vezes porque o mesmo cliente pode ser o **menor** (zero ou um vínculo) ou o **responsável** (zero ou muitos, um pai com vários filhos).
2. **CONTA e TRANSACAO**, no meio: a conta se divide em uma ou mais carteiras e movimenta transações. Cada transação **gera 2 ou mais lançamentos** (partidas dobradas, ADR-007) e "pode ser" um dos quatro subtipos (Pix enviado, Pix recebido, boleto, movimento de meta).
3. **FUNCIONARIO**, embaixo à esquerda: decide análises de risco, atende chamados e deixa registros de auditoria. Nas duas primeiras a participação do funcionário é opcional (`|o`), porque um Pix pode ser liberado automaticamente e um chamado pode estar aberto sem atendente.

Cardinalidades que merecem explicação

Relação	Cardinalidade	Por quê
CLIENTE possui CONTA	1 para 0..1	O cliente existe antes da conta (cadastro em análise, US01). Depois de aprovado, uma conta por cliente no MVP
CONTA divide em CARTEIRA	1 para 1..N	Toda conta nasce com a carteira principal; cada meta acrescenta uma
META guarda em CARTEIRA	1 para 1	A carteira da meta nasce com a meta (composição)
TRANSACAO gera LANCAMENTO	1 para 2..N	O mínimo de 2 é garantido pelo gatilho do banco
TRANSACAO pode ser PIX_ENVIO	1 para 0..1	Generalização: cada transação tem exatamente um subtipo, indicado por `tipo`
TRANSACAO avaliada por ANALISE_RISCO	1 para 0..1	Só o Pix enviado passa pelo antifraude
TRANSACAO aguarda PEDIDO_APROVACAO	1 para 0..1	Só a transação do menor acima do limite gera pedido
CHAMADO contém MENSAGEM_CHAMADO	1 para 1..N	O chamado abre com a primeira mensagem

`OUTBOX_EVENTO` aparece solta de propósito: ela guarda eventos de qualquer agregado (`agregado_id`), sem chave estrangeira, porque o evento pode vir de transação, conta, vínculo ou chamado (ADR-008).

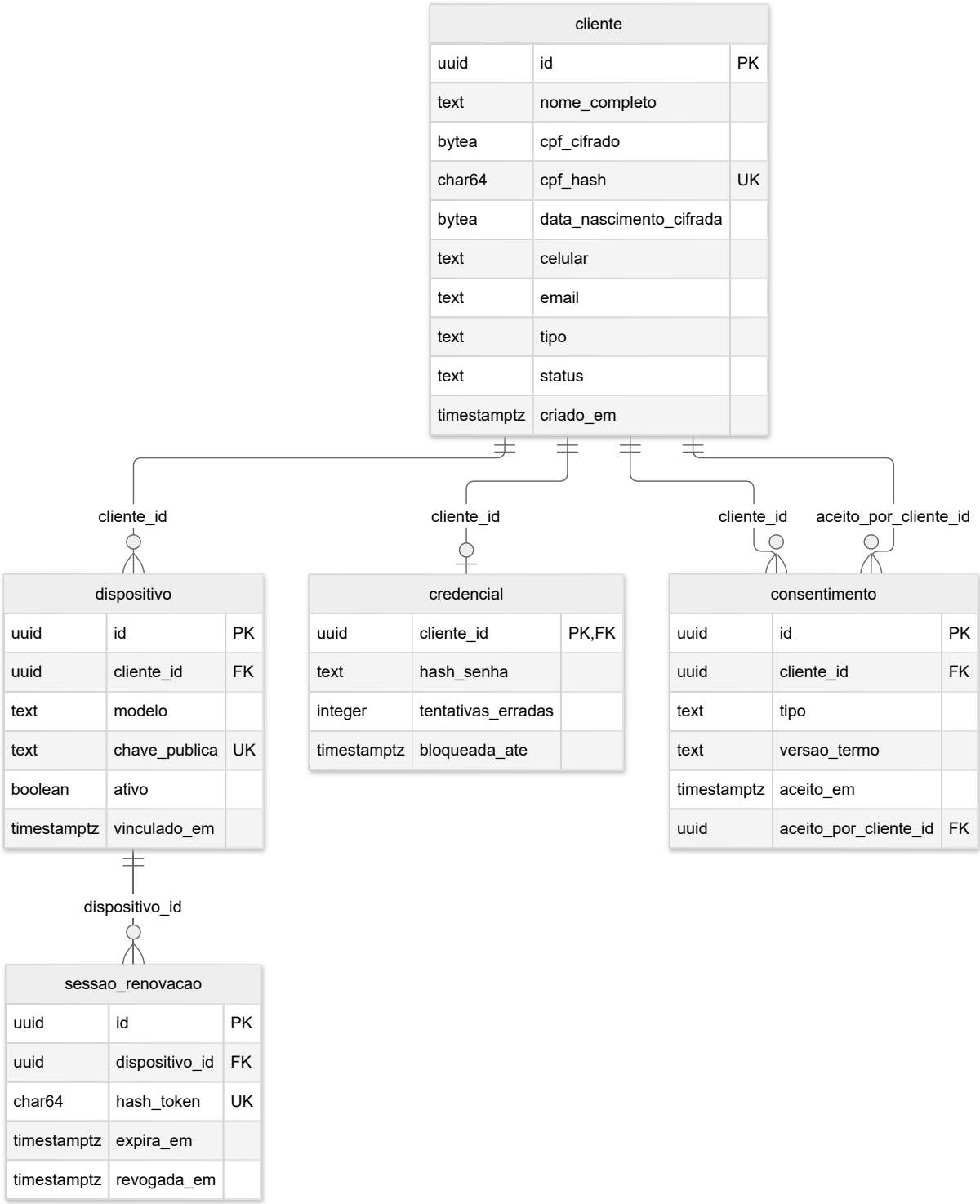
5.3 Modelo lógico

27 tabelas no esquema `rufus`, com atributos, tipos, chaves primárias (PK), chaves estrangeiras (FK), chaves únicas (UK) e índices. Para caber legível, o modelo está em seis diagramas por área. Os seis são **gerados do catálogo do banco real** pelo mesmo script do dicionário de dados, então o desenho não diverge do SQL.

Diagramas

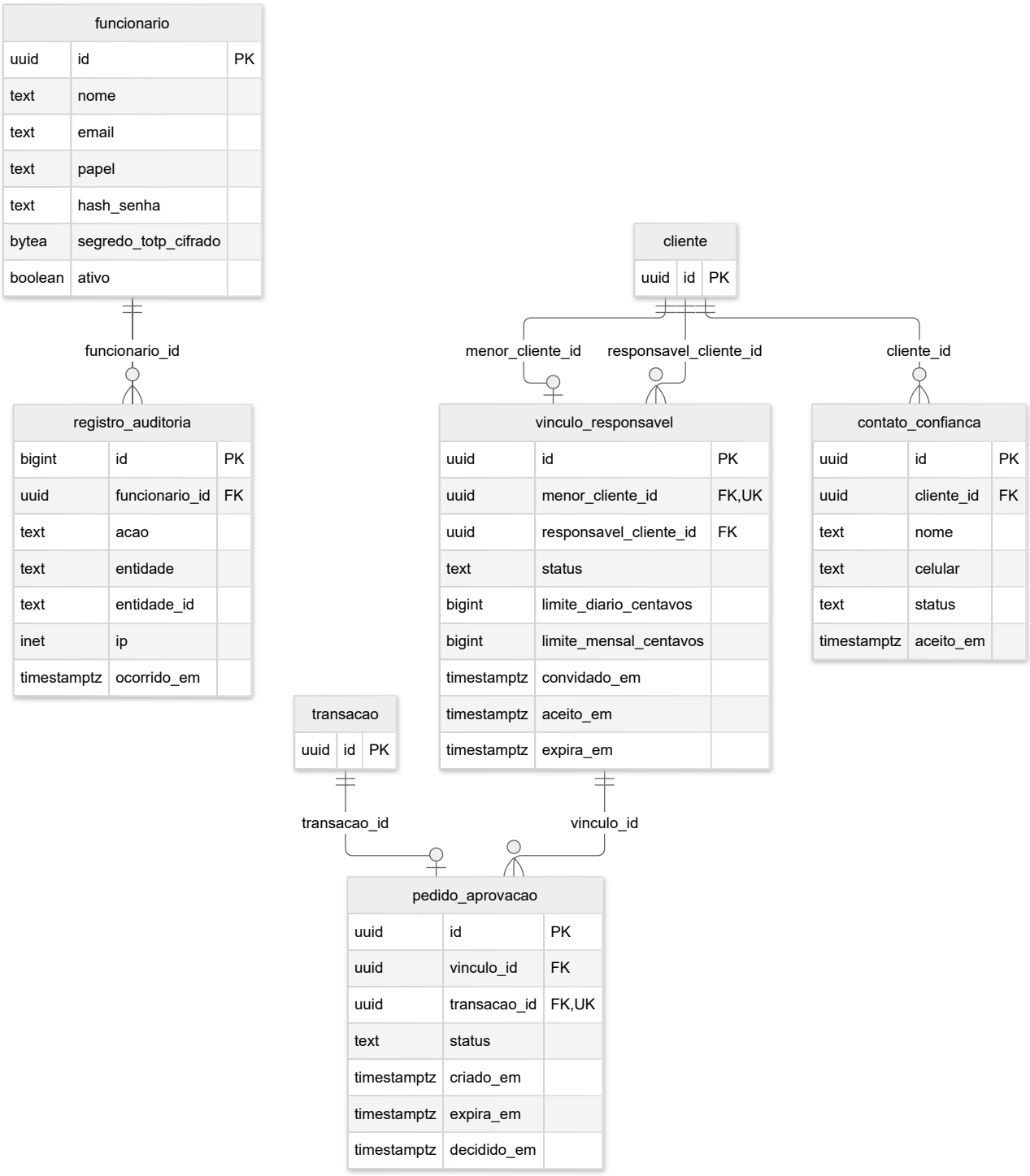
Modelo lógico 1 de 6: identidade

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



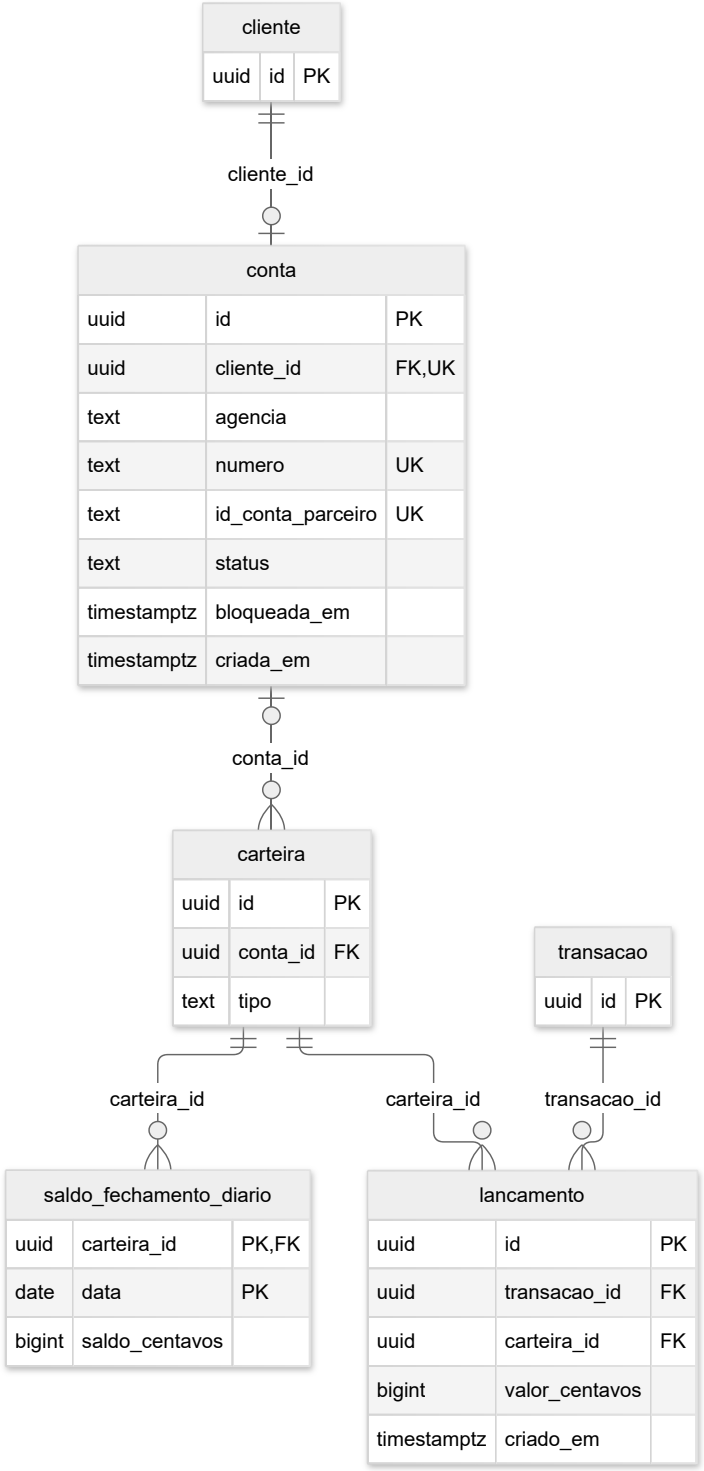
Modelo lógico 2 de 6: família e backoffice

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



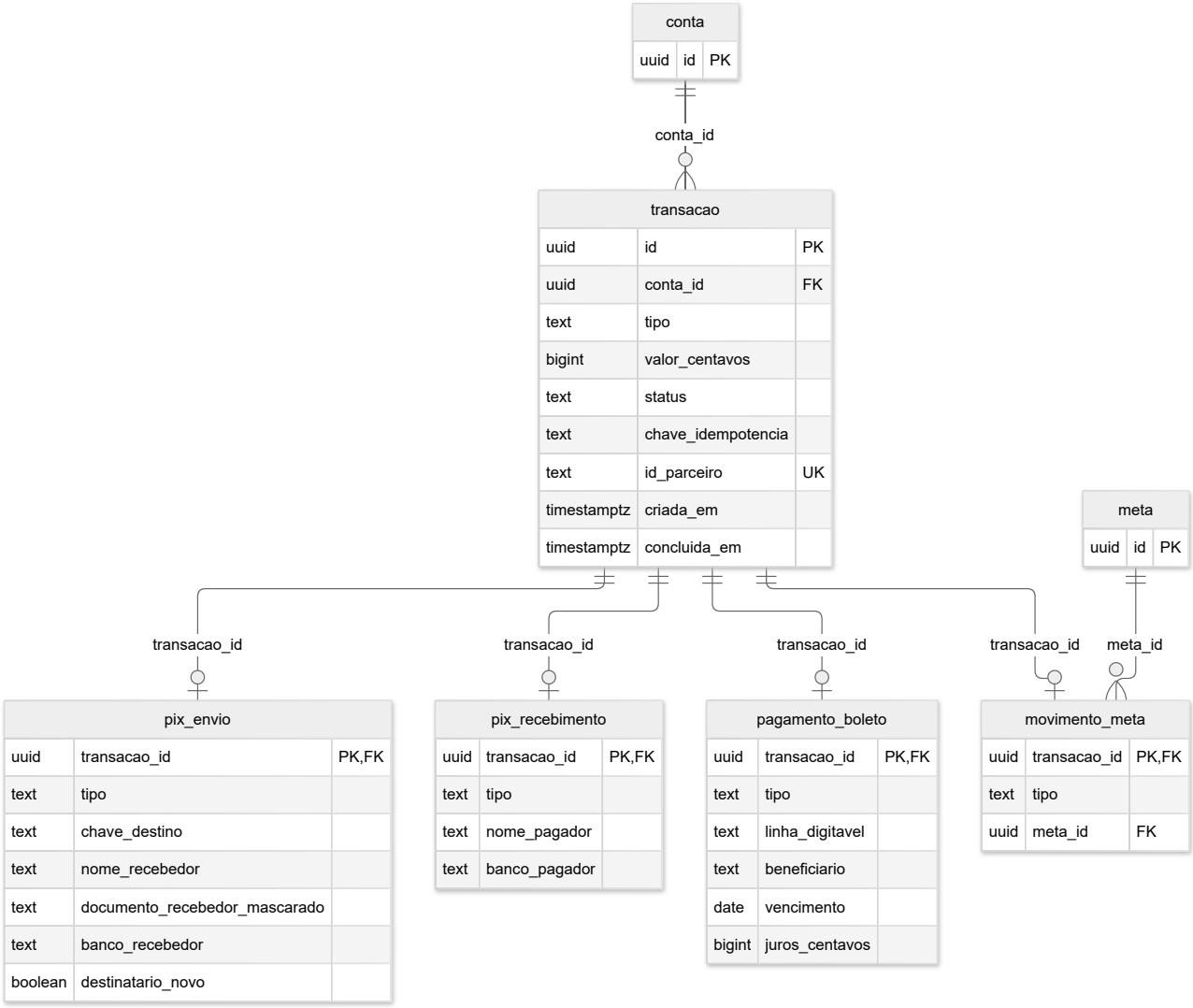
Modelo lógico 3 de 6: contas e razão contábil

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



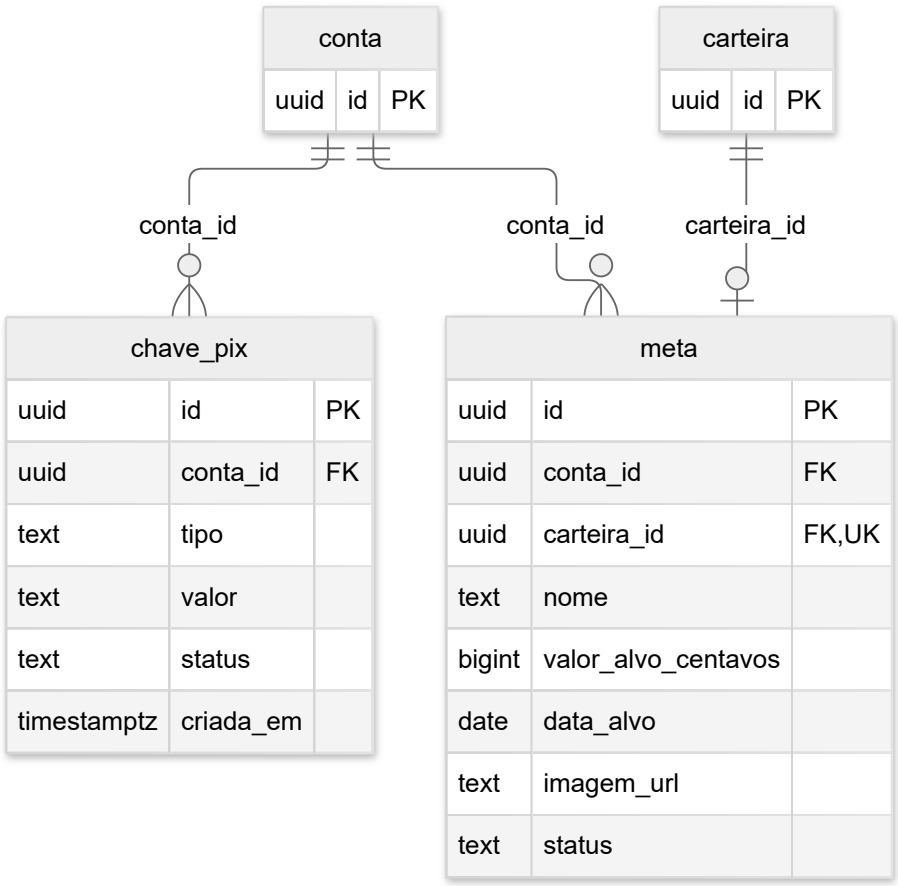
Modelo lógico 4 de 6: transações e subtipos

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



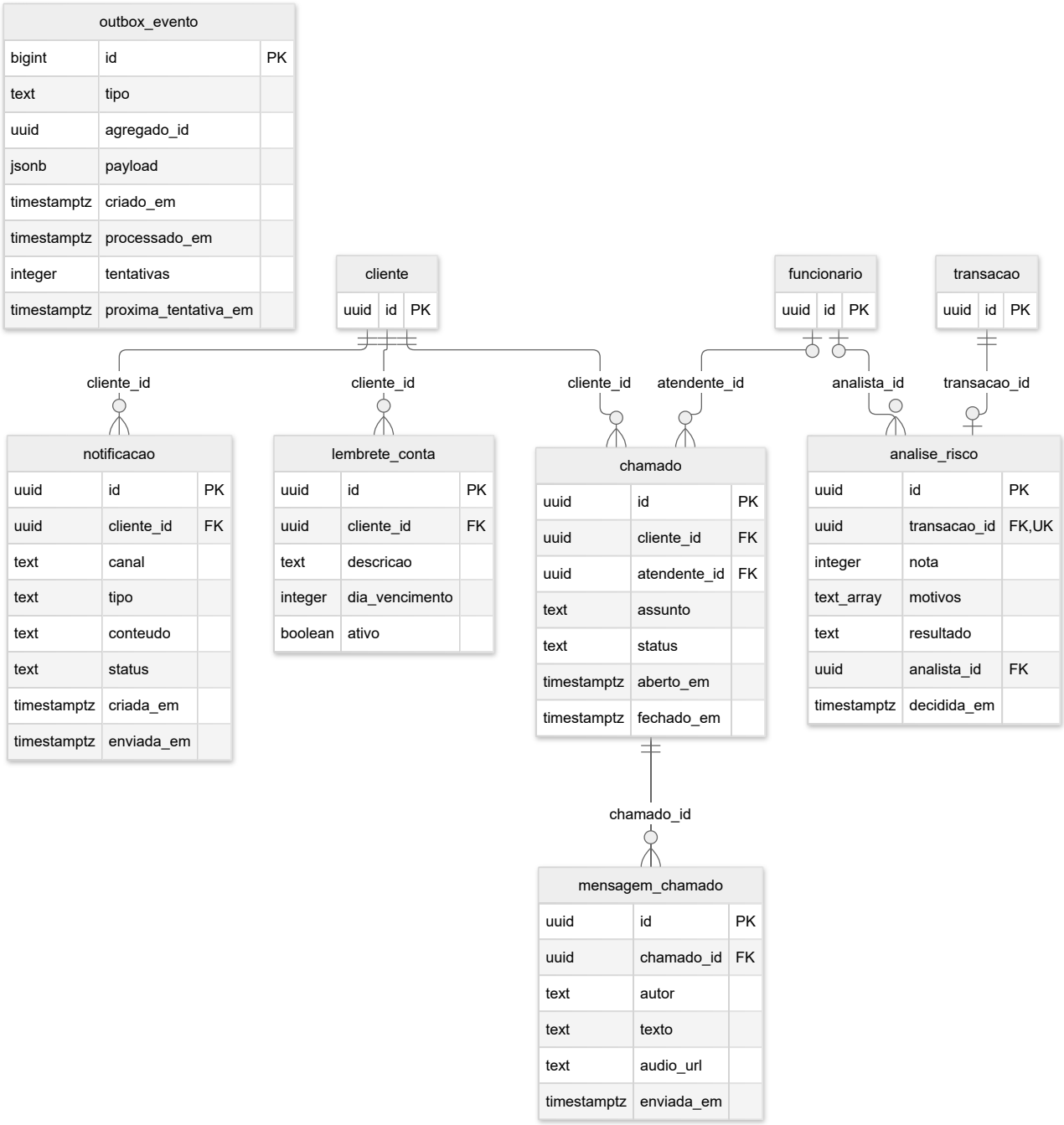
Modelo lógico 5 de 6: chaves Pix e metas

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Modelo lógico 6 de 6: antifraude, atendimento e notificação

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Tabelas de outra área aparecem só com a chave (`id`), para mostrar a ligação. O tipo `char64` no desenho é `char(64)` no banco (o Mermaid não aceita parênteses).

Tabelas por módulo

Módulo	Tabelas
identidade	cliente, dispositivo, credencial, sessao_renovacao, consentimento
contas	conta, carteira, lancamento, saldo_fechamento_diario
pagamentos	transacao, pix_envio, pix_recebimento, pagamento_boleto, movimento_meta, chave_pix, meta
familia	vinculo_responsavel, pedido_aprovacao, contato_confianca

Módulo	Tabelas
antifraude	analise_risco
atendimento	chamado, mensagem_chamado
backoffice	funcionario, registro_auditoria
notificacao	notificacao, outbox_evento, lembrete_conta

Decisões de tipo

Decisão	Por quê
Dinheiro em `bigint` de centavos	Sem erro de arredondamento de ponto flutuante. `bigint` vai até 92 quatrilhões de centavos. Bate com o ADR-004 (dinheiro como inteiro no TypeScript)
Chave `uuid` com `gen_random_uuid()`	O app e o worker podem gerar a chave antes de gravar, e o identificador não revela quantos clientes existem
`bigint GENERATED ALWAYS AS IDENTITY` em `registro_auditoria` e `outbox_evento`	Tabelas só de inserção com volume alto, onde a ordem de chegada importa. Mais compacto que `uuid`
`timestamptz` em todas as datas com hora	Grava em UTC e converte para o horário de Brasília na exibição. Evita erro no horário do Pix noturno (RF019)
`text` com `CHECK ... IN (...)` para os estados	Mais simples de evoluir que `ENUM` do PostgreSQL: incluir um estado novo é trocar a restrição, sem bloquear a tabela inteira
`bytea` nos campos cifrados	Guarda o texto cifrado pela aplicação (AES-256-GCM), ver estratégia
`char(64)` em `cpf_hash` e `hash_token`	Tamanho fixo do SHA-256 em hexadecimal, conferido por `CHECK`

Índices justificados

Além das PKs e UKs (que já criam índice), cada índice abaixo existe para uma consulta ou regra específica.

Índice	Tabela (colunas)	Consulta ou regra que atende
`ux_dispositivo_um_ativo_por_cliente`	dispositivo (cliente_id) WHERE ativo	**Regra RF005**: no máximo 1 celular ativo. Único e parcial: aparelhos antigos não contam
`ix_sessao_dispositivo_validas`	sessao_renovacao (dispositivo_id) WHERE revogada_em IS NULL	Revogar todas as sessões na troca de aparelho (US03) sem varrer as revogadas
`ux_funcionario_email`	funcionario (lower(email))	Login do painel (RF030) e e-mail único sem diferenciar maiúsculas
`ix_auditoria_entidade`	registro_auditoria (entidade, entidade_id, ocorrido_em DESC)	"Quem consultou este cliente?" em fiscalização ou pedido do titular (RNF14)
`ix_auditoria_funcionario`	registro_auditoria (funcionario_id, ocorrido_em DESC)	"O que este funcionário fez?" em investigação interna
`ux_carteira_principal_por_conta`	carteira (conta_id) WHERE tipo = 'PRINCIPAL'	**Regra**: uma carteira principal por conta
`ux_carteira_tecnica_unica`	carteira (tipo) WHERE conta_id IS NULL	**Regra**: uma carteira de liquidação de cada tipo no sistema
`ix_carteira_conta`	carteira (conta_id)	Listar as carteiras (principal e metas) da conta na tela inicial
`ux_transacao_idempotencia`	transacao (conta_id, chave_idempotencia), declarado como restrição UNIQUE	**Regra RNF11**: o mesmo pedido repetido não cria um segundo Pix. É a consulta feita a cada `POST /pix/envios`
`ix_transacao_extrato`	transacao (conta_id, criada_em DESC, id DESC)	Extrato paginado por cursor, mais recentes primeiro (`GET /contas/me/extrato` , **RNF01**)

Índice	Tabela (colunas)	Consulta ou regra que atende
`ix_transacao_pendentes`	transacao (status, criada_em) WHERE status IN ('PENDENTE','RETIDA','ENVIADA')	Worker que reprocessa Pix sem resposta do parceiro e conciliação. Parcial: as concluídas, que são quase todas, ficam fora
`ix_lancamento_carteira`	lancamento (carteira_id, criado_em)	**Saldo** (soma depois do último fechamento) e fechamento diário (**RNF01**)
`ix_lancamento_transacao`	lancamento (transacao_id)	O gatilho de partidas dobradas soma os lançamentos da transação a cada inserção
`ux_chave_pix_valor_ativa`	chave_pix (valor) WHERE status = 'ATIVA'	**Regra**: chave ativa única. Depois de excluída pode ser cadastrada de novo
`ix_chave_pix_conta`	chave_pix (conta_id)	Listar as chaves da conta (`GET /pix/chaves`)
`ix_meta_conta`	meta (conta_id) WHERE status = 'ATIVA'	Listar as metas ativas (`GET /metas`)
`ix_movimento_meta_meta`	movimento_meta (meta_id)	Histórico de uma meta
`ix_vinculo_responsavel`	vinculo_responsavel (responsavel_cliente_id) WHERE status = 'ATIVO'	O responsável vê os filhos vinculados e os limites (US10)
`ix_pedido_pendente`	pedido_aprovacao (vinculo_id, expira_em) WHERE status = 'PENDENTE'	Pedidos que o responsável precisa decidir e worker que expira os de mais de 24 horas (US18)
`ix_contato_confianca_cliente`	contato_confianca (cliente_id) WHERE status = 'ATIVO'	Quem avisar quando sai um Pix fora do padrão (US13)
`ix_analise_fila`	analise_risco (resultado, id) WHERE resultado = 'RETIDA'	**Fila do painel antifraude** (`GET /admin/analises` , RF027)
`ix_chamado_cliente`	chamado (cliente_id, aberto_em DESC)	Histórico de chamados do cliente para o atendente (US15)
`ix_chamado_fila`	chamado (status, aberto_em) WHERE status <> 'RESOLVIDO'	Fila do SAC (`GET /admin/chamados`)
`ix_mensagem_chamado`	mensagem_chamado (chamado_id, enviada_em)	Conversa do chamado em ordem
`ix_notificacao_cliente`	notificacao (cliente_id, criada_em DESC)	Últimas notificações do cliente
`ix_outbox_pendentes`	outbox_evento (proxima_tentativa_em) WHERE processado_em IS NULL	**Worker do outbox** (ADR-008): busca só os pendentes. Parcial: a tabela cresce, mas o índice fica pequeno
`ix_lembrete_dia`	lembrete_conta (dia_vencimento) WHERE ativo	Job diário que acha os lembretes de daqui a 3 dias (US23)

O que **não** indexamos de propósito: colunas de baixa seletividade sozinhas (como `status` ou `tipo` sem outra coluna) e campos que nunca aparecem em `WHERE` (como `nome_recebedor`). Cada índice custa espaço e deixa a escrita mais lenta, e o Rufus escreve muito no razão.

O detalhe de cada coluna (tipo, nulo, padrão, restrição, descrição e LGPD) está no dicionário de dados.

5.5 Dicionário de dados

> Arquivo **gerado** por `bloco-5-banco/ferramentas/gerar_dicionario.py` a partir do catálogo de um PostgreSQL 17.11 real com o `sql/001_schema.sql` aplicado. Não editar à mão: mude o SQL (inclusive os `COMMENT ON`) e rode o script de novo.

27 tabelas, 170 colunas, **26 colunas com marcação LGPD**.

Legenda da coluna LGPD: **PII** = dado pessoal (LGPD art. 5º, I) · **PII-SENSIVEL** = dado pessoal sensível (art. 5º, II) · **CIFRADO** = guardado cifrado pela aplicação · **SEGREDO** = credencial, nunca sai do banco nem vai para log · **-** = sem tratamento especial.

Campos com tratamento LGPD

Tabela	Campo	Marcação
chamado	assunto	PII
chave_pix	valor	PII
cliente	nome_completo	PII
cliente	cpf_cifrado	PII, CIFRADO
cliente	cpf_hash	PII
cliente	data_nascimento_cifrada	PII, CIFRADO
cliente	celular	PII
cliente	email	PII
conta	numero	PII
contato_confianca	nome	PII
contato_confianca	celular	PII
credencial	hash_senha	SEGREDO
funcionario	nome	PII
funcionario	email	PII
funcionario	hash_senha	SEGREDO
funcionario	segredo_totp_cifrado	SEGREDO, CIFRADO
mensagem_chamado	texto	PII
mensagem_chamado	audio_url	PII
notificacao	conteudo	PII
outbox_evento	payload	PII
pix_envio	chave_destino	PII
pix_envio	nome_recebedor	PII
pix_envio	documento_recebedor_mascarado	PII
pix_recebimento	nome_pagador	PII
registro_auditoria	ip	PII
sessao_renovacao	hash_token	SEGREDO

Módulo identidade

cliente

Pessoa física com conta no Rufus: titular adulto ou menor de 16 e 17 anos.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador interno do cliente.	-
`nome_completo`	text	não	-	(length(TRIM(BOTH FROM nome_completo)) >= 3)	Nome civil completo, como no documento.	PII
`cpf_cifrado`	bytea	não	-	-	CPF cifrado pela aplicação. Só é decifrado para envio ao parceiro BaaS.	PII, CIFRADO
`cpf_hash`	character(64)	não	-	cpf_hash ~ '^[0-9a-f]{64}$' UNIQUE	HMAC-SHA256 do CPF com chave secreta, em hexadecimal. Permite buscar e garantir CPF único sem decifrar.	PII
`data_nascimento_cifrada`	bytea	não	-	-	Data de nascimento cifrada. Usada para checar a idade do menor.	PII, CIFRADO
`celular`	text	não	-	celular ~ '^+55[0-9]{10,11}$'	Celular no formato +55DDNNNNNNNNN. Recebe SMS e convites.	PII
`email`	text	sim	-	(email IS NULL) OR (email ~ '^[^@\s]+@[^@\s]+.[^@\s]+$')	E-mail opcional.	PII
`tipo`	text	não	-	(tipo IN ('TITULAR', 'MENOR'))	TITULAR (adulto) ou MENOR (16 e 17 anos, exige vínculo com responsável).	-
`status`	text	não	'EM_ANALISE'::text	(status IN ('EM_ANALISE', 'ATIVO', 'BLOQUEADO', 'ENCERRADO'))	Situação do cadastro: EM_ANALISE, ATIVO, BLOQUEADO ou ENCERRADO.	-
`criado_em`	timestamp with time zone	não	now()	-	Momento do início do cadastro.	-

Índices:

- `CREATE UNIQUE INDEX cliente_cpf_hash_key ON cliente USING btree (cpf_hash)`

dispositivo

Celular vinculado à conta. Só um ativo por cliente (RF005).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do vínculo do aparelho.	-
`cliente_id`	uuid	não	-	FK → cliente UNIQUE parcial (ativo)	Cliente dono do aparelho.	-

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`modelo`	text	não	-	-	Modelo informado pelo aparelho, para o cliente reconhecer na lista.	-
`chave_publica`	text	não	-	UNIQUE	Chave pública do par gerado no Keystore ou Keychain. Confere a assinatura do login (ADR-010).	-
`ativo`	boolean	não	true	-	Verdadeiro para o aparelho em uso. A troca desativa o anterior.	-
`vinculado_em`	timestamp with time zone	não	now()	-	Momento do vínculo.	-

Índices:

- `CREATE UNIQUE INDEX dispositivo_chave_publica_key ON dispositivo USING btree (chave_publica)`
- `CREATE UNIQUE INDEX ux_dispositivo_um_ativo_por_cliente ON dispositivo USING btree (cliente_id) WHERE ativo`

credencial

Senha de 6 números do cliente e controle de tentativas (RF003, RF004).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`cliente_id`	uuid	não	-	PK FK → cliente	Cliente dono da senha.	-
`hash_senha`	text	não	-	hash_senha ~~ '$argon2id$%'	Hash Argon2id da senha (RNF05). A senha nunca é guardada.	SEGREDO
`tentativas_erradas`	integer	não	0	(tentativas_erradas >= 0) AND (tentativas_erradas <= 5)	Senhas erradas seguidas. Na 5ª o acesso é bloqueado por 30 minutos.	-
`bloqueada_ate`	timestamp with time zone	sim	-	-	Fim do bloqueio por tentativas, quando houver.	-

sessao_renovacao

Token de renovação de 30 dias, trocado a cada uso (RNF05).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador da sessão.	-
`dispositivo_id`	uuid	não	-	FK → dispositivo	Aparelho que recebeu o token.	-
`hash_token`	character(64)	não	-	UNIQUE	SHA-256 do token de renovação. O token em si só existe no celular.	SEGREDO
`expira_em`	timestamp with time zone	não	-	-	Fim da validade (30 dias após a emissão).	-
`revogada_em`	timestamp with time zone	sim	-	-	Momento em que o token foi trocado, usado no logout ou revogado na troca de aparelho.	-

Regras entre colunas:

- `(revogada_em IS NULL) OR (revogada_em <= (expira_em + '30 days'))`

Índices:

- `CREATE UNIQUE INDEX sessao_renovacao_hash_token_key ON sessao_renovacao USING btree (hash_token)`
- `CREATE INDEX ix_sessao_dispositivo_validas ON sessao_renovacao USING btree (dispositivo_id) WHERE (revogada_em IS NULL)`

consentimento

Registro de cada aceite de termo, com versão e quem aceitou (LGPD art. 8º e 14, risco R08).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do aceite.	-
`cliente_id`	uuid	não	-	UNIQUE (cliente_id, tipo, versao_termo) FK → cliente	Titular a quem o consentimento se refere.	-
`tipo`	text	não	-	(tipo IN ('TERMOS_DE_USO', 'PRIVACIDADE', 'BIOMETRIA', 'RESPONSAVEL_LEGAL')) UNIQUE (cliente_id, tipo, versao_termo)	Termo aceito: TERMOS_DE_USO, PRIVACIDADE, BIOMETRIA ou RESPONSAVEL_LEGAL.	-
`versao_termo`	text	não	-	UNIQUE (cliente_id, tipo, versao_termo)	Versão do texto aceito. Termo novo exige novo aceite.	-
`aceito_em`	timestamp with time zone	não	now()	-	Momento do aceite.	-
`aceito_por_cliente_id`	uuid	não	-	FK → cliente	Quem aceitou: o próprio titular ou, no caso do menor, o responsável.	-

Índices:

- `CREATE UNIQUE INDEX consentimento_cliente_id_tipo_versao_termo_key ON consentimento USING btree (cliente_id, tipo, versao_termo)`

Módulo contas

conta

Conta de pagamento do cliente, mantida no parceiro BaaS. Uma por cliente no MVP.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador interno da conta.	-
`cliente_id`	uuid	não	-	UNIQUE FK → cliente	Cliente dono da conta. Único: uma conta por cliente.	-
`agencia`	text	não	-	agencia ~ '^[0-9]{4}$'	Agência no parceiro (4 dígitos).	-
`numero`	text	não	-	numero ~ '^[0-9]{5,12}-[0-9X]$' UNIQUE	Número da conta com dígito, por exemplo 123456-7.	PII
`id_conta_parceiro`	text	não	-	UNIQUE	Identificador da conta no parceiro BaaS, usado na conciliação.	-
`status`	text	não	'ATIVA'::text	(status IN ('ATIVA', 'BLOQUEADA_TOTAL', 'ENCERRADA'))	ATIVA, BLOQUEADA_TOTAL (botão bloquear tudo, RF020) ou ENCERRADA.	-
`bloqueada_em`	timestamp with time zone	sim	-	-	Momento do bloqueio total. Preenchido se e somente se o status é BLOQUEADA_TOTAL.	-

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`criada_em`	timestamp with time zone	não	now()	-	Momento da abertura.	-

Regras entre colunas:

- `(status = 'BLOQUEADA_TOTAL') = (bloqueada_em IS NOT NULL)`

Índices:

- `CREATE UNIQUE INDEX conta_cliente_id_key ON conta USING btree (cliente_id)`
- `CREATE UNIQUE INDEX conta_numero_key ON conta USING btree (numero)`
- `CREATE UNIQUE INDEX conta_id_conta_parceiro_key ON conta USING btree (id_conta_parceiro)`

carteira

Divisão contábil do dinheiro: principal, meta ou carteira técnica de liquidação com o parceiro (ADR-007).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador da carteira.	-
`conta_id`	uuid	sim	-	FK → conta UNIQUE parcial ((tipo = 'PRINCIPAL'::text))	Conta dona. Vazio nas carteiras técnicas de liquidação.	-
`tipo`	text	não	-	(tipo IN ('PRINCIPAL', 'META', 'LIQUIDACAO_PIX', 'LIQUIDACAO_BOLETO')) UNIQUE parcial ((conta_id IS NULL))	PRINCIPAL (1 por conta), META (1 por meta), LIQUIDACAO_PIX ou LIQUIDACAO_BOLETO (do sistema).	-

Regras entre colunas:

- `(tipo ~~ 'LIQUIDACAO_%') = (conta_id IS NULL)`

Índices:

- `CREATE UNIQUE INDEX ux_carteira_principal_por_conta ON carteira USING btree (conta_id) WHERE (tipo = 'PRINCIPAL'::text)`
- `CREATE UNIQUE INDEX ux_carteira_tecnica_unica ON carteira USING btree (tipo) WHERE (conta_id IS NULL)`
- `CREATE INDEX ix_carteira_conta ON carteira USING btree (conta_id)`

lancamento

Razão contábil em partidas dobradas: a soma dos lançamentos de uma transação é zero (ADR-007). Só inserção.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do lançamento.	-
`transacao_id`	uuid	não	-	FK → transacao	Transação que originou o lançamento.	-
`carteira_id`	uuid	não	-	FK → carteira	Carteira afetada.	-
`valor_centavos`	bigint	não	-	valor_centavos <> 0	Positivo é crédito, negativo é débito. Nunca zero.	-
`criado_em`	timestamp with time zone	não	now()	-	Momento do lançamento.	-

Índices:

- `CREATE INDEX ix_lancamento_carteira ON lancamento USING btree (carteira_id, criado_em)`
- `CREATE INDEX ix_lancamento_transacao ON lancamento USING btree (transacao_id)`

saldo_fechamento_diario

Saldo de fechamento por carteira e dia, calculado pelo worker. Desnormalização para o RNF01.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`carteira_id`	uuid	não	-	PK FK → carteira	Carteira fechada.	-
`data`	date	não	-	PK	Dia do fechamento (horário de Brasília).	-
`saldo_centavos`	bigint	não	-	-	Soma de todos os lançamentos da carteira até o fim do dia. Pode ser negativo só em carteira técnica.	-

Módulo pagamentos

transacao

Toda movimentação de dinheiro. Classe base dos subtipos pix_envio, pix_recebimento, pagamento_boleto e movimento_meta.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	UNIQUE (id, tipo) PK	Identificador da transação.	-
`conta_id`	uuid	não	-	FK → conta UNIQUE (conta_id, chave_idempotencia)	Conta do cliente que fez ou recebeu a transação.	-
`tipo`	text	não	-	UNIQUE (id, tipo) (tipo IN ('PIX_ENVIO', 'PIX_RECEBIMENTO', 'BOLETO', 'META_GUARDAR', 'META_RESGATAR'))	PIX_ENVIO, PIX_RECEBIMENTO, BOLETO, META_GUARDAR ou META_RESGATAR. Define qual subtipo existe.	-
`valor_centavos`	bigint	não	-	valor_centavos > 0	Valor em centavos, sempre maior que zero. O sentido do dinheiro está nos lançamentos.	-
`status`	text	não	'PENDENTE'::text	(status IN ('PENDENTE', 'RETIDA', 'ENVIADA', 'CONCLUIDA', 'RECUSADA', 'ESTORNADA'))	Estado (padrão State): PENDENTE, RETIDA, ENVIADA, CONCLUIDA, RECUSADA ou ESTORNADA.	-
`chave_idempotencia`	text	não	-	(length(chave_idempotencia) >= 8) AND (length(chave_idempotencia) <= 128) UNIQUE (conta_id, chave_idempotencia)	Valor do cabeçalho Idempotency-Key. Único por conta (RNF11).	-
`id_parceiro`	text	sim	-	UNIQUE	Identificador fim a fim no parceiro BaaS. Preenchido quando o parceiro aceita.	-
`criada_em`	timestamp with time zone	não	now()	-	Momento do pedido.	-
`concluida_em`	timestamp with time zone	sim	-	-	Momento do estado final (concluída, recusada ou estornada).	-

Regras entre colunas:

- `(concluida_em IS NULL) OR (status IN ('CONCLUIDA', 'RECUSADA', 'ESTORNADA'))`

Índices:

- `CREATE UNIQUE INDEX transacao_id_parceiro_key ON transacao USING btree (id_parceiro)`
- `CREATE UNIQUE INDEX ux_transacao_idempotencia ON transacao USING btree (conta_id, chave_idempotencia)`
- `CREATE UNIQUE INDEX ux_transacao_id_tipo ON transacao USING btree (id, tipo)`
- `CREATE INDEX ix_transacao_extrato ON transacao USING btree (conta_id, criada_em DESC, id DESC)`
- `CREATE INDEX ix_transacao_pendentes ON transacao USING btree (status, criada_em) WHERE (status = ANY (ARRAY['PENDENTE'::text, 'RETIDA'::text, 'ENVIADA'::text]))`

pix_envio

Subtipo de transação: Pix enviado. A chave (transacao_id, tipo) garante que a transação base é PIX_ENVIO.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`transacao_id`	uuid	não	-	PK FK → transacao (transacao_id, tipo)	Transação base.	-
`tipo`	text	não	'PIX_ENVIO'::text	tipo = 'PIX_ENVIO' FK → transacao (transacao_id, tipo)	Sempre PIX_ENVIO. Existe para a chave estrangeira composta.	-
`chave_destino`	text	não	-	-	Chave Pix de quem recebe.	PII
`nome_recebedor`	text	não	-	-	Nome do recebedor devolvido pelo parceiro, falado antes da confirmação (RF009).	PII
`documento_recebedor_mascarado`	text	não	-	documento_recebedor_mascarado ~ '^[*0-9./-]+$'	CPF ou CNPJ do recebedor com dígitos ocultos, por exemplo ***.456.789-**.	PII
`banco_recebedor`	text	não	-	-	Nome da instituição do recebedor.	-
`destinatario_novo`	boolean	não	-	-	Verdadeiro se a conta nunca enviou Pix para essa chave. Aciona limite menor (RF019).	-

pix_recebimento

Subtipo de transação: Pix recebido, avisado pelo webhook do parceiro.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`transacao_id`	uuid	não	-	PK FK → transacao (transacao_id, tipo)	Transação base.	-

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`tipo`	text	não	'PIX_RECEBIMENTO'::text	tipo = 'PIX_RECEBIMENTO' FK → transacao (transacao_id, tipo)	Sempre PIX_RECEBIMENTO.	-
`nome_pagador`	text	não	-	-	Nome de quem pagou, falado no aviso (US05).	PII
`banco_pagador`	text	não	-	-	Instituição de quem pagou.	-

pagamento_boleto

Subtipo de transação: boleto ou conta de consumo paga (RF013).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`transacao_id`	uuid	não	-	PK FK → transacao (transacao_id, tipo)	Transação base.	-
`tipo`	text	não	'BOLETO'::text	tipo = 'BOLETO' FK → transacao (transacao_id, tipo)	Sempre BOLETO.	-
`linha_digitavel`	text	não	-	linha_digitavel ~ '^[0-9]{44,48}$'	Código lido pela câmera ou digitado, só números (44 a 48 dígitos).	-
`beneficiario`	text	não	-	-	Empresa que recebe, falada antes da confirmação.	-
`vencimento`	date	não	-	-	Data de vencimento do boleto.	-
`juros_centavos`	bigint	não	0	juros_centavos >= 0	Juros e multa por atraso, mostrados separados (US08).	-

movimento_meta

Subtipo de transação: guardar ou resgatar dinheiro de uma meta.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`transacao_id`	uuid	não	-	PK FK → transacao (transacao_id, tipo)	Transação base.	-
`tipo`	text	não	-	(tipo IN ('META_GUARDAR', 'META_RESGATAR')) FK → transacao (transacao_id, tipo)	META_GUARDAR ou META_RESGATAR.	-
`meta_id`	uuid	não	-	FK → meta	Meta movimentada.	-

Índices:

- `CREATE INDEX ix_movimento_meta_meta ON movimento_meta USING btree (meta_id)`

chave_pix

Chaves Pix da conta para receber (RF011).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador da chave.	-
`conta_id`	uuid	não	-	FK → conta	Conta dona da chave.	-
`tipo`	text	não	-	(tipo IN ('CPF', 'CELULAR', 'ALEATORIA'))	CPF, CELULAR ou ALEATORIA.	-
`valor`	text	não	-	UNIQUE parcial ((status = 'ATIVA'::text))	Valor da chave (CPF, celular ou código aleatório).	PII

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`status`	text	não	'ATIVA'::text	(status IN ('ATIVA', 'EXCLUIDA'))	ATIVA ou EXCLUIDA.	-
`criada_em`	timestamp with time zone	não	now()	-	Momento do cadastro.	-

Índices:

- `CREATE UNIQUE INDEX ux_chave_pix_valor_ativa ON chave_pix USING btree (valor) WHERE (status = 'ATIVA'::text)`
- `CREATE INDEX ix_chave_pix_conta ON chave_pix USING btree (conta_id)`

meta

Meta de economia (RF017). O dinheiro guardado fica na carteira META da própria meta.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador da meta.	-
`conta_id`	uuid	não	-	FK → conta	Conta dona da meta.	-
`carteira_id`	uuid	não	-	UNIQUE FK → carteira	Carteira META criada junto com a meta (composição).	-
`nome`	text	não	-	(length(TRIM(BOTH FROM nome)) >= 1) AND (length(TRIM(BOTH FROM nome)) <= 60)	Nome dado pelo cliente, por exemplo Notebook.	-
`valor_alvo_centavos`	bigint	não	-	valor_alvo_centavos > 0	Quanto o cliente quer juntar.	-
`data_alvo`	date	sim	-	-	Data desejada, opcional.	-
`imagem_url`	text	sim	-	-	Endereço da imagem da meta no S3.	-
`status`	text	não	'ATIVA'::text	(status IN ('ATIVA', 'CONCLUIDA', 'CANCELADA'))	ATIVA, CONCLUIDA ou CANCELADA.	-

Índices:

- `CREATE UNIQUE INDEX meta_carteira_id_key ON meta USING btree (carteira_id)`
- `CREATE INDEX ix_meta_conta ON meta USING btree (conta_id) WHERE (status = 'ATIVA'::text)`

Módulo familia

vinculo_responsavel

Liga o menor ao responsável legal e guarda os limites (RF014, RF015).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do vínculo.	-
`menor_cliente_id`	uuid	não	-	UNIQUE FK → cliente	Cliente menor. Único: um responsável vinculado por menor.	-
`responsavel_cliente_id`	uuid	não	-	FK → cliente	Cliente titular que é o responsável legal.	-
`status`	text	não	'CONVIDADO'::text	(status IN ('CONVIDADO', 'ATIVO', 'ENCERRADO'))	CONVIDADO, ATIVO ou ENCERRADO.	-

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`limite_diario_centavos`	bigint	não	-	limite_diario_centavos >= 0	Saída máxima por dia da conta do menor.	-
`limite_mensal_centavos`	bigint	não	-	limite_mensal_centavos >= 0	Saída máxima por mês. Não pode ser menor que o diário.	-
`convidado_em`	timestamp with time zone	não	now()	-	Momento do convite.	-
`aceito_em`	timestamp with time zone	sim	-	-	Momento do aceite do responsável. Obrigatório para ATIVO.	-
`expira_em`	timestamp with time zone	não	(now() + '7 days'::interval)	-	Prazo do convite: 7 dias (US09).	-

Regras entre colunas:

- `menor_cliente_id <> responsavel_cliente_id`
- `limite_diario_centavos <= limite_mensal_centavos`
- `(status = 'ATIVO') <= (aceito_em IS NOT NULL)`

Índices:

- `CREATE UNIQUE INDEX vinculo_responsavel_menor_cliente_id_key ON vinculo_responsavel USING btree (menor_cliente_id)`
- `CREATE INDEX ix_vinculo_responsavel ON vinculo_responsavel USING btree (responsavel_cliente_id) WHERE (status = 'ATIVO'::text)`

pedido_aprovacao

Pedido do menor para uma saída acima do limite (RF016).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do pedido.	-
`vinculo_id`	uuid	não	-	FK → vinculo_responsavel	Vínculo entre o menor e o responsável.	-
`transacao_id`	uuid	não	-	UNIQUE FK → transacao	Transação que aguarda a decisão.	-
`status`	text	não	'PENDENTE'::text	(status IN ('PENDENTE', 'APROVADO', 'RECUSADO', 'EXPIRADO'))	PENDENTE, APROVADO, RECUSADO ou EXPIRADO.	-
`criado_em`	timestamp with time zone	não	now()	-	Momento do pedido.	-
`expira_em`	timestamp with time zone	não	(now() + '24:00:00'::interval)	-	Prazo de 24 horas para decidir (US18).	-
`decidido_em`	timestamp with time zone	sim	-	-	Momento da decisão ou da expiração.	-

Regras entre colunas:

- `(status = 'PENDENTE') = (decidido_em IS NULL)`

Índices:

- `CREATE UNIQUE INDEX pedido_aprovacao_transacao_id_key ON pedido_aprovacao USING btree (transacao_id)`
- `CREATE INDEX ix_pedido_pendente ON pedido_aprovacao USING btree (vinculo_id, expira_em) WHERE (status = 'PENDENTE'::text)`

contato_confianca

Pessoa avisada em movimentação fora do padrão, sem acesso à conta (RF018).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do contato.	-
`cliente_id`	uuid	não	-	FK → cliente	Cliente protegido.	-
`nome`	text	não	-	-	Nome do contato (terceiro, não cliente).	PII
`celular`	text	não	-	celular ~ '^+55[0-9]{10,11}$'	Celular do contato, que recebe o aviso por SMS.	PII
`status`	text	não	'CONVIDADO'::text	(status IN ('CONVIDADO', 'ATIVO', 'REMOVIDO'))	CONVIDADO, ATIVO ou REMOVIDO.	-
`aceito_em`	timestamp with time zone	sim	-	-	Momento do aceite do convite.	-

Índices:

- `CREATE INDEX ix_contato_confianca_cliente ON contato_confianca USING btree (cliente_id) WHERE (status = 'ATIVO'::text)`

Módulo antifraude

analise_risco

Nota de risco de cada Pix enviado e decisão automática ou do analista (RF026, RF027).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador da análise.	-
`transacao_id`	uuid	não	-	UNIQUE FK → transacao	Pix analisado.	-
`nota`	integer	não	-	(nota >= 0) AND (nota <= 100)	Soma das regras de risco, de 0 a 100. A partir de 70 o Pix é retido.	-
`motivos`	text[]	não	'{}'::text[]	-	Regras que pontuaram, por exemplo DESTINATARIO_NOVO e HORARIO_NOTURNO.	-
`resultado`	text	não	-	(resultado IN ('LIBERADA_AUTOMATICA', 'RETIDA', 'LIBERADA_ANALISTA', 'RECUSADA_ANALISTA'))	LIBERADA_AUTOMATICA, RETIDA, LIBERADA_ANALISTA ou RECUSADA_ANALISTA.	-
`analista_id`	uuid	sim	-	FK → funcionario	Analista que decidiu, quando a decisão é humana.	-
`decidida_em`	timestamp with time zone	sim	-	-	Momento da decisão humana.	-

Regras entre colunas:

- `(resultado IN ('LIBERADA_ANALISTA', 'RECUSADA_ANALISTA')) = ((analista_id IS NOT NULL) AND (decidida_em IS NOT NULL))`
- `(NOT ((resultado = 'LIBERADA_AUTOMATICA') AND (nota >= 70)))`

Índices:

- `CREATE UNIQUE INDEX analise_risco_transacao_id_key ON analise_risco USING btree (transacao_id)`
- `CREATE INDEX ix_analise_fila ON analise_risco USING btree (resultado, id) WHERE (resultado = 'RETIDA'::text)`

Módulo atendimento

chamado

Atendimento humano aberto de dentro do app (RF024, RF025).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do chamado.	-
`cliente_id`	uuid	não	-	FK → cliente	Cliente que abriu.	-
`atendente_id`	uuid	sim	-	FK → funcionario	Atendente responsável. Obrigatório a partir de EM_ATENDIMENTO.	-
`assunto`	text	não	-	-	Assunto escrito pelo cliente. Pode conter dado pessoal.	PII
`status`	text	não	'ABERTO'::text	(status IN ('ABERTO', 'EM_ATENDIMENTO', 'RESOLVIDO'))	ABERTO, EM_ATENDIMENTO ou RESOLVIDO.	-
`aberto_em`	timestamp with time zone	não	now()	-	Momento da abertura.	-
`fechado_em`	timestamp with time zone	sim	-	-	Momento da resolução.	-

Regras entre colunas:

- `(status = 'RESOLVIDO') = (fechado_em IS NOT NULL)`
- `(status = 'ABERTO') OR (atendente_id IS NOT NULL)`

Índices:

- `CREATE INDEX ix_chamado_cliente ON chamado USING btree (cliente_id, aberto_em DESC)`
- `CREATE INDEX ix_chamado_fila ON chamado USING btree (status, aberto_em) WHERE (status <> 'RESOLVIDO'::text)`

mensagem_chamado

Mensagens de um chamado. Composição: apagar o chamado apaga as mensagens.

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador da mensagem.	-
`chamado_id`	uuid	não	-	FK → chamado	Chamado a que pertence.	-
`autor`	text	não	-	(autor IN ('CLIENTE', 'ATENDENTE'))	CLIENTE ou ATENDENTE.	-
`texto`	text	sim	-	-	Texto da mensagem. Pode conter dado pessoal.	PII
`audio_url`	text	sim	-	-	Endereço no S3 do áudio enviado pelo cliente (US14). Voz é dado pessoal.	PII

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`enviada_em`	timestamp with time zone	não	now()	-	Momento do envio.	-

Regras entre colunas:

- `(texto IS NOT NULL) OR (audio_url IS NOT NULL)`

Índices:

- `CREATE INDEX ix_mensagem_chamado ON mensagem_chamado USING btree (chamado_id, enviada_em)`

Módulo backoffice

funcionario

Usuário interno dos painéis de SAC, antifraude e administração (RF030).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do funcionário.	-
`nome`	text	não	-	-	Nome do funcionário.	PII
`email`	text	não	-	email ~ '^[^@\s]+@[^@\s]+.[^@\s]+$'	E-mail corporativo, usado no login. Único sem diferenciar maiúsculas.	PII
`papel`	text	não	-	(papel IN ('ATENDENTE', 'ANALISTA_ANTIFRAUDE', 'ADMINISTRADOR'))	Papel de acesso (RBAC): ATENDENTE, ANALISTA_ANTIFRAUDE ou ADMINISTRADOR.	-
`hash_senha`	text	não	-	hash_senha ~~ '$argon2id$%'	Hash Argon2id da senha.	SEGREDO
`segredo_totp_cifrado`	bytea	sim	-	-	Segredo do segundo fator TOTP. Vazio até o primeiro login (US22).	SEGREDO, CIFRADO
`ativo`	boolean	não	true	-	Falso para funcionário desligado. Nunca se apaga, por causa da auditoria.	-

Índices:

- `CREATE UNIQUE INDEX ux_funcionario_email ON funcionario USING btree (lower(email))`

registro_auditoria

Toda consulta e alteração feita por funcionário. Só inserção. Guardado 5 anos (RNF14, Lei 9.613/1998).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	bigint	não	identity	PK	Número sequencial do registro.	-
`funcionario_id`	uuid	não	-	FK → funcionario	Quem fez a ação.	-
`acao`	text	não	-	-	O que foi feito, por exemplo CONSULTAR_HISTORICO ou DECIDIR_ANALISE.	-
`entidade`	text	não	-	-	Tabela afetada, por exemplo cliente ou analise_risco.	-
`entidade_id`	text	não	-	-	Identificador da linha afetada.	-
`ip`	inet	não	-	-	Endereço IP de onde veio a ação.	PII

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`ocorrido_em`	timestamp with time zone	não	now()	-	Momento da ação.	-

Índices:

- `CREATE INDEX ix_auditoria_entidade ON registro_auditoria USING btree (entidade, entidade_id, ocorrido_em DESC)`
- `CREATE INDEX ix_auditoria_funcionario ON registro_auditoria USING btree (funcionario_id, ocorrido_em DESC)`

Módulo notificacao

notificacao

Push e SMS enviados ao cliente (RF028).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador da notificação.	-
`cliente_id`	uuid	não	-	FK → cliente	Cliente que recebe.	-
`canal`	text	não	-	(canal IN ('PUSH', 'SMS'))	PUSH ou SMS.	-
`tipo`	text	não	-	-	Evento que gerou a notificação, por exemplo PixConcluido.	-
`conteudo`	text	não	-	-	Texto enviado. Pode ter valor e nome de terceiro.	PII
`status`	text	não	'PENDENTE'::text	(status IN ('PENDENTE', 'ENVIADA', 'FALHOU'))	PENDENTE, ENVIADA ou FALHOU.	-
`criada_em`	timestamp with time zone	não	now()	-	Momento da criação.	-
`enviada_em`	timestamp with time zone	sim	-	-	Momento da entrega ao provedor de push ou SMS.	-

Regras entre colunas:

- `(status = 'ENVIADA') = (enviada_em IS NOT NULL)`

Índices:

- `CREATE INDEX ix_notificacao_cliente ON notificacao USING btree (cliente_id, criada_em DESC)`

outbox_evento

Eventos gravados na mesma transação do dado e entregues pelo worker (ADR-008).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	bigint	não	identity	PK	Número sequencial do evento.	-
`tipo`	text	não	-	-	Nome do evento de domínio, por exemplo PixConcluido ou ContaBloqueada.	-
`agregado_id`	uuid	não	-	-	Identificador da entidade de origem.	-
`payload`	jsonb	não	-	-	Dados do evento em JSON. Pode conter dado pessoal; apagado 90 dias após processado.	PII
`criado_em`	timestamp with time zone	não	now()	-	Momento da gravação.	-
`processado_em`	timestamp with time zone	sim	-	-	Momento em que o worker terminou. Vazio enquanto pendente.	-

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`tentativas`	integer	não	0	tentativas >= 0	Quantas vezes o worker tentou.	-
`proxima_tentativa_em`	timestamp with time zone	não	now()	-	Quando tentar de novo (espera crescente após erro).	-

Índices:

- `CREATE INDEX ix_outbox_pendentes ON outbox_evento USING btree (proxima_tentativa_em) WHERE (processado_em IS NULL)`

lembrete_conta

Conta a vencer cadastrada pelo cliente para receber lembrete (RF022).

Campo	Tipo	Nulo	Padrão	Restrições	Descrição de negócio	LGPD
`id`	uuid	não	gen_random_uuid()	PK	Identificador do lembrete.	-
`cliente_id`	uuid	não	-	FK → cliente	Cliente lembrado.	-
`descricao`	text	não	-	-	Nome da conta, por exemplo Conta de água.	-
`dia_vencimento`	integer	não	-	(dia_vencimento >= 1) AND (dia_vencimento <= 31)	Dia do mês do vencimento (1 a 31). O lembrete sai 3 dias antes.	-
`ativo`	boolean	não	true	-	Falso quando o cliente desliga o lembrete.	-

Índices:

- `CREATE INDEX ix_lembrete_dia ON lembrete_conta USING btree (dia_vencimento) WHERE ativo`

5.6 Estratégia: backup, LGPD e migrações

Backup

O objetivo vem do RNF12: perder no máximo **5 minutos** de dados (RPO) e voltar ao ar em até **1 hora** (RTO).

Camada	Frequência	Retenção	Para quê	Custo
Réplica síncrona Multi-AZ	Contínua (cada transação confirmada nas 2 zonas)	Não é backup, é cópia viva	Falha de máquina ou de zona: failover automático, sem perda de dado	Já no preço da instância Multi-AZ (Bloco 1)
Backup automático do RDS com log de transações	Snapshot diário + log contínuo enviado a cada 5 minutos	**35 dias** (o máximo do RDS)	Restaurar a **qualquer ponto** dos últimos 35 dias. Erro humano ou de software que apagou ou corrompeu dados	50 GB além da cota grátis já custeados no Bloco 1 (US$ 4,75/mês)
Snapshot manual mensal, copiado para uma conta AWS separada (cofre de backup com bloqueio contra exclusão) na mesma região	Todo dia 1º, às 3h	**12 meses**	Comparar com meses anteriores, responder auditoria fora da janela de 35 dias e ter uma cópia que um invasor da conta principal não consegue apagar	Custeado no item 4.4 do Bloco 1: 12 x 50 GB a US$ 0,095 por GB-mês, US$ 57,00 por mês

Por que não copiar para outra região. A única região da AWS no Brasil é São Paulo. Copiar o backup para os EUA seria transferência internacional de dados pessoais (LGPD, art. 33), o que o ADR-013 evita. A proteção contra desastre fica nas 3 zonas de disponibilidade da região e na conta separada. Se um dia for exigida uma cópia fora de São Paulo, ela entra como decisão nova, com as salvaguardas do art. 33.

Teste de restauração a cada 3 meses (RNF12): restaurar o backup de um ponto no tempo em uma instância temporária, rodar `bloco-5-banco/testes/testar-schema.sh` adaptado para a cópia e conferir a soma de lançamentos por carteira contra o saldo de fechamento. O tempo medido do início ao fim precisa ficar abaixo de 1 hora. O resultado é registrado; se passar de 1 hora, vira item de sprint.

O que não é backup: os registros que a lei manda guardar por 5 anos (lançamentos, transações, auditoria) **não** dependem do backup. Eles ficam no próprio banco, protegidos contra alteração por gatilho (`tg_lancamento_imutavel` , `tg_auditoria_imutavel`).

LGPD

Quais campos são dado pessoal

O dicionário de dados marca **26 colunas** com tratamento LGPD (22 de dado pessoal e 4 de segredo; uma coluna pode ter mais de uma marcação), todas tiradas do `COMMENT ON` do SQL. Resumo:

Marcação	Quantidade	Exemplos	Tratamento
PII	22 (inclui as 2 cifradas do cliente)	`cliente.nome_completo` , `cliente.celular` , `pix_envio.nome_recebedor` , `contato_confianca.celular` , `registro_auditoria.ip`	Acesso só pelo módulo dono. Nunca vai para log (o logger mascara esses nomes de campo). Entra na exportação do titular (RF029)
CIFRADO	3	`cliente.cpf_cifrado` , `cliente.data_nascimento_cifrada` , `funcionario.segredo_totp_cifrado`	Cifrado pela aplicação antes de gravar (abaixo)
SEGREDO	4	`credencial.hash_senha` , `funcionario.hash_senha` , `sessao_renovacao.hash_token` , `funcionario.segredo_totp_cifrado`	Só hash (Argon2id ou SHA-256). Nunca devolvido pela API, nunca exportado
PII-SENSIVEL	0		**Nenhum dado sensível é guardado.** A biometria fica no celular (ADR-010) e a selfie do cadastro vai direto para o parceiro de validação de identidade, sem ser gravada pelo Rufus

A tabela completa está no começo do dicionário de dados.

Como o CPF e a data de nascimento são cifrados

1. A aplicação pede ao **AWS KMS** uma chave de dados (envelope encryption). A chave mestra nunca sai do KMS.
2. Cifra o valor com **AES-256-GCM** e grava o resultado em `cpf_cifrado` (`bytea`), junto com o vetor de inicialização.
3. Para buscar um cliente pelo CPF sem decifrar a base inteira, grava também `cpf_hash` : **HMAC-SHA256** do CPF com uma chave secreta guardada no Secrets Manager. O HMAC (e não um SHA-256 puro) impede descobrir o CPF testando todos os 10^11 CPFs possíveis, porque sem a chave o atacante não consegue calcular o hash.
4. `cpf_hash` é `UNIQUE` : o banco recusa CPF repetido mesmo sem enxergar o CPF.
5. Quem vazar o banco sem a chave do KMS e a do HMAC não lê CPF nem data de nascimento.

Por que só esses dois: são os campos que, junto com o nome, permitem abrir conta ou crédito em nome da pessoa em outro lugar. Cifrar tudo impediria as buscas e o extrato sem ganho proporcional; os demais PII ficam protegidos pela criptografia do disco do RDS (ativada por padrão com KMS), pela rede privada e pelo controle de acesso.

Encerramento de conta e anonimização (RF029, US17)

Quando o titular encerra a conta com saldo zero:

Dado	O que acontece	Por quê
`cliente.nome_completo` , `celular` , `email`	Substituídos por `ANONIMIZADO` e valores nulos	Não há mais finalidade (LGPD art. 16)
`cliente.cpf_cifrado` , `data_nascimento_cifrada`	**Mantidos cifrados por 5 anos** e depois apagados	A prevenção à lavagem de dinheiro (Lei 9.613/1998) exige identificar o titular das movimentações por 5 anos. Base legal: cumprimento de obrigação legal (LGPD art. 16, I)
`transacao` , `lancamento` , subtipos	Mantidos 5 anos	Mesma obrigação legal. O razão não pode ser alterado
`contato_confianca` , `lembrete_conta` , `dispositivo` , `sessao_renovacao` , `credencial`	Apagados (ON DELETE CASCADE ou exclusão direta)	Sem obrigação de guarda
`chamado` , `mensagem_chamado`	Texto anonimizado depois de 5 anos	Prova de atendimento em caso de reclamação
`outbox_evento.payload`	Apagado 90 dias depois de processado, para todos os clientes	O evento já cumpriu a função
`registro_auditoria`	Mantido 5 anos (RNF14)	Obrigação legal

Um job mensal apaga o que passou dos 5 anos. Pedido de exportação ou de informação do titular é respondido em até **15 dias** (RNF07, LGPD art. 19).

Migrações sem parar o sistema

As migrações são arquivos SQL versionados (`node-pg-migrate` , Bloco 3), revisados em pull request e aplicados pelo pipeline antes de trocar as tarefas da API. Como as tarefas antigas e as novas rodam juntas por alguns minutos durante o deploy, **toda migração precisa funcionar com as duas versões do código ao mesmo tempo**. Para isso usamos o padrão **expandir e contrair**.

Regras

1. Nunca renomear nem apagar coluna no mesmo deploy que muda o código.
2. Coluna nova entra **nula ou com valor padrão constante** (no PostgreSQL isso é só metadado, sem reescrever a tabela).
3. Índice em tabela grande sempre com `CREATE INDEX CONCURRENTLY` (não trava escrita). Essa migração roda fora de transação.

4. Restrição nova em tabela grande: `ADD CONSTRAINT ... NOT VALID` e depois `VALIDATE CONSTRAINT` em outro passo (valida sem travar escrita).
5. Toda migração tem tempo limite: `SET lock_timeout = '5s'`. Se não conseguir a trava, falha e o pipeline para, em vez de travar o banco.

Exemplo concreto: separar o celular do contato de confiança em DDD e número

Pedido hipotético: o SMS do provedor passa a exigir DDD separado.

Passo	Deploy	Banco	Código
1. Expandir	v1.5	`ALTER TABLE contato_confianca ADD COLUMN ddd char(2), ADD COLUMN numero text;` (nulas, instantâneo)	Grava nas **duas** formas (`celular` e `ddd` + `numero`), lê da antiga
2. Migrar dados	job	`UPDATE contato_confianca SET ddd = substr(celular, 4, 2), numero = substr(celular, 6) WHERE ddd IS NULL` em lotes de 1.000 linhas	Sem mudança
3. Trocar a leitura	v1.6	`ALTER TABLE contato_confianca ADD CONSTRAINT ck_ddd CHECK (ddd IS NOT NULL) NOT VALID;` e depois `VALIDATE CONSTRAINT ck_ddd;`	Lê de `ddd` + `numero` , ainda grava nas duas
4. Contrair	v1.7	`ALTER TABLE contato_confianca DROP COLUMN celular;`	Para de gravar em `celular`

Em nenhum momento uma versão do código encontra o banco num formato que ela não entende, e cada passo pode ser revertido voltando só o código.

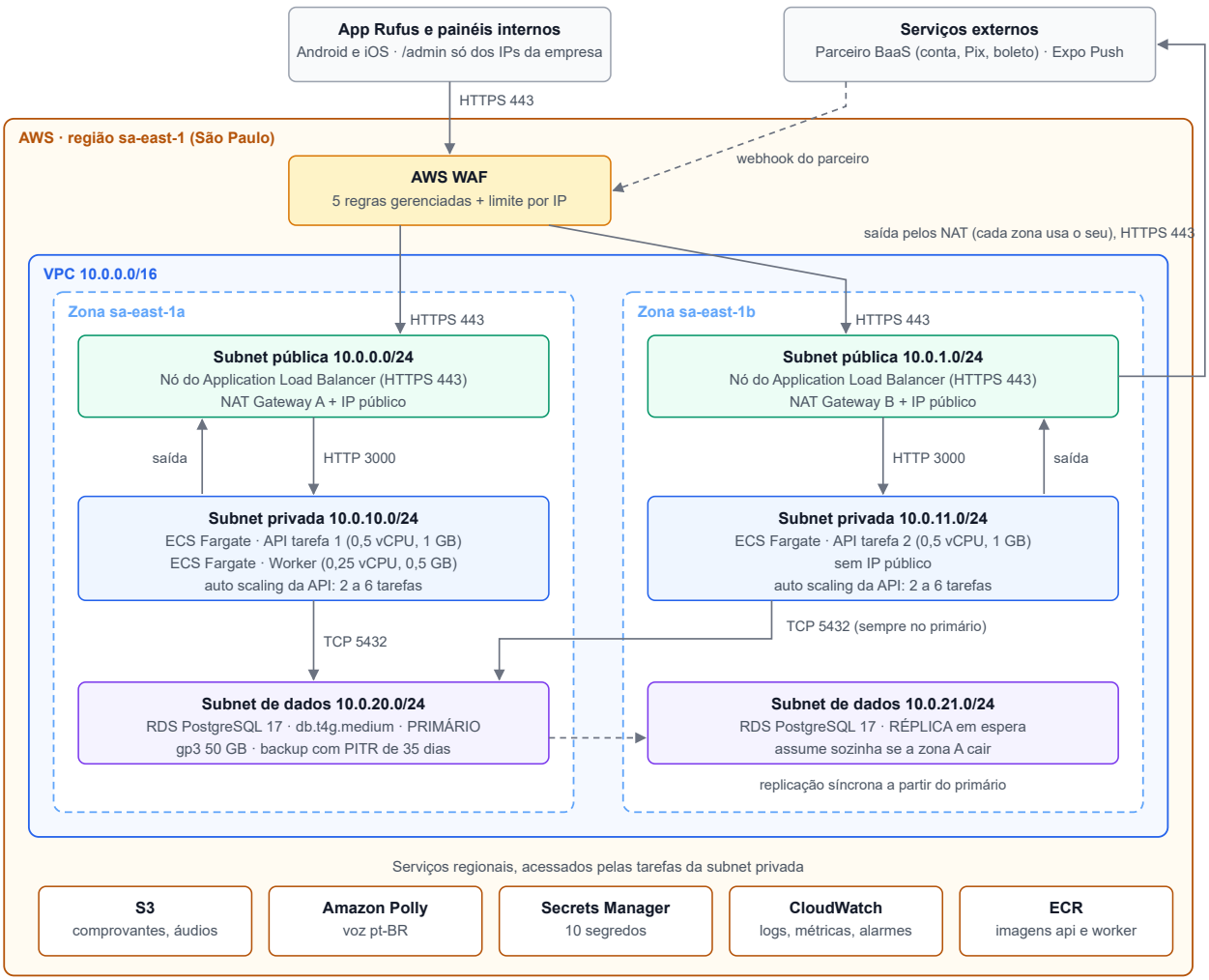
Bloco 6 · Infraestrutura e DevOps

6.1 Diagrama de infraestrutura e custo por ambiente

Produção roda na AWS em São Paulo (ADR-013), em **2 zonas de disponibilidade**, com balanceador, 2 tarefas da API, banco Multi-AZ e 1 NAT por zona. Esse desenho é o que sustenta os 99,9% do RNF03: se uma zona inteira cair, a outra continua atendendo e o banco troca para a réplica sozinho.

Diagrama de infraestrutura · produção na AWS sa-east-1 (São Paulo)

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Fonte do diagrama: diagramas/infra-producao.svg.py, gerado por `ferramentas/diagramas/gerar.py`.

Como ler o diagrama

Camada	O que tem	Por quê
Borda	AWS WAF na frente do balanceador	Bloqueia ataques comuns (injeção, robôs) e limita requisições por IP antes de chegar à API (R02). O caminho `/admin` só aceita os IPs da empresa (ADR-012)
Subnet pública (1 por zona)	Nós do Application Load Balancer e NAT Gateway	O ALB termina o HTTPS (porta 443) e distribui entre as zonas. O NAT é a única saída para a internet das tarefas, que não têm IP público
Subnet privada (1 por zona)	Tarefas do ECS Fargate: API e worker	Sem IP público: só o ALB chega nelas, na porta 3000
Subnet de dados (1 por zona)	RDS PostgreSQL 17 primário na zona A, réplica síncrona na zona B	Só as subnets privadas chegam na porta 5432. Nenhum acesso da internet
Serviços regionais	S3, Polly, Secrets Manager, CloudWatch, ECR	Usados pelas tarefas; não ficam dentro da VPC

Regras de rede (security groups):

Origem	Destino	Porta	Regra
Internet	ALB	443 (e 80 só para redirecionar para 443)	Permitido, passando pelo WAF
ALB	Tarefas da API	3000	Só o security group do ALB
Tarefas (API e worker)	RDS	5432	Só o security group das tarefas
Tarefas	Internet (parceiro BaaS, Expo)	443	Saída pelo NAT da própria zona
Qualquer outro	Qualquer outro		Negado

Ambientes

Ambiente	Onde	Diferença para produção
Desenvolvimento	Máquina do dev, Docker Compose (`codigo/docker-compose.yml`)	Tudo local, parceiro BaaS e voz com adaptador falso (ADR-002)
Homologação	AWS sa-east-1, conta separada	1 zona, 1 tarefa da API, banco db.t4g.small sem réplica, 1 NAT. Mesmo código e mesmo Terraform com outros parâmetros
Produção	AWS sa-east-1	O diagrama acima

Custo mensal real por ambiente

Valores de ferramentas/custo-infra/custo-gerado.md, calculados pelo script `custo_infra.py` com a tabela oficial de preços da AWS e a PTAX de 04/11/2026 (R$ 5,4778). A tabela item a item está lá.

Ambiente	USD por mês	R$ por mês
Produção	691,91	3.790,14
Homologação	207,50	1.136,64
Serviços fora da AWS (Expo EAS, GitHub, domínio, lojas)	94,86	519,62
Desenvolvimento	0,00	0,00
Total	**994,27**	**5.446,39**

Onde está o dinheiro de produção: banco Multi-AZ com disco, backup e snapshots (US$ 284,40), os 2 NAT Gateways (US$ 137,64) e os containers (US$ 62,00). O NAT duplicado e o Multi-AZ são o preço da disponibilidade: com 1 NAT só, uma falha na zona A cortaria a comunicação com o parceiro BaaS também na zona B.

E se a demanda triplicar?

A API escala de 2 para até 6 tarefas sozinha (auto scaling por CPU acima de 60% e por requisições por tarefa). O banco é o limite: o próximo passo é trocar o db.t4g.medium por um db.m7g.large Multi-AZ, que a tabela da AWS em São Paulo cobra US$ 0,442 por hora, e só depois considerar réplica de leitura para o extrato. Nada muda no código.

6.2 Pipeline CI/CD

Oito etapas, sempre nesta ordem: **lint, testes, build, segurança, homologação, smoke test, aprovação, produção**. Cada etapa tem um critério de bloqueio: se ele falhar, nada segue para a próxima. Os arquivos reais estão em codigo/.github/workflows/ci.yml e cd.yml, no GitHub Actions (stack, Bloco 3).

```
pull request → 1 lint → 2 testes → 3 build → 4 segurança → merge na main
                                                                  |
main → 5 publicar no ECR → 6 homologação → 7 smoke → 8 aprovação → produção → smoke em produção
```

Etapas e critérios de bloqueio

#	Etapa	O que roda	Bloqueia quando	Ligado a
1	Lint e tipos	ESLint, `tsc --noEmit`, `redocly lint` do contrato	Qualquer erro de lint, inclusive import entre módulos (ADR-001), erro de tipo ou contrato OpenAPI inválido	Guia de padrões (Bloco 8)
2	Testes	Jest de unidade e integração contra um PostgreSQL 17 real com o schema do Bloco 5	Qualquer teste falhando ou cobertura de linhas abaixo de 80%	Bloco 9
3	Build	`docker build` das imagens da API e do worker	Imagem que não compila	Containerização (6.3)
4	Segurança	`npm audit --audit-level=high`, gitleaks no histórico, Trivy nas duas imagens	Vulnerabilidade alta ou crítica com correção disponível, ou segredo encontrado no código	RNF06, R02
5	Publicar	Build e envio das imagens para o ECR, com a tag igual ao hash do commit	Só começa se as etapas 1 a 4 ficaram verdes	
6	Homologação	Migração do banco (só a parte que expande, Bloco 5) e atualização dos serviços no ECS	As tarefas novas não passam no `/health` do balanceador. O circuit breaker do ECS desfaz o deploy sozinho	RNF03
7	Smoke test	`/health` com a versão certa e carga curta com k6 (20 usuários por 30 s)	`/health` diferente de 200, versão diferente do commit, p95 acima de 300 ms ou mais de 1% de erro	RNF01
8	Aprovação e produção	Aprovação manual de 1 revisor (que não seja o autor) no ambiente `producao` do GitHub, depois o mesmo deploy da etapa 6 e um `/health` final	Sem aprovação nada sai. Smoke de produção falhando abre incidente e a task definition anterior é reaplicada	

Decisões do pipeline

Decisão	Alternativa descartada	Por quê
Credenciais da AWS por OIDC, com uma role por ambiente	Chave de acesso guardada nos segredos do GitHub	Chave fixa vaza e não expira. Com OIDC o GitHub recebe uma credencial temporária a cada execução (R02)
A mesma imagem vai de homologação para produção	Fazer build de novo para produção	O que foi testado em homologação é exatamente o que sobe
Deploy rolling com 100% das tarefas saudáveis no mínimo	Parar tudo e subir de novo	Zero tempo fora do ar a cada versão (RNF03)
Migração só "expandir" antes do deploy	Migração que apaga coluna junto com o deploy	A versão antiga continua funcionando enquanto a nova sobe (Bloco 5, migrações sem downtime)
Aprovação humana antes de produção	Deploy automático em produção	Sistema financeiro: um olho humano no que vai para o cliente. A aprovação leva minutos e não trava o resto

Onde os workflows ficam

O GitHub só executa workflows na pasta `.github/` da raiz do repositório. Nesta entrega a pasta está em `codigo/.github/` de propósito, para o repositório acadêmico não tentar fazer deploy numa AWS que não existe. Quando `codigo/` virar o

repositório do produto, a pasta vai para a raiz sem mudança. As variáveis `AWS_ROLE_ARN` e `REDE_TAREFA` e os ambientes `homologacao` e `producao` (com o revisor obrigatório) são configurados no GitHub pelo Tech Lead no Sprint 1.

6.3 Containerização

Um Dockerfile por serviço, os dois com build em 3 estágios, e um Docker Compose que sobe banco, API e worker no computador do dev. Testado em 04/11/2026: as duas imagens compilam, o `docker compose up --build` aplica o schema do Bloco 5, `/health` responde 200 e o worker processa um evento do outbox.

Arquivo	Serviço
codigo/docker/api.Dockerfile	API (porta 3000)
codigo/docker/worker.Dockerfile	Worker do outbox (sem porta)
codigo/docker-compose.yml	Ambiente local: PostgreSQL 17, API e worker

Os 3 estágios

Estágio	Base	O que faz	Vai para a imagem final?
`build`	node:22-alpine	`npm ci` com tudo e compila o TypeScript	Só a pasta `dist/`
`deps`	node:22-alpine	`npm ci --omit=dev`, só dependências de produção	Só `node_modules/`
`final`	node:22-alpine	Junta `dist/` e `node_modules/`, roda como usuário `node` (sem root)	É a imagem

Resultado: compilador, testes e ferramentas de lint ficam fora da imagem de produção. Menos coisa instalada é menos vulnerabilidade para o Trivy encontrar (RNF06). As duas imagens ficaram com 286 MB no build local.

Escolhas

Escolha	Alternativa descartada	Por quê
Dois Dockerfiles, mesmo código	Uma imagem só com comando diferente	O enunciado pede um por serviço, e assim cada um tem a sua variável `SERVICO`, o seu healthcheck e pode mudar de base sem afetar o outro
`node:22-alpine`	`node:22` (Debian completo)	Imagem base bem menor, com menos pacotes para manter atualizados
Rodar como usuário `node`	Root, padrão do Docker	Se alguém explorar a API, não tem root dentro do container (OWASP A05)
Sistema de arquivos só de leitura na AWS (`readonlyRootFilesystem` nas definições de tarefa)	Gravável	A API não grava em disco: log vai para a saída padrão e arquivo vai para o S3
Healthcheck no container e no ALB batendo em `/health`	Só checar se o processo está vivo	`/health` testa o banco também: tarefa sem banco sai da rotação do balanceador
SQL do Bloco 5 montado em `/docker-entrypoint-initdb.d` no Compose	Banco vazio e migração manual	O dev tem o banco com schema e dados de exemplo no primeiro `up`

Compose de desenvolvimento

```
cd codigo
cp .env.example .env
docker compose up --build
```

Serviço	Imagem	Porta no seu computador	Depende de
banco	postgres:17-alpine	5432	
api	build de `api.Dockerfile`	3000	banco saudável (`pg_isready`)
worker	build de `worker.Dockerfile`	nenhuma	banco saudável

O contrato OpenAPI é montado só no Compose (`/contrato/openapi.yaml`) para o Swagger UI em `/docs` . Em produção o `/docs` não é publicado.

6.4 Segurança

Quatro camadas: quem é o usuário (autenticação), o que ele pode fazer (autorização), onde ficam os segredos e como cada item do OWASP Top 10 é tratado. As decisões vêm do ADR-010 e dos RNF05, RNF06, RNF07 e RNF14.

Autenticação

Cliente (app)

Passo	Como funciona
Cadastro	O app gera um par de chaves no Keystore (Android) ou Keychain (iOS). A chave privada nunca sai do celular. A pública vai para o servidor e fica em `Dispositivo.chavePublica`
Login	CPF, senha de 6 números (guardada com Argon2id) e a assinatura de um desafio com a chave do aparelho. Aparelho diferente do vinculado é recusado (RF005)
Tokens	Token de acesso JWT assinado, válido por **15 minutos**. Token de renovação aleatório, válido por **30 dias**, guardado no servidor só como hash (`SessaoRenovacao.hashToken`)
Renovação	Cada uso do token de renovação gera outro e invalida o anterior. Se um token já trocado for usado de novo, todas as sessões do aparelho são revogadas: sinal de que alguém copiou o token
Biometria	Só destrava a chave do aparelho no próprio celular. Nada biométrico chega ao servidor (R08)
Operação com dinheiro	Pix e boleto exigem, além do token, a assinatura do pedido com a chave do aparelho, destravada pela biometria na hora
Bloqueio	5 senhas erradas bloqueiam o acesso por 30 minutos (RF004)

Funcionário (painéis internos)

Senha com Argon2id e código TOTP de 6 dígitos (segundo fator obrigatório desde o primeiro acesso, US22). Token de 15 minutos, sem renovação por mais de 8 horas, e acesso a `/admin` só pelos IPs da empresa (regra do WAF).

Autorização: RBAC e ABAC

RBAC (por papel) para funcionários. O papel vem no token e cada rota `/admin` declara o papel que aceita (`x-papel` no OpenAPI).

Papel	Pode	Não pode
ATENDENTE	Ver e responder chamados, ver histórico do cliente (registrado na auditoria)	Fila antifraude, decidir Pix, cadastrar funcionário
ANALISTA_ANTIFRAUDE	Ver a fila de Pix retidos, liberar, recusar e bloquear conta	Responder chamado, cadastrar funcionário
ADMINISTRADOR	Cadastrar funcionários e papéis	Movimentar dinheiro de cliente (nenhum papel pode)

ABAC (por atributo) para clientes. O que um cliente pode fazer depende da relação entre ele e o recurso, não de um papel fixo:

Regra	Atributos checados
Ver e mexer na própria conta	`conta.clienteId == token.clienteId`
Responsável muda limite do menor	Existe `VinculoResponsavel` com `responsavelClienteId == token.clienteId` , `menorClienteId == alvo` e `status == ATIVO`
Responsável decide pedido de aprovação	Mesmo vínculo ativo e `pedido.status == PENDENTE` e `agora < pedido.expiraEm`
Menor envia Pix	`cliente.tipo == MENOR` obriga checar `limiteDiarioCentavos` e `limiteMensalCentavos` do vínculo
Contato de confiança	Recebe aviso, mas não tem nenhuma permissão de leitura na conta

Toda negação devolve 403 com o código `SEM_PERMISSAO` e o mesmo texto, para não revelar se o recurso existe.

Gestão de segredos

Segredo	Onde fica	Rotação
Senha do banco	Secrets Manager, rotação automática do RDS	30 dias
Segredo de assinatura do JWT	Secrets Manager	90 dias, com 2 chaves válidas na troca
Chave de cifra de CPF e data de nascimento	Chave de dados gerada pelo AWS KMS, guardada cifrada	Anual
Credenciais e segredo do webhook do parceiro BaaS	Secrets Manager	Conforme o contrato com o parceiro
Token do Expo	Secrets Manager	Quando alguém sai da equipe

As tarefas do ECS recebem os segredos como variáveis de ambiente injetadas pela AWS no momento da partida (campo `secrets` das definições em `codigo/infra/ecs/`). Nenhum segredo fica no repositório: o `.env` está no `.gitignore` e o gitleaks bloqueia o pipeline se encontrar algum (etapa 4 do CI). São os 10 segredos já custeados no Bloco 1.

OWASP Top 10 (2021), item por item

Item	Risco no Rufus	Medida
A01 Quebra de controle de acesso	Cliente ver a conta de outro, atendente decidir Pix	ABAC em toda rota de cliente, RBAC em toda rota `/admin`, testes de autorização por rota no CI, ids em UUID (não sequenciais)
A02 Falhas de criptografia	Vazamento de CPF, senha ou token	TLS 1.2 ou superior no ALB, Argon2id para senha, CPF e data de nascimento cifrados no banco, RDS e S3 cifrados em repouso com KMS, token de renovação guardado só como hash
A03 Injeção	SQL injection no extrato ou na busca de cliente	Kysely com parâmetros em toda consulta (sem concatenar SQL), validação de entrada com DTOs, regras gerenciadas de SQL injection no WAF
A04 Design inseguro	Golpe da falsa central, Pix por engano	Confirmação falada do recebedor (RF009), limites para destinatário novo e noturno (RF019), "bloquear tudo agora" (RF020), atendimento só de dentro do app (RF024)
A05 Configuração insegura	Porta do banco exposta, `/docs` em produção, cabeçalho revelando framework	Banco e tarefas em subnet privada, `/docs` desligado em produção, cabeçalho `X-Powered-By` removido, container sem root e com disco só de leitura
A06 Componentes vulneráveis	Dependência npm com falha conhecida	`npm audit` e Trivy bloqueando alta e crítica no CI (RNF06), Dependabot abrindo PR de atualização toda semana
A07 Falhas de identificação e autenticação	Força bruta na senha de 6 números, roubo de sessão	Bloqueio após 5 erros (RF004), limite por IP no WAF, aparelho vinculado com assinatura, token curto, renovação de uso único com revogação em caso de reuso
A08 Falhas de integridade de software e dados	Webhook falso dizendo que um Pix chegou, imagem adulterada	Webhook com assinatura HMAC conferida e idempotente pelo `idEvento`, imagem com tag igual ao hash do commit e publicada só pelo pipeline, `package-lock.json` com `npm ci`
A09 Falhas de log e monitoramento	Golpe ou invasão percebidos só pela reclamação	Log estruturado com correlação em 100% das requisições (RNF13), auditoria de toda ação de funcionário por 5 anos (RNF14), alarmes do item 6.5
A10 SSRF	API buscar uma URL enviada pelo usuário (imagem da meta, áudio do chamado)	A API não baixa URL arbitrária: imagem e áudio sobem para o S3 por URL pré-assinada, e a saída da rede só vai para os domínios do parceiro e do Expo

6.5 Observabilidade

Três coisas: **log estruturado** para investigar, **métricas** técnicas e de negócio para acompanhar, e **alertas** com limiar ligado aos RNFs para avisar antes do cliente reclamar. Tudo no CloudWatch, já custeado no Bloco 1 (logs, 10 alarmes e 20 métricas de negócio).

Log estruturado

Uma linha JSON por evento, gerada pelo pino (`codigo/backend/src/compartilhado/log.ts`). Exemplo real tirado do compose em 04/11/2026:

```
{"level":30,"time":"2026-11-05T00:13:44.254Z","servico":"rufus-api","ambiente":"desenvolvimento","correlacaoId":"23ac0913-1d2a-4bf7-a357-b28a50b321e2","metodo":"GET","rota":"/health","status":200,"duracaoMs":2.336995,"msg":"requisicao"}
```

Regra	Como
Identificador de correlação em 100% das requisições (RNF13)	O middleware usa o `X-Correlation-Id` recebido ou gera um, devolve na resposta e põe em toda linha. O app envia o mesmo id para o chamado do SAC, então o atendente acha o erro do cliente
Nenhum dado pessoal no log (LGPD)	CPF, senha, token, celular e o cabeçalho Authorization são apagados pela lista `redact` do pino, mesmo se alguém os passar por engano
Campos fixos	`servico` , `ambiente` , `correlacaoId` , `rota` , `status` , `duracaoMs`
Retenção	90 dias no CloudWatch Logs. Auditoria de funcionário fica no banco por 5 anos (RNF14), não no log

Métricas

Técnicas

Métrica	Fonte	Para quê
Latência p95 por rota	ALB (TargetResponseTime) e log da API	RNF01: abaixo de 300 ms
Taxa de erro 5xx	ALB	RNF13: alerta acima de 2%
Disponibilidade externa	Health check do Route 53 no `/health` , de fora da AWS (pendente de entrar no script de custo)	RNF03: 99,9% ao mês
CPU e memória por tarefa	ECS	Auto scaling e dimensionamento
Conexões e CPU do banco, espaço livre	RDS	Saber a hora de trocar a instância
Idade do evento mais antigo pendente no outbox	Consulta do worker publicada como métrica	Worker parado ou atrasado
Circuit breaker do parceiro aberto	API	Parceiro BaaS fora do ar

De negócio (as 20 métricas personalizadas do Bloco 1)

Métrica	Por quê
Pix enviados, concluídos e recusados por minuto	Saúde do fluxo principal
Tempo do toque em "confirmar" até a resposta do Pix (p95)	RNF02: até 10 s
Pix retidos pelo antifraude e tempo até a decisão do analista	RF026 e RF027
Uso do "bloquear tudo agora" por hora	Pico indica onda de golpe (R01)
Contas abertas por dia e cadastros abandonados por etapa	Adesão (R06) e telas difíceis (R03)
Uso do botão "Ouvir" por tela	Quais telas mais precisam de áudio (RNF09)
Diferença na conciliação diária com o parceiro	RNF11: tem de ser zero

Métrica	Por quê
Pedidos de aprovação do responsável sem resposta em 24 h	Fluxo da Camila (RF016)

Alertas

Os 10 alarmes do custo do Bloco 1. Cada um tem o limiar tirado de um RNF e vai para o canal de plantão (e-mail e push do celular do responsável da semana).

#	Alarme	Limiar	Janela	RNF
1	API fora do ar	`/health` externo falhando 2 vezes seguidas	2 min	RNF03
2	Taxa de erro alta	5xx acima de 2% das requisições	5 min	RNF13
3	API lenta	p95 acima de 300 ms nas rotas de leitura	10 min	RNF01
4	Pix lento	p95 do Pix acima de 10 s	10 min	RNF02
5	Parceiro BaaS fora	Circuit breaker aberto	2 min	RNF02
6	Outbox atrasado	Evento pendente mais antigo com mais de 5 min	5 min	ADR-008
7	Banco no limite	CPU acima de 80% ou espaço livre abaixo de 10 GB	15 min	RNF04
8	Divergência na conciliação	Qualquer diferença diferente de zero	diário	RNF11
9	Onda de golpe	"Bloquear tudo agora" ou Pix retidos 3 vezes acima da média da mesma hora na semana anterior	30 min	R01
10	Custo	Fatura projetada acima de 80% do orçamento do mês (AWS Budgets)	diário	R05

Os alarmes 1 a 6 acordam o plantão a qualquer hora. Os outros abrem tarefa para o próximo dia útil, menos o 9, que avisa também a equipe antifraude.

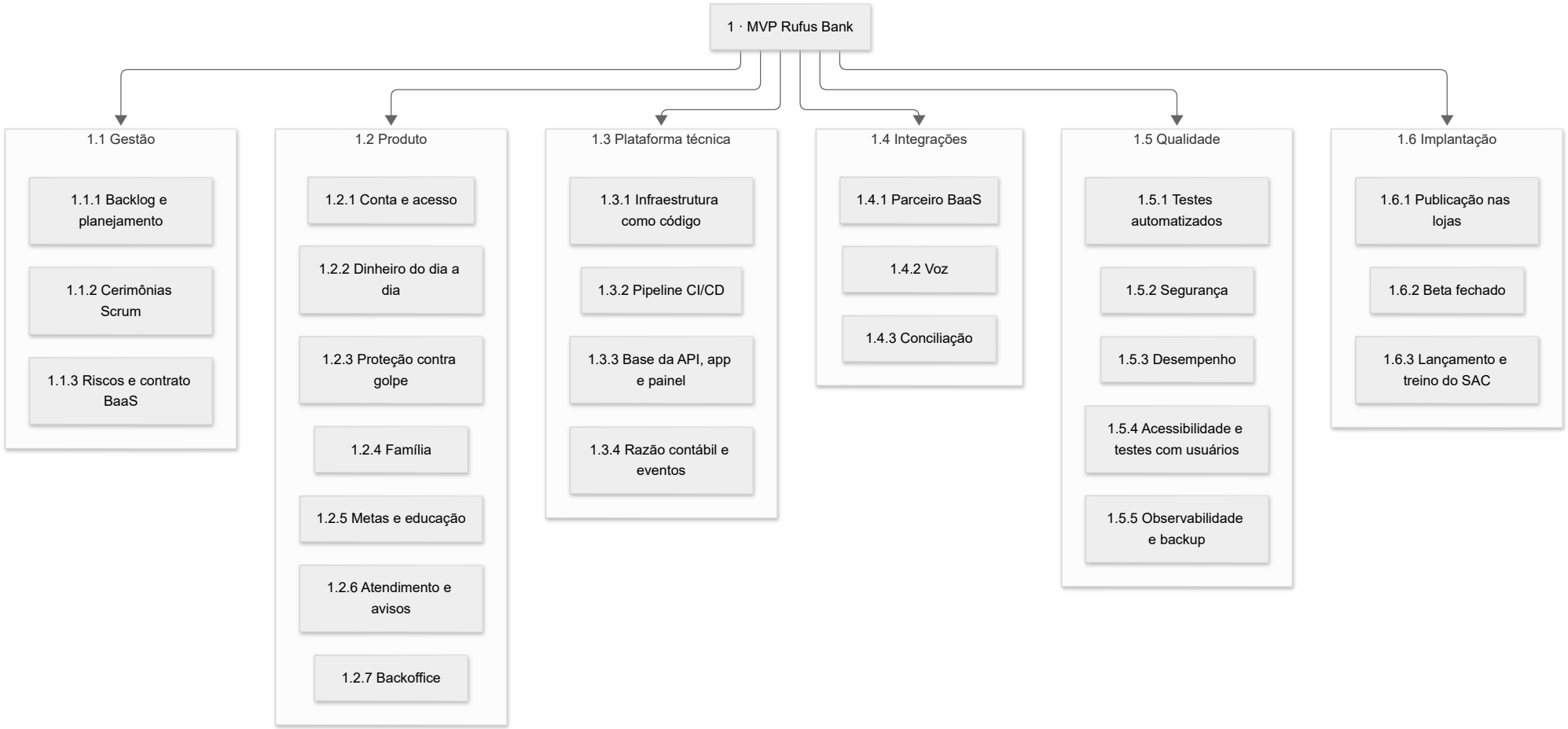
Bloco 7 · Planejamento e Estimativas

7.1 WBS (Work Breakdown Structure)

A WBS divide o MVP em **6 grupos de entrega**, 25 pacotes de trabalho e, no 4º nível, as 24 histórias do Bloco 2 e as 20 tarefas técnicas. São 4 níveis de decomposição; o enunciado pede no mínimo 3.

WBS do MVP do Rufus Bank (níveis 1 a 3)

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



Fonte do diagrama: diagramas/wbs.mmd. O diagrama mostra os níveis 1 a 3. O nível 4 está na tabela abaixo.

Nível 4: o que tem dentro de cada pacote

Os pontos são os mesmos das histórias (Bloco 2) e das tarefas técnicas (02-estimativas.md).

Nível 2	Nível 3	Nível 4 (itens)	SP
1.1 Gestão	1.1.1 Backlog e planejamento	Refinamento semanal, priorização MoSCoW, replanejamento depois do sprint 2	Sem SP
	1.1.2 Cerimônias Scrum	Planning, daily, review e retrospectiva de cada sprint (bloco-9-qualidade/06-metodologia.md)	Sem SP
	1.1.3 Riscos e contrato BaaS	Revisão quinzenal da matriz de riscos, negociação com 2 parceiros (R04)	Sem SP
1.2 Produto	1.2.1 Conta e acesso	US01 Abrir conta, US02 Entrar com digital, US03 Trocar de celular, US17 Encerrar conta e baixar dados	23
	1.2.2 Dinheiro do dia a dia	US04 Ouvir saldo, US05 Receber por QR Code, US06 Enviar Pix, US07 Pix sem duplicar, US08 Pagar boleto, US21 Comprovante	32
	1.2.3 Proteção contra golpe	US11 Limite para destinatário novo e à noite, US12 Bloquear tudo, US13 Contato de confiança, US16 Analisar Pix retido	17
	1.2.4 Família	US09 Conta da Camila, US10 Limite do responsável, US18 Pedir aprovação	16
	1.2.5 Metas e educação	US19 Meta do notebook, US23 Lembrete de conta, US24 Dicas em áudio	10
	1.2.6 Atendimento e avisos	US14 Chat no app, US15 Painel do SAC, US20 Aviso de movimentação	13
	1.2.7 Backoffice	US22 Cadastrar funcionário com papel	3
1.3 Plataforma técnica	1.3.1 Infraestrutura como código	T02 Terraform de rede, T03 Terraform de containers e banco	16
	1.3.2 Pipeline CI/CD	T04 CI, T05 CD	10
	1.3.3 Base da API, app e painel	T01 Repositório e padrões, T06 Esqueleto da API, T12 Esqueleto do app, T13 Esqueleto do painel	21
	1.3.4 Razão contábil e eventos	T07 Razão com partidas dobradas, T08 Outbox e worker	13
1.4 Integrações	1.4.1 Parceiro BaaS	T09 Porta e adaptador falso, T10 Adaptador real e webhooks	13
	1.4.2 Voz	T11 Polly, cache e voz do celular	5
	1.4.3 Conciliação	T18 Conciliação diária	5
1.5 Qualidade	1.5.1 Testes automatizados	Dentro da Definition of Done de cada item, sem SP próprio (bloco-9-qualidade/02-estrategia-de-testes.md)	Sem SP
	1.5.2 Segurança	T15 Revisão OWASP e teste de invasão	8
	1.5.3 Desempenho	T16 Teste de carga k6	3
	1.5.4 Acessibilidade e testes com usuários	T17 Auditoria WCAG. Os testes de uso de cada sprint estão na DoD	5
	1.5.5 Observabilidade e backup	T14 Painéis e alarmes, T20 Teste de restauração	5
1.6 Implantação	1.6.1 Publicação nas lojas	T19 Contas e revisão Apple e Google	3
	1.6.2 Beta fechado	50 pessoas do perfil das personas no sprint 8	Reserva do sprint 8
	1.6.3 Lançamento e treino do SAC	Liberação gradual e treino do SAC com os golpes mais comuns	Reserva do sprint 8
Total		24 histórias e 20 tarefas técnicas	**221**

Conferência: Produto soma 114 SP (igual ao Bloco 2) e Plataforma, Integrações, Qualidade e Implantação somam 107 SP (igual às tarefas técnicas do item 7.2).

7.2 Estimativa de esforço e cronograma

> Arquivo gerado por `ferramentas/planejamento.py`. Não editar à mão: mude o plano no script e rode de novo.

O MVP soma **221 story points**: 114 das 24 histórias do Bloco 2 e 107 de tarefas técnicas. Com a equipe do item 7.3, isso cabe em **9 sprints de 2 semanas**, de 16/11/2026 a 19/03/2027 (18 semanas). O último sprint não tem item novo: é a reserva para o beta fechado e as correções.

Velocidade da equipe

- Quem produz story points: 2 devs de back-end, 2 devs mobile e o DevOps em meio período, **4,5 pessoas equivalentes**. PO, QA e UX não pontuam: o trabalho deles está dentro da Definition of Done de cada história.
- Velocidade de referência: **34 SP por sprint** com a equipe completa e 10 dias úteis, cerca de 7,6 pontos por pessoa por sprint. A referência de tamanho é a US13 = 3 pontos (Bloco 2).
- Rampa: o sprint 0 rende 60% (montagem do ambiente, equipe nova) e o sprint 1 rende 80%.
- Feriados e pontos facultativos reduzem a capacidade na proporção dos dias úteis perdidos.
- A velocidade é uma hipótese. Depois do sprint 2 ela é recalculada pela média dos sprints anteriores e o plano é refeito.

Sprints

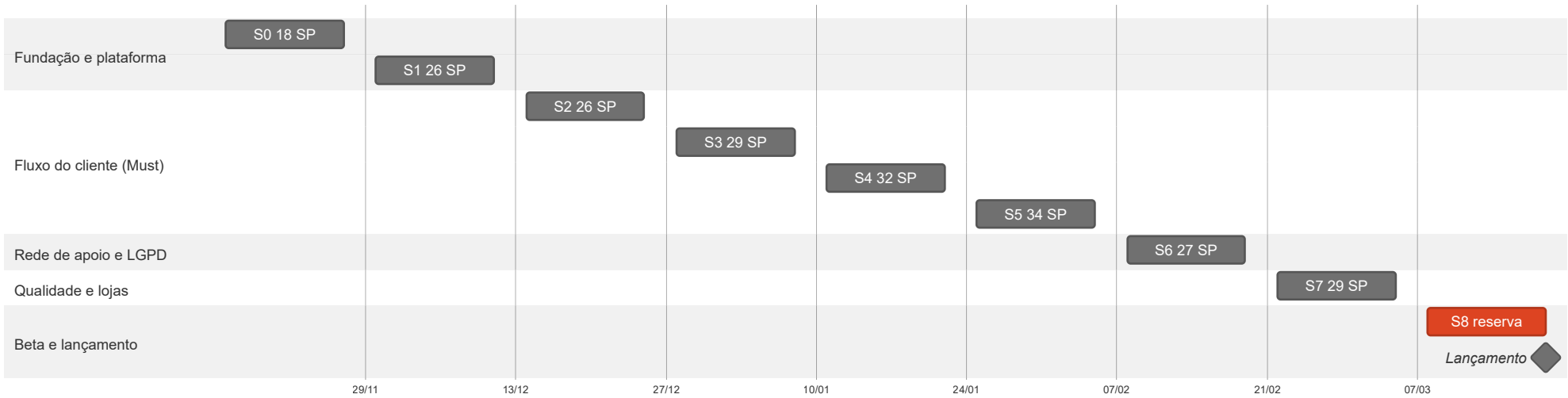
Sprint	Período	Dias úteis	Capacidade (SP)	Planejado (SP)	Uso	Objetivo
0	16/11/2026 a 27/11/2026	9	18,4	18	98%	Fundação: repositório, pipeline de CI, esqueleto da API e parceiro falso
1	30/11/2026 a 11/12/2026	10	27,2	26	96%	Antônio entra com a digital num app que fala
2	14/12/2026 a 25/12/2026	9	30,6	26	85%	Antônio abre a conta sozinho e ouve o saldo
3	28/12/2026 a 08/01/2027	9	30,6	29	95%	Enviar e receber Pix sem duplicar, com infraestrutura de homologação e produção
4	11/01/2027 a 22/01/2027	10	34,0	32	94%	Boleto, proteção contra golpe e primeira fila do antifraude
5	25/01/2027 a 05/02/2027	10	34,0	34	100%	Conta da Camila e atendimento humano, já com o parceiro real
6	08/02/2027 a 19/02/2027	8	27,2	27	99%	Rede de apoio (contato de confiança, aprovação) e direitos LGPD
7	22/02/2027 a 05/03/2027	10	34,0	29	85%	Segurança, carga, acessibilidade e publicação nas lojas
8	08/03/2027 a 19/03/2027	10	34,0	0	reserva	Beta fechado com 50 pessoas do perfil, correções e lançamento
Total			**270,0**	**221**		

Nos sprints com itens (0 a 7), a capacidade é de 236,0 SP para 221 planejados: uso de 94%, com folga para imprevisto. O sprint 8 inteiro é reserva.

Feriados considerados: 20/11/2026 Dia da Consciência Negra (feriado nacional); 25/12/2026 Natal; 01/01/2027 Confraternização Universal; 08/02/2027 Carnaval (segunda, ponto facultativo); 09/02/2027 Carnaval (terça, ponto facultativo); 26/03/2027 Sexta-feira Santa.

Cronograma do MVP em sprints de 2 semanas

Rufus Bank · Autores: Pedro Oliveira, Izaias Terto, Carlos Eduardo · 04/11/2026



O que entra em cada sprint

Ordem: primeiro o que destrava os outros (plataforma, razão contábil, parceiro falso), depois os Must na ordem do fluxo do cliente, depois os Should e por fim os Could. Todo Must termina até o sprint 6.

Sprint 0 · 16/11 a 27/11/2026 · Fundação: repositório, pipeline de CI, esqueleto da API e parceiro falso

Item	Descrição	Tipo	Prioridade	SP
T01	Repositório, padrões de código, lint e estrutura de módulos	Plataforma	Técnica	3
T06	Esqueleto da API NestJS: módulos, configuração, /health, log pino com correlação	Plataforma	Técnica	5
T04	Pipeline de CI: lint, testes, build e varredura de segurança	Plataforma	Técnica	5
T09	Porta bancária e adaptador falso do parceiro BaaS	Integrações	Técnica	5
	Total			**18**

Sprint 1 · 30/11 a 11/12/2026 · Antônio entra com a digital num app que fala

Item	Descrição	Tipo	Prioridade	SP
T12	Esqueleto do app Expo: navegação, tema acessível, botão Ouvir, cliente gerado do OpenAPI	Plataforma	Técnica	8
T07	Migrações e razão contábil: carteiras, lançamentos e gatilho de soma zero	Plataforma	Técnica	8
US02	Antônio: Entrar com a digital	Produto	Must	5
T11	Porta de voz: Amazon Polly, cache no S3 e voz do celular sem internet	Integrações	Técnica	5
	Total			**26**

Sprint 2 · 14/12 a 25/12/2026 · Antônio abre a conta sozinho e ouve o saldo

Item	Descrição	Tipo	Prioridade	SP
US01	Antônio: Abrir a conta ouvindo cada passo	Produto	Must	8
US04	Antônio: Ouvir o saldo e o extrato	Produto	Must	5
T08	Outbox transacional e worker com SKIP LOCKED e nova tentativa	Plataforma	Técnica	5
T02	Terraform: VPC, subnets, NAT, ALB e WAF	Plataforma	Técnica	8
	Total			**26**

Sprint 3 · 28/12 a 08/01/2027 · Enviar e receber Pix sem duplicar, com infraestrutura de homologação e produção

Item	Descrição	Tipo	Prioridade	SP
T03	Terraform: ECS Fargate, RDS Multi-AZ, Secrets Manager e ECR (homologação e produção)	Plataforma	Técnica	8
T05	Pipeline de CD: homologação, smoke test, aprovação e produção	Plataforma	Técnica	5
US06	Dona Marlene: Enviar Pix ouvindo para quem vai	Produto	Must	8
US07	Antônio: Não mandar Pix em dobro	Produto	Must	3
US05	Antônio: Receber a diária por QR Code	Produto	Must	5
	Total			**29**

Sprint 4 · 11/01 a 22/01/2027 · Boleto, proteção contra golpe e primeira fila do antifraude

Item	Descrição	Tipo	Prioridade	SP
US08	Dona Marlene: Pagar boleto pela câmera	Produto	Must	8

Item	Descrição	Tipo	Prioridade	SP
US11	Antônio: Ter limite menor com destinatário novo e à noite	Produto	Must	3
US12	Dona Marlene: Bloquear tudo na hora	Produto	Must	3
US03	Dona Marlene: Trocar de celular com selfie	Produto	Must	5
T13	Esqueleto do painel interno (React e Vite) com login e TOTP	Plataforma	Técnica	5
US16	Analista antifraude: Analisar Pix retido	Produto	Must	8
	Total			**32**

Sprint 5 · 25/01 a 05/02/2027 · Conta da Camila e atendimento humano, já com o parceiro real

Item	Descrição	Tipo	Prioridade	SP
T10	Adaptador real do parceiro BaaS (sandbox) e webhooks com HMAC	Integrações	Técnica	8
US09	Camila: Abrir conta com aceite do pai	Produto	Must	8
US10	Responsável da Camila: Definir limite da conta da filha	Produto	Must	3
US14	Antônio: Falar com uma pessoa pelo chat do app	Produto	Must	5
US15	Atendente do SAC: Responder chamado vendo o histórico	Produto	Must	5
T18	Conciliação diária com o parceiro (RNF11)	Integrações	Técnica	5
	Total			**34**

Sprint 6 · 08/02 a 19/02/2027 · Rede de apoio (contato de confiança, aprovação) e direitos LGPD

Item	Descrição	Tipo	Prioridade	SP
US17	Antônio: Encerrar a conta e baixar meus dados	Produto	Must	5
US13	Dona Marlene: Cadastrar o neto como contato de confiança	Produto	Should	3
US18	Camila: Pedir aprovação acima do limite	Produto	Should	5
US20	Camila: Receber aviso de cada movimentação	Produto	Should	3
US21	Dona Marlene: Ouvir e compartilhar o comprovante	Produto	Should	3
US22	Administrador: Cadastrar funcionário com papel	Produto	Should	3
US19	Camila: Criar a meta do notebook	Produto	Should	5
	Total			**27**

Sprint 7 · 22/02 a 05/03/2027 · Segurança, carga, acessibilidade e publicação nas lojas

Item	Descrição	Tipo	Prioridade	SP
T15	Segurança: revisão OWASP, teste de invasão externo e correções	Qualidade	Técnica	8
T16	Teste de carga com k6 (RNF04)	Qualidade	Técnica	3
T17	Auditoria de acessibilidade WCAG 2.2 AA (RNF09)	Qualidade	Técnica	5
T20	Teste de restauração de backup (RNF12)	Qualidade	Técnica	2
T14	Observabilidade: painéis e alarmes do RNF01, RNF03 e RNF13	Qualidade	Técnica	3
US23	Dona Marlene: Lembrete de conta a vencer	Produto	Could	3
US24	Camila: Ouvir dicas de dinheiro	Produto	Could	2
T19	Publicação nas lojas: contas, revisão da Apple e do Google	Implantação	Técnica	3
	Total			**29**

Sprint 8 · 08/03 a 19/03/2027 · Beta fechado com 50 pessoas do perfil, correções e lançamento

Sem item novo. Beta fechado com 50 pessoas do perfil das personas, correção do que aparecer, lançamento controlado.

Tarefas técnicas

ID	Tarefa	Grupo da WBS	SP	Sprint
T01	Repositório, padrões de código, lint e estrutura de módulos	Plataforma	3	0
T02	Terraform: VPC, subnets, NAT, ALB e WAF	Plataforma	8	2
T03	Terraform: ECS Fargate, RDS Multi-AZ, Secrets Manager e ECR (homologação e produção)	Plataforma	8	3
T04	Pipeline de CI: lint, testes, build e varredura de segurança	Plataforma	5	0
T05	Pipeline de CD: homologação, smoke test, aprovação e produção	Plataforma	5	3
T06	Esqueleto da API NestJS: módulos, configuração, /health, log pino com correlação	Plataforma	5	0
T07	Migrações e razão contábil: carteiras, lançamentos e gatilho de soma zero	Plataforma	8	1
T08	Outbox transacional e worker com SKIP LOCKED e nova tentativa	Plataforma	5	2
T09	Porta bancária e adaptador falso do parceiro BaaS	Integrações	5	0
T10	Adaptador real do parceiro BaaS (sandbox) e webhooks com HMAC	Integrações	8	5
T11	Porta de voz: Amazon Polly, cache no S3 e voz do celular sem internet	Integrações	5	1
T12	Esqueleto do app Expo: navegação, tema acessível, botão Ouvir, cliente gerado do OpenAPI	Plataforma	8	1
T13	Esqueleto do painel interno (React e Vite) com login e TOTP	Plataforma	5	4
T14	Observabilidade: painéis e alarmes do RNF01, RNF03 e RNF13	Qualidade	3	7
T15	Segurança: revisão OWASP, teste de invasão externo e correções	Qualidade	8	7
T16	Teste de carga com k6 (RNF04)	Qualidade	3	7
T17	Auditoria de acessibilidade WCAG 2.2 AA (RNF09)	Qualidade	5	7
T18	Conciliação diária com o parceiro (RNF11)	Integrações	5	5
T19	Publicação nas lojas: contas, revisão da Apple e do Google	Implantação	3	7
T20	Teste de restauração de backup (RNF12)	Qualidade	2	7
	Total		**107**	

Histórias por prioridade

Prioridade	Histórias	SP	Último sprint
Must	16	87	6
Should	6	22	6
Could	2	5	7

7.3 Composição da equipe e custo do projeto

> Arquivo gerado por `ferramentas/planejamento.py`. Não editar à mão.

A equipe tem **8 pessoas** e custa **R$ 132.664,11 por mês** com encargos. O projeto dura 4,05 meses (16/11/2026 a 19/03/2027) e custa **R$ 572.922,39** no total, somando equipe, infraestrutura do Bloco 1, serviços fora da AWS e custos únicos (conta do Google Play e teste de invasão).

Pesquisa salarial: de onde vêm os números

- **Glassdoor (exigido no enunciado):** tentamos abrir as páginas de salário do glassdoor.com.br em 04/11/2026 e o site recusou o acesso automático (HTTP 403, exige login). Por isso nenhum valor desta tabela é do Glassdoor. A coluna de conferência abaixo fica para o grupo preencher abrindo o Glassdoor logado.
- **Robert Half, Guia Salarial 2027:** fonte principal. Usamos a **mediana (50º percentil) de Recife**, a cidade com página no guia mais próxima de João Pessoa, onde a equipe trabalha.
- **Designer UX/UI:** o guia da Robert Half não tem o cargo em Recife. Usamos a média do Novo CAGED para Designer de UX em Recife (Salário.com.br). É a média de todos os níveis com carteira assinada e provavelmente fica abaixo de um pleno: é o número mais fraco da tabela.

Perfil	Salário mensal (R$)	Fonte	Glassdoor (conferência do grupo)
Product Owner	8.750,00	Robert Half, Guia Salarial 2027, Product Owner - PO, Recife, 50º percentil	a preencher
Tech Lead (Back-End Sênior)	14.500,00	Robert Half, Guia Salarial 2027, Desenvolvedor(a) Back-End Sênior, Recife, 50º percentil	a preencher
Desenvolvedor Back-End Pleno	11.800,00	Robert Half, Guia Salarial 2027, Desenvolvedor(a) Back-End Pleno, Recife, 50º percentil	a preencher
Desenvolvedor Mobile Pleno (React Native)	13.800,00	Robert Half, Guia Salarial 2027, Desenvolvedor(a) Mobile Pleno, Recife, 50º percentil	a preencher
Analista de DevOps	14.600,00	Robert Half, Guia Salarial 2027, Analista de DevOps, Recife, 50º percentil	a preencher
Analista de Testes Pleno (QA)	9.100,00	Robert Half, Guia Salarial 2027, Analista de Testes Pleno, Recife, 50º percentil	a preencher
Designer UX/UI	3.344,38	Salário.com.br, Designer de UX (CBO 2624-10), Recife-PE, média com dados do Novo CAGED, atualizado em 08/10/2026	a preencher

Perfis, quantidade e por quê

Perfil	Qtd.	Alocação	O que faz no projeto
Product Owner	1	100%	Backlog, prioridade MoSCoW, aceite das histórias, contato com o parceiro BaaS
Tech Lead (Back-End Sênior)	1	100%	Arquitetura, revisão de código, módulos de identidade, contas e pagamentos
Desenvolvedor Back-End Pleno	1	100%	Módulos de família, antifraude, atendimento, notificação e integrações
Desenvolvedor Mobile Pleno (React Native)	2	100%	App Android e iOS, acessibilidade, voz, câmera e biometria
Analista de DevOps	1	50%	Terraform, pipeline, observabilidade, backup. Meio período: o trabalho pesado é no início e no lançamento
Analista de Testes Pleno (QA)	1	100%	Testes automatizados de integração e E2E, teste de carga, testes com usuários junto com o UX
Designer UX/UI	1	100%	Telas, linguagem simples, roteiro de áudio, teste de uso com pessoas do perfil a cada sprint

Por que este tamanho: o ADR-001 (monolito modular) foi pensado para **2 devs de back-end**, e o app em React Native (ADR-005) para **2 devs mobile** cobrirem Android e iOS. Sem QA dedicado, os testes de integração e com usuários (RNF08) ficariam com os devs e a velocidade cairia. O UX é tempo integral porque a interface é o produto: cada tela passa por teste de uso com pessoas do perfil das personas.

Encargos CLT: a conta

Empresa no Lucro Presumido ou Real, sem a desoneração da folha (CPRB). Encargos sobre o salário:

Encargo	Alíquota	Base legal
INSS patronal	20,0%	Lei 8.212/1991, art. 22, I
RAT/SAT (risco leve, FAP 1,0)	1,0%	Lei 8.212/1991, art. 22, II, a. Desenvolvimento de software é grau de risco leve
Terceiros (Sistema S, salário-educação, INCRA)	5,8%	Contribuições de terceiros da guia do INSS
FGTS	8,0%	Lei 8.036/1990, art. 15
Total de encargos	**34,8%**	

Provisões mensais:

Provisão	% do salário por mês	Conta
13º salário	8,33%	1 salário por ano, dividido em 12 meses
Férias + 1/3	11,11%	1 salário + 1/3 por ano, dividido em 12 meses
Total de provisões	**19,44%**	

Os encargos também incidem sobre o 13º e as férias, então o fator é:

custo mensal = salário x (1 + 0,1944) x (1 + 0,348) = salário x 1,6101

Custo mensal da equipe

Perfil	Qtd.	Alocação	Salário (R$)	Custo mensal com encargos (R$)
Product Owner	1	100%	8.750,00	14.088,47
Tech Lead (Back-End Sênior)	1	100%	14.500,00	23.346,61
Desenvolvedor Back-End Pleno	1	100%	11.800,00	18.999,31
Desenvolvedor Mobile Pleno (React Native)	2	100%	13.800,00	44.439,07
Analista de DevOps	1	50%	14.600,00	11.753,81
Analista de Testes Pleno (QA)	1	100%	9.100,00	14.652,01
Designer UX/UI	1	100%	3.344,38	5.384,82
Total	**8**			**132.664,11**

Custo total do projeto

Item	Conta	R$
Equipe	R$ 132.664,11 x 4,05 meses	537.945,66
Infra de homologação (Bloco 1)	R$ 1.136,64 x 4,05 meses	4.609,01
Infra de produção (Bloco 1), a partir do sprint 7	R$ 3.790,14 x 0,82 meses	3.123,74
Serviços fora da AWS (Bloco 1, item 4.4: Expo, GitHub, domínio, conta Apple)	R$ 519,62 x 4,05 meses	2.107,03
Custos únicos (Bloco 1, item 4.4: conta do Google Play e teste de invasão)	valor único	25.136,94

Item	Conta	R$
Total		**572.922,39**

Meses = semanas do projeto x 12 / 52. Câmbio: a mesma PTAX do Bloco 1.

Fora deste valor (precisa de cotação)

- Benefícios (vale-refeição, plano de saúde) e provisão de multa rescisória do FGTS.
- Cotação formal do teste de invasão (tarefa T15): o total usa o teto da faixa de mercado, R$ 25.000.
- Tarifa do parceiro BaaS por conta e por transação: depende de contrato (Bloco 1).
- Equipamentos, licenças de software e espaço de trabalho.

Depois do lançamento, a operação custa por mês a infraestrutura do Bloco 1 (R$ 5.446,39) mais a equipe que ficar para evoluir o produto.

Conferência com a referência da disciplina (Aula 01)

A Aula 01 traz faixas de custo por tipo de software e três regras práticas. O Rufus conferido contra elas:

Referência da Aula 01	No Rufus
Sistema complexo (app mobile, alta segurança): R$ 150 mil a R$ 500 mil	R$ 572.922,39, um pouco acima do teto. Faz sentido: é app em Android e iOS (a aula diz que multiplataforma dobra ou triplica o esforço), tem painéis internos na web (SAC, antifraude e administração) e integração com parceiro BaaS (a aula diz que integrações de API somam de 20% a 30%)
Manutenção pós-lançamento: 15% a 20% do custo inicial por ano	De R$ 85.938,36 a R$ 114.584,48 por ano, além da infraestrutura mensal
Cerca de 60% do custo em desenvolvimento e 40% em teste	O teste não fica só com o QA: entra na Definition of Done de cada item (Bloco 9), com cobertura mínima no pipeline. A referência é o motivo de o QA ser dedicado desde o sprint 0, e não cortado para baratear
Corrigir depois de ir ao ar custa muito mais do que testar antes	Teste de uso a cada sprint (RNF08), teste de invasão antes do lançamento e o sprint 8 inteiro reservado para o beta

7.4 Matriz de riscos

12 riscos, de 5 categorias. Os 8 primeiros vêm do Bloco 1 (mesmo ID, mesma nota). Os 4 novos (R09 a R12) apareceram ao montar o cronograma e a equipe. Escala igual à do Bloco 1: probabilidade e impacto de 1 a 3, severidade = probabilidade x impacto.

Riscos

ID	Risco	Categoria	Prob.	Impacto	Sev.	Mitigação	Dono	Gatilho de ação
R01	Fraudes e golpes contra usuários vulneráveis	Segurança	3	3	**9**	Confirmação falada do recebedor (US06), limite para destinatário novo e à noite (US11), bloquear tudo (US12), contato de confiança (US13), fila do antifraude (US16), aviso "o Rufus nunca liga"	PO e Tech Lead	Contestações de Pix acima de 0,5% dos envios no mês
R06	Baixa adesão	Negócio	3	3	**9**	Conta sem tarifa, indicação pela família, parceria com associações e com quem paga diária. Beta com 50 pessoas no sprint 8 para medir antes de lançar	PO	Menos de 30 dos 50 do beta usando por 2 semanas seguidas
R02	Vazamento de dados pessoais	Segurança	2	3	**6**	Cifra de CPF e data de nascimento, segredos no Secrets Manager, WAF, 2 fatores no painel, log sem dado pessoal, teste de invasão (T15)	Tech Lead e DevOps	Achado crítico ou alto no teste de invasão ou na varredura do pipeline
R03	Usuários não conseguem usar o app	Negócio	2	3	**6**	Teste de uso com 5 pessoas do perfil a cada sprint (RNF08), áudio em toda tela, auditoria WCAG (T17)	UX	Menos de 4 de 5 pessoas pagando boleto sozinhas em até 3 minutos
R04	Não conseguir parceria regulatória (BaaS)	Regulatório	2	3	**6**	Negociar com 2 parceiros. Porta e adaptador (ADR-002) com parceiro falso até o sprint 4, adaptador real no sprint 5 (T10)	PO	Nenhum contrato assinado até o fim do sprint 3 (08/01/2027)
R08	Erro no tratamento de dados de menores e biometria	Regulatório	2	3	**6**	Consentimento do responsável com versão do termo, biometria só no celular, campos marcados no dicionário de dados	Tech Lead	Pedido de titular sem resposta em 15 dias (RNF07)
R09	Sandbox do parceiro BaaS instável ou diferente da produção	Técnico	2	3	**6**	Testes de contrato contra o sandbox no pipeline, adaptador falso para os testes do dia a dia, conciliação (T18) desde o primeiro Pix real	Dev Back-End	Mais de 2 diferenças entre sandbox e documentação do parceiro no mesmo sprint
R10	Saída de pessoa-chave (Tech Lead ou dev mobile)	Equipe	2	3	**6**	Revisão de código obrigatória (ninguém é o único que conhece um módulo), ADRs e README do Bloco 8, pareamento nas partes críticas (razão, identidade)	Tech Lead	Aviso de saída. Plano: redistribuir e usar a reserva do sprint 8
R05	Custo de infraestrutura maior que o esperado	Técnico	2	2	**4**	Alerta de orçamento em 80%, tag por ambiente, homologação desligada fora do horário, cache de áudio	DevOps	Gasto parcial acima de 80% do previsto antes do dia 20 do mês
R07	Equipe sem experiência em acessibilidade e produto financeiro	Equipe	2	2	**4**	Guia de acessibilidade no padrão de código, revisão de acessibilidade na DoD, UX em tempo integral	UX	Tela reprovada no teste de uso por acessibilidade em 2 sprints seguidos

ID	Risco	Categoria	Prob.	Impacto	Sev.	Mitigação	Dono	Gatilho de ação
R11	Mudança nas regras do Pix pelo Banco Central durante o projeto	Regulatório	2	2	**4**	Limites e regras de horário configuráveis, não fixos no código. Acompanhar as resoluções do BC na revisão quinzenal de riscos	PO	Publicação de resolução do BC que afete limites, horários ou devolução do Pix
R12	Revisão da Apple ou do Google atrasar a publicação	Negócio	2	2	**4**	Primeira versão de teste enviada ao TestFlight e ao teste interno do Google Play no sprint 4 para conhecer a revisão cedo, publicação no sprint 7 com o sprint 8 de folga	Dev Mobile	App recusado na revisão

Matriz

	Impacto 1	Impacto 2	Impacto 3
Probabilidade 3			R01, R06
Probabilidade 2		R05, R07, R11, R12	R02, R03, R04, R08, R09, R10
Probabilidade 1			

Cobertura por categoria

Categoria	Riscos
Técnico	R05, R09
Negócio	R03, R06, R12
Equipe	R07, R10
Segurança	R01, R02
Regulatório	R04, R08, R11

Como a matriz é usada

A matriz é revisada a cada 2 sprints, na retrospectiva. Risco com severidade 6 ou mais tem dono e gatilho, e o gatilho disparado vira item no próximo sprint. A reserva do sprint 8 (34 SP) é o colchão para os riscos que se confirmarem.

Bloco 8 · Documentação para o Dev

8.1 README do desenvolvedor (do clone ao localhost)

O README completo está em codigo/README.md, que é o primeiro arquivo que o dev abre. Ele cobre: o que instalar, os 4 comandos do clone ao sistema no ar, onde fica cada coisa, como rodar testes e lint, e os problemas comuns.

Resumo dos comandos:

```
git clone https://github.com/pethrus07/aps-projeto-final-rufus-bank.git
cd aps-projeto-final-rufus-bank/codigo
cp .env.example .env
docker compose up --build
```

Depois disso, http://localhost:3000/health responde e http://localhost:3000/docs mostra o contrato da API.

Prova de que funciona

Executado em 04/11/2026 numa máquina com Docker 29.6.2, a partir de um volume vazio (`docker compose down -v` antes):

```
$ curl -si http://localhost:3000/health
HTTP/1.1 200 OK
X-Correlation-Id: dd340018-b12b-400f-94c0-3a87972b7fba
{"status":"ok","banco":"ok","versao":"dev"}

$ curl -s -o /dev/null -w "docs %{http_code}\n" http://localhost:3000/docs/
docs 200
```

O Swagger UI em `/docs` carregou as **50 operações** do contrato. No log do banco, os dois arquivos do Bloco 5 rodaram na primeira subida:

```
banco-1 | running /docker-entrypoint-initdb.d/001_schema.sql
banco-1 | running /docker-entrypoint-initdb.d/002_seed_dev.sql
```

E o worker processou um evento de teste inserido no outbox:

```
$ psql -c "insert into rufus.outbox_evento (tipo, agregado_id, payload) values ('ExemploPing', gen_random_uuid(), '{}')"
worker-1 | {"level":30,...,"servico":"rufus-worker","eventoId":"1","msg":"ping recebido"}
$ psql -c "select count(*) filter (where processado_em is not null) from rufus.outbox_evento"
1
```

Testes e lint do esqueleto, no mesmo dia:

```
$ npm run lint        (sem erros)
$ npm run typecheck   (sem erros)
$ npm run test:cov
Tests:       12 passed, 12 total
All files    |   99.25 |    89.28 |   94.44 |   99.02 |
$ npm audit --omit=dev --audit-level=high
found 0 vulnerabilities
```

O que o dev encontra no dia 1

Já pronto	Entra no Sprint 1
API NestJS com `/health`, log estruturado e correlação	Casos de uso dos módulos (`bloco-8-dev/06-sprint-1.md`)
Os 8 módulos com pastas, `index.ts` público e README	App mobile com Expo (`codigo/mobile/README.md`)
Worker com o laço do outbox, nova tentativa e espera crescente	Painéis internos com Vite (`codigo/admin/README.md`)
Banco com o schema completo do Bloco 5 e dados de exemplo	Terraform da AWS e migrações versionadas
Contrato OpenAPI validado e navegável	Cliente da API gerado a partir do contrato
CI e CD em GitHub Actions, Dockerfiles, `.env.example`	

8.2 Variáveis de ambiente

Todas as variáveis estão em codigo/.env.example, com comentário. O dev copia para `.env` (que não vai para o git). Em homologação e produção os valores vêm das definições de tarefa do ECS e os segredos do Secrets Manager (Bloco 6, segurança).

A API valida as obrigatórias na partida: se faltar uma, ela não sobe e diz qual falta (`codigo/backend/src/compartilhado/config.ts`).

Variável	Obrigatória	Valor local	Em produção	Para quê
NODE_ENV	não	desenvolvimento	producao	Liga ou desliga `/docs` e ajusta o log
PORTA	não	3000	3000	Porta HTTP da API
LOG_NIVEL	não	debug	info	Nível do log
VERSAO	não	dev	hash do commit (pipeline)	Mostrada em `/health`. O smoke test confere
POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	só no Compose	rufus, rufus, rufus_local	não usadas	Criação do banco no container
DATABASE_URL	**sim**	postgres://rufus:rufus_local@banco:5432/rufus	segredo	Conexão da API e do worker
JWT_SEGREDO	**sim**	texto local longo	segredo, rotação a cada 90 dias	Assinatura do token de acesso (ADR-010)
JWT_ACESSO_MINUTOS	não	15	15	Validade do token de acesso (RNF05)
JWT_RENOVACAO_DIAS	não	30	30	Validade do token de renovação (RNF05)
LOGIN_TENTATIVAS_MAX	não	5	5	Senhas erradas antes do bloqueio (RF004)
LOGIN_BLOQUEIO_MINUTOS	não	30	30	Duração do bloqueio (RF004)
CHAVE_CIFRA_PII	sim no Sprint 1	chave local de teste	chave de dados do KMS	Cifra de CPF e data de nascimento (RNF07)
OUTBOX_INTERVALO_MS	não	1000	1000	Pausa entre leituras do outbox (ADR-008)
OUTBOX_LOTE	não	50	50	Eventos por leitura
BAAS_ADAPTADOR	não	falso	http	Escolhe o adaptador do parceiro (ADR-002)
BAAS_URL, BAAS_CLIENT_ID, BAAS_CLIENT_SECRET	só com `http`	vazio	segredo	Acesso ao parceiro
BAAS_WEBHOOK_SEGREDO	sim no Sprint 1	texto local	segredo	Confere a assinatura HMAC do webhook

Variável	Obrigatória	Valor local	Em produção	Para quê
BAAS_TIMEOUT_MS	não	8000	8000	Tempo máximo de espera do parceiro, dentro dos 10 s do RNF02
VOZ_ADAPTADOR	não	falso	polly	Voz falsa local ou Amazon Polly (ADR-011)
AWS_REGION	não	sa-east-1	sa-east-1	Região dos serviços AWS (ADR-013)
S3_BUCKET_AUDIOS	não	rufus-audios-local	rufus-audios-prd	Cache dos áudios
PUSH_ADAPTADOR	não	falso	expo	Push falso (só log) ou Expo
EXPO_ACCESS_TOKEN	só com `expo`	vazio	segredo	Envio de push
TOTP_EMISSOR	não	Rufus Bank (local)	Rufus Bank	Nome no app autenticador dos funcionários
OPENAPI_CAMINHO	não	definido no Compose	não usada	Arquivo do contrato para o `/docs`

Regra para variável nova: entra no `.env.example` com comentário e nesta tabela no mesmo pull request. O revisor recusa o PR se faltar uma das duas.

8.3 Guia de padrões de código

Regras que valem para todo código do Rufus. As que uma ferramenta consegue checar são checadas no CI (etapa 1, lint); as outras são conferidas na revisão do pull request.

Linguagem e nomes

Regra	Exemplo	Checado por
Domínio em português, igual ao modelo de domínio (`bloco-3-arquitetura/05-modelo-de-dominio.md`)	`Transacao` , `valorCentavos` , `ServicoEnvioPix`	Revisão
Termos técnicos consagrados ficam em inglês	`Controller` , `Repository` , `middleware`	Revisão
Classe e tipo em PascalCase, função e variável em camelCase, constante em MAIUSCULAS	`MotorAntifraude` , `processarLote` , `HORAS`	ESLint
Banco em snake_case, com os mesmos nomes do domínio	`valor_centavos` , `chave_idempotencia`	Revisão
Arquivo em kebab-case com o tipo no fim	`envio-pix.servico.ts` , `conta.repositorio.ts` , `pix.controller.ts`	Revisão
Código de erro em MAIUSCULAS_COM_SUBLINHADO, estável	`LIMITE_DESTINATARIO_NOVO`	Contrato OpenAPI

TypeScript

- Modo estrito ligado (`strict` , `noUncheckedIndexedAccess`). Proibido `any` (regra de lint como erro).
- **Dinheiro sempre em centavos inteiros.** Nunca `number` com casas decimais para valor. No banco, `bigint` . Na API, inteiro (ADR-004).
- Data e hora sempre em UTC (`timestamptz`). Conversão para o horário de Brasília só na tela.
- Sem `console.log` : use o `log` do pino (regra de lint como erro).
- Entrada da API validada por DTO antes de chegar ao caso de uso.

Arquitetura no código

Regra	Por quê
Cada módulo tem `dominio/` , `aplicacao/` , `infraestrutura/` e `http/`	Mesma estrutura em todo lugar (ADR-001)
Outro módulo só pelo `index.ts` público	Fronteira do monolito modular. **O lint bloqueia** import de camada interna de outro módulo
`dominio/` não importa NestJS, Kysely nem SDK de fornecedor	O domínio é testado sem banco e sem rede (ADR-002)
Tudo que é externo passa por uma porta (`PortaBancaria` , `PortaVoz` ...)	Trocar de fornecedor é escrever outro adaptador
Um módulo só escreve nas tabelas que são dele	Tabela de dono em `05-modelo-de-dominio.md`
Operação que move dinheiro: `UnidadeDeTrabalho` + chave de idempotência + evento no outbox, na mesma transação	RNF11 e ADR-008
Saldo nunca é atualizado: é soma de lançamentos	ADR-007
Consulta SQL com parâmetros, nunca texto concatenado	OWASP A03

Mensagens para o cliente

Toda mensagem que o cliente vê também é falada, então segue a régua de linguagem simples do produto (RNF08):

- Frase com até 15 palavras, na ordem direta.
- Dizer o que aconteceu e o que fazer: "A foto ficou escura, tente perto da janela."
- Sem jargão: "dinheiro que você tem" no lugar de "saldo disponível", "código" no lugar de "token".

- Erro de dinheiro diz se o dinheiro saiu ou não: "Seu dinheiro não saiu."

Log

- Uma linha JSON por evento, com `correlacaoId`.
- Nunca logar CPF, nome, celular, senha ou token. O `redact` do pino apaga esses campos, mas a regra é não mandá-los.
- Nível `error` só para o que alguém precisa olhar. Regra de negócio recusada (saldo, limite) é `info`.

Testes

- Todo caso de uso tem teste de unidade com repositório em memória.
- Razão, outbox e idempotência têm teste de integração contra PostgreSQL real.
- Nome do teste em português, descrevendo o comportamento: `it('não cria um segundo Pix com a mesma chave de idempotência')`.
- Cobertura mínima de 80% de linhas no CI. Metas por módulo no Bloco 9.

Formatação

ESLint com as regras de `codigo/backend/eslint.config.mjs`. Indentação de 2 espaços, aspas simples, ponto e vírgula, linha de até 120 caracteres. O editor formata ao salvar; o CI recusa o que o lint recusa.

8.4 Git workflow

Trunk-based com branches curtas: a `main` está sempre pronta para ir a produção, cada mudança nasce numa branch que vive no máximo 2 dias e entra por pull request revisado e com o CI verde.

Por que esse fluxo

Escolha	Alternativa descartada	Por quê
Trunk-based com branches curtas	GitFlow (develop, release, hotfix)	Com deploy contínuo (Bloco 6) não há "versão de release" para estabilizar. Branch longa gera conflito grande, e GitFlow tem 5 tipos de branch para uma equipe de 4 devs (Bloco 7)
Squash merge	Merge commit	Cada PR vira 1 commit na `main`, com mensagem no padrão. O histórico fica legível e fácil de reverter
Feature flag para o que não está pronto	Segurar a branch até terminar	O código entra cedo, desligado, e não envelhece fora da `main`

Branches

Tipo	Nome	Exemplo
Funcionalidade	`feat/US<número>-<resumo>`	`feat/US06-pix-com-confirmacao-falada`
Correção	`fix/<resumo>`	`fix/extrato-ordem-errada`
Tarefa técnica	`chore/<resumo>`	`chore/terraform-vpc`
Documentação	`docs/<resumo>`	`docs/adr-014-cache`

Criada a partir da `main` atualizada. Vida máxima de 2 dias; se passar disso, a história era grande demais e deve ser quebrada.

Commits: Conventional Commits

```
<tipo>(<módulo>): <o que mudou, no imperativo>

<por que, quando não for óbvio>

Refs: US06
```

Tipo	Quando
feat	Comportamento novo para o usuário
fix	Correção de defeito
refactor	Muda o código sem mudar o comportamento
test	Só testes
docs	Só documentação
chore	Build, dependências, infraestrutura
perf	Melhoria de desempenho

Exemplos: `feat(pagamentos): exige Idempotency-Key no envio de Pix`, `fix(contas): soma do saldo ignorava lançamento estornado`. Mudança que quebra o contrato da API leva `!` (`feat(pagamentos)!: ...`) e o rodapé `BREAKING CHANGE:`.

Pull request

1. Título no padrão do commit (vira a mensagem do squash).

2. Descrição: o que muda, a história (US) e o critério de aceite atendido, como testar, print da tela quando houver interface.
3. Checklist da Definition of Done (Bloco 9) marcada pelo autor.
4. Pelo menos **1 aprovação** de outra pessoa. Mudança em `identidade`, `contas`, `pagamentos` ou na infraestrutura exige **2 aprovações**, uma delas do Tech Lead.
5. CI verde nos 4 jobs (lint, testes, build, segurança).
6. Até 400 linhas alteradas. Maior que isso, dividir.

Proteção da main

Configurada no GitHub (Settings, Branches, regra para `main`):

- Proibido push direto, inclusive para administradores.
- Exige pull request com as aprovações acima e descarta aprovação quando chega commit novo.
- Exige os checks `1. lint e tipos`, `2. testes`, `3. build das imagens` e `4. segurança` passando, com a branch atualizada.
- Exige conversas resolvidas.
- Proíbe force push e exclusão da branch.
- `CODEOWNERS` define os revisores obrigatórios por pasta (`/codigo/backend/src/modulos/pagamentos/` para o Tech Lead, por exemplo).

Versionamento

Versão semântica (`MAJOR.MINOR.PATCH`) para a API e para o app:

Parte	Sobe quando
MAJOR	Quebra o contrato da API (commit com `!`). A API mantém `/v1` e cria `/v2` em paralelo
MINOR	Funcionalidade nova compatível (`feat`)
PATCH	Correção (`fix`)

Cada deploy em produção ganha uma tag `vX.Y.Z` no commit que subiu, e as notas da versão saem das mensagens de commit. O app mobile segue a mesma versão nas lojas; correções só de texto e acessibilidade vão por EAS Update sem nova versão na loja (ADR-005).

Hotfix

Mesmo fluxo, com prioridade: branch `fix/` a partir da `main`, PR com 2 aprovações, pipeline completo. Não existe atalho que pule testes ou segurança. Se a correção não puder esperar o pipeline, a saída é reverter o último deploy (reaplicar a task definition anterior), não subir código sem teste.

8.5 Contrato de API (OpenAPI 3.0)

O contrato está em openapi.yaml, no formato **OpenAPI 3.0.3**, e cobre as **50 rotas** da lista canônica (`bloco-3-arquitetura/06-rotas-da-api.md`). Conferido por script em 04/11/2026: nenhuma rota faltando, nenhuma a mais.

Como ver

- Com o sistema no ar: http://localhost:3000/docs (Swagger UI).
- Sem o sistema: abrir o `openapi.yaml` no Swagger Editor.

Como validar

```
cd bloco-8-dev
npx -y @redocly/cli lint openapi.yaml
```

Resultado em 04/11/2026: `Your API description is valid.` sem erros nem avisos, com as regras recomendadas do Redocly (`redocly.yaml` só desliga o aviso de servidor `localhost` , que é o servidor do Docker Compose de propósito). O CI roda o mesmo comando e bloqueia o merge se o contrato quebrar.

Regras do contrato

Regra	Como aparece no contrato
Dinheiro em centavos inteiros	Todo campo `valorCentavos` é `integer` com `format: int64`
Idempotência em toda operação que move dinheiro	Parâmetro `Idempotency-Key` (UUID, obrigatório) em `POST /pix/envios` , `POST /boletos/pagamentos` , `POST /metas/{id}/depositos` e `POST /metas/{id}/resgates` . Repetir a chave devolve 200 com a transação original
Autenticação	`bearerAuth` (JWT) por padrão. Rotas públicas com `security: []` . Cadastro com `tokenCadastro` . Webhook com `assinaturaParceiro` (HMAC)
Papel exigido nas rotas `/admin`	Extensão `x-papel` (ATENDENTE, ANALISTA_ANTIFRAUDE, ADMINISTRADOR)
Erro padronizado	Toda resposta de erro usa o schema `Erro` : `codigo` estável, `mensagem` simples para mostrar e falar, `correlacaoId`
Texto para o áudio	Respostas que o app fala trazem `textoFalado` (saldo, extrato, recebedor, boleto)
Paginação	Por cursor (`cursor` e `limite` , até 50 itens)

Respostas de erro reutilizadas

Status	Nome	Quando
400	Invalido	Campo faltando ou fora do formato
401	NaoAutenticado	Token ausente, vencido ou inválido
403	Proibido	Sem o papel ou sem o vínculo (ABAC)
404	NaoEncontrado	Recurso não existe ou não é do cliente
409	Conflito	Já existe ou já foi decidido
422	RegraDeNegocio	Saldo, limite, conta bloqueada
429	MuitasRequisicoes	Limite de requisições (WAF e login)
503	ParceiroIndisponivel	Parceiro BaaS ou voz fora do ar, com a frase "Seu dinheiro não saiu"

8.6 Sprint 1 detalhado

Período	30/11/2026 (segunda) a 11/12/2026 (sexta), 10 dias úteis
Meta do sprint	**Antônio entra com a digital num app que fala**
Capacidade	27,2 SP (velocidade de 34 SP x 80% de rampa, ver Bloco 7)
Planejado	26 SP em 4 itens
Pré-requisito	Sprint 0 concluído: repositório e padrões (T01), esqueleto da API com `/health` (T06), pipeline de CI (T04), porta bancária com adaptador falso (T09)

Itens do sprint

Item	Descrição	SP	Responsável principal	Apoio
T12	Esqueleto do app Expo: navegação, tema acessível, botão "Ouvir", cliente gerado do OpenAPI	8	Dev Mobile 1	UX (tema, tamanhos e contraste)
T07	Migrações e razão contábil: carteiras, lançamentos e gatilho de soma zero	8	Tech Lead	DevOps (banco do Docker Compose e do pipeline)
US02	Antônio: entrar com a digital	5	Dev Mobile 2 (app) e Dev Back-End (API)	QA (E2E), UX (texto e áudio)
T11	Porta de voz: Amazon Polly, cache no S3 e voz do celular sem internet	5	Dev Back-End	Dev Mobile 1 (botão "Ouvir")
	Total	**26**		

Como ainda não existe a abertura de conta (US01, sprint 2), a US02 usa um cliente de teste criado pelo script de dados do ambiente de desenvolvimento.

Critérios de aceite

T12 · Esqueleto do app

- O app abre no Android 10 e no iOS 16 com as telas de entrada e inicial vazias.
- Todo componente base (botão, campo, texto) tem rótulo de acessibilidade, área de toque de 48 dp e texto base de 18 sp (RNF09).
- O botão "Ouvir" aparece fixo em todas as telas e toca o texto da tela pelo serviço de voz.
- O cliente HTTP do app é gerado a partir do `openapi.yaml` e o build falha se o contrato mudar sem gerar de novo.

T07 · Razão contábil

- A migração cria `carteira`, `transacao` e `lancamento` conforme o Bloco 5 e roda do zero no Docker Compose.
- Teste de integração: uma transação com lançamentos que não somam zero é **recusada pelo banco**.
- Teste de integração: o saldo de uma carteira é a soma dos seus lançamentos e a rota `GET /v1/contas/me/saldo` responde em menos de 300 ms no p95 com 10.000 lançamentos na carteira (RNF01).

US02 · Entrar com a digital (copiado do Bloco 2)

Como **Antônio**, quero entrar no app com a minha digital, para não precisar lembrar nem digitar senha.

- **Dado** que criei a senha de 6 números, **quando** aceito usar a biometria, **então** os próximos acessos pedem só a digital.
- **Dado** que errei a senha 5 vezes, **quando** tento a 6ª, **então** o acesso fica bloqueado por 30 minutos e o app explica por voz.
- **Dado** que meu celular não tem biometria, **quando** entro, **então** o app pede a senha de 6 números com teclado grande.

Critérios técnicos que vêm dos RNF e do ADR-010:

- A senha é guardada com Argon2id. O token de acesso vale 15 minutos e o de renovação, 30 dias, trocado a cada uso (RNF05).
- O login exige a assinatura da chave do aparelho guardada no Keystore ou Keychain. A biometria nunca sai do celular.

T11 · Porta de voz

- `POST /v1/falas` devolve a URL de um áudio para um texto em português.
- Frase fixa repetida é servida do cache no S3 sem chamar o Polly de novo (conferido pelo log).
- Sem internet, o app usa a voz do próprio celular e o usuário não fica sem áudio.
- Em desenvolvimento e no pipeline roda o adaptador falso, sem custo e sem rede.

Tarefas por pessoa

Perfil	Tarefas no sprint
Product Owner	Aceite dos 4 itens na Review. Refinamento de US01, US04, T08 e T02 para o sprint 2. Retomar a conversa com os 2 parceiros BaaS (R04)
Tech Lead	T07. Revisão de todo PR de identidade e contas. Scrum Master do sprint
Dev Back-End Pleno	Rotas `/auth/login`, `/auth/renovacao`, `/auth/logout` da US02. T11
Dev Mobile 1	T12 e integração do botão "Ouvir" com o T11
Dev Mobile 2	Telas de senha, biometria e bloqueio da US02
DevOps (meio período)	PostgreSQL no Docker Compose e no pipeline para o T07. Bucket S3 de áudio em homologação
QA	Teste E2E de login com Maestro. Testes de integração do razão junto com o Tech Lead
UX	Textos e roteiro de áudio das telas de entrada. Preparar e conduzir o teste com 5 pessoas do perfil

Cerimônias e datas

Cerimônia	Data	Horário
Sprint Planning	30/11/2026	9h às 11h
Daily	Todos os dias úteis	9h30
Refinamento do sprint 2	02/12/2026	14h às 15h
Teste com 5 usuários	10/12/2026	9h às 13h
Sprint Review	11/12/2026	10h às 11h
Retrospectiva	11/12/2026	15h às 16h

Como saber se o sprint deu certo

Na Review, uma pessoa do perfil do Antônio, com um celular Android de entrada, entra no app só com a digital e ouve a tela inicial sem ninguém ler para ela.

Bloco 9 · Qualidade e Processo

9.1 Glossário e modelo de domínio

O modelo de domínio completo (entidades, atributos, tipos e relações) está em bloco-3-arquitetura/05-modelo-de-dominio.md e é a fonte única. Este glossário explica os termos em linguagem simples e aponta para lá; ele não redefine nada.

Regra para o time: no código, no banco, nas telas internas e nas conversas usamos **estas palavras e só estas**. Se um termo novo aparecer numa história, ele entra aqui antes de entrar no código.

Termos do negócio

Termo	O que significa	No modelo
Cliente	Pessoa que tem conta no Rufus. Pode ser titular (adulto) ou menor (16 ou 17 anos)	`Cliente`, campo `tipo`
Titular	Cliente adulto, responsável pela própria conta	`TipoCliente.TITULAR`
Menor	Cliente de 16 ou 17 anos, com conta vinculada a um responsável	`TipoCliente.MENOR`
Responsável	Cliente adulto que aceitou o vínculo com um menor e define os limites dele	`VinculoResponsavel.responsavelClienteId`
Vínculo	Ligação entre menor e responsável, com os limites diário e mensal	`VinculoResponsavel`
Pedido de aprovação	Pedido do menor para fazer uma saída acima do limite. Vale 24 horas	`PedidoAprovacao`
Contato de confiança	Pessoa avisada quando sai dinheiro fora do padrão. Não vê o saldo nem movimenta a conta	`ContatoConfianca`
Conta	A conta de pagamento do cliente, mantida pelo parceiro BaaS	`Conta`
Carteira	Divisão interna do dinheiro de uma conta: a principal e uma por meta. Também existem carteiras técnicas de liquidação	`Carteira`
Saldo	Soma dos lançamentos de uma carteira. Nunca é um número editado	Soma de `Lancamento.valorCentavos`
Lançamento	Registro de crédito ou débito numa carteira. Toda transação tem pelo menos dois e a soma é zero	`Lancamento`
Partidas dobradas	Regra contábil: todo dinheiro que sai de um lugar entra em outro	ADR-007
Transação	Qualquer movimentação: Pix enviado, Pix recebido, boleto, guardar ou resgatar da meta	`Transacao` e subtipos
Pix enviado	Transferência instantânea para outra pessoa, por chave, contato ou QR Code	`PixEnvio`
Pix recebido	Dinheiro que chega por Pix	`PixRecebimento`
Chave Pix	Identificador para receber Pix: CPF, celular ou chave aleatória	`ChavePix`
Cobrança (QR Code)	QR Code gerado para receber um valor	rota `POST /pix/cobrancas`
Destinatário novo	Quem nunca recebeu Pix daquele cliente. Tem limite menor nas primeiras 24 horas	`PixEnvio.destinatarioNovo`
Boleto	Conta paga pela linha digitável ou pelo código de barras	`PagamentoBoleto`

Termo	O que significa	No modelo
Meta	Objetivo de economia, com valor, data e imagem. O dinheiro guardado fica numa carteira própria	`Meta`
Bloquear tudo	Botão que suspende todas as saídas de dinheiro na hora	`Conta.status = BLOQUEADA_TOTAL`
Pix retido	Pix que o antifraude segurou para análise humana	`StatusTransacao.RETIDA`
Nota de risco	Pontuação de 0 a 100 dada pelo antifraude. 70 ou mais retém o Pix	`AnaliseRisco.nota`
Chamado	Conversa de atendimento aberta pelo cliente dentro do app	`Chamado`, `MensagemChamado`
Funcionário	Pessoa da equipe interna: atendente, analista antifraude ou administrador	`Funcionario`, `Papel`
Consentimento	Aceite registrado de termo de uso, privacidade, biometria ou do responsável legal	`Consentimento`
Dado pessoal (PII)	Informação que identifica a pessoa. Marcado no dicionário de dados do Bloco 5	Coluna LGPD do dicionário

Termos técnicos

Termo	O que significa	Onde aparece
Parceiro BaaS	Instituição autorizada pelo Banco Central que mantém a conta e liquida o Pix e o boleto. O Rufus é a interface	ADR-002, Bloco 1
Porta e adaptador	Interface nossa (porta) e a tradução para cada fornecedor (adaptador)	`PortaBancaria`, `PortaVoz`
Adaptador falso	Implementação da porta que responde sem rede, usada no localhost e nos testes	Bloco 8
Chave de idempotência	Código enviado no cabeçalho `Idempotency-Key`. A mesma chave nunca cria duas transações	RNF11, US07
Outbox	Tabela onde a API grava os eventos na mesma transação do dado. O worker lê e executa	ADR-008, `OutboxEvento`
Worker	Processo separado da API que executa os eventos do outbox	ADR-009
Saga	Sequência de passos com compensação: reservar saldo, enviar ao parceiro, confirmar ou estornar	Bloco 3, padrões
Estorno	Compensação que devolve o dinheiro reservado quando o parceiro recusa ou não responde	`StatusTransacao.ESTORNADA`
Conciliação	Comparação diária entre o nosso razão e o extrato do parceiro	RNF11, T18
Token de acesso	JWT de 15 minutos enviado em cada requisição	ADR-010
Token de renovação	Token de 30 dias, trocado a cada uso, que gera um novo token de acesso	`SessaoRenovacao`
Dispositivo vinculado	O único celular autorizado a usar a conta. Assina o login com uma chave guardada no próprio aparelho	`Dispositivo`, RF005
TOTP	Código de 6 dígitos que muda a cada 30 segundos, usado como segundo fator dos funcionários	`Funcionario.segredoTotp`
p95	Tempo dentro do qual 95% das requisições respondem	RNF01, RNF02
Story point (SP)	Medida relativa de esforço. Referência: US13 = 3 SP	Bloco 2, Bloco 7
DoR e DoD	Definition of Ready (pronto para entrar no sprint) e Definition of Done (pronto de verdade)	9.4 e 9.5

Modelo de domínio em uma página

Módulo	Entidades	Regra mais importante
identidade	Cliente, Dispositivo, Credencial, SessaoRenovacao, Consentimento	Só 1 dispositivo ativo por cliente
contas	Conta, Carteira, Lancamento	A soma dos lançamentos de uma transação é zero

Módulo	Entidades	Regra mais importante
pagamentos	Transacao (PixEnvio, PixRecebimento, PagamentoBoleto, MovimentoMeta), ChavePix, Meta	A mesma chave de idempotência não cria duas transações
familia	VinculoResponsavel, PedidoAprovacao, ContatoConfianca	O menor não passa do limite sem aprovação do responsável
antifraude	AnaliseRisco	Nota 70 ou mais retém o Pix
atendimento	Chamado, MensagemChamado	Toda consulta de funcionário a dados do cliente vai para a auditoria
backoffice	Funcionario, RegistroAuditoria	Registro de auditoria nunca é alterado e fica 5 anos
notificacao	Notificacao, OutboxEvento, LembreteConta	Evento só é marcado como processado depois de executado

O diagrama de classes com tipos, visibilidades e relações está no Bloco 4.

9.2 Estratégia de testes

O Rufus mexe com dinheiro de quem tem medo de errar, então a estratégia testa mais onde o erro custa mais: razão contábil, Pix, identidade e acessibilidade. Cada tipo de teste diz o que cobre, a ferramenta, quando roda e qual requisito protege.

Pirâmide

Nível	Quantidade relativa	Tempo no pipeline	Ferramenta
Unidade	Muitos	Menos de 2 minutos	Jest
Integração com banco real	Médio	Menos de 6 minutos	Jest com Testcontainers (PostgreSQL 17)
Contrato da API	Médio	Menos de 2 minutos	Validação das respostas contra o `openapi.yaml`
E2E mobile	Poucos, só os fluxos principais	Menos de 15 minutos, em homologação	Maestro
Carga	1 cenário por versão grande	20 minutos, em homologação	k6
Acessibilidade	Em toda tela	Automático no PR e manual por sprint	Lint de acessibilidade do React Native, TalkBack e VoiceOver
Teste com usuários	5 pessoas por sprint	Fora do pipeline	Roteiro do UX com cronômetro

Tipos de teste

Tipo	O que cobre	Exemplo no Rufus	Requisito protegido
Unidade	Regras de domínio sem banco nem rede: estados da transação, regras do antifraude, limites	`RegraDestinatarioNovo` devolve a nota certa. Transação CONCLUIDA não pode virar ESTORNADA	RF019, RF026
Integração	Repositórios, SQL, gatilho do razão, outbox, chave de idempotência, contra um PostgreSQL de verdade em container	Transação com lançamentos que não somam zero é recusada pelo banco. Dois `POST /pix/envios` com a mesma chave criam uma transação só	RNF11, ADR-007, ADR-008
Contrato	Toda resposta da API confere com o `openapi.yaml`. O cliente do app é gerado do mesmo arquivo	Campo novo sem estar no contrato quebra o teste	ADR-003
Contrato com o parceiro	Adaptador real contra o sandbox do parceiro BaaS, toda noite	Mudança de formato no sandbox é detectada antes de chegar à produção	R09
E2E	Fluxos do cliente no app, em homologação, com o adaptador falso	Abrir conta, entrar com senha, enviar Pix, pagar boleto, bloquear tudo	US01, US02, US06, US08, US12
Carga	100 requisições por segundo por 10 minutos nas rotas de leitura e de Pix	p95 das leituras abaixo de 300 ms e taxa de erro abaixo de 1%	RNF01, RNF04
Segurança	Varredura de dependências e da imagem (npm audit, Trivy), busca de segredos no código (gitleaks) e teste de invasão antes do lançamento	Nenhuma vulnerabilidade crítica ou alta	RNF05, RNF06
Acessibilidade	Rótulo de acessibilidade em todo elemento tocável, contraste, área de toque, botão "Ouvir"	Botão sem rótulo reprova o PR	RNF09
Usabilidade com usuários	5 pessoas do perfil das personas por sprint, com tarefas cronometradas	Pagar um boleto sem ajuda em até 3 minutos	RNF08
Restauração	Restaurar o backup do RDS num banco novo e conferir os saldos	A cada 3 meses e antes do lançamento (T20)	RNF12

Ambientes de teste

Onde	O que roda	Parceiro BaaS
Máquina do dev	Unidade, integração (Testcontainers)	Adaptador falso
Pipeline (GitHub Actions)	Unidade, integração, contrato, lint, varredura de segurança	Adaptador falso
Homologação	E2E, carga, contrato com o parceiro	Sandbox do parceiro (a partir do sprint 5)
Produção	Smoke test depois de cada deploy (`/health` e login de uma conta de teste)	Parceiro real

Dados de teste

- Nunca usar dado real de cliente fora da produção.
- Dados de teste gerados por script com CPFs válidos de teste, recriados a cada execução.
- Contas de teste em produção marcadas e excluídas da conciliação e das métricas de negócio.

A lição do Ariane 5: redundância não protege contra defeito de software

O caso da Aula 02: em 1996 o foguete Ariane 5 explodiu 40 segundos depois da decolagem porque uma conversão de número com ponto flutuante para inteiro de 16 bits recebeu um valor fora da faixa. O computador reserva rodava **o mesmo software** e falhou junto. E o código com defeito nem era mais necessário.

O Rufus tem a mesma armadilha: a disponibilidade de 99,9% vem de redundância (2 zonas, banco Multi-AZ, 2 NAT, Bloco 6), mas as duas zonas rodam **a mesma imagem**. Um defeito de software derruba as duas ao mesmo tempo. Por isso a proteção contra defeito de software é outra:

Proteção	Onde
Testes em camadas, com o razão e o outbox testados contra PostgreSQL real	Pirâmide e tipos de teste, acima
Regras críticas garantidas pelo banco, não só pelo código (partidas dobradas, idempotência, um aparelho ativo)	Bloco 5, regras RN02, RN12, RN13
Dinheiro sempre em centavos inteiros (`bigint`), nunca em ponto flutuante: o mesmo tipo de erro de conversão do Ariane	ADR-004, Bloco 5
Homologação e smoke test antes de produção	Bloco 6, etapas 6 e 7 do pipeline
Deploy gradual com circuit breaker do ECS, que desfaz a versão nova sozinho se ela não passar no `/health`	Bloco 6, etapa 8
Menos código que não precisa existir: o lint barra variável e import sem uso (`no-unused-vars` do typescript-eslint) e import entre módulos, e a revisão de código pede a remoção do que sobrou	`codigo/backend/eslint.config.mjs` , guia de padrões (Bloco 8)

9.3 Metas de cobertura de código

A meta é maior onde o erro mexe com dinheiro ou com acesso à conta. Cobertura mede **linhas e ramos** executados pelos testes de unidade e de integração juntos.

Módulo	Linhas	Ramos	Por que esse número
identidade	90%	85%	Login, troca de aparelho e sessão são a principal barreira contra golpe (ADR-010, R01)
contas	90%	85%	Razão contábil e saldo. Erro aqui é dinheiro errado (RNF11)
pagamentos	90%	85%	Pix, boleto, idempotência e saga com estorno
antifraude	85%	80%	Cada regra de risco é uma Strategy pequena e fácil de testar
familia	80%	75%	Limites e aprovações. Mexe com dinheiro, mas sempre passa pelo módulo de pagamentos
atendimento	70%	60%	Fluxo de chamado, pouca regra de negócio
notificacao	70%	60%	Envio é idempotente e testado na integração do outbox
backoffice	70%	60%	Cadastro de funcionário e auditoria
app mobile (lógica, fora das telas)	70%	60%	Estados de tela e chamadas à API. As telas são cobertas pelo E2E e pelos testes com usuários
Projeto inteiro (mínimo)	**80%**	**70%**	Média ponderada esperada com as metas acima

Como o pipeline cobra

1. O Jest roda com `--coverage` e um `coverageThreshold` **por pasta de módulo** com os números acima.
2. Se qualquer módulo ficar abaixo da meta, o job de testes falha e o pull request **não pode ser mesclado** (regra de proteção da branch `main`).
3. Pull request que **diminui** a cobertura de identidade, contas ou pagamentos é bloqueado mesmo se continuar acima da meta.
4. O relatório de cobertura fica anexado a cada execução do pipeline.

O que não conta

- Código gerado (cliente do OpenAPI, migrações) fica fora da medição.
- Cobertura alta não substitui o teste de integração com o banco real: o gatilho do razão e o outbox só são aceitos com teste de integração (DoD).

9.4 Definition of Ready (DoR)

Uma história ou tarefa só entra no Sprint Planning se cumprir **todos** os itens abaixo. Quem confere é o PO, no refinamento da semana anterior.

Para histórias de usuário

- [ ] Está no formato "Como [persona], quero [ação], para [objetivo]" e a persona é uma das do Bloco 1 ou um papel do modelo de domínio.
- [ ] Tem pelo menos 2 critérios de aceite em Dado/Quando/Então, incluindo pelo menos 1 caso de erro.
- [ ] Está ligada a pelo menos um RF do Bloco 2 e tem prioridade MoSCoW.
- [ ] Foi estimada pela equipe em story points e tem **8 SP ou menos**. Com 13, é quebrada antes.
- [ ] As telas foram desenhadas pelo UX, com o **texto de cada tela e o roteiro do áudio** do botão "Ouvir" escritos em linguagem simples.
- [ ] As rotas que ela usa existem no `openapi.yaml` ou a mudança no contrato está descrita.
- [ ] As dependências estão prontas ou estão no mesmo sprint, antes dela (por exemplo, o razão contábil antes do saldo).
- [ ] Se mexe com dado pessoal, os campos estão marcados no dicionário de dados (Bloco 5).
- [ ] Se mexe com dinheiro, diz qual transação cria e quais lançamentos (débito e crédito).

Para tarefas técnicas

- [ ] Tem objetivo e critério de pronto verificável (por exemplo, "`terraform apply` cria a VPC com 2 subnets privadas e 2 públicas em homologação").
- [ ] Está ligada a um ADR, RNF ou risco.
- [ ] Foi estimada em story points e tem 8 SP ou menos.

9.5 Definition of Done (DoD)

Um item só está **pronto** quando cumpre todos os itens que se aplicam a ele. Item que não cumpre volta para o backlog e não conta na velocidade do sprint.

Código

- [ ] Código revisado e aprovado por pelo menos 1 pessoa que não escreveu. Mudança em identidade, contas ou pagamentos precisa da aprovação do Tech Lead.
- [ ] Segue o guia de padrões de código e passa no lint sem aviso.
- [ ] Nenhum módulo acessa tabela de outro módulo (regra de dependência do ADR-001 no lint).
- [ ] Nenhum segredo, senha ou dado pessoal no código ou no log.

Testes

- [ ] Testes de unidade das regras novas.
- [ ] Teste de integração com PostgreSQL real quando mexe com banco, razão contábil, outbox ou idempotência.
- [ ] Cobertura do módulo dentro da meta de 03-metas-de-cobertura.md.
- [ ] Todos os critérios de aceite da história testados, de preferência automatizados.
- [ ] Pipeline verde: lint, testes, build, varredura de segurança.

Segurança

- [ ] **Nenhuma vulnerabilidade crítica ou alta** aberta em dependência ou imagem (RNF06).
- [ ] Rota nova protegida por autenticação e papel corretos, ou marcada como pública no OpenAPI com o motivo.
- [ ] Ação de funcionário gera registro de auditoria (RNF14).

Contrato e documentação

- [ ] `openapi.yaml` atualizado e o teste de contrato passando.
- [ ] Migração de banco no padrão expandir e contrair, sem parar o sistema (Bloco 5).
- [ ] Dicionário de dados atualizado se houver coluna nova, com a marcação LGPD.
- [ ] README e `.env.example` atualizados se houver variável nova.

Acessibilidade e linguagem (toda tela)

- [ ] Botão "Ouvir" funcionando e o áudio confere com o texto da tela.
- [ ] Todo elemento tocável tem rótulo para o TalkBack e o VoiceOver, área de toque de pelo menos 48 dp e contraste de pelo menos 4,5:1 (RNF09).
- [ ] Texto revisado pelo UX: frases curtas e sem jargão.
- [ ] A ação principal da tela cabe no limite de 3 toques depois do login, quando for uma das ações principais (RNF08).
- [ ] Testado em um Android de entrada real com 3 GB de RAM (RNF10).

Aceite

- [ ] Funcionando em homologação, e não só no localhost.
- [ ] Demonstrado na Sprint Review e aceito pelo PO.
- [ ] Se a tela é nova, passou no teste de uso do sprint com pessoas do perfil das personas, ou o problema encontrado virou item no backlog.

9.6 Metodologia de desenvolvimento

Decisão: Scrum com sprints de 2 semanas, com um quadro Kanban dentro de cada sprint para acompanhar o fluxo das tarefas.

Primeiro: o modelo de ciclo de vida

A Aula 03 lembra que o modelo de ciclo de vida é a **primeira escolha** do processo de software. Pela NBR ISO/IEC 12207, ciclo de vida é a estrutura de processos, atividades e tarefas do desenvolvimento, da operação e da manutenção, da definição dos requisitos até o fim do uso. O critério de escolha (Wazlawick) é o quanto os requisitos são conhecidos e estáveis, quando o cliente precisa ver a primeira versão e o tamanho e o risco do sistema.

Decisão: modelo Incremental e iterativo, executado com Scrum, com duas práticas emprestadas de outros modelos: a análise de risco a cada ciclo (do Espiral) e o teste planejado junto com cada requisito (do modelo V).

Modelo	Serve quando	No Rufus	Escolha
Cascata	Requisitos bem conhecidos e congelados, sistema simples	A usabilidade só se confirma com usuário real (RNF08), e a 1ª versão só sairia no fim. O público nunca foi ouvido por um banco	Descartado
Modelo V	Mesmo caso do cascata, com foco em planejar testes	Qualquer mudança no meio obriga a refazer documentos de teste, análise e requisitos. Ficamos só com a ideia de planejar o teste junto do requisito (critérios Given/When/Then e DoR)	Descartado, com uma prática aproveitada
Incremental	Requisitos tratados em módulos, valor entregue cedo	Os 7 épicos (E1 a E7) viram incrementos. O beta com usuários reais chega no sprint 8, sem esperar o fim de tudo	**Escolhido**
Evolutivo	Requisitos pouco conhecidos, que surgem do uso	O núcleo é conhecido e regulado (Pix, LGPD, regras do Bloco 2). E o evolutivo dificulta limitar escopo e prazo, o que o parceiro BaaS e os investidores exigem	Descartado
Espiral (Boehm, 1988)	Sistema grande e de alto risco, guiado por análise de risco	O risco é alto (golpe, dinheiro, regulação), mas o espiral completo é caro e não tem número de voltas definido. Ficamos com a análise de risco a cada ciclo: a matriz do Bloco 7 é revista a cada 2 sprints	Descartado, com uma prática aproveitada

As 3 etapas macro do ciclo de vida no projeto:

Etapa	No Rufus
1. Definição	Este trabalho: problema, requisitos, arquitetura, banco e plano (Blocos 1 a 9)
2. Desenvolvimento	Sprints 0 a 8, de 16/11/2026 a 19/03/2027 (Bloco 7)
3. Operação	Depois do lançamento: sustentação em Kanban, correções e novos requisitos. Pela referência da Aula 01, a manutenção custa de 15% a 20% do custo inicial por ano (Bloco 7, item 7.3)

Por que Scrum e não Kanban puro

Critério	Scrum	Kanban puro	Por que pesa para o Rufus
Entrega com data	Sprint de duração fixa, meta por sprint	Fluxo contínuo, sem data por pacote	O parceiro BaaS, as lojas e os investidores precisam de datas (risco R04, R12). O cronograma do Bloco 7 é por sprint
Teste com usuários	A Review de cada sprint é o momento natural do teste com 5 pessoas do perfil	Precisaria de uma cadência criada à parte	O RNF08 exige teste de uso a cada ciclo, e a interface é o produto
Equipe nova	Cerimônias fixas ajudam um time que nunca trabalhou junto a criar ritmo	Exige um time já maduro para se auto-organizar só pelo quadro	Os 8 perfis do Bloco 7 começam juntos no sprint 0

Critério	Scrum	Kanban puro	Por que pesa para o Rufus
Medição de velocidade	Velocidade por sprint, usada para recalcular o plano depois do sprint 2	Mede tempo de ciclo, bom para demanda contínua	O plano do Bloco 7 depende de velocidade em SP
Trabalho imprevisível	Mudança espera o próximo sprint	Absorve demanda que chega a qualquer hora	Pesa para Kanban **depois do lançamento**, quando aparecem incidentes e pedidos do SAC

Kanban seria a melhor escolha para a fase de operação, com muitos pedidos pequenos e incidentes. Para construir o MVP com data e com teste de uso a cada ciclo, Scrum encaixa melhor. Depois do lançamento a equipe reavalia, e o mais provável é passar a Kanban para a sustentação.

Papéis

Papel do Scrum	Quem	Observação
Product Owner	Product Owner (Bloco 7)	Dono do backlog e do aceite. Na equipe do trabalho de faculdade: Pedro Oliveira
Scrum Master	Tech Lead, acumulando	Equipe de 8 pessoas não justifica um Scrum Master dedicado. Se a velocidade cair dois sprints seguidos, a decisão é revista
Time de desenvolvimento	2 back-end, 2 mobile, DevOps, QA, UX	QA e UX fazem parte do time e respondem pela DoD

Cerimônias

Cerimônia	Quando	Duração	Resultado
Sprint Planning	Segunda de manhã, 1º dia do sprint	2 horas	Meta do sprint e itens escolhidos dentro da capacidade
Daily	Todo dia às 9h30	15 minutos	Impedimentos visíveis
Refinamento	Quarta da 1ª semana	1 hora	Itens do próximo sprint cumprindo a DoR
Teste com usuários	Quinta da 2ª semana	Meio dia	5 sessões com pessoas do perfil das personas
Sprint Review	Sexta da 2ª semana, de manhã	1 hora	Demonstração em homologação e aceite do PO
Retrospectiva	Sexta da 2ª semana, à tarde	1 hora	Até 3 ações de melhoria. A cada 2 sprints também revisa a matriz de riscos

Artefatos

- **Product Backlog:** as 24 histórias do Bloco 2 e as 20 tarefas técnicas do Bloco 7, em ordem de prioridade.
- **Sprint Backlog:** os itens do sprint, num quadro com as colunas A fazer, Fazendo, Em revisão, Em homologação e Pronto. O limite é de 2 itens em "Em revisão" por pessoa, para a revisão não virar gargalo.
- **Incremento:** o que cumpre a DoD e está em homologação ao fim do sprint.

Fontes e referências

Toda fonte usada no projeto, agrupada pelo que ela sustenta. As páginas foram abertas e os números e artigos conferidos nelas em 04/11/2026. O texto das leis foi conferido no site do Planalto e o da Resolução BCB 142 na API de normativos do Banco Central.

Problema e público (Bloco 1)

Dado usado	Fonte
21,1 milhões de adultos sem usar produto financeiro em 2024; 53% de acerto em conhecimento financeiro; 87% erraram juros simples; letramento cai com a idade; 59% dos idosos usam internet	Banco Central do Brasil. *Relatório de Cidadania Financeira 2025*, seções 1.1 e 1.2. https://www.bcb.gov.br/content/cidadaniafinanceira/documentos_cidadania/RIF/relatorio_de_cidadania_financeira_2025.pdf
29% de analfabetos funcionais (15 a 64 anos); 51% entre 50 e 64 anos; 73% dos analfabetos funcionais usam internet e 60% deles estão no nível mais baixo de habilidade digital	Inaf 2024 (Ação Educativa e Conhecimento Social), divulgado em 05/05/2025. Comunicado do UNICEF Brasil: https://www.unicef.org/brazil/comunicados-de-imprensa/analfabetismo-funcional-nao-apresenta-melhora-e-alcanca-29-por-cento-dos-brasileiros-mesmo-patamar-de-2018-aponta-novo-levantamento-do-inaf
139 mil relatos do golpe da falsa central ou falso funcionário no 1º semestre de 2025, alta de 195,7%	Dados da Febraban divulgados em 11/11/2025: https://conexaoto.com.br/2025/11/11/golpe-da-falsa-venda-e-o-mais-aplicado-contra-clientes-bancarios-no-1o-semestre-do-ano

Custos de infraestrutura (Blocos 1, 3 e 6)

Dado usado	Fonte
Preço de cada serviço da AWS em São Paulo (Fargate, RDS, ALB, NAT, IPv4, WAF, Secrets Manager, CloudWatch, S3, ECR, Polly, saída de dados)	AWS Price List API, um arquivo oficial por serviço, região sa-east-1. Lista completa com a data de publicação de cada arquivo no item 4.4 (custo item a item). Endereço base: https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/index.json
EKS: US$ 0,10 por hora por cluster em São Paulo (ADR-009)	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEKS/current/sa-east-1/index.json

Dado usado	Fonte
Lambda: US$ 0,0000166667 por GB-segundo e US$ 0,20 por milhão de requisições em São Paulo (ADR-009)	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AWSLambda/current/sa-east-1/index.json
RDS db.m7g.large Multi-AZ a US$ 0,442 por hora (crescimento, arguição)	Mesma tabela do RDS listada no item 4.4
Dólar a R$ 5,4778 (PTAX de venda de 04/11/2026)	Banco Central, API PTAX: https://olinda.bcb.gov.br/olinda/servico/PTAX/versao/v1/odata/
Conferência manual dos custos	Calculadora oficial da AWS: https://calculator.aws
SMS para o Brasil: US$ 0,03 por mensagem	CSV oficial de preços de SMS por país da AWS End User Messaging: https://d1.awsstatic.com/onedam/marketing-channels/website/aws/en_US/business-applications/approved/documents/End-User-Messaging-SMS-Prices.ebc340b4d416d90832dd59629c4792b0deb6f8bc.csv
Route 53: US$ 0,50 por zona e US$ 0,40 por milhão de consultas	https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonRoute53/current/index.json
Expo EAS Starter: US$ 19 por mês, 3.000 usuários incluídos e US$ 0,005 por usuário a mais	https://expo.dev/pricing
Expo Push sem custo por envio	https://docs.expo.dev/push-notifications/faq/
GitHub Team: US$ 4 por usuário por mês, 3.000 minutos de Actions	https://github.com/pricing
Domínio .com.br: R$ 40,00 por ano	https://registro.br/
Teste de invasão: R$ 10.000 a R$ 25.000 para aplicação, API e autenticação (referência de mercado 2026, não cotação)	https://www.freelasemcrise.com.br/quanto-cobrar/pentest-teste-seguranca

Leis e normas

Norma	Onde usamos	Fonte
Lei 13.709/2018 (LGPD): art. 5º, II (biometria é dado pessoal sensível), art. 14 (dados de crianças e adolescentes no seu melhor interesse), art. 18 (direitos do titular), art. 19, II (resposta completa em até 15 dias), art. 33 (transferência internacional)	RNF07, RF029, ADR-013, Bloco 5	https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm
Lei 9.613/1998, art. 10, §2º: cadastros e registros guardados por no mínimo 5 anos a partir do encerramento da conta ou da conclusão da transação	RNF14, Bloco 5	https://www.planalto.gov.br/ccivil_03/leis/l9613.htm
Lei 10.097/2000 (aprendizagem)	Persona Camila	https://www.planalto.gov.br/ccivil_03/leis/l10097.htm
Código Civil, art. 4º (maiores de 16 e menores de 18 anos assistidos pelos pais)	RF014	https://www.planalto.gov.br/ccivil_03/leis/2002/l10406compilada.htm
Resolução BCB nº 142/2021, art. 2º, I: limite de R$ 1.000 entre 20h e 6h por conta, entre pessoas físicas	RF019, US11	https://www.bcb.gov.br/estabilidadefinanceira/exibenormativo?tipo=Resolu%C3%A7%C3%A3o%20BCB&numero=142

Padrões técnicos

Padrão	Onde usamos	Fonte
WCAG 2.2 (contraste mínimo de 4,5:1, nível AA)	RNF09	https://www.w3.org/TR/WCAG22/
Área de toque de 48 dp no Android	RNF09	https://developer.android.com/guide/topics/ui/accessibility/apps
OWASP Password Storage Cheat Sheet (Argon2id)	RNF05, ADR-010	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
OWASP Top 10 (2021)	Bloco 6	https://owasp.org/Top10/
Teste de usabilidade com 5 usuários (Nielsen)	RNF08	https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/
Restauração a ponto no tempo do RDS	RNF12, Bloco 5	https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_PIT.html
Modelo C4	Bloco 3	https://c4model.com/
Architecture Decision Records	Bloco 3	https://adr.github.io/
OpenAPI 3.0	Bloco 8	https://spec.openapis.org/oas/v3.0.3
Conventional Commits	Bloco 8	https://www.conventionalcommits.org/pt-br/v1.0.0/

Salários e custo da equipe (Bloco 7)

Consultadas em 04/11/2026. O Glassdoor (glassdoor.com.br), indicado no enunciado, recusou o acesso automático (HTTP 403, exige login); a conferência no Glassdoor está na tabela do item 7.3, em coluna própria.

Perfil	Valor usado	Fonte
Product Owner	R$ 8.750,00	Robert Half, Guia Salarial 2027, Recife, mediana: https://www.roberthalf.com/br/pt/vagas-detalhes/product-owner-po/recife
Tech Lead (Back-End Sênior)	R$ 14.500,00	https://www.roberthalf.com/br/pt/vagas-detalhes/desenvolvedora-back-end-senior/recife
Desenvolvedor Back-End Pleno	R$ 11.800,00	https://www.roberthalf.com/br/pt/vagas-detalhes/desenvolvedora-back-end-pleno/recife
Desenvolvedor Mobile Pleno	R$ 13.800,00	https://www.roberthalf.com/br/pt/vagas-detalhes/desenvolvedora-mobile-pleno/recife
Analista de DevOps	R$ 14.600,00	https://www.roberthalf.com/br/pt/vagas-detalhes/analista-de-devops/recife
Analista de Testes Pleno	R$ 9.100,00	https://www.roberthalf.com/br/pt/vagas-detalhes/analista-de-testes-pleno/recife
Designer UX/UI	R$ 3.344,38	Salário.com.br, média do Novo CAGED em Recife: https://www.salario.com.br/profissao/designer-ux-cbo-262410/recife-pe/
Conta de desenvolvedor Apple	US$ 99 por ano	https://developer.apple.com/support/enrollment/
Conta de desenvolvedor Google Play	US$ 25, uma vez	https://support.google.com/googleplay/android-developer/answer/6112435

Encargos trabalhistas (INSS patronal, RAT, terceiros, FGTS, 13º, férias e um terço) calculados item a item no item 7.3, a partir das alíquotas legais.

Bibliografia e material da disciplina

Referência	Onde usamos
CUNHA, Israel. Slides das Aulas 01 a 05 e enunciado do Projeto Final. Análise e Projeto de Sistemas, UNIPÊ, 2026.2	Todo o projeto. O mapa "A disciplina no projeto" liga cada conceito ao bloco onde ele aparece

Referência	Onde usamos
SOMMERVILLE, Ian. *Engenharia de Software*. 9. ed. São Paulo: Pearson, 2011	Classificação dos RNF em produto, organizacional e externo (2.2). Processo de engenharia de requisitos (2.0)
PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016	Dificuldade de entender requisitos (2.0)
WAZLAWICK, Raul S. *Engenharia de Software: conceitos e práticas*. Rio de Janeiro: Elsevier, 2013	Critério de escolha do ciclo de vida pelo nível de conhecimento e estabilidade dos requisitos (9.6)
BOEHM, Barry W. A spiral model of software development and enhancement. *Computer*, IEEE, v. 21, n. 5, p. 61-72, 1988	Modelo espiral e análise de risco por ciclo (9.6)
ABNT. *NBR ISO/IEC 12207: Tecnologia de informação, processos de ciclo de vida de software*. Rio de Janeiro, 1998	Definição de ciclo de vida (9.6)
DENNIS, Alan; WIXOM, Barbara H.; ROTH, Roberta M. *Análise e Projeto de Sistemas*. 5. ed. Rio de Janeiro: LTC, 2014	Estrutura geral de análise e projeto
Definição de requisito do IEEE (2008), como apresentada na Aula 04	Abertura do item 2.0